



accredited
program

ACQUIN



MODULE HANDBOOK



Program Code: 60610400 – Software Engineering

Program: Bachelor's Program

Submitted to **ACQUIN**
by Termez State University
as part of the Accreditation
Process



Termez city, "Barkamol avlod"
at street, house number 43



+998 76 221 74 55



info@tersu.uz



www.tersu.uz



@tersu340



@tersu340



@terdu_rasmiy



@Termizdavlatuniversitet

No	Module	Page
1ST ACADEMIC YEAR		4
1.	THE CONTEMPORARY HISTORY OF UZBEKISTAN	4
2.	RELIGION STUDIES	10
3.	PHILOSOPHY	18
4.	FOREIGN LANGUAGE 1 (ENGLISH)	24
5.	FOREIGN LANGUAGE 2 (ENGLISH)	31
6.	CALCULUS	37
7.	PHYSICS 1	46
8.	PHYSICS 2	54
9.	PROGRAMMING 1	59
10.	PROGRAMMING 2	65
11.	ACADEMIC WRITING	72
12.	DATA STRUCTURES AND ALGORITHMS	79
2nd ACADEMIC YEAR		88
13.	DIFFERENTIAL EQUATIONS	88
14.	LINEAR ALGEBRA	97
15.	DATABASE	107
16.	CYBERSECURITY FUNDAMENTALS	115
17.	ELECTRONICS AND CIRCUITS 1	126
18.	DISCRETE STRUCTURES	132
19.	COMPUTER NETWORKS	141
20.	ARTIFICIAL INTELLIGENCE FUNDAMENTALS	148
21.	PROBABILITY AND STATISTICS	156
22.	PROFESSIONAL SPEECH CULTURE	165
23.	USE OF THE UZBEK LANGUAGE IN PROFESSIONAL CONTEXTS(ELECTIVE)	171
24.	PHYSICAL CULTURE	176
25.	PHYSICAL TRAINING	182
3rd ACADEMIC YEAR		188
26.	ALGORITHM DESIGN	188
27.	SOFTWARE SYSTEM DESIGN	198
28.	INTRODUCTION TO SOFTWARE ENGINEERING	207
29.	OPERATING SYSTEMS	215
30.	ECOLOGY AND ENVIRONMENTAL PROTECTION	223
31.	PROTECTION OF THE ENVIRONMENT AND GREEN DEVELOPMENT	240
32.	MATHEMATICAL PROGRAMMING	246
33.	MATHEMATICAL MODELING	255

34.	PARALLEL COMPUTER ARCHITECTURE AND PROGRAMMING	264
35.	MOBILE APPLICATION DEVELOPMENT	272
36.	SOFTWARE REQUIREMENTS ANALYSIS	280
37.	NUMERICAL METHODS AND ALGORITHMS	288
38.	COMPUTATIONAL THINKING AND INTRODUCTION TO PROGRAMMING	296
39.	PYTHON PROGRAMMING LANGUAGE	302
40.	R PROGRAMMING LANGUAGE	309
41.	BIG DATA FUNDAMENTALS AND DATA ANALYSIS	316
42.	SOFTWARE PROJECT MANAGEMENT	324
43.	PROGRAMMING STYLES AND PARADIGMS	331
44.	PRACTICAL APPLICATIONS SUITE	339
45.	ARDUINO PROGRAMMING LANGUAGE	346
4th ACADEMIC YEAR		354
46.	INDIVIDUAL PROJECT	354
47.	SOFTWARE TESTING	360
48.	SOFTWARE ARCHITECTURE	367
49.	MODERN METHODS OF COMPUTATIONAL MATHEMATICS	373
50.	COMPUTER GAME DEVELOPMENT	381
51.	CREATION OF E-LEARNING RESOURCES	387
52.	COMPUTER GRAPHICS	395
53.	INTRODUCTION TO MACHINE LEARNING	403
54.	ROBOTICS AND INTELLIGENT SYSTEMS	410
55.	SOFTWARE ARCHITECTURE AND EVOLUTION	416
56.	WEB APPLICATION DEVELOPMENT	422

1ST ACADEMIC YEAR

Field	Description
Module Title	THE CONTEMPORARY HISTORY OF UZBEKISTAN
Module Code	UEY1104
Credit Points (ECTS)	4 Credits
Workload	Total workload: 120 hours Contact hours: 60 hours (30 hours Lecture, 30 hours Seminar lessons) Self-Assignments: 60 hours
Frequency/Duration	I semester
Module objective	
Module Learning Outcomes	Knowledge: L D D D Skills: 4 Significance. 607. Can grasp the essence of events taking place in the global community. Competencies: L 009. Independently analyzes and evaluates the contribution of historical events and individuals to the development of society.
Content	Lectures Introduction. The subject, purpose, and objectives of the "Recent History of Uzbekistan" course, as well as its theoretical and methodological principles. Introduces the scope, objectives, significance, and methodological foundations of the course. Students become familiar with the principles of studying contemporary history, historical sources, and modern approaches to historical research. Historical foundations of independence ideas in Uzbekistan. Examines the historical development of national identity, the struggle for independence, and the intellectual and political movements that contributed to the emergence of independence ideas in Uzbekistan. Socio-political processes in Uzbekistan on the eve of independence. Explores the political, economic, and social

conditions of the late Soviet period, including the challenges and reforms that shaped the transition toward national independence.

The establishment of the independent Republic of Uzbekistan and its historical significance. Uzbekistan's distinctive path to independence and development. Discusses the declaration of independence, the formation of state institutions, and the adoption of the "Uzbek Model" of development based on gradual reforms and national priorities.

Formation of the foundations of a democratic, civil society in Uzbekistan and the political reforms implemented.

Analyzes constitutional reforms, the establishment of democratic institutions, political pluralism, electoral reforms, and the development of civil society organizations.

Economic reforms, the formation of private property, and the development of market relations in Uzbekistan.

Examines the transition from a centrally planned economy to a market-oriented system, privatization, entrepreneurship, investment policies, and sustainable economic development.

Social changes in the Republic of Uzbekistan. Reviews reforms in healthcare, employment, social protection, demographic development, gender equality, and improvements in the quality of life during the years of independence.

Spiritual and cultural development in Uzbekistan during the years of independence. Explores policies aimed at preserving national heritage, promoting cultural values, strengthening national identity, and supporting literature, arts, and religious tolerance.

Reforms implemented in the field of education in the Republic of Uzbekistan. Examines educational reforms, modernization of higher education, curriculum improvements, digital learning initiatives, and policies for developing human capital.

Uzbekistan and the world community. Analyzes Uzbekistan's foreign policy priorities, international cooperation, regional integration, diplomatic relations, and participation in international organizations.

Uzbekistan and the world community (Contemporary cooperation and international partnerships). Focuses on Uzbekistan's expanding economic, political, cultural, and strategic partnerships with neighboring countries and the global community in recent years.

Reflection of the changes taking place in the social, political, economic, and cultural life of the New Uzbekistan

in international rankings and indices. Examines how reforms in New Uzbekistan are reflected in international indices related to governance, economic freedom, human development, education, innovation, and competitiveness.

Reforms being implemented in the New Uzbekistan under the leadership and initiative of President Sh. M. Mirziyoyev.

Discusses comprehensive reforms introduced since 2016, including public administration, judicial reforms, economic liberalization, digital transformation, and improvements in public services.

Reforms being implemented in the New Uzbekistan under the leadership and initiative of President Sh. M. Mirziyoyev (Sectoral reforms). Explores reforms in education, healthcare, agriculture, industry, environmental protection, investment policy, and regional development under the New Uzbekistan strategy.

The socio-political significance of building a people-oriented and humane state in the New Uzbekistan.

Examines the principles of a people-centered state, good governance, transparency, protection of human rights, public participation, and the role of reforms in strengthening democratic development and sustainable progress.

Seminar lessons

Subject, purpose, and objectives of the course "The Latest History of Uzbekistan," and its theoretical-methodological principles. Introduces the course objectives, scope, research methods, and the importance of studying Uzbekistan's contemporary history.

Historical foundations of independence ideas in Uzbekistan. Examines the historical development of national identity and the ideas that led to Uzbekistan's independence.

Socio-political processes in Uzbekistan on the eve of independence. Discusses the political, economic, and social changes that occurred before independence.

The establishment of the independent Republic of Uzbekistan and its historical significance. Uzbekistan's unique path to independence and development. Explores the declaration of independence and Uzbekistan's model of political and economic development.

The formation of the foundations of a democratic, civil society in Uzbekistan and the political reforms implemented. Examines the establishment of democratic institutions and major political reforms.

Economic reforms, the emergence of private property, and the development of market relations in Uzbekistan. Reviews the transition to a market economy, privatization, and entrepreneurship development.

Social changes in the Republic of Uzbekistan. Analyzes reforms in healthcare, social protection, employment, and public welfare.

Spiritual and cultural development in Uzbekistan during the years of independence. Discusses the revival of national values, cultural heritage, and spiritual development.

Reforms implemented in the education sector of the Republic of Uzbekistan. Examines modernization of the education system and higher education reforms.

Uzbekistan and the world community. Introduces Uzbekistan's foreign policy and international cooperation.

Uzbekistan and the world community. Explores the country's growing role in regional and global partnerships.

Reflection of the changes taking place in the social, political, economic, and cultural life of the New Uzbekistan in international rankings and indices. Reviews Uzbekistan's progress as reflected in international rankings and development indicators.

Reforms being implemented in the New Uzbekistan under the leadership and initiative of President Sh. M. Mirziyoyev. Examines key political, economic, and administrative reforms introduced in New Uzbekistan.

Reforms being implemented in the New Uzbekistan under the leadership and initiative of President Sh. M. Mirziyoyev. Focuses on reforms in education, healthcare, investment, and digital transformation.

The socio-political significance of building a people-oriented and humane state in the New Uzbekistan. Explores the principles of a people-centered state, the rule of law, and sustainable national development.

Independent study.
 Historical stages of Uzbek statehood (in the territory of Uzbekistan states in the territory of Uzbekistan).
 Steps toward state independence.
 Changes in Uzbekistan's political system.
 Introduction of the Presidential Institution in the Republic of Uzbekistan.
 Adoption of the Declaration of Independence.

	Reforms implemented in the system of state governance during the years of independence. The principle of separation of powers in an independent Uzbekistan. Non-governmental, non-profit organizations in Uzbekistan and their role in political, social, and economic processes. The essence of the newly revised Law on Education. New developments in the Republic of Karakalpakstan International situation. The role of Uzbekistan in foreign policy. Brotherly Cooperation Relations Among the Central Asian Republics New Uzbekistan Development Strategy The Position of the Republic of Uzbekistan in International Ratings and Indices (Case of Economic Ratings) The Position of the Republic of Uzbekistan in International Ratings and Indices (Case of Higher Education)
Teaching Methods	The module is delivered through interactive lectures, seminars, inquiry-based and discussion-based learning, historical document analysis, case studies, group discussions, presentations, and project-based activities. Students examine historical sources, legal documents, archival materials, maps, and statistical data to develop critical thinking, analytical reasoning, and historical interpretation skills. Independent learning, digital educational resources, and research-based assignments further promote academic writing, historical inquiry, effective communication, and a deeper understanding of the contemporary history of Uzbekistan and its role in national and global development.
Prerequisites	History of the pre-independence period T
Exams and Assessment	Student performance is assessed through continuous assessment (20%), midterm assessment (30%), and a final examination (50%). The continuous assessment (20%) evaluates students' attendance, active participation in classroom discussions, preparation for seminars, completion of independent study tasks, presentations, historical source analysis, map-based activities, quizzes, and written assignments on key events and processes in the contemporary history of Uzbekistan.

	The midterm assessment (30%) consists of written tests, essay questions, document-based analysis, oral presentations, and analytical assignments designed to evaluate students' understanding of the major stages of Uzbekistan's contemporary history, political and economic reforms, nation-building processes, foreign policy, and social development. The final examination (50%) is a comprehensive written examination that assesses students' knowledge of the entire course. It includes multiple-choice questions, short-answer questions, essay writing, analysis of historical documents and events, and evaluation of students' ability to critically interpret historical developments, explain cause-and-effect relationships, and assess the significance of contemporary reforms and national development in the Republic of Uzbekistan.
Recommended Literature	1. Mirziyoyev, Sh. M. We will resolutely continue our national development path and elevate it to a new stage. Volume 1. - Tashkent: Uzbekistan, 2017. 2. Mirziyoyev Sh. M. We will build our great future together with our brave and noble people. - Tashkent: Uzbekistan, 2017. 3. I. K. G'ulomov, G. Sh. Ibadullayeva, S. F. Norbekov, X. Muzaffarov Textbook: The Latest History of Uzbekistan (for students of all educational fields at higher education institutions). - T. : "TAMADDUN", 2025. 4. The Latest History of Uzbekistan. Textbook. /R. H. Murtazayeva and others. –Tashkent, 2024. History of Uzbekistan. Edited by A. S. Sagdullayev. Volume 1. Textbook. Tashkent: VNESHINVESTPROM, 2019. Savriyev J. F. , Iskandarova Sh. I. 5. The Latest History of Uzbekistan [Text] : textbook / Savriyev J. F. , Iskandarova Sh. I. – Bukhara: "Sadriddin Salim Bukhari" Durdona Publishing House, - 2024. 6. N. Oblamurodov. F. Tolipov. The Latest History of Uzbekistan. Textbook. – Tashkent: Iqtisod-moliya, 2022. 7. B. J. Eshov, A. A. Odilov. History of Uzbekistan. Volume 2. Textbook. – Tashkent: Donishmand Ziyosi, 2020. 8. Khalid A. The Birth of Uzbekistan: Nation, Empire, and Revolution in the Early Soviet Period / trans. Sardor Salim. – Tashkent: Akademnashr, 2022. – 424 p.
Responsible Lecturer	
Field	Description
Module Title	RELIGION STUDIES

Module Code	DINB204
Credit Points (ECTS)	4 Credits
Workload	Total workload: 120 hours Contact hours: 60 hours (30 hours Lecture, 30 hours Practical lessons) Self-Assignments: 60 hours
Frequency/Duration	I semester
Module objective	Religious studies is a discipline that studies the historical, philosophical, and social essence of various religions, religious institutions, traditions, and sacred texts. It explores the understanding of the essence of religious texts, the use of religious knowledge to ensure the purity of the human soul, and the development of a culture of respect for the current constitutional system of governance and the rule of law alongside religious faith. Religious studies serves to ensure the integrity of faith and the purity of conscience of specialists in all fields, achieve a balance between secular and religious knowledge, and foster a sense of belonging to the maintenance of peace in the country. The purpose of teaching this subject is to develop theoretical knowledge and practical skills in students and young people concerning the essence of religion, religious picture of the world, the relationship between religious and philosophical views, the negative impact of religious fanaticism, extremism and terrorism on the security of the state and society in the context of globalization, as well as counteraction to them.
Module Learning Outcomes	Knowledge: LO1. The essence of religion, its role in society and the life of an individual, how people with religious beliefs understand the world, the balance of secular and religious knowledge. LO2. Have an understanding of the relationship between religion and science, the goals of religious and secular education. LO3. By studying religious studies, students will become familiar with the essence of various religions, the differences and similarities between their traditions and beliefs, the role of religious and secular knowledge in fostering tolerance and mutual respect among people, and an analysis of national and world

	religions, beliefs and customs, traditions, and religious and moral concepts. LO4. Knowledge of the basic concepts of religious studies: God, prophets, man, destiny, the afterlife, the essence of faith, the history of Buddhism and its ideas on human education, the role of Christian religious ideas in cultivating the spiritual and moral perfection of man, the essence of the Sunni, Shiite and Kharijite branches of Islam, the formation of theology, as well as the essence of the teachings of Sufism and its application. LO5. understanding of the processes of secularization, a culture of mutual understanding and communication with believers and non-believers of different religions. LO6. understanding of the processes of secularization, a culture of mutual understanding and communication with believers and non-believers of different religions. Skills: LO7. They will be able to scientifically substantiate and express their views on spiritual, national and universal issues, and will also adhere to an active lifestyle consistent with the ideas of national independence. Competencies: LO8. will receive competitive general professional training corresponding to the specialty obtained in the bachelor's degree. LO9. be able to independently acquire new knowledge, organize one's activities on a scientific basis and work on self-improvement. LO10. will have experience of independent work in vacant positions for individuals with higher education in their field.
Content	Lectures Subject, goals and objectives of religious studies. Religion as a form of social consciousness, way of life, way of thinking, and spiritual values. Areas of religious study: religious studies, philosophy of religion, theology, Theosophy. The development of religious studies as a science, its subject matter, functions, and categories. The essence of religion. The relationship and difference between religiosity and piety. The structure and functions of religion. Types of religion: polytheism,

monotheism, and henotheism. Historical forms of religion: primitive religions, national religions, and world religions. The importance of religion in the upliftment of society and individual spirituality, in the development of the perfect individual.

National religions. Definition of national religions. National religions of India: Vedas, Upanishads, Brahmanism. And Hinduism. The teachings of Jainism and Sikhism. Chinese national religions: the essence of Confucianism and Taoism. The origin and development of the Japanese national religion of Shinto. The history, sacred texts, theological foundations, and rituals of the national religion of Judaism. The essence of faith in gods and the teachings of Zoroastrianism. The educational value of the ideas contained in the sacred source of the Avesta. The role of national religions in national identity and national consciousness.

History and main ideas of Buddhism. The status of Buddhism as a world religion. The history of the emergence of Buddhism and Its founder. The Tripitaka is the sacred source of Buddhism. Concepts of the world and man in Buddhist teachings. The teachings of Mahayana, Nichiyana, Lamaism, and Zen Buddhism. The penetration of Buddhism into Central Asia. The development of traditional Buddhism. The influence of Chan Buddhism on cultural life. Distinctive features of modern Buddhism. Global Buddhism and its trends. The educational significance of Buddhist teachings.

History and main ideas of Christianity. The status of Christianity as a world religion. The origin and spread of Christianity. The holy book of Christianity. The teachings, rituals, and holidays of Christianity. The main directions of Christianity: specific features of the ideas of Catholicism. The essence of the Orthodox religion. Formation and Development. History and modernity of Christianity in Uzbekistan. In Christianity. The social significance of ideas about humanitarian education.

History and essence of Islam. Social, economic, political and cultural conditions that influenced the emergence of Islam. Pre-Islamic religious beliefs and ideas. The Hanifa movement.

	Arabian Peninsula. The religious landscape of the island. The sacred origins of Islam. The science of interpretation. The Islamic religion. Doctrine. Classification of acts in Islam. The spread of Islam in Transoxiana and The impact of the formation of the madrasah system on social and cultural life. The significance of Islamic civilization in the modern era. Sources of Islam, schools of thought and their significance in the modern era. The history of the development of the sciences of the Quran, hadith, jurisprudence, and aqida. The meaning of the concepts of the Quran, surah, and verse, the formation and development of Quranic studies. The essence of the concept of hadith, the types and classification of hadith. The lives and works of great hadith scholars. The development of hadith studies in Uzbekistan. The emergence of the Sunni and Shia schools and the differences in their beliefs. The schools of Muturidism and Ash'arism. Philosophy of the Word. The essence of eschatology: resurrection and the afterlife. Non-traditional beliefs: the Arji, Mu'tazili, and Qarmatian movements.
--	---

Islamic legal schools. The history of the formation of the first Islamic schools of jurisprudence and their classification. The main ideas of the Hanafi, Maliki, Shafi'i, and Hanbali schools of jurisprudence and methods of legal reasoning. Categories of Sharia: halal, mandub, muba, makruh, haram. The role of qadi courts in enforcing Islamic law. The role of Muwarunnahr jurists in the development of jurisprudence in the Middle Ages. "Al-Characteristics" by Burhan ad-Din Marjinani. The importance of the work "Hidaya" in the development of jurisprudence.

The influence of Sufism on Eastern culture. The history, essence, and fundamental ideas of the formation of Sufism. The development of ideas of soul education, asceticism, and asceticism in the Quran in Sufism. The essence of the idea of divine love in Sufism. Interpretation of early Sufi literature. An overview of the concept of the saint in Islam. The teachings of Yusuf Hamadani and the social significance of the Kubrawiyya, Yasaviyya, and Naqshbandiyya orders. The influence of Sufism on fiction. The stages and status of spiritual education in Sufism. Modern Sufi orders.

The importance of Islamic values and education in the modern era. Ideas of social equality, justice, tolerance and patriotism in Islam The essence. Islam is about human rights. Five values in the protection of Sharia: religion, Life, mind, property, and offspring. Priority areas of environmental protection in Islam. Islam is about marriage and family relations, the rights and responsibilities of spouses. Islamic concepts of respect for parents and responsibility for raising children. Islamic moral values. Islamic values of friendship, mutual respect, and Promotion of a healthy lifestyle. Attitudes toward science and education, culture and art in Islam. Interpretation of the concepts of peace, good neighborliness, knowledge, and enlightenment in Islamic sources. Islamic architecture. Sacred sites and rituals of worship in Mecca and Medina. The role of Islam in the development of world civilization. Analysis of literature on peace, knowledge, enlightenment, and tolerance written by Uzbek scholars in the 21st century.

Globalization and religion. Contents and stages of globalization. Social, political and Influence on cultural life. The place and role of religion in the process of globalization. The role of religion. Transnational character. Interaction of world religions and global culture. Globalization. changes in religious values, traditions and customs in the context of secularism and Globalization. The spread of religion and religious ideas in the global information space. Religious Identity and Global Culture. New Forms of Religion in the Context of Globalization: New Religious movements and syncretic currents. Religion and global issues: radicalism. extremism, terrorism, ecology, climate change, migration.

Religion in Cyberspace. The Socio-Political Context of Missionary Work and Proselytism. Concepts of cyberspace, cyberbullying and cybersecurity. The essence. The negative impact of extremist groups and individuals operating on websites and social media on public life. The importance of uniting international forces in the fight against virtual religious extremism and terrorism in the modern era. The essence and development of the concept of missionary work and proselytism. Stages. Areas of activity of international missionary organizations, their Propaganda methods. Modernization of missionary propaganda methods and tools. The influence of religious manipulation on the human mind. The threat of missionary activity and proselytism in cyberspace to spiritual stability and the need to eradicate them. Mechanisms. Cybersecurity methods.

History of religious fanaticism, extremism, fundamentalism, terrorism and their threat to security in the modern era. History and stages of development of religious fanaticism, extremism, fundamentalism. Religious extremist movements are observed in the Central Asian region. Objectives. The nature and forms of terrorism. The consequences of terrorist activity under the guise of religion. The security threat posed by international religious terrorist groups—Al-Qaeda, the Muslim Brotherhood, ISIS, Boko Haram, and the Taliban. Terrorist attacks committed in

Uzbekistan and their impact on sustainable development. Influence. The history of the emergence of the Turkestan Islamic Movement, Akromiya, pseudo-Salafiism, Wahhabism, the Hizb ut-Tahrir and Nur movements, and their refutation by Islamic organizations and Muslim scholars. The directions and legal framework for combating radical movements in Uzbekistan.

Religious organizations operating in Uzbekistan. Key aspects of attitudes toward religion in Soviet Uzbekistan. Muslims of Uzbekistan. The history and significance of the establishment of the Directorate. The goals and objectives of the International Center for Islamic Civilization and the Imam Bukhari International Research Center. The role of religious organizations in adapting to modern secular standards and ensuring societal stability. The socio-political significance of the proposal of the President of Uzbekistan to adopt a resolution on religious tolerance at the 72nd session of the UN. The essence of the right to freedom of conscience in different countries and the Constitution of Uzbekistan. The history of the adoption of the Law on Freedom of Conscience and Religious Organizations in Uzbekistan and its significance. Definitions of the concepts of freedom of conscience and religion. The relationship of religious education to freedom of conscience. The essence of the concept of confession. Classification of religious denominations and the main directions of confessional processes. Types of regulation of relations between the state and religion (identification, separation). and cooperative models).

Interreligious dialogue and tolerance. The concept, goals, and objectives of interreligious dialogue. Between representatives of different religions. The need for dialogue. The content and historical development of the idea of tolerance. The importance of religious tolerance in social and cultural life. Contemporary forms and institutions of interreligious dialogue. Interreligious dialogue in the activities of international organizations such as UNESCO, the UN, and the OSCE. The experience of religious tolerance and

	interreligious cooperation in Uzbekistan. The role of interreligious dialogue in preventing extremism and religious conflicts. The problem of holistic education of the individual through interreligious dialogue. The relationship between religion and science. The Historical Relationship between Religion and Science. From Antiquity to the Modern Age: Religion and Science. The development of scientific thought. Religious schools and scientific discoveries in the Middle Ages. The confrontation between science and religion during the Renaissance. Philosophy and religion of the Enlightenment. The harmony of science and religion in Islamic civilization. The relationship between religion and science in the modern era: secular scientific thought, atheism, and postsecular theories. Controversial issues between religion and science: cosmology, biological evolution, genetics, bioethics. The idea of harmony between religion and science: the development of human community, the role of spiritual values in scientific progress.
Teaching Methods	The module is delivered through interactive lectures, inquiry-based and problem-based learning, seminars, group discussions, and project-based activities. Independent learning and digital educational resources further promote critical thinking, scientific communication, and the application of religious.
Prerequisites	PHILOSOPHY – Fals1204 Recent History of Uzbekistan OEY 1204 Scientific Heritage of Eastern Scholars SH 1306
Exams and Assessment	Type of examination (test, oral, project) and its weight Student performance is assessed through continuous assessment (20%), midterm assessment (30%), and a final test examination (50%). The continuous assessment (20%) evaluates classroom participation, seminar presentations, and the successful completion of independent study assignments. The midterm assessment (30%) consists of written assignments, short in-class quizzes, and the evaluation of independent study projects, a video presentation and an essay on a religious topic.

	The final assessment (50%) is a test examination consisting of theoretical questions and problem-solving tasks designed to assess students' understanding of religious, analytical skills, and their ability to apply scientific principles. Successful completion of all independent study assignments is required as an essential component of the module assessment.
Recommended Literature	1. Jalilov B. Philosophy of Religions. Textbook. - Tashkent: Publishing House, 2019. - 148 p. 2. Kamilov D. Religious Studies. Textbook. - T.: Lesson Press, 2021. - 128 p. 3. Muratov D., Alimova M., Karimov Zh. Religious studies, textbook. – Tashkent, publishing house “Navruz”, 2019. – 264 p. 4. Rakhimjanov D., Ernazarov O. Introduction to Religious Studies. Textbook. - T.: Publishing House "National Society of Philosophers of Uzbekistan", 2018. -304 s. 5. Shermukhamedova N.A. Phenomenon of religious fanaticism // Philosophy of man. - Textbook. - Tashkent: Innovation-Ziyo, 2021.B.314-499.
Responsible Lecturer	Mustafayev Ulash Utayevich, PhD in Philosophy Sciences, Associate Professor.
Field	Description
Module Title	PHILOSOPHY
Module Code	Philosophy 1204
Credit Points (ECTS)	4 Credits
Workload	Total workload: 120 hours Contact hours: 60 hours (30 hours Lecture, 30 hours Seminar lessons) Self-Assignments: 60 hours
Frequency/Duration	II semester
Module objective	To form philosophical, scientific, and moral knowledge in students, philosophical, the social significance of natural and scientific knowledge, the evolution of techniques and technologies as a response to human pragmatic needs, the interrelationship and distinction between national values, culture, and mass culture, the importance of ensuring the harmony of practice and activity in the knowledge and understanding of the world, the role of correct thinking in achieving positive goals, knowing the causes and forms of global corruption and combating them, It consists of fully demonstrating the world knowledge and skills in the course of reforms in Uzbekistan that are updating them, and of fostering a sense of social and

	professional responsibility and of making a worthy contribution to t
Module Learning Outcomes	Knowledge: L Skills: 1 Competencies: 2 O
Content	Lectures The essence of philosophy and its role in societal development. Introduces the nature of philosophy and its role in developing critical thinking, ethics, and social progress. The essence of eastern philosophy and its role in social development. Examines the main ideas of Eastern philosophy and their influence on morality, culture, and society. The main directions of western philosophy and their impact on the development of science. Explores major schools of Western philosophy and their contribution to science and rational thinking. Philosophy of being (Ontology). Studies the nature of existence, reality, and the relationship between mind and matter. Philosophy of development. Examines the principles of change, progress, and development in nature, society, and human thought. Philosophy of knowledge. Explores the origin, nature, and validity of knowledge and scientific inquiry. Logic. Introduces the principles of correct reasoning, argumentation, and problem-solving. Philosophy of society. Examines the structure of society, justice, freedom, and social responsibility. Philosophy of history, civilization, and culture. Discusses historical development, civilizations, cultural values, and globalization. Philosophy of man (Philosophical Anthropology). Explores human nature, consciousness, freedom, and personal identity. Philosophy of values. Examines moral, cultural, aesthetic, and social values and their role in human life. Ethics. Studies moral principles, responsibility, justice, and professional conduct. Aesthetics. Explores beauty, art, creativity, and their importance in culture and society.

Philosophy of global processes and sustainable development. Examines globalization, environmental challenges, and sustainable development.

The nature of corruption, its consequences, and the fight against it mechanisms. Explores the causes and effects of corruption and the importance of transparency, accountability, and anti-corruption measures.

Seminar lessons

The essence of philosophy and its role in societal development. Philosophy is the study of fundamental questions concerning existence, knowledge, values, reason, and reality. It encourages critical thinking, logical reasoning, and ethical decision-making.

The essence of eastern philosophy and its role in social development. Eastern philosophy includes the philosophical traditions of China, India, Central Asia, Japan, and other Eastern civilizations. It emphasizes harmony, morality, self-discipline, spiritual growth, and balance between humans and nature.

The main directions of western philosophy and their impact on the development of science. Western philosophy developed from Ancient Greece through the Middle Ages, the Renaissance, and the Modern era. Major schools include Rationalism, Empiricism, Idealism, Materialism, Existentialism, and Pragmatism.

Philosophy of being (Ontology). Ontology is the branch of philosophy that studies the nature of existence and reality. It explores questions such as what exists, the relationship between mind and matter, and the structure of the universe.

Philosophy of development. The philosophy of development examines the principles and laws of change, growth, and transformation in nature, society, and human thinking.

Philosophy of knowledge. Philosophy of knowledge, also known as epistemology, investigates the origin, nature, limits, and validity of knowledge.

Logic. Logic is the science of correct reasoning and valid argumentation.

Philosophy of society. The philosophy of society examines the structure, development, and functioning of human society.

Philosophy of history, civilization, and culture. This branch studies historical development, the rise and fall of civilizations, and the role of culture in shaping human identity.

Philosophy of man (philosophical anthropology).

Philosophical anthropology explores the nature, origin, purpose, and destiny of human beings.

Philosophy of values. The philosophy of values, or axiology, studies moral, cultural, aesthetic, social, and spiritual values.

Ethics. Ethics is the philosophical study of morality and human conduct.

Aesthetics. Aesthetics is the branch of philosophy that studies beauty, art, creativity, and artistic expression.

Philosophy of global processes and sustainable development. This field examines globalization, environmental challenges, technological progress, international cooperation, and sustainable development.

The nature of corruption, its consequences, and the fight against it mechanisms. Corruption is the misuse of entrusted power for personal gain.

Independent study.

Read Seneca's "Moral Letters to Lucilius" and, based on those letters, have students prepare an essay evaluating their own lives and activities.

Comparative analysis of Abu Nasr al-Farabi's and al-Ghazali's views on the science of philosophy (based on al-Farabi's Politics of Philosophy and al-Ghazali's The Chemistry of Happiness)

Writing an essay on the directions and essence of non-classical ontology (based on M. Heidegger's Being and Time)

Write an essay on "The Classification of Categories in the Ontologies of Aristotle, al-Farabi, and Kant" (and prepare a crossword puzzle).

Prepare an essay on the family-building procedures and ideas about child-rearing in A. Fitrat's work "Oila. "

Prepare an essay on A. Fitrat's views on the "Two Categories of Unhappy People" in the treatise "Guide to Salvation" (), and hold a video clip competition.

Compare and analyze Avloniy's "Waves of Culture" article and E. Toffler's "The Third Wave," and prepare a presentation.

Write an essay on "The Informational Society and Its Prospects" (based on a comparative analysis of the works of A. Toynbee and E. Toffler).

Write an essay on the topic "The Importance of Artistic Works in the Aesthetic Perception of the World. "

Prepare a video clip on the topic "Similarities and differences in the cultures of the world's peoples. "

	Prepare a crossword puzzle about "Myths of the World's Peoples. " Analyze the essence and pedagogical significance of J. Rumi's "The Hunter and the Falcon" tale (based on the work "The Inner Within Is Within"). Write an essay on the topic "The Advantages and Disadvantages of Modern Moral Education. " Write an essay on "The Dialectics of Coevolution and Environmentalism" (identify information about animals and plants included in Uzbekistan's Red Book). Prepare a crossword puzzle on "The Nature of Uzbekistan" (). Analyze M. Behbudiy's articles "Address to the Youth" and "Address to the Honorable Youth" and prepare an essay.
Teaching Methods	The module is delivered through interactive lectures, inquiry-based and problem-based learning, seminars, group discussions, and project-based activities. Independent learning and digital educational resources further promote critical thinking, scientific communication, and the application of modern philosophical concepts.
Prerequisites	
Exams and Assessment	Type of examination (test, oral, project) and its weight Student performance is assessed through continuous assessment (20%), midterm assessment (30%), and a final test examination (50%). The continuous assessment (20%) evaluates classroom participation, seminar presentations, and the successful completion of independent study assignments. The midterm assessment (30%) consists of written assignments, short in-class quizzes, and the evaluation of independent study projects, a video presentation and an essay on a philosophy topic. The final assessment (50%) is a test examination consisting of theoretical questions and problem-solving tasks designed to assess students' understanding of philosophy, analytical skills, and their ability to apply scientific principles. Successful completion of all independent study assignments is required as an essential component of the module assessment.
Recommended Literature	hermukhamedova N. Philosophy, textbook - Tashkent: Idris Publishing House Abdurauf , 2021. - 667 p. avronov Z., Shermukhamedova N., Kahkharova M., Nurmatova M., Khusanov B., Sultonova A. Philosophy, textbook - Tashkent:

	TMU, 2019. - 375 p. aifnazarov I., Mukhtorov A., Ernazarov D., Usmanov F. Philosophy, textbook - Tashkent: Innovative Development Publishing House, 2021. - 424 p. adaeva Sh., Shermukhamedova N., et al. Philosophy, textbook. - Tashkent: 2019. 5. Chumakov A.N. Philosophy textbook - M.: INFRA. 2018. - 320 p.
Responsible Lecturer	Mamatkulov Eldor, PhD in Philosophy Sciences.
Field	Description
Module Title	FOREIGN LANGUAGE 1 (ENGLISH)
Module Code	XOT1108
Credit Points (ECTS)	4 Credits
Workload	Total workload: 120 hours Contact hours: 60 hours (60 hours Practical lessons) Self-Assignments: 60 hours
Frequency/Duration	I semester
Module objective	The main purpose of teaching a foreign language in non-philological fields is to train future specialists who can use a foreign language freely in everyday life and in academic and professional activities. The module also aims to develop students' patriotism, independent learning, knowledge and skills beyond classroom hours and after graduation, as well as their ability to write academic articles on universal and field-related topics and to strengthen logical thinking.
Module Learning Outcomes	Knowledge: L O 1 · LO2. Develop oral and written communication skills related to demonstrate professional knowledge of specialty topics (reading, writing, listening and speaking), language competencies (lexical and grammatical), tolerance and intercultural friendship, pragmatic communication. On Learning the terms and expressions used in academic and professional activity.

LO5. Organize independent-study assignments according to students' academic and professional fields through clear and accurate instructions.

Skills:

LO6. Develop reading, writing, listening and speaking skills in an integrated manner during practical classes.

LO7. Select and prioritize the relevant language skill in class according to the essence of the topic and the features of the specialization, using appropriate pedagogical methods and technologies.

LO8. Conduct conversations and informal dialogues on social topics; participate in formal and informal professional discussions; manage discussions, interviews, negotiations and telephone communication.

LO9. Prepare and deliver presentations or lectures on specialty-related topics; support arguments with evidence; summarize information, write and discuss articles.

LO10. Develop skimming, scanning and intensive reading skills; read correspondence, emails, authentic texts, scientific and professional literature, electronic sources and press materials containing specialized terms.

LO11. Produce written correspondence, messages, special reports, CVs and notes; write essays, summaries, resumes and research papers.

LO12. Understand the main content of authentic materials after listening twice, identify necessary information and present it orally or in writing; understand news, reports and characters' speech in films.

Competencies:

LO13. Use a foreign language practically in professional contexts across speech activities; develop as a creative personality; develop skills for translating field-specific literature.

LO14. Reading: work weekly throughout the academic year with texts on various general and field-specific topics (1,200-1,500 characters).

L
O
1

LO16. Listening: understand radio, internet and television programs and interviews; understand complex authentic speech in familiar and unfamiliar contexts.

Writing: Speaking: meeting candidates for general and formal occasions, write educational, of texts related various society, workplace topics, essays, of 200g

	daily life, global issues, education systems, scientific writing, internet technologies, professional ethics, negotiations, innovations and leading specialists in the field.
Content	Seminar lessons Reading: Meeting people Listening: The letters of the alphabet Writing: Write about yourself Speaking: Talking about you Specialty text: Working in the IT industry Grammar: verb “to be”, possessive pronoun. Reading: Jobs in IT Listening: Common abbreviation Writing: Write about your hometown Speaking: Talking about your hometown Specialty text: The Software engineer Grammar: Verb Forms I (to have). Reading: Schedules Listening: Names of places, Data collection Writing: Describing your daily routine. Speaking: Talking about family Specialty text: Types of Computers Grammar: Present simple(1). Reading: Spelling IT acronyms Listening: Getting to know each other Writing: Problems of working on the Net. Speaking: Talk to your partner about the problem with Computer Specialty text: Inside the computer Grammar: Present Simple (2). Reading: Computer hardware Listening: The maps of some countries Writing: Describing computer hardware. Speaking: Role-play about assignments you found difficult and subjects you enjoy studying. Specialty text: System Software 1. Grammar: Present Simple (3). Reading: Computer software Listening: That sounds interesting Writing: Complete the e-mail from a software engineer Speaking: Talk about ways to prevent security problems. My weekend Specialty text: System Software 2 Grammar: Present continuous.

Reading: Working with computers
 Listening: Reading the numbers
 Writing: Write about your friend
 Speaking: Talk about security threats to your system. A nice place
 Specialty text: Programming Software
 Grammar: Present continuous II.
 Reading: Computer usage
 Listening: Number practice
 Writing: Complete the company's software product index.
 Speaking: Talk about new developments at your software company.
 Specialty text: Application Software 1
 Grammar: There is/are.
 Reading: Website analytics
 Listening: Discriminating between teens and tens
 Writing: Formal letters. Write an e-mail on software developments to the owner of a company
 Speaking: Talk about software engineering projects.
 Specialty text: Application Software 2
 Grammar: Past Simple I.
 Reading: Website development Function of a website
 Listening: Number dictation
 Writing: Complete the troubleshooting guide
 Speaking: Talk about a problem that you are having with a program
 Specialty text: The Desktop and GUI
 Grammar: Past Simple II.
 Reading: The best websites
 Listening: Circulation of British newspapers
 Writing: Your favorite websites. Use the conversation from task 8 to complete the email.
 Speaking: Talk about a problem that you are having with a program
 Specialty text: Basic Numbers and Math
 Grammar: Was/were.
 Reading: Database basics
 Listening: Practical problem-solving
 Writing: Complete the progress report.
 Speaking: A man talking about his job
 Specialty text: Analyzing Numbers and Qualities
 Grammar: Past Continuous.

Reading: Data processing
 Listening: Ordinal numbers
 Writing: Write an email to a business associate
 Speaking: Talk about a new software on the market. Talking about you.
 Specialty text: Describing Change
 Grammar: Present Perfect (I have done).
 Reading: Data storage and back-up.
 Listening: Ordinal numbers dictation
 Writing: Complete the presentation feedback form.
 Speaking: Talk about problems with your communication style.
 Specialty text: Presentations and Communication
 Grammar: Present Perfect II (Have you ever...).
 Reading: Education
 Listening: Practice with ordinals
 Writing: Write about your university.
 Speaking: Can you tell me the time?
 Specialty text: Database systems benefits.
 Grammar: Present Perfect III (How long have you been...?).
 Reading: E-commerce companies
 Listening: Traffic at Britain`s main airports
 Writing: Neighborhood.
 Speaking: Talk about your Neighborhood.
 Specialty text: Information Technology. IT Department
 Restructuring
 Grammar: It is done/ It was done (passive).
 Reading: E-commerce features
 Listening: Household expenditure
 Writing: How to access computers.
 Speaking: What computers they have
 Specialty text: Introduction to computing systems
 Grammar: Will/shall.
 Reading: Transaction security.
 Listening: Migration.
 Writing: Describing the transaction process
 Speaking: Talk about taking out an old hard drive
 Specialty text: Inside the computer
 Grammar: To be going to...
 Reading: Online transactions.
 Listening: Interview about the earthquake
 Writing: What devices can you attach to computers?
 Speaking: What kind of hobbies have people?

Specialty text: Computing devices
 Grammar: Can and could.
 Reading: Types of network.
 Listening: British trade
 Writing: Types of network system.
 Speaking: How does a computer network help a business?
 Specialty text: Networking
 Grammar: May and might.
 Reading: Networking hardware
 Listening: Money
 Writing: Write a web comment about work.
 Speaking: What is a User Interface?
 Specialty text: The user interface
 Grammar: Must.
 Reading: Talking about the past
 Listening: How much is this?
 Writing: Report writing
 Speaking: When do people use word processing programs?
 Our university
 Specialty text: Word processing
 Grammar: Should.
 Reading: Modern Examinations
 Listening: Living in London
 Writing: How is email helpful to people and businesses?
 Speaking: What are some problems with email?
 Specialty text: email. Network range and speed.
 Grammar: Have to...
 Reading: Fault diagnosis
 Listening: Can you tell me the time?
 Writing: Review writing
 Speaking: When do you browse the web at work?
 Specialty text: Web browsing
 Grammar: Would you like...?
 Reading: Software repair
 Listening: The telephone
 Writing: How do people edit their pictures?
 Speaking: Talk about: • what the editing software is?
 Specialty text: Images and graphic design
 Grammar: Adjectives
 Reading: Hardware repair.
 Listening: Dates
 Writing: Old cities
 Speaking: What kind of information do you find in databases

	Specialty text: databases vs. spreadsheets. Grammar: Antonyms and synonyms Reading: Customer service Listening: Survival English Writing: Shops and shopping Speaking: What kind of websites do you like to use? Specialty text: Web design vs. development. Grammar: Linking words (and, so, but because) Reading: IT security and safety, Security solutions Listening: University campus Writing: What skills are needed in publishing? Speaking: hobbies and interests Specialty text: Desktop publishing Grammar: Phrasal verb “look” Reading: Workstation health and safety Listening: TV and radio Writing: What can video conferencing be used for? Speaking: How the world spends its free time Specialty text: Videoconferencing, Workstation health and safety Grammar: too/ much/ many/ enough/ very. Reading: Security procedures Listening: Sports Writing: How does online shopping help businesses? Speaking: What do you buy online? Specialty text: E-commerce Grammar: Abbreviations and abbreviated words
Teaching Methods	The module is delivered entirely through practical classes using communicative language teaching, task-based learning, pair and group work, role-playing, discussions, presentations, problem-solving activities, and project-based learning. Students develop their listening, speaking, reading, and writing skills through interactive exercises, vocabulary and grammar practice, real-life communication tasks, and digital language-learning resources. Continuous practice, independent learning, and classroom interaction promote fluency, accuracy, confidence, and effective communication in English for academic and professional purposes.
Prerequisites	Students should have a general understanding and basic knowledge of English Students should be able to read short texts composed of previously learned and currently studied language material. Students should have an initial understanding of present, past and future verb forms.

	Students should be able to express ideas coherently in oral and written speech in simple situations.
Exams and Assessment	Student performance is assessed through continuous assessment (20%), midterm assessment (30%), and a final examination (50%). The continuous assessment (20%) evaluates classroom participation, speaking activities, listening exercises, reading tasks, writing assignments, vocabulary and grammar quizzes, pair and group work, presentations, and the successful completion of independent learning tasks. The midterm assessment (30%) consists of written tests, listening and reading comprehension exercises, speaking tasks, and the evaluation of practical assignments designed to assess students' progress in the four language skills: listening, speaking, reading, and writing. The final assessment (50%) is a comprehensive examination consisting of written and oral components that assess students' proficiency in grammar, vocabulary, reading, writing, listening, and speaking. Successful completion of all practical and independent learning assignments is required as an essential component of the module assessment.
Recommended Literature	1. Alireza Memarzadeh. IELTS Maximiser Speaking, Writing, Reading. 2016 2. Zhang Juan et al. Basic IELTS Reading, Listening, Writing, Speaking. Beijing Language and Culture University Press, 2013. 3. Raymond Murphy. English Grammar in Use. Elementary and Intermediate books. Cambridge University Press, 2019. 4. Constitution of the Republic of Uzbekistan. National Database of Legislation, 01. 05. 2023, No. 03/23/837/0241. https://lex.uz/uz/docs/-6445145 5. Labor Code of the Republic of Uzbekistan. National Database of Legislation, 29. 10. 2022, No. 02/22/798/0972; 12. 04. 2023, No. 03/23/829/0208. https://lex.uz/ru/docs/-6257288. 6. Mirziyoyev Sh. M. Decree No. PF-60 dated 28 January 2022 "On the Development Strategy of New Uzbekistan for 2022-2026". 7. Chris Gough. English Vocabulary Organiser. LTP Language, 2001.
Responsible Lecturer	<i>Ibodullayeva Go'zal Jo'rayevna</i>
Field	Description

Module Title	FOREIGN LANGUAGE 2 (ENGLISH)
Module Code	XOT1108
Credit Points (ECTS)	4 Credits
Workload	Total workload: 120 hours Contact hours: 30 hours (30 hours Practical lessons) Self-Assignments: 90 hours
Frequency/Duration	II semester
Module objective	The main purpose of teaching a foreign language in non-philological fields is to train future specialists who can use a foreign language freely in everyday life and in academic and professional activities. The module also aims to develop students' patriotism, independent learning, knowledge and skills beyond classroom hours and after graduation, as well as their ability to write academic articles on universal and field-related topics and to strengthen logical thinking.
Module Learning Outcomes	Knowledge: L O 1 L O 2003. Obtain knowledge with respect to universal and particular aspects of English (reading, writing, listening and speaking) and integrate them into competencies (lexical and grammatical). 2004. Learn the terms and expressions used in academic and professional activity. written communication skills related to professional activity. 2005. Organize independent study assignments according to students' academic and professional fields through clear and accurate instructions. Skills: 2006. Develop reading, writing, listening and speaking skills in an integrated manner during practical classes. 2006. Select and prioritize the relevant language skill in class according to the essence of the topic and the features of the specialization, using appropriate pedagogical methods and technologies. 2007. Conduct conversations and informal dialogues on social topics; participate in formal and informal professional discussions; manage discussions, interviews, negotiations and telephone communication.

	LO8. Prepare and deliver presentations or lectures on specialty-related topics; support arguments with evidence; summarize information, write and discuss articles. LO9. Develop skimming, scanning and intensive reading skills; read correspondence, emails, authentic texts, scientific and professional literature, electronic sources and press materials containing specialized terms. LO10. Produce written correspondence, messages, special reports, CVs and notes; write essays, summaries, resumes and research papers. LO11. Understand the main content of authentic materials after listening twice, identify necessary information and present it orally or in writing; understand news, reports and characters' speech in films. Competencies: LO12. Use a foreign language practically in professional contexts across speech activities; develop as a creative personality; develop skills for translating field-specific literature. LO13. Reading: work weekly throughout the academic year with texts on various general and field-specific topics (1,200-1,500 characters). LO14. Writing: write greeting cards, informal and formal letters; write summaries of texts on various topics; compose essays of 200-250 words. LO15. Listening: understand radio, internet and television programs and interviews; understand complex authentic speech in familiar and unfamiliar contexts. LO16. Speaking: communicate on general, social, socio-cultural, educational, ICT-related and specialty-related topics, including daily life, global issues, education systems, scientific writing, internet technologies, professional ethics, negotiations, innovations and leading specialists in the field.
Content	PRACTICAL Reading: Job-Sharing: Half a Job Is Better than None Listening: Who invented popcorn? Writing: something you love doing. Speaking: Online shopping Specialty text: Computer memory Vocabulary: Meeting people. Reading: Food Agency Takes on Industry over Junk Labels Listening: Where do ants go in winter? Writing: Abstract Speaking: Entertainment

Specialty text: Programming languages
 Vocabulary: Family.
 Reading: How Shops Can Exploit People's Herd Mentality to Increase Sales
 Listening: How crossword puzzle
 Writing: Health problem.
 Speaking: Health and illness
 Specialty text: ISPs and Internet access
 Vocabulary: Describing people: age and appearance.
 Reading: The Air around Us
 Listening: Stamp collecting
 Writing: Traffic where you live
 Speaking: Best cities of young people
 Specialty text: Storage devices
 Vocabulary: House and home.
 Reading: Closing the Gender Gap
 Listening: Inventor of the telephone
 Writing: report writing.
 Speaking: Crime murder, prison, guilty.
 Specialty text: Peripherals .
 Vocabulary: Test yourself.
 Reading: Health Care: Helping the Sick
 Listening: Inventors of the airplane
 Writing: Internet Fame
 Speaking: Favorite film.
 Specialty text: MP3 players
 Vocabulary: Life at home.
 Reading: Victoria READ Society
 Listening: Inventors of the telegraph
 Writing: The world's favorite food
 Speaking: Food and cooking
 Specialty text: Cell phones
 Vocabulary: School.
 Reading: Hire Me!
 Listening: University's calendar
 Writing: My university
 Speaking: Disasters
 Specialty text: Communications
 Vocabulary: Sport.
 Reading: Demand for Water in the United States
 Listening: Being involved in campus life
 Writing: Rubric Writing

	Speaking: Earning a Degree Specialty text: Entertainment, gaming and social networking Vocabulary: Test yourself Reading: A. Temur created a great Kingdom Listening: The first-year undergraduates Writing: How do you imagine Tashkent after 10 or more than 10 years? Speaking: Tashkent - the Capital Specialty text: GPS systems Vocabulary: Free time. Reading: Nice day, isn't it? Listening: Be a successful student Writing: My dreams for the future Speaking: Leisure Specialty text: Banking. Vocabulary: Fashion. Reading: Niagara Falls Listening: Oxford Writing: Give your views on life in the city or the country Speaking: Nature Specialty text: Education and research Vocabulary: Food and drink. Reading: The First Woman On Mount Everest Listening: Computing service Writing: Email and Internet Speaking: Communication Specialty text: Robotics Vocabulary: Countries, nationalities and languages. Reading: Uzbekistan – a milestone of the history Listening: English letter writing Writing: Formal letters Speaking: Money Specialty text: Electronic publishing Vocabulary: Test yourself 3. Reading: The metric system Listening: How to become a confident student Writing: Similar sounding words Speaking: Nature Specialty text: Artificial intelligence.
Teaching Methods	The module is delivered entirely through practical classes using communicative language teaching, task-based learning, pair and group work, role-playing, discussions, presentations, problem-solving activities, and project-based learning. Students develop

	their listening, speaking, reading, and writing skills through interactive exercises, vocabulary and grammar practice, real-life communication tasks, and digital language-learning resources. Continuous practice, independent learning, and classroom interaction promote fluency, accuracy, confidence, and effective communication in English for academic and professional purposes.
Prerequisites	Students should have a general understanding and basic knowledge of English. Students should be able to read short texts composed of previously learned and currently studied language material. Students should have an initial understanding of present, past and future verb forms. Students should be able to express ideas coherently in oral and written speech in simple situations.
Exams and Assessment	Student performance is assessed through continuous assessment (20%), midterm assessment (30%), and a final examination (50%). The continuous assessment (20%) evaluates classroom participation, speaking activities, listening exercises, reading tasks, writing assignments, vocabulary and grammar quizzes, pair and group work, presentations, and the successful completion of independent learning tasks. The midterm assessment (30%) consists of written tests, listening and reading comprehension exercises, speaking tasks, and the evaluation of practical assignments designed to assess students' progress in the four language skills: listening, speaking, reading, and writing. The final assessment (50%) is a comprehensive examination consisting of written and oral components that assess students' proficiency in grammar, vocabulary, reading, writing, listening, and speaking. Successful completion of all practical and independent learning assignments is required as an essential component of the module assessment.
Recommended Literature	1. Alireza Memarzadeh. IELTS Maximiser Speaking, Writing, Reading. 2016. 2. Zhang Juan et al. Basic IELTS Reading, Listening, Writing, Speaking. Beijing Language and Culture University Press, 2013. 3. Raymond Murphy. English Grammar in Use. Elementary and Intermediate books. Cambridge University Press, 2019. 4. Constitution of the Republic of Uzbekistan. National Database of Legislation, 01. 05. 2023, No. 03/23/837/0241. https://lex.uz/uz/docs/-6445145.

	5. Labor Code of the Republic of Uzbekistan. National Database of Legislation, 29. 10. 2022, No. 02/22/798/0972; 12. 04. 2023, No. 03/23/829/0208. https://lex.uz/ru/docs/-6257288. 6. Mirziyoyev Sh. M. Decree No. PF-60 dated 28 January 2022 "On the Development Strategy of New Uzbekistan for 2022-2026". 7. Chris Gough. English Vocabulary Organiser. LTP Language, 2001. 8. Materials and Man's Needs: Materials Science and Engineering. Supplementary Report of the Committee on the Survey of Materials Science and Engineering. Volume I. National Academy of Sciences, Washington, D. C. , 1975. 9. William D. Callister Jr. , David G. Rethwisch. Materials Science and Engineering: An Introduction. Wiley, 2018.
Responsible Lecturer	<i>Ibodullayeva Go'zal Jo'rayevna</i>
Field	Description
Module Title	CALCULUS
Module Code	HIS1112
Credit Points (ECTS)	12 Credits
Workload	Total workload: 360 hours Contact hours: 180 hours (90 hours Lecture, 90 hours Practical lessons) Self-Assignments: 180 hours
Frequency/Duration	I-II semester
Module objective	.
Module Learning Outcomes	Knowledge: LO1 Have an understanding of the theoretical and scientific foundations of the mathematical and natural sciences required for the engineering work education field. LO2. Be able to apply theorems on function limits and the fundamental rules of differentiation. LO3. Visualizing a function's graph using asymptotes, inflection points, and the derivative for increasing (decreasing) functions. LO4. Solve practical problems involving maxima (minima) using differentiation methods. LO5. Linear approximation; optimization methods (optimization problems, Newton's method). LO6. Application of the L'Hospital's rule when taking derivatives of certain functions, and use of differential calculus in solving problems.

	Skills: LO7. Knowledge of the concepts of elementary functions and definite integrals. LO8. Calculating integrals using substitution, simple fractions, and piecewise integration methods. LO9. Determine whether improper integrals are convergent (divergent) and evaluate convergent improper integrals. LO10. Calculating exact integrals using theorems of real analysis and Riemann sums; applying exact integrals to geometry (calculating the area of a plane figure, the volume of a solid of revolution, the length of a planar curve, and the differential of the arc length of a curve). LO11. Application of definite integrals to solving mechanics and physics problems (determining the static moments and center of mass of curved and planar shapes, calculating work), and determining whether numerical series are convergent (divergent). Competencies: LO12. Becomes competent in analyzing practical problems related to engineering and information technology and finding solutions using methods of mathematical analysis. LO13. Gains skills in mathematical modeling of processes and analysis of results by applying differential and integral calculus methods. LO14. Develops skills in independent mathematical modeling of logical systems and the effective application of multivariable functions in professional activities.
Content	Lectures The concept of a function. Methods of defining functions, evenness and oddness, periodicity. Introduces functions, methods of representation, and their basic properties. Elementary functions and their graphs. Examines elementary functions and techniques for graphing them. The concept of a sequence of numbers. The limit of a sequence of numbers. Explores numerical sequences and the concept of convergence.

Properties of convergent sequences. Calculating the limit of sequences of numbers. Studies convergence properties and methods for evaluating sequence limits.

Existence of a series limit. Important limits. Introduces fundamental limits and their applications.

Definitions of the function limit. Infinitely small and infinitely large functions. Explains the concept of function limits and infinite quantities.

Properties of functions with finite limits, important limits, and computing the limit of a function. Covers the properties and calculation of function limits.

The concept of a function's continuity. Operations on continuous functions. A function's discontinuity and types of discontinuities. Examines continuity, discontinuities, and operations involving continuous functions.

The concept of a function derivative. The derivative's geometric and mechanical meanings. Introduces derivatives and their interpretations in geometry and physics.

Calculating derivatives of inverse and composite functions. Studies differentiation techniques for inverse and composite functions.

Differential of a function. Approximate formulas. Explores differentials and their use in approximation methods.

Higher-order derivatives and differentials. Introduces higher-order derivatives and their applications.

Properties of differentiable functions. Taylor's formula. Examines differentiability and the use of Taylor's formula for function approximation.

Determining the increasing, decreasing, and extremum points of functions using derivatives. Applies derivatives to analyze function behavior and optimization.

Convexity, concavity, inflection point, and asymptotes of a function graph. Studies graph analysis using derivatives and asymptotic behavior.

Improper integral. Basic properties of integrals and integration methods. Introduces improper integrals and basic integration techniques.

Integration of simple fractions. Covers methods for integrating simple rational fractions.

Integration of rational functions. Examines techniques for integrating rational functions.

Integrating some irrational functions. Introduces integration methods for selected irrational functions.

Integration of trigonometric functions. Studies integration techniques involving trigonometric functions.

The concept of the definite integral. Properties of the definite integral. Table of integrals. Introduces definite integrals, their properties, and standard integration formulas.

Calculation of the definite integral. Approximate calculation of the definite integral. Covers analytical and numerical methods for evaluating definite integrals.

Some applications of the definite integral. Explores applications of definite integrals in geometry, physics, and engineering.

Second Semester

Special functions. Calculation of special functions. Introduces special mathematical functions and methods for evaluating them.

Concept of a numerical series and its convergence and divergence. Positive-term series and their convergence criteria. Examines numerical series and the main tests for convergence and divergence.

Functional series and their uniform convergence.

Studies functional series and the concept of uniform convergence.

Convergence domain of power series, functional properties of power series. Explores power series, their convergence intervals, and functional properties.

Taylor series. Expansion of elementary functions into power series. Introduces Taylor series and the expansion of elementary functions.

Preliminary information about Fourier series. Presents the basic concepts and applications of Fourier series.

A function of several variables and its limit and continuity.

Derivatives and differentials of a function of two variables. Examines multivariable functions, limits, continuity, and partial derivatives.

Higher-order derivatives and differentials. Gradient. Studies higher-order derivatives, differentials, and the gradient of multivariable functions.

Taylor formula for a function of two variables. Introduces Taylor's formula for approximating functions of two variables.

Extrema of a function. Examines methods for finding local and global extrema of multivariable functions.

The concept of a multiple integral. Properties and calculation of integrals. Introduces multiple integrals and their basic properties.

Double integrals over domains that can be decomposed into simple domains. Covers methods for evaluating double integrals over simple regions.

Some applications of the double integral. Explores applications of double integrals in geometry and physics.

A function of three variables, its partial derivatives and integrals. Examines functions of three variables, partial derivatives, and multiple integrals.

Some applications of the triple integral. Studies applications of triple integrals in science and engineering.

The concept of first-order curvilinear integrals and their calculation. Introduces line integrals of the first kind and methods of calculation.

Some applications of first-kind curvilinear integrals. Explores practical applications of first-kind line integrals.

The concept of second-order curvilinear integrals and their calculation. Examines line integrals of the second kind and computational techniques.

Some applications of second-order curvilinear integrals. Studies applications of second-kind line integrals in vector fields.

Surface integrals. First type surface integrals. Introduces surface integrals of the first kind and their properties.

Some applications of first-kind surface integrals. Explores applications of first-kind surface integrals.

Surface integrals of the second kind. Calculation of surface integrals of the second kind. Examines second-kind surface integrals and methods for their evaluation.

Practical lessons

The concept of a function. Methods of defining functions, evenness and oddness, periodicity. Introduces functions, methods of representation, and their basic properties.

Elementary functions and their graphs. Examines elementary functions and techniques for graphing them.

The concept of a sequence of numbers. The limit of a sequence of numbers. Explores numerical sequences and the concept of convergence.

Properties of convergent sequences. Calculating the limit of sequences of numbers. Studies convergence properties and methods for evaluating sequence limits.

Existence of a series limit. Important limits. Introduces fundamental limits and their applications.

Definitions of the function limit. Infinitely small and infinitely large functions. Explains the concept of function limits and infinite quantities.

Properties of functions with finite limits, important limits, and computing the limit of a function. Covers the properties and calculation of function limits.

The concept of a function's continuity. Operations on continuous functions. A function's discontinuity and types of discontinuities. Examines continuity, discontinuities, and operations involving continuous functions.

The concept of a function derivative. The derivative's geometric and mechanical meanings. Introduces derivatives and their interpretations in geometry and physics.

Calculating derivatives of inverse and composite functions. Studies differentiation techniques for inverse and composite functions.

Differential of a function. Approximate formulas. Explores differentials and their use in approximation methods.

Higher-order derivatives and differentials. Introduces higher-order derivatives and their applications.

Properties of differentiable functions. Taylor's formula. Examines differentiability and the use of Taylor's formula for function approximation.

Determining the increasing, decreasing, and extremum points of functions using derivatives. Applies derivatives to analyze function behavior and optimization.

Convexity, concavity, inflection point, and asymptotes of a function graph. Studies graph analysis using derivatives and asymptotic behavior.

Improper integral. Basic properties of integrals and integration methods. Introduces improper integrals and basic integration techniques.

Integration of simple fractions. Covers methods for integrating simple rational fractions.

Integration of rational functions. Examines techniques for integrating rational functions.

Integrating some irrational functions. Introduces integration methods for selected irrational functions.

Integration of trigonometric functions. Studies integration techniques involving trigonometric functions.

The concept of the definite integral. Properties of the definite integral. Table of integrals. Introduces definite integrals, their properties, and standard integration formulas.

Calculation of the definite integral. Approximate calculation of the definite integral. Covers analytical and numerical methods for evaluating definite integrals.

Some applications of the definite integral. Explores applications of definite integrals in geometry, physics, and engineering.

Second Semester

Special functions. Calculation of special functions. Introduces special mathematical functions and methods for evaluating them.

Concept of a numerical series and its convergence and divergence. Positive-term series and their convergence criteria. Examines numerical series and the main tests for convergence and divergence.

Functional series and their uniform convergence. Studies functional series and the concept of uniform convergence.

Convergence domain of power series, functional properties of power series. Explores power series, their convergence intervals, and functional properties.

Taylor series. Expansion of elementary functions into power series. Introduces Taylor series and the expansion of elementary functions.

Preliminary information about Fourier series. Presents the basic concepts and applications of Fourier series.

A function of several variables and its limit and continuity. Derivatives and differentials of a function of two variables. Examines multivariable functions, limits, continuity, and partial derivatives.

Higher-order derivatives and differentials. Gradient. Studies higher-order derivatives, differentials, and the gradient of multivariable functions.

Taylor formula for a function of two variables. Introduces Taylor's formula for approximating functions of two variables.

Extrema of a function. Examines methods for finding local and global extrema of multivariable functions.

The concept of a multiple integral. Properties and calculation of integrals. Introduces multiple integrals and their basic properties.

Double integrals over domains that can be decomposed into simple domains. Covers methods for evaluating double integrals over simple regions.

Some applications of the double integral. Explores applications of double integrals in geometry and physics.

A function of three variables, its partial derivatives and integrals. Examines functions of three variables, partial derivatives, and multiple integrals.

Some applications of the triple integral. Studies applications of triple integrals in science and engineering.

The concept of first-order curvilinear integrals and their calculation. Introduces line integrals of the first kind and methods of calculation.

Some applications of first-kind curvilinear integrals. Explores practical applications of first-kind line integrals.

The concept of second-order curvilinear integrals and their calculation. Examines line integrals of the second kind and computational techniques.

Some applications of second-order curvilinear integrals. Studies applications of second-kind line integrals in vector fields.

Surface integrals. First type surface integrals. Introduces surface integrals of the first kind and their properties.

Some applications of first-kind surface integrals. Explores applications of first-kind surface integrals.

Surface integrals of the second kind. Calculation of surface integrals of the second kind. Examines second-kind surface integrals and methods for their evaluation.

Independent study.

The concept of a function and its methods of definition.

Elementary functions and their graphs.

Numerical sequences and their properties.

Methods for calculating limits of sequences.

The concept of a function limit and rules for computing limits.

Impressive limits and their applications.

Continuity of a function and points of discontinuity.

Basic properties of continuous functions.

Methods for studying functions in mathematical analysis.

The concept of a derivative and its geometric meaning.

Rules for calculating derivatives.

Derivatives of composite and inverse functions.

The differential of a function and its applications.

Higher-order derivatives.

Taylor's formula and approximations.

	Finding Extremums of a Function. Investigating a function's graph using the derivative. Application of derivatives to practical and engineering problems. The concept of the improper integral and basic formulas. Integration methods: substitution and partial fractions. Integration of rational and trigonometric functions. The concept of the definite integral and its properties. Application of the Newton–Leibniz formula. Geometric applications of the definite integral. Singular integrals and their convergence. Numerical series and convergence criteria. Power series and their convergence domain. Taylor series and expansion of functions into series. Introduction to Fourier series. Application of integral calculus to practical problems. The concept of multiple-variable functions. Limits and continuity of two-variable functions. Partial derivatives and the total derivative. Gradient and its applications. Extrema of multiple-variable functions. Double integrals and their computation. Triple integrals and their applications. Application of multiple integrals in geometric and physical problems. The concept of curvilinear integrals. Line integrals of the first kind. Curvilinear integrals of the second kind. Surface integrals and their properties. Vector fields and integral formulas. Application of mathematical analysis methods in modern technology and information technology. Independent solution of mathematical modeling and computational problems.
Teaching Methods	The module is delivered through interactive lectures, practical problem-solving sessions, tutorials, computer-assisted learning, group discussions, and project-based activities. Students develop mathematical reasoning and analytical skills by solving theoretical and applied problems, using mathematical software for visualization and computation, and participating in collaborative exercises. Independent learning, digital educational resources, and regular practice further enhance critical thinking,

	problem-solving abilities, and the application of calculus concepts to engineering, science, and real-world problems.
Prerequisites	<i>Linear Algebra</i> <i>Probability and Statistics</i> <i>Differential Equations</i>
Exams and Assessment	Student performance is assessed through continuous assessment (20%), midterm assessment (30%), and a final written examination (50%). The continuous assessment (20%) evaluates classroom participation, practical problem-solving activities, quizzes, assignments, and the successful completion of independent study tasks, including exercises on limits, derivatives, integrals, series, multivariable calculus, and mathematical modeling. The midterm assessment (30%) consists of written tests, in-class problem-solving exercises, and the evaluation of independent assignments, such as analytical calculations, applications of differentiation and integration, series convergence problems, and multivariable calculus tasks. The final assessment (50%) is a written examination consisting of theoretical questions and mathematical problem-solving tasks designed to assess students' understanding of calculus concepts, analytical reasoning, computational skills, and their ability to apply mathematical methods to scientific and engineering problems. Successful completion of all independent study assignments is required as an essential component of the module assessment.
Recommended Literature	1. Alimov. Sh. O. , Ashurov. R. R. Mathematical Analysis 1,2,3 vols. T. "Mumtoz so'z", 2018. 2. Fichtenholz G. M. Course of Differential and Integral Calculus, Vol. 1, 2, 3. Moscow: Fizmatlit, 2007. 3. Azlarov T. A. , Mansurov X. T. Mathematical Analysis, Vol. 1, 2. Tashkent: "O'qituvchi", 1994, 1995. 4. Demidovich B. P. Collection of Problems in Mathematical Analysis. Moscow: Nauka, 1997. 5 6 7 .
Responsible Lecturer	Sh. M. Yuldoshev - Head of the Department of Mathematical Modeling and Computer Sciences, TerDU, PhD.
Field	Description
Module Title	PHYSICS 1

Module Code	PHYS1110
Credit Points (ECTS)	4 Credits
Workload	Total workload: 120 hours Contact hours: 60 hours (30 hours Lecture, 30 hours Practical lessons) Self-Assignments: 60 hours
Frequency/Duration	I semester
Module objective	
Module Learning Outcomes	Knowledge: LO 1. They must have knowledge of the meaning, content, laws, and graphs of physical processes in general physics. LO 2. They must have knowledge of the meaning, content, laws, and graphs of physical processes, their formulas, and how to compare them. LO 3. They must have knowledge of the basic physical laws and processes and be able to apply them to ecological phenomena. Skills: LO 4. Understands the meaning of laws and formulas related to general physics. LO 5. Knows how to apply general physics laws and formulas to non-standard problems. LO 6. Applies physical laws and formulas to laboratory work and knows how to draw conclusions. LO 7. Knows how to express and compare physical quantities in the SI and SGS systems of measurement. Competencies: LO 8. Be proficient in scientifically analyzing physical phenomena and processes, applying physical laws to solve practical problems, and interpreting the resulting findings. LO 9. Be able to independently organize physical experiments, effectively use measuring instruments, process
Content	Lectures The science of physics and its objectives. Motion. Space. Time. Physics is the fundamental natural science that studies matter, energy, motion, and the interactions between them. Its primary objective is to understand the laws governing the physical universe and to apply these laws to solve scientific and engineering problems. Motion describes the change in an

object's position over time, while space provides the framework in which objects exist, and time measures the sequence and duration of events. Together, these concepts form the foundation of classical and modern physics.

Circular motion of a particle. Circular motion refers to the movement of a particle along a circular path around a fixed center. The motion may be uniform or non-uniform depending on whether the speed remains constant. A particle undergoing circular motion experiences centripetal acceleration directed toward the center of the circle, requiring a centripetal force to maintain its path. Circular motion is widely observed in planetary orbits, rotating machinery, and satellites.

Dynamics of a material point. Dynamics is the branch of mechanics that studies the relationship between forces and the motion of objects. A material point is an idealized object whose size is negligible compared to the distance it travels. Newton's three laws of motion explain how external forces influence acceleration and movement. Dynamics forms the basis for understanding mechanical systems in engineering, transportation, and robotics.

Rotational motion of a rigid body. Rotational motion describes the movement of a rigid body around a fixed axis. Important quantities include angular displacement, angular velocity, angular acceleration, torque, and moment of inertia. Newton's laws have rotational equivalents that explain how applied torques produce angular acceleration. Rotational dynamics is essential in the design of engines, turbines, gears, and mechanical equipment.

Law of conservation of mechanical energy

The law of conservation of mechanical energy states that the total mechanical energy of an isolated system remains constant if only conservative forces act on it. Mechanical energy consists of kinetic energy and potential energy, which can transform into each other while their sum remains unchanged. This principle is widely applied in mechanics, engineering, and the analysis of moving systems.

Inertial and non-inertial reference frames. A reference frame is a coordinate system used to describe the position and motion of objects. In an inertial reference frame, Newton's laws of motion hold without modification because the frame moves at constant velocity. A non-inertial reference frame accelerates relative to an inertial frame, requiring the introduction of fictitious forces such as centrifugal and Coriolis forces. Understanding reference

frames is important in mechanics, astronomy, and aerospace engineering.

Molecular physics. Molecular physics studies the structure, properties, and behavior of matter based on the motion and interaction of atoms and molecules. It explains physical phenomena such as pressure, temperature, diffusion, and phase transitions using kinetic molecular theory. Molecular physics provides the microscopic foundation for thermodynamics and many applications in chemistry, materials science, and nanotechnology.

Thermodynamics. Thermodynamics is the study of heat, work, energy, and their transformations. It is based on four fundamental laws that describe energy conservation, entropy, and thermal equilibrium. Thermodynamics explains the operation of heat engines, refrigerators, power plants, and industrial processes. It is one of the most important branches of physics and engineering.

Electrostatic interaction. Electrostatics studies stationary electric charges and the forces between them. According to Coulomb's law, the force between two charges depends on the magnitude of the charges and the distance separating them. Electrostatic interactions are responsible for many natural phenomena and are widely used in technologies such as photocopiers, electrostatic painting, and particle accelerators.

Work done in moving a charge in an electric field. An electric field exerts a force on charged particles, causing them to move and perform work. The work done depends on the charge, the electric field strength, and the distance moved. This work is related to electric potential difference (voltage), which represents the energy transferred per unit charge. These concepts are fundamental to electric circuits and electrical engineering.

Dielectrics and conductors in an electric field. Materials respond differently when placed in an electric field. Conductors contain free electrons that move easily, causing charge redistribution and shielding the interior from electric fields. Dielectrics are insulating materials whose molecules become polarized in an electric field, reducing the field intensity within the material. These properties are essential in capacitors, electrical insulation, and electronic devices.

Electric current. condition for the generation of current, its voltage, and current density. Electric current is the directed flow of electric charges through a conductor. For current to exist, a potential difference (voltage) and a closed conducting path are

required. Current density measures the amount of current flowing through a unit cross-sectional area of a conductor. Understanding these concepts is fundamental to electrical circuits, power transmission, and electronic systems.

Magnetic field and the magnetic properties of matter.

Magnetic field induction vector. A magnetic field is produced by moving electric charges and permanent magnets. The magnetic induction vector (**B**) describes the strength and direction of the magnetic field. Materials exhibit different magnetic properties and are classified as diamagnetic, paramagnetic, or ferromagnetic depending on their response to magnetic fields. Magnetic fields play a crucial role in electric motors, generators, transformers, and communication systems.

Biot-savart-laplace law. Current loops. Magnetic flux.

Circulation of the magnetic induction vector. The Biot-Savart-Laplace law describes the magnetic field generated by an electric current. Current loops produce magnetic fields similar to those of permanent magnets. Magnetic flux measures the total magnetic field passing through a surface, while the circulation of the magnetic induction vector is described by Ampère's circuital law. These principles are fundamental to electromagnetism and electrical engineering.

The phenomenon of electromagnetic induction. Inductance electromotive force. Faraday and lenz's laws.

Electromagnetic induction is the generation of an electromotive force (EMF) when the magnetic flux through a conductor changes. Faraday's law states that the induced EMF is proportional to the rate of change of magnetic flux. Lenz's law determines the direction of the induced current, showing that it opposes the change causing it. Inductance is the property of a circuit that resists changes in current. These principles form the basis of transformers, electric generators, inductors, and many modern electrical technologies.

Practical lessons

Kinetics of forward motion. Forward motion describes the movement of an object along a straight path. Kinematics studies displacement, velocity, and acceleration without considering the forces causing the motion.

Kinematics of rigid body rotational motion. Rotational kinematics studies the motion of rigid bodies around a fixed axis using angular displacement, angular velocity, and angular acceleration.

Forward motion dynamics. Dynamics explains the relationship between force and motion. Newton's laws describe how forces cause acceleration and determine the motion of objects.

Dynamics of rigid body rotational motion. Rotational dynamics examines how torque causes angular acceleration. The moment of inertia determines a body's resistance to rotational motion.

Laws of conservation of momentum, angular momentum, and energy. These conservation laws state that momentum, angular momentum, and total energy remain constant in an isolated system. Mechanical energy includes kinetic and potential energy.

Laws of ideal gases. Molecular kinetic theory. The ideal gas laws describe the relationship between pressure, volume, and temperature. Molecular kinetic theory explains gas behavior through the motion of molecules.

Work done by a gas. Heat capacity. A gas performs work when it expands or compresses. Heat capacity measures the amount of heat required to change the temperature of a substance by one degree.

First law of thermodynamics. Entropy. The First Law of Thermodynamics states that energy is conserved. Entropy measures the degree of disorder and determines the direction of natural processes.

Electrostatic field in a vacuum. An electrostatic field is produced by stationary electric charges. It is characterized by electric field strength and electric potential.

Dielectrics and conductors in an electric field. Conductors allow free movement of electric charges, while dielectrics become polarized in an electric field and act as electrical insulators.

Electric capacitance. Capacitors. Capacitance is the ability of a system to store electric charge. Capacitors are devices used to store electrical energy in an electric field.

Laws of direct current. Kirchhoff's rules. Direct current flows in one direction. Kirchhoff's current and voltage laws are used to analyze complex electrical circuits.

Magnetic interactions. Magnetic field in a vacuum. Magnetic fields are produced by moving charges and electric currents. Their strength and direction can be calculated using electromagnetic laws.

Electromagnetic induction and magnetic field of matter. Electromagnetic induction occurs when a changing magnetic field

induces an electric current. Materials respond differently to magnetic fields depending on their magnetic properties.

Magnetic field energy. Magnetic fields store energy, especially in inductors and electromagnets. This energy plays an important role in electrical and electronic systems.

S

Forward motion kinematics. Study the basic concepts of displacement, velocity, acceleration, and equations of motion. Analyze examples of straight-line motion and explain their practical applications.

Kinematics of rigid body rotational motion
Study angular displacement, angular velocity, and angular acceleration. Explain the principles of rotational motion using practical examples.

Forward motion dynamics. Examine Newton's laws of motion and the relationship between force, mass, and acceleration. Analyze the dynamics of moving bodies.

Dynamics of the rotational motion of a rigid body. Study torque, moment of inertia, and angular acceleration. Explain how rotational forces affect the motion of rigid bodies.

Laws of conservation of momentum, angular momentum, and energy. Kinetic and potential energies of a mechanical system. Study the conservation laws of mechanics and explain the transformation between kinetic and potential energy in physical systems.

Laws of ideal gas. the fundamental equation of molecular kinetic theory. Distribution of the velocities and energies of ideal gas molecules. Analyze the behavior of ideal gases, the molecular kinetic theory, and the statistical distribution of molecular velocities and energies.

Work done by a gas. Heat capacity. Study the work performed during gas expansion and compression. Explain the concepts of specific heat and heat capacity.

The first law of thermodynamics, its application to isochoric and adiabatic processes. Entropy. Study the law of energy conservation in thermodynamic processes and explain the concept of entropy and its significance.

Electrostatic field in a vacuum. Field strength. Potential of an electric field. Study the properties of electrostatic fields, electric field intensity, and electric potential in a vacuum.

Properties of dielectrics in an electric field. Conductors in an electric field. Compare the behavior of conductors and

	dielectric materials when placed in an electric field and explain their practical applications. Electric capacitance. Capacitors. Study the concept of capacitance, the construction of capacitors, and their applications in electrical circuits. Laws of direct current. Kirchhoff's laws. Study the characteristics of direct current and apply Kirchhoff's current and voltage laws to analyze electrical circuits. Magnetic interactions. Magnetic field in a vacuum. Calculation of the simplest magnetic fields. Study magnetic forces, magnetic fields produced by electric currents, and methods for calculating simple magnetic field configurations. Electromagnetic induction and its manifestations. Magnetic field of matter. Study the principles of electromagnetic induction and explain the magnetic properties of different materials. Magnetic field energy. Study the storage of energy in magnetic fields and explain its applications in electrical and electronic devices
Teaching Methods	The Physics module is delivered through interactive lectures, problem-based and inquiry-based learning, laboratory experiments, numerical problem-solving, computer simulations, seminars, group discussions, and project-based activities. Students investigate physical phenomena through experiments, analyze scientific data using modern software and measurement tools, and develop analytical, practical, and research skills. Independent learning, virtual laboratories, and digital educational resources further enhance critical thinking, scientific communication, and the application of fundamental physical principles to real-world engineering and technological problems.
Prerequisites	
Exams and Assessment	Type of examination (written, oral, project) and its weight Student performance is assessed through continuous assessment (20%), midterm assessment (30%), and a final written examination (50%). The continuous assessment (20%) evaluates classroom participation, laboratory performance, problem-solving activities, seminar presentations, and the successful completion of independent study assignments, including numerical exercises,

	laboratory reports, computer simulations, data analysis, and practical physics experiments. The midterm assessment (30%) consists of written assignments, in-class quizzes, laboratory reports, and the evaluation of independent study projects, such as experimental investigations, analytical reports, simulation-based tasks, and presentations on selected topics in physics. The final assessment (50%) is a written examination consisting of theoretical questions, numerical calculations, and problem-solving tasks designed to assess students' understanding of fundamental physics concepts, analytical thinking, mathematical reasoning, and their ability to apply physical principles to scientific and engineering problems. Successful completion of all independent study assignments is required as an essential component of the module assessment.
Recommended Literature	1. Sivuxin D. V. General Physics Course. Moscow. Phys. Math. Lit. 2005. 2. Qosimov A. S. General Physics Textbook. IFRON-PRINT Publishing House, Termiz 2023. - Qosimov A. S. , Sh. Jo'rayev, B. Qo'yliyev. Laboratory Work in General Physics. Tashkent. 2019. - 309 p. 3. O'. N. Sultonova, D. A. Begmatova, Y. SH. Dusov, J. X. Xaliyarov, B. B. Ismailov, M. Yo'lchiyev. Collection of Problems in General Physics. Ifron Print Publishing House. Termez-2022. 4. I. G. Tursunov, D. A. Begmatov. "Physics" Teaching and Methodological Guide. "Excellent Polygraphy". Tashkent 2020. 5. O. Qodirov, "Course of Physics" Part 1 (Mechanics, Molecular Physics), Fan va Texnologiya Publishing House. Tashkent 2005. 6. S. Polvonov, X. Dalliyev, E. Bozorov, G. Polvonova. "Collection of Problems in General Physics. " Tashkent 2020.
Responsible Lecturer	Tursunov Alisher Isoqovich (Associate Professor, PhD)
Field	Description
Module Title	PHYSICS 2
Module Code	PHYS1110
Credit Points (ECTS)	2 Credits
Workload	Total workload: 120 hours Contact hours: 30 hours (16 hours Lecture, 14 hours Practical lessons) Self-Assignments: 90 hours
Frequency/Duration	II semester

Module objective Module Learning Outcomes	Knowledge: LO 1. They must have knowledge of the meaning, content, laws, and graphs of physical processes in general physics. LO 2. They must have knowledge of the meaning of the laws and formulas of physical quantities, their units, and how to compare them. LO 3. They must have knowledge of the basic physical laws and processes and be able to apply them to ecological phenomena. Skills: LO 4. Understands the meaning of laws and formulas related to general physics. LO 5. Knows how to apply general physics laws and formulas to non-standard problems. LO 6. Applies physical laws and formulas to laboratory work and knows how to draw conclusions. LO 7. Knows how to express and compare physical quantities in the SI and SGS systems of measurement. Competencies: LO 8. Be proficient in scientifically analyzing physical phenomena and processes, applying physical laws to solve practical problems, and interpreting the resulting findings. LO 9. Be able to independently organize physical experiments, effectively use measuring instruments, process
Content	Lecture Oscillatory motions. – Kinematics and dynamics of harmonic oscillation; oscillatory energy; mathematical, physical, and spring pendulums; addition of oscillations; damped and forced oscillations; the phenomenon of resonance. Wave phenomena. – Differential equation of a plane wave; amplitude, phase, period, frequency, and wavelength; superposition and interference of waves; standing waves; electromagnetic wave energy; the Poynting vector. Beams of light. – Nature of light, coherence and monochromaticity; interference and diffraction (Fresnel, Fraunhofer); diffraction grating; dispersion; absorption, scattering and polarization; thermal radiation. Quantum Optics. – Photoelectric effect and its quantum theory; light pressure; Compton effect; Schrödinger equation; depth of

potential and barrier (tunneling effect); hydrogen atom; quantum numbers; Pauli principle.

Line spectra of atoms. – Corpuscular-wave dualism; de Broglie wave; Heisenberg uncertainty relation; wave function; spectra of the hydrogen atom; Rydberg constant; Bohr postulates; periodic system.

Physics of solids. – Bonding forces; crystal lattice; energy bands; conductors, semiconductors, and dielectrics according to band theory; donor and acceptor levels; Fermi level; temperature dependence of conductivity.

Contact phenomena. – Electrical conductivity of metals; Fermi gas; superconductivity; magnetic properties of atoms; work function; metal-metal, metal-semiconductor contacts; electron-hole transition.

Atomic and Nuclear Physics. – Composition, charge, size, and mass of the atomic nucleus; interaction of nucleons; mass defect and binding energy; nuclear forces; nuclear fission; radioactivity; elementary particles.

Practical lessons

Harmonic oscillations. Superposition of oscillations. – Solving problems on harmonic oscillations and their superposition.

Damped and forced oscillations. – Problems on mechanical and electromagnetic damped and forced oscillations.

Mechanical and electromagnetic waves. – Solving problems on wave parameters and interference.

Wave optics. Quantum properties of light. – Problems related to interference, diffraction, and the photoelectric effect.

Elements of quantum mechanics. – Problems on the de Broglie wave, the uncertainty relation, and the Schrödinger equation.

Physics of solids. Contact phenomena. – Solving problems on band theory and conductivity.

Physics of the Atomic Nucleus. – Problems on mass defect, binding energy, and nuclear reactions.

Independent study.

Holography; optical instruments based on interference. Problems on holography, interference, and optical instruments.

Problem solving on vibrations (harmonic, damped, forced). Problems on harmonic, damped, and forced oscillations.

X-rays and their practical applications. Problems on the properties and applications of X-rays.

	Problem solving in wave phenomena and wave optics. Problems on interference, diffraction, polarization, and wave optics. The absolute black body and the Stefan–Boltzmann law. Problems on black-body radiation and the Stefan–Boltzmann law. Scattering of light; spectroscopy; types of spectra. Problems on light scattering, spectroscopy, and spectral analysis. Problem solving on the quantum properties of light (photoelectric effect, Compton effect). Problems on the photoelectric effect and Compton scattering. Lasers and their operating principles. Problems on laser operation and practical applications. Problem solving in quantum mechanics. Problems on wave functions, probability, and quantum states. Bohr theory for the hydrogen atom; Franck–Hertz experiment. Problems on Bohr's atomic model and the Franck–Hertz experiment. Problem solving in solid-state physics and contact phenomena. Problems on semiconductors, crystal structures, and contact phenomena. Radioactivity; law of radioactive decay. Problems on radioactive decay and half-life calculations. Nuclear and thermonuclear reactions; chain reactions. Problems on nuclear fission, fusion, and chain reactions. Doppler effect and the Large Hadron Collider. Problems on the Doppler effect and elementary particle physics. Problem solving in atomic and nuclear physics. Problems on atomic structure, nuclear reactions, and radioactive processes.
Teaching Methods	The Physics module is delivered through interactive lectures, problem-based and inquiry-based learning, laboratory experiments, numerical problem-solving, computer simulations, seminars, group discussions, and project-based activities. Students investigate physical phenomena through experiments, analyze scientific data using modern software and measurement tools, and develop analytical, practical, and research skills. Independent learning, virtual laboratories, and digital educational resources further enhance critical thinking, scientific communication, and the application of fundamental physical principles to real-world engineering and technological problems.
Prerequisites	

Exams and Assessment	Type of examination (written, oral, project) and its weight Student performance is assessed through continuous assessment (20%), midterm assessment (30%), and a final written examination (50%). The continuous assessment (20%) evaluates classroom participation, laboratory performance, problem-solving activities, seminar presentations, and the successful completion of independent study assignments, including numerical exercises, laboratory reports, computer simulations, data analysis, and practical physics experiments. The midterm assessment (30%) consists of written assignments, in-class quizzes, laboratory reports, and the evaluation of independent study projects, such as experimental investigations, analytical reports, simulation-based tasks, and presentations on selected topics in physics. The final assessment (50%) is a written examination consisting of theoretical questions, numerical calculations, and problem-solving tasks designed to assess students' understanding of fundamental physics concepts, analytical thinking, mathematical reasoning, and their ability to apply physical principles to scientific and engineering problems. Successful completion of all independent study assignments is required as an essential component of the module assessment.
Recommended Literature	1. Sivuxin D. V. General Physics Course. Moscow. Phys. Math. Lit. 2005. 2. Qosimov A. S. General Physics Textbook. IFRON-PRINT Publishing House, Termiz 2023. - Qosimov A. S. , Sh. Jo'rayev, B. Qo'yliyev. Laboratory Work in General Physics. Tashkent. 2019. - 309 p. 3. O'. N. Sultonova, D. A. Begmatova, Y. SH. Dusov, J. X. Xaliyarov, B. B. Ismailov, M. Yo'ldiyev. Collection of Problems in General Physics. Ifron Print Publishing House. Termez-2022. 4. I. G. Tursunov, D. A. Begmatov. "Physics" Teaching and Methodological Guide. "Excellent Polygraphy". Tashkent 2020. 5. O. Qodirov, "Course of Physics" Part 1 (Mechanics, Molecular Physics), Fan va Texnologiya Publishing House. Tashkent 2005. 6. S. Polvonov, X. Daliyev, E. Bozorov, G. Polvonova. "Collection of Problems in General Physics. " Tashkent 2020.
Responsible Lecturer	Tursunov Alisher Isoqovich (Associate Professor, PhD)
Field	Description
Module Title	PROGRAMMING 1

Module Code	DAS1110
Credit Points (ECTS)	6 Credits
Workload	Total workload: 180 hours Contact hours: 60 hours (30 hours Lecture, 30 hours Practical lessons) Self-Assignments: 120 hours
Frequency/Duration	I semester
Module objective	The goal of the Programming course is to develop algorithmic thinking in students, teach the theoretical foundations of programming, and develop skills in solving practical problems using modern programming languages. Within the course, students will learn to write programs, programming, data processing, developing and analyzing algorithms, effective use of programming tools, and creating software solutions for practical problems in various fields. Additionally, the course helps develop students' competencies in independently designing, testing, and improving software projects.
Module Learning Outcomes	Knowledge: LO1. Knows the fundamental concepts of the science of programming, algorithms, programming languages, data types, and operators. LO 2. Understands program structure, variables, expressions, input/output statements, and the principles of working in a programming environment. LO3. Knows programming techniques for linear, branching, and iterative processes and how to apply them to solve practical problems. LO4. Has theoretical knowledge of developing algorithms, analyzing them, and implementing them using programming language tools. Skills: LO5. Has the skill to develop algorithms, express them using flowcharts and programming language tools, and create software solutions for practical problems. LO 6. Master the skills to program linear, branching, and iterative processes, and to construct programs using conditional statements and loops. LO 7. Will be able to solve problems using variables, data types, arithmetic and logical operations, strings, and standard functions.

	LO 8. Develops skills in creating functions (subroutines), testing programs, identifying and debugging errors, and developing software products for practical problems. Competencies: LO 9. Becomes proficient in analyzing algorithms, evaluating their efficiency, and developing optimal software solutions for various practical problems. LO 10. Develops the ability to design, create, test, and refine programs using the fundamental tools of programming languages. LO 11. Becomes proficient in effectively using functions (subroutines), modules, and standard libraries in software development, as well as structuring program code. LO12. Gains the ability to analyze practical problems encountered in professional activities, develop their algorithms, and solve them using software tools.
Content	Lectures Subject matter, purpose of programming, and an introduction to the python programming language. This topic introduces the fundamentals of programming, its role in solving computational problems, and the basic features of the Python programming language. Students become familiar with Python's syntax, applications, and advantages in software development, data analysis, and artificial intelligence. C++ environment and program structure. This topic explains how to install and configure the Python environment and introduces the basic structure of a C++ program. Students learn how to write, save, and execute Python scripts using different development environments. Variables, identifiers, and data types. This topic covers the concepts of variables, naming rules for identifiers, and the basic data types in C++, including integers, floating-point numbers, strings, and Boolean values. Input-output operators and formatted output. This topic introduces methods for receiving user input and displaying output. Students learn how to use the input() and print() functions and apply formatted output techniques to improve program readability. Numeric data types and operations on them. This topic focuses on Python's numeric data types, including integers, floating-point numbers, and complex numbers. Students perform mathematical calculations and type conversions.

Arithmetic operators and mathematical functions. This topic explains arithmetic operators and commonly used mathematical functions available in Python. Students solve numerical problems using built-in functions and mathematical modules.

Comparison, logical, and bitwise operators. This topic introduces comparison, logical, and bitwise operators. Students learn how to evaluate expressions, combine conditions, and manipulate binary data.

Strings and string operations. This topic covers string creation, indexing, slicing, concatenation, and other fundamental string operations. Students learn how to process and manipulate textual data.

String methods and formatting techniques. This topic explains built-in string methods and formatting techniques such as f-strings, format(), and string manipulation functions to produce clear and readable output.

Conditional statements (if, elif, else). This topic introduces decision-making structures using if, elif, and else statements. Students learn how programs make decisions based on logical conditions.

Complex conditions and nested branching. This topic focuses on combining multiple logical conditions and creating nested conditional statements to solve more complex programming problems.

For loop and range() function. This topic explains iterative programming using the for loop and the range() function. Students learn how to automate repetitive tasks efficiently.

While loops and iterative algorithms. This topic introduces the while loop and demonstrates how iterative algorithms are used when the number of repetitions is not known in advance.

Break, continue, and pass operators. This topic explains the use of break, continue, and pass statements for controlling loop execution and improving program flow.

Complexity and efficiency of algorithms. This topic introduces the concepts of algorithm complexity and efficiency. Students learn how to evaluate the performance of algorithms using time and space complexity and understand the importance of writing optimized programs.

Practical lessons

Working with the python environment and creating programs. This practical session introduces students to the Python development environment. Students create, save,

execute, and debug simple Python programs using an integrated development environment (IDE).

Working with variables and data types. Students practice declaring variables, assigning values, and working with different data types such as integers, floating-point numbers, strings, and Boolean values.

Input-output operators. This practical session focuses on using the input() and print() functions to receive user input and display formatted program output.

Programming arithmetic expressions. Students solve programming tasks involving arithmetic operations, operator precedence, and mathematical calculations using Python.

Using mathematical functions. This session provides practice in applying built-in mathematical functions and the math module to solve computational problems.

Logical and comparison operators. Students write programs that use logical and comparison operators to evaluate conditions and control program execution.

Problems on bitwise operators. This practical session develops skills in applying bitwise operators for binary calculations and low-level data manipulation.

Working with strings. Students perform practical exercises involving string creation, indexing, slicing, concatenation, and other common string operations.

Problems on string methods. This session focuses on solving programming problems using built-in string methods such as searching, replacing, splitting, joining, and formatting text.

Simple conditional statements. Students develop programs using simple if, elif, and else statements to make decisions based on given conditions.

Compound conditional statements. This practical session covers writing programs with multiple and nested conditions to solve more complex decision-making problems.

Solving problems using the for loop. Students solve programming exercises involving repetition and iteration using the for loop and the range() function.

Solving problems using the while loop. This session provides practice in developing algorithms with while loops for tasks where the number of iterations is not predetermined.

Programs involving nested loops. Students write programs using nested loops to solve multidimensional and pattern-based programming problems.

	Break and continue operators. This practical session demonstrates the use of break and continue statements to control loop execution and improve program efficiency. Independent study. Working with the python environment and creating programs. This practical session introduces students to the Python development environment. Students create, save, execute, and debug simple Python programs using an integrated development environment (IDE). Working with variables and data types. Students practice declaring variables, assigning values, and working with different data types such as integers, floating-point numbers, strings, and Boolean values. Input-output operators. This practical session focuses on using the <code>input()</code> and <code>print()</code> functions to receive user input and display formatted program output. Programming arithmetic expressions. Students solve programming tasks involving arithmetic operations, operator precedence, and mathematical calculations using Python. Using mathematical functions. This session provides practice in applying built-in mathematical functions and the <code>math</code> module to solve computational problems.
--	---

	Logical and comparison operators. Students write programs that use logical and comparison operators to evaluate conditions and control program execution. Problems on bitwise operators. This practical session develops skills in applying bitwise operators for binary calculations and low-level data manipulation. Working with strings. Students perform practical exercises involving string creation, indexing, slicing, concatenation, and other common string operations. Problems on string methods. This session focuses on solving programming problems using built-in string methods such as searching, replacing, splitting, joining, and formatting text. Simple conditional statements. Students develop programs using simple <code>if</code>, <code>elif</code>, and <code>else</code> statements to make decisions based on given conditions. Compound conditional statements. This practical session covers writing programs with multiple and nested conditions to solve more complex decision-making problems. Solving problems using the for loop. Students solve programming exercises involving repetition and iteration using the <code>for</code> loop and the <code>range()</code> function. Solving problems using the while loop. This session provides practice in developing algorithms with <code>while</code> loops for tasks where the number of iterations is not predetermined. Programs involving nested loops. Students write programs using nested loops to solve multidimensional and pattern-based programming problems. Break and continue operators. This practical session demonstrates the use of <code>break</code> and <code>continue</code> statements to control loop execution and improve program efficiency.
Teaching Methods	The module is delivered through interactive lectures, practical programming sessions, live coding demonstrations, problem-based learning, project-based learning, pair programming, seminars, group discussions, and computer laboratory activities. Students develop programming skills by writing, testing, debugging, and optimizing Python programs to solve practical problems. Independent learning, coding assignments, online programming platforms, and digital educational resources further enhance algorithmic thinking, problem-solving abilities, and software development skills.
Prerequisites	Physics (PHYS16MBK) Differential Equations(DFEQ14MBK)

Exams and Assessment	Student performance is assessed through continuous assessment (20%), midterm assessment (30%), and a final written and practical examination (50%). The continuous assessment (20%) evaluates classroom participation, laboratory performance, coding exercises, programming assignments, quizzes, and the successful completion of independent study tasks, including algorithm design, debugging, and problem-solving activities. The midterm assessment (30%) consists of written tests, practical programming tasks, coding assignments, and the evaluation of mini-projects that assess students' understanding of Python programming concepts and their ability to develop correct and efficient programs. The final assessment (50%) consists of a written examination and practical programming tasks designed to evaluate students' knowledge of Python programming, algorithm development, problem-solving skills, code analysis, and the ability to apply programming concepts to real-world problems. Successful completion of all independent study assignments is required as an essential component of the module assessment.
Recommended Literature	1. Eric Matthes, Python Crash Course. – 3rd Edition. – San Francisco: No Starch Press, 2023. -544 p. 2. Narzullayev Anvar, Fundamentals of Programming in Python. -Tashkent: Akademnashr, 2021. -336p. 3. Fundamentals of Programming in Python. A Textbook. – Samarkand: SamDU Publishing, 2020, – 180 p. 4. Allen B. Downey, "Think Python: How to Think Like a Computer Scientist". 3rd Edition, O'Reilly Media, 2024, 325 p. 5. David D. Riley, Kenny A. Hunt, "Computational Thinking for the Modern Problem Solver", CRC Press, 2014, 406 p. 6. Al Sweigart. "Automate the Boring Stuff with Python". 2nd Edition. No Starch Press, 2020. 568 p. 7. Thomas Mailund, "Introduction to Computational Thinking". Apress, 2021, 657 p.
Responsible Lecturer	Y u
Field	Description
Module Title	PROGRAMMING 2
Module Code	DAS1110
Credit Points (ECTS)	4 Credits

Workload	Total workload: 120 hours Contact hours: 60 hours (30 hours Lecture, 30 hours Practical lessons) Self-Assignments: 60 hours
Frequency/Duration	I semester
Module objective	The goal of the Programming course is to develop algorithmic thinking in students, teach the theoretical foundations of programming, and develop skills in solving practical problems using modern programming languages. Within the course, students will learn to write programs, programming, data processing, developing and analyzing algorithms, effective use of programming tools, and creating software solutions for practical problems in various fields. Additionally, the course helps develop students' competencies in independently designing, testing, and improving software projects.
Module Learning Outcomes	Knowledge: LO1. Knows the fundamental concepts of the science of programming, algorithms, programming languages, data types, and operators. LO 2. Understands program structure, variables, expressions, input/output statements, and the principles of working in a programming environment. LO3. Knows programming techniques for linear, branching, and iterative processes and how to apply them to solve practical problems. LO4. Has theoretical knowledge of developing algorithms, analyzing them, and implementing them using programming language tools. Skills: LO5. Has the skill to develop algorithms, express them using flowcharts and programming language tools, and create software solutions for practical problems. LO 6. Master the skills to program linear, branching, and iterative processes, and to construct programs using conditional statements and loops. LO 7. Will be able to solve problems using variables, data types, arithmetic and logical operations, strings, and standard functions. LO 8. Develops skills in creating functions (subroutines), testing programs, identifying and debugging errors, and developing software products for practical problems. Competencies:

	LO 9. Becomes proficient in analyzing algorithms, evaluating their efficiency, and developing optimal software solutions for various practical problems. LO 10. Develops the ability to design, create, test, and refine programs using the fundamental tools of programming languages. LO 11. Becomes proficient in effectively using functions (subroutines), modules, and standard libraries in software development, as well as structuring program code. LO12. Gains the ability to analyze practical problems encountered in professional activities, develop their algorithms, and solve them using software tools.
Content	Lectures Functions and program modularization. This topic introduces functions as reusable blocks of code that improve program organization and maintainability. Students learn how modular programming simplifies software development. Parameters, arguments, and returning values. This topic explains how functions receive input through parameters and arguments and return results using the return statement. Students practice designing flexible and reusable functions. Local and global variables. This topic covers variable scope in Python. Students learn the difference between local and global variables and how variable scope affects program execution. Recursive functions. This topic introduces recursion as a programming technique in which a function calls itself to solve complex problems by breaking them into smaller subproblems. Lambda functions and anonymous functions. This topic explains lambda expressions for creating small anonymous functions. Students learn how lambda functions simplify code in functional programming tasks. Lists and their properties. This topic introduces C++ lists as mutable data structures used to store collections of elements. Students learn how to create, access, and modify lists. Lists, methods and slicing. This topic covers common list methods and slicing techniques for efficiently manipulating list elements and extracting subsets of data. Tuples and sets. This topic explains tuples as immutable collections and sets as unordered collections of unique elements. Students learn their properties and practical applications. Dictionaries and working with them. This topic introduces dictionaries as key-value data structures. Students practice creating, updating, searching, and manipulating dictionary data.

Search and sorting algorithms. This topic presents basic searching and sorting algorithms, including linear search, binary search, and common sorting techniques. Students analyze algorithm efficiency and performance.

File system technologies. This topic explains how to create, read, write, and manage files in Python. Students learn file handling techniques for processing and storing data.

Exception handling. This topic introduces exception handling using try, except, else, and finally blocks. Students learn how to write reliable programs that handle runtime errors effectively.

Object-oriented programming fundamentals. This topic introduces the principles of object-oriented programming (OOP), including objects, classes, and code reusability. Students understand how OOP improves software design.

Classes, objects, constructors, and methods. This topic explains how to define classes, create objects, use constructors, and implement methods. Students practice developing object-oriented Python programs.

Fundamentals of inheritance, polymorphism and encapsulation. This topic introduces the core principles of object-oriented programming. Students learn how inheritance promotes code reuse, polymorphism enables flexible program behavior, and encapsulation improves data security and software maintainability.

Practical lessons

Creating and Calling Functions. Practical exercises on creating, calling, and testing user-defined functions in Python.

Parameterized Functions. Practical exercises on developing functions with parameters and arguments to solve programming problems.

Using the return Operator. Practical exercises on returning values from functions and using returned results in programs.

Creating Recursive Functions. Practical exercises on implementing recursive algorithms to solve mathematical and logical problems.

Working with Lambda Functions. Practical exercises on creating and applying lambda (anonymous) functions in Python.

Working with Lists. Practical exercises on creating, modifying, and manipulating lists using Python.

Lists, Methods, and Slices. Practical exercises on applying list methods and slicing techniques for efficient data processing.

Working with Tuples. Practical exercises on creating, accessing, and manipulating tuples in Python programs.

Working with Sets. Practical exercises on performing set operations such as union, intersection, difference, and membership testing.

Search and Sorting Algorithms. Practical exercises on implementing and analyzing basic searching and sorting algorithms.

Working with Files. Practical exercises on reading, writing, and managing files using Python file-handling techniques.

Handling Exceptions. Practical exercises on detecting and handling runtime errors using exception-handling mechanisms.

Object-Oriented Programming Fundamentals. Practical exercises on creating simple classes and objects using object-oriented programming concepts.

Classes, Objects, Constructors, and Methods. Practical exercises on defining classes, creating objects, implementing constructors, and using class methods.

Fundamentals of Inheritance, Polymorphism, and Encapsulation.

- Practical exercises on implementing inheritance, polymorphism, and encapsulation using Python classes to develop reusable, flexible, and maintainable object-oriented programs.

Independent study.

Principles of Function Organization in the C++/C# Programming Language. – Study the principles of organizing programs using functions and explain how modular programming improves code readability, reusability, and maintainability.

Methods for Passing Parameters and Arguments. – Analyze different methods of passing parameters and arguments to functions and evaluate their practical applications.

Application of Local and Global Variables. – Study the scope of local and global variables and explain their role in program execution and variable management.

Recursion and the Advantages of Recursive Algorithms. – Investigate recursion and analyze its advantages in solving complex computational problems.

Comparison of Recursive and Iterative Algorithms. – Compare recursive and iterative approaches by evaluating their efficiency, readability, and memory usage.

Practical Application of Lambda Functions and Anonymous Functions. – Study the use of lambda expressions and demonstrate their practical applications in functional programming.

Basic Principles of Modularization. – Investigate modular programming techniques and explain how modules improve software development and code organization.

Lists and Operations on Them. – Study list data structures and analyze common operations such as insertion, deletion, updating, and traversal.

Practical Application of List Methods. – Investigate built-in list methods and demonstrate their use in solving practical programming tasks.

Tuples and Their Properties. – Study the characteristics of tuples and compare their advantages and limitations with lists.

Sets and Working with Them. – Investigate set operations and explain their applications in managing unique collections of data.

Dictionaries and Associative Data Storage. – Study dictionary data structures and analyze their use for efficient key-value data storage and retrieval.

List, Set, and Dictionary Comprehension Constructs. – Investigate comprehension techniques and demonstrate how they simplify data processing and improve code readability.

Analysis of the Linear Search Algorithm. – Analyze the linear search algorithm and evaluate its time complexity and practical applications.

Efficiency of the Binary Search Algorithm. – Study the binary search algorithm and compare its efficiency with linear search for sorted data.

Comparison of Sorting Algorithms. – Compare common sorting algorithms based on their efficiency, complexity, and practical use cases.

File Handling Technologies. – Investigate techniques for reading, writing, and managing files using Python and explain their practical applications.

Python Standard Modules and Their Capabilities. – Study the functionality of Python's standard library modules and demonstrate their use in software development.

Exception Handling Mechanisms. – Analyze exception handling techniques and explain how they improve the reliability and robustness of Python programs.

Basic Principles of Object-Oriented Programming: Class, Object, Encapsulation, Inheritance, and Polymorphism. – Study the core concepts of object-oriented programming and explain how they support the development of reusable, maintainable, and scalable software systems.

Teaching Methods	The module is delivered through interactive lectures, practical programming sessions, live coding demonstrations, problem-based learning, project-based learning, pair programming, computer laboratory activities, seminars, and group discussions. Students develop programming skills by designing algorithms, writing, testing, debugging, and optimizing C++/C# programs to solve real-world problems. Practical exercises, mini-projects, coding challenges, and independent learning activities strengthen computational thinking, problem-solving abilities, and software development competencies. The use of integrated development environments (IDEs), version control tools, online coding platforms, and digital educational resources further enhances students' programming proficiency and collaborative learning skills.
Prerequisites	Physics (PHYS16MBK) Differential Equations(DFEQ14MBK)
Exams and Assessment	Student performance is assessed through continuous assessment (20%), midterm assessment (30%), and a final written and practical examination (50%). The continuous assessment (20%) evaluates classroom participation, laboratory performance, coding exercises, programming assignments, quizzes, and the successful completion of independent study tasks, including algorithm design, debugging, and problem-solving activities. The midterm assessment (30%) consists of written tests, practical programming tasks, coding assignments, and the evaluation of mini-projects that assess students' understanding of Python programming concepts and their ability to develop correct and efficient programs. The final assessment (50%) consists of a written examination and practical programming tasks designed to evaluate students' knowledge of Python programming, algorithm development, problem-solving skills, code analysis, and the ability to apply programming concepts to real-world problems. Successful completion of all independent study assignments is required as an essential component of the module assessment.
Recommended Literature	1. Eric Matthes, Python Crash Course. – 3rd Edition. – San Francisco: No Starch Press, 2023. -544 p. 2. Narzullayev Anvar, Fundamentals of Programming in Python. -Tashkent: Akademnashr, 2021. -336p.

	3. Fundamentals of Programming in Python. A Textbook. – Samarkand: SamDU Publishing, 2020, – 180 p. 4. Allen B. Downey, "Think Python: How to Think Like a Computer Scientist". 3rd Edition, O'Reilly Media, 2024, 325 p. 5. David D. Riley, Kenny A. Hunt, "Computational Thinking for the Modern Problem Solver", CRC Press, 2014, 406 p. 6. Al Sweigart. "Automate the Boring Stuff with Python". 2nd Edition. No Starch Press, 2020. 568 p. 7. Thomas Mailund, "Introduction to Computational Thinking". Apress, 2021, 657 p.
Responsible Lecturer	Y u
Field	Description
Module Title	ACADEMIC WRITING
Module Code	AKY1104
Credit Points (ECTS)	4 Credits
Workload	Total workload: 120 hours Contact hours: 60 hours (30 hours Lecture, 30 hours Practical lessons) Self-Assignments: 60 hours
Frequency/Duration	II semester
Module objective	
Module Learning Outcomes	Knowledge: LO1. Students must know the basic rules, structures, and styles of academic writing. This includes the requirements of this type of writing and methods for clear and logical expression. LO 2. Ability to express complex topics clearly and logically in writing, to analyze the subject in depth, and to think coherently. LO3. Writing in a scientific and academic style, constructing clear and meaningful sentences, and correctly using the necessary lexical and grammatical tools. LO4. Ability to properly cite sources, avoid plagiarism, and correctly format a list of scholarly sources. Skills: LO5. Ability to edit written work, identifying and correcting grammatical, stylistic, and logical errors. LO 6. Ability to express complex topics clearly and logically in writing, to analyze a topic in depth, and to think coherently.

	LO 7. Writing in a scientific and academic style, constructing clear and meaningful sentences, and correctly using the necessary lexical and grammatical tools. LO 8. Ability to properly cite sources, avoid plagiarism, and correctly list scholarly sources in the appropriate format. Competencies: LO 9. Becomes proficient in analyzing algorithms, evaluating their efficiency, and developing optimal software solutions for various practical problems. LO 10. Develops the ability to design, create, test, and refine programs using the fundamental tools of programming languages. LO 11. Becomes proficient in effectively using functions (subroutines), modules, and standard libraries in software development, as well as structuring program code. LO12. Gains the ability to analyze practical problems encountered in professional activities, develop their algorithms, and solve them using software tools.
Content	Lectures Basic concepts and principles of academic writing. This topic introduces the fundamental concepts, objectives, and ethical principles of academic writing. Students learn the characteristics of academic texts and the importance of clarity, accuracy, objectivity, and evidence-based communication in scholarly work. Types of academic texts: article, report, dissertation. This topic examines the major types of academic writing, including research articles, reports, and dissertations. Students explore their structures, purposes, and writing conventions in academic and professional contexts. Choosing a topic and writing a plan. This topic explains how to select an appropriate research topic and prepare a well-structured writing plan. Students learn techniques for narrowing research topics and organizing ideas effectively. Scientific style and language features. This topic focuses on the characteristics of scientific writing, including formal language, objectivity, precision, coherence, and appropriate academic vocabulary. Students learn how to communicate research findings clearly and professionally. Developing research questions and hypotheses. This topic introduces the process of formulating research questions and developing testable hypotheses. Students learn how these elements guide the direction and methodology of scientific research.

Searching for and evaluating scientific sources. This topic develops students' skills in locating, selecting, and critically evaluating reliable academic sources using scientific databases, digital libraries, and scholarly search engines.

Citing scholarly sources and style. This topic introduces the principles of academic citation and referencing. Students learn how to apply standard citation styles such as APA, IEEE, and MLA while maintaining academic integrity.

Plagiarism and how to avoid it in academic writing. This topic explains the concept of plagiarism, its ethical and academic consequences, and effective strategies for avoiding it through proper citation, paraphrasing, and original writing.

Analysis and commentary in academic writing. This topic teaches students how to critically analyze information, interpret research findings, and present logical commentary supported by evidence and scholarly arguments.

Drawing conclusions in academic writing. This topic explains how to write effective conclusions that summarize key findings, answer research questions, and provide meaningful recommendations for future research.

The logical structure of academic writing. This topic examines the organization of academic texts, emphasizing logical flow, coherence, paragraph structure, and the effective connection of ideas throughout a document.

Editing and revising written work. This topic develops students' editing and proofreading skills by focusing on grammar, style, coherence, consistency, formatting, and the overall quality of academic writing.

Introduction to the scientific paper and its importance. This topic introduces the structure and purpose of scientific papers. Students learn the essential components of a research article and understand the role of scientific publications in advancing knowledge.

Results and their presentation. This topic focuses on presenting research findings clearly and accurately using text, tables, figures, and graphs. Students learn how to interpret and communicate results effectively.

The concluding part of academic writing: analysis and conclusion. This topic explains how to prepare the final section of an academic paper by integrating analysis, discussing the significance of research findings, drawing well-supported conclusions, and suggesting directions for future research.

Practical lessons

Comparing academic and literary texts, identifying their specific principles. This practical session develops students' ability to compare academic and literary texts by identifying their structure, writing style, purpose, and distinctive characteristics.

Analysis of article and dissertation texts. This practical session focuses on analyzing the structure, organization, and academic writing style of research articles and dissertations.

Developing outlines for given topics and evaluating them. This practical session enables students to prepare logical outlines for academic papers and evaluate their clarity, coherence, and organization.

Conducting a stylistic analysis of the given scientific text. This practical session develops students' skills in analyzing the language, style, vocabulary, and objectivity of scientific texts.

Formulating hypotheses and linking them to research questions. This practical session provides experience in developing research questions and formulating appropriate hypotheses for academic research.

Finding and analyzing sources from an academic database. This practical session focuses on searching academic databases, selecting reliable scholarly sources, and evaluating their relevance and credibility.

Learning citation rules and applying them in practice. This practical session develops students' skills in applying academic citation and referencing styles correctly in research writing.

Discussing methods for detecting plagiarism in given texts and preventing it.

This practical session introduces plagiarism detection methods and develops skills in proper citation, paraphrasing, and maintaining academic integrity.

Analyzing and commenting on the given text. This practical session enhances students' critical thinking by analyzing academic texts and presenting well-supported comments and interpretations.

Analyzing various conclusions and formulating them correctly. This practical session develops the ability to evaluate and write effective conclusions based on research findings and logical arguments.

Analyzing the logical structure of the text. This practical session focuses on evaluating the organization, coherence, and logical flow of academic texts to improve writing quality.

Editing and analyzing the given text. This practical session develops editing and proofreading skills by correcting grammar, style, formatting, and overall academic quality.

Identifying important aspects in the introduction of a scientific paper. This practical session focuses on analyzing the introduction of a scientific paper by identifying the research background, objectives, significance, and problem statement.

Exploring methods for presenting results in visual and textual formats. This practical session develops students' ability to present research findings effectively using tables, figures, graphs, and well-structured written explanations.

Analyzing final conclusions in given texts. This practical session develops students' ability to evaluate and improve the conclusion section of academic texts by assessing its clarity, completeness, and logical consistency.

Independent study.

Write an essay based on the main principles of academic writing. Study the fundamental principles of academic writing and prepare an essay that demonstrates clarity, objectivity, logical organization, and academic integrity.

Write a short text in the selected academic text type. Prepare a short academic text in the chosen format, such as an article, report, or essay, following appropriate academic writing conventions.

Independently develop a topic and outline for a research paper. Select an appropriate research topic and prepare a well-organized outline that reflects the logical structure of a scientific paper.

Write a short article in a scientific style on a chosen topic. Write a concise scientific article using formal academic language, logical argumentation, and evidence-based discussion.

Exercises on developing research questions and hypotheses. Develop relevant research questions and formulate clear, testable hypotheses based on a selected research topic.

Find three reliable sources on a topic and analyze them. Identify three credible academic sources related to a chosen topic and critically evaluate their relevance, reliability, and contribution to the research.

Preparing a reference list for a scholarly article. Prepare a correctly formatted reference list using an accepted academic citation style while following the principles of academic integrity.

	Writing an essay on the rules of academic writing to prevent plagiarism. Write an essay explaining the importance of academic integrity, proper citation practices, and effective strategies for preventing plagiarism. Write an analytical article on the chosen topic. Prepare an analytical article that critically examines a selected topic using logical reasoning, scholarly evidence, and well-supported arguments. Writing the conclusion for a scientific article. Write a clear and concise conclusion that summarizes the main findings, answers the research questions, and highlights the significance of the study. Write a logical text based on a topic outline. Develop a coherent academic text by following a prepared outline and maintaining logical organization throughout the writing. Editing and revising the initial version of a scientific article. Review and improve the first draft of a scientific article by correcting language, structure, formatting, coherence, and academic style. Write an introduction for the text you have written. Prepare an effective introduction that presents the research background, objectives, significance, and scope of the study. Present the results of the scientific work using graphs or diagrams. Organize and present research findings using appropriate graphs, charts, diagrams, and explanatory text to improve clarity and understanding. Write the conclusion and analysis section for the chosen topic. Prepare the final analysis and conclusion by interpreting the research findings, discussing their significance, and providing recommendations for future research.
Teaching Methods	The module is delivered through interactive lectures, writing workshops, practicals, group discussions, case studies, problem-based learning, project-based learning, peer review activities, and independent research tasks. Students develop academic writing skills by analyzing scholarly texts, preparing essays, research papers, literature reviews, and scientific articles, as well as practicing citation and referencing techniques using internationally recognized academic styles. Practical writing exercises, editing and proofreading activities, presentations, and peer feedback strengthen critical thinking, academic communication, and research competencies. The use of digital libraries, academic databases, plagiarism detection tools, reference management software, and other digital

	educational resources further enhances students' ability to produce high-quality academic writing in accordance with international standards.
Prerequisites	Mother Tongue and Literature Textual Studies Stylistics Logic Research methodology
Exams and Assessment	Student performance is assessed through continuous assessment (20%), midterm assessment (30%), and a final written assessment (50%). The continuous assessment (20%) evaluates classroom participation, seminar performance, writing exercises, text analysis activities, presentations, citation and referencing tasks, and the successful completion of independent study assignments, including essays, literature reviews, and academic writing exercises. The midterm assessment (30%) consists of written assignments, quizzes, analytical writing tasks, and the evaluation of independent projects, such as research proposals, annotated bibliographies, article reviews, and short scientific papers. The final assessment (50%) is a written examination and academic writing task designed to assess students' understanding of the principles of academic writing, research methodology, citation and referencing standards, critical analysis, and their ability to prepare well-structured academic texts. Successful completion of all independent study assignments is required as an essential component of the module assessment.
Recommended Literature	1. Kadirova Z. Z. Academic Writing. A Textbook. –Termiz: "Globe Edit", 2024. –110 p. 2. Короткина И. Б. Academic Writing: Process, Product, etc. A Textbook for Higher Education Institutions. Moscow: Yurayt, 2021 3. Kuvshinskaya Y. M. , Zevakhina N. A. , Akhapkina Y. E. , Gordienko E. N. Academic Writing: From Research to Text. Textbook and Workbook for Universities. Moscow: Yurayt, 2022 4. Korotkina I. B. A Model for Teaching Academic Writing. A Textbook for Higher Education Institutions. Moscow: Yurayt, 20215. David D. Riley, Kenny A. Hunt, "Computational Thinking for the Modern Problem Solver", CRC Press, 2014, 406 p. 5. N. Jiyanova, O. Mo'ydinova. Speech Culture. T. : "Ilm ziyo zakovat", 2019. 221 p.

	6. Qo'ng'urov R. , Begmatov E. , Tojiyev Yo. Fundamentals of Speech Culture and Stylistics, T. : "Fan", 1992, 254 p. 7. Husanov, N. , Rasulov, Mo'ydinov, Q. The Culture of Speech and the Art of Oratory. T. : "IQTISOD-MOLIYA", 2006. 8. M. Akhmedova. Speech Culture and Professional Public Speaking. TATU, 2017, 114 pp.
Responsible Lecturer	Zaynura Kadirova, PhD in Philology, Associate Professor.
Field	Description
Module Title	DATA STRUCTURES AND ALGORITHMS
Module Code	MTA1204
Credit Points (ECTS)	4 Credits
Workload	Total workload: 120 hours Contact hours: 60 hours (30 hours Lecture, 30 hours Practical lessons) Self-Assignments: 60 hours
Frequency/Duration	II semester
Module objective	The course "Data Structures and Algorithms" 60610400 is intended for students in the Software Engineering program, The course is designed to provide students with knowledge of systematic programming for managing computers in the digital world and of ways to solve the problems they encounter, based on modern requirements.
Module Learning Outcomes	Knowledge: LO1. Provide information on data structures and algorithms, demonstrate the role of information technologies in each person's life and in the development of society, and reveal the capabilities of data structures and algorithms. LO2. Develop practical skills in creating programs using modern programming languages; LO3. Develop practical skills in computer data structures and algorithms, as well as in working with Internet networks; LO4. Formulate an understanding of when and for what purposes to use data structures and algorithms. LO5. Formulate theoretical knowledge about the structure and development principles of modern information systems; LO6. Formulate knowledge about the application and development in the field of data structures and algorithms; LO7. Forming an understanding of digital technologies, data structures and algorithms, and programming technologies;

	LO8. Familiarize with methods for ensuring information security in the context of processing data structures and algorithms. Skills: LO9. In today's "Information" age, students not only increase their computer literacy but also learn about opportunities to work with data structures and algorithms; LO10. Studies the theoretical foundations of using personal computers in data structures and algorithms processes and the methods for applying them to various areas of data structures; LO11. Studies methods for the effective use of data structures and algorithms in management systems; LO12. Learns the skills to work with Internet systems and data structure technologies. Competencies: LO13. Studies the development trends of data structures and algorithms and their characterization; LO14. It involves forming concepts about when and for what purposes to use data structures and algorithms, and providing knowledge about how data can be generated; LO15. Become familiar with implementing methods to ensure security when working with data structures and algorithms and with their functional capabilities.
Content	Lectures Introduction. The Concept of Algorithms and Data Structures. Formalization of the Concept of an Algorithm. Computational Models. Comparing Algorithms. Determining the Computation Time for a Program. Criteria for Evaluating Algorithm Complexity This lecture introduces the fundamental concepts of algorithms and data structures and explains their importance in computer science and software development. Abstract Data Types and Data Structures. Stack. Queue. Vector. List. This lecture focuses on abstract data types (ADTs) and their implementation through various data structures. Students study the characteristics, operations, and applications of stacks, queues, vectors, and lists. Sorting Algorithms. The Simplest Algorithms. Low-Complexity. Statement of the Inner Sorting Problem and Approaches to Performance Evaluation. Simple Sorting Algorithms and Their Analysis. Operations on Binary Search Trees. This lecture introduces the fundamentals of sorting algorithms and discusses the internal sorting problem and methods for evaluating sorting performance.

Merge Sort Algorithms. Merge Sort Algorithm. Evaluation of the Merge Sort Algorithm. This lecture examines the Merge Sort algorithm as an efficient divide-and-conquer sorting technique. Students learn the recursive process of dividing data into smaller subproblems, merging sorted subarrays, and evaluating the algorithm's efficiency.

Quick Sort Algorithm. General Description. General Sorting Mechanism. "Divide and Conquer" Algorithm. "Divide and Conquer" Algorithm Pseudocode. QuickSort Algorithm Analysis. This lecture presents the Quick Sort algorithm and its divide-and-conquer strategy for efficient sorting. Students study the partitioning process, pivot selection methods, recursive implementation, and algorithm pseudocode.

Elements of Graph Theory and Traversal Algorithms. Representation of a Graph. Graph. Basic Terms of Graph Theory. Basic Concepts of Graph Theory. Trees and Chains. Methods of Graph Representation. This lecture introduces the fundamental concepts of graph theory, including vertices, edges, paths, cycles, trees, and connected components.

Graph Traversal Algorithms. Depth-First Search (DFS) Algorithm. Breadth-First Search (BFS) Algorithm. This lecture focuses on graph traversal techniques using Depth-First Search (DFS) and Breadth-First Search (BFS). Students learn the principles, implementation methods, and complexity analysis of both algorithms.

Trees as a Special Case of Graphs. Binary Trees. Methods of Representing Trees in a Machine. Prufer Code. Reconstructing a Tree from Prufer Code. This lecture explores trees as a special category of graphs and introduces binary trees and their properties. Students study different methods for representing trees in computer memory and learn the concept of Prufer codes for encoding labeled trees.

Sorted and Balanced Trees. Balancing Nodes. Theorem on Balanced Trees. AVL Tree. Performance Analysis of AVL Trees. Node Key Search Function. Tree Display Function. This lecture introduces balanced search trees with emphasis on AVL trees. Students learn balancing techniques, tree rotations, insertion, deletion, and search operations while maintaining tree balance.

B-Trees. Operations on a B-Tree. Search Algorithm in a B-Tree. Inserting an Element in a B-Tree. Deleting an Element. Implementation of a B-Tree. This lecture presents the structure and functionality of B-Trees used in databases and file systems.

Students study searching, insertion, deletion, node splitting, and merging operations while maintaining tree balance

Priority Queues. Binary Heap – Pyramid. Creating an Empty Heap. Inserting an Element into a Binary Heap. Sorting with Heap Sort. This lecture introduces priority queues and binary heaps as efficient data structures for managing prioritized elements. Students learn heap construction, insertion, deletion, heapify operations, and the implementation of Heap Sort.

Computational Geometry Algorithms. Convex Hull Problems. Sweep Line Algorithm for Finding Line Intersections in the Plane. This lecture introduces computational geometry and its applications in computer science. Students study algorithms for solving convex hull problems and the Sweep Line algorithm for detecting line segment intersections.

Hash Tables. Hash Tables and Their Organization. Implementation of Hash Tables in the C++ Programming Language. The Collision Problem. This lecture explains the principles of hash tables and their role in efficient data storage and retrieval. Students study hash table organization, insertion, deletion, and search operations, along with implementation techniques in C++.

Hash Function. Types of Hash Functions. Applications of Hash Functions and Their Role in Information Security. This lecture examines the design and characteristics of hash functions and their importance in computing. Students learn different types of hash functions, their mathematical properties, and their applications in indexing, authentication, digital signatures, password protection, and information security.

SHA-1 Hash Function Algorithm. This lecture focuses on the SHA-1 cryptographic hash algorithm, its internal structure, and the process of generating fixed-length hash values from input data. Students study the algorithm's operational steps, security properties, and practical applications in data integrity verification and digital authentication.

Practical lessons

Computation models. Complexity of algorithms. Determining the program's execution time. Criteria for evaluating algorithm complexity. This lecture introduces computation models used to describe algorithm execution and analyze computational processes. Students learn the concepts of time and space complexity, methods for determining program

execution time, and the criteria used to evaluate algorithm efficiency.

Abstract data types and data structures. Stack. Queue. Vector. List. This lecture focuses on abstract data types (ADTs) and their implementation through fundamental data structures. Students study the characteristics, operations, and applications of stacks, queues, vectors, and linked lists.

Simple sorting algorithms and their analysis. This lecture introduces the basic principles of sorting algorithms and examines simple sorting techniques, including Bubble sort, Selection sort, and Insertion sort.

Merge sort algorithm. Merging sort algorithm. Evaluation of the merge sort algorithm. This lecture presents the Merge sort algorithm as an efficient recursive sorting technique based on the divide-and-conquer paradigm.

Divide and conquer algorithm. Pseudo-code for the divide and conquer algorithm. Analysis of the Quicksort algorithm. This lecture explains the divide-and-conquer strategy and its application in the Quicksort algorithm. Students study the partitioning process, pivot selection, recursive implementation, and pseudocode of the algorithm.

Trees and chains. Graph representation methods. This lecture introduces trees and chains as fundamental graph structures and examines different methods for representing graphs in computer memory.

Depth-first search (DFS) algorithm on a graph. This lecture focuses on the Depth-first search (DFS) algorithm for graph traversal.

Methods for representing trees in a computer. Prufer code. Restoring a tree using Prufer code. This lecture explains different techniques for representing tree structures in computer memory, including sequential and linked representations. Students learn the concept of Prufer code for encoding labeled trees and study algorithms for reconstructing a tree from a given Prufer sequence. The lecture highlights applications of tree encoding in graph theory and network analysis.

Analysis of AVL tree efficiency. Tree display function. This lecture examines AVL trees as self-balancing binary search trees. Students analyze the efficiency of insertion, deletion, and search operations while maintaining balance through tree rotations.

Adding elements in B-trees. Deleting elements. Implementation of B-trees. This lecture introduces the structure and implementation of B-trees used in databases and file

systems. Students study algorithms for inserting and deleting elements while maintaining tree balance through node splitting and merging.

Inserting an element into a binary heap. Sorting using a heap (Heap sort). This lecture focuses on binary heaps and their applications in priority queues and sorting. Students learn how to insert elements while preserving the heap property and implement the Heap sort algorithm.

Sweep line algorithm for finding intersections in the plane. This lecture introduces the Sweep line algorithm as an efficient computational geometry technique for detecting line segment intersections in a two-dimensional plane. Students study the event-driven approach, active data structures, and computational complexity of the algorithm.

Implementation of hash tables in the C++ programming language. The collision problem. This lecture explains the implementation of hash tables using the C++ programming language. Students study hash table construction, insertion, deletion, and search operations, as well as methods for resolving collisions, including separate chaining and open addressing

Application of hash functions and their role in information security. SHA-1 hash function algorithm. This lecture explores the principles of hash functions and their applications in information security. Students study the characteristics of cryptographic hash functions, their use in data integrity verification, digital signatures, password protection, and authentication systems.

Independent study.

Formalization of the concept of an algorithm. This lecture introduces the formal concept of an algorithm and its fundamental properties. Students study the characteristics of algorithms, including finiteness, definiteness, correctness, effectiveness, and generality.

Comparison of algorithms. Determining the execution time of a program. This lecture focuses on comparing algorithms based on their efficiency and computational performance. Students learn techniques for measuring program execution time and evaluating algorithm performance using time and space complexity.

Abstract data types and data structures. This lecture explains the concepts of abstract data types and their implementation through various data structures. Students study the relationship between logical data models and physical implementations while

learning the characteristics, operations, and applications of commonly used data structures in programming.

Formulation of the internal sorting problem and approaches for performance evaluation. This lecture introduces the internal sorting problem and discusses different approaches for evaluating the performance of sorting algorithms. Students learn how to measure sorting efficiency based on the number of comparisons, data movements, execution time, and memory usage.

Merge sort algorithms. This lecture presents the Merge sort algorithm as an efficient recursive sorting technique based on the divide-and-conquer strategy. Students study the process of dividing data into smaller subproblems, merging sorted sequences, and analyzing the algorithm's computational complexity.

Quick sort algorithm. General description. This lecture introduces the Quick sort algorithm and explains its working principles. Students learn about partitioning, pivot selection, recursive implementation, and the divide-and-conquer strategy.

Elements of graph theory and traversal algorithms. This lecture introduces the basic concepts of graph theory, including vertices, edges, paths, cycles, connected graphs, and trees.

Graph traversal algorithms. This lecture focuses on graph traversal techniques, including Depth-first search (DFS) and Breadth-first search (BFS).

Trees as a special case of graphs. This lecture examines trees as a special class of graphs with unique structural properties.

Sorted and balanced trees. This lecture introduces sorted and balanced tree structures used for efficient searching and data management.

B-trees. Operations on a B-tree. Search algorithm on a B-tree. This lecture explains the structure and operation of B-trees as balanced multiway search trees.

Priority queues. Binary heap (stack). This lecture introduces priority queues and binary heaps as efficient data structures for managing prioritized data.

Computational geometry algorithms. This lecture introduces the basic concepts of computational geometry and examines algorithms for solving geometric problems.

Hash tables and their organization. explains the organization and implementation of hash tables for efficient data storage and retrieval.

	Types of hash functions. introduces different types of hash functions and their characteristics.
Teaching Methods	The course is delivered through a combination of student-centered and practice-oriented teaching methods to develop both theoretical knowledge and practical programming skills. Lectures are used to introduce fundamental concepts, algorithm design techniques, and data structures. Practical sessions provide opportunities for solving algorithmic problems, implementing data structures, and analyzing algorithm performance using the C++ programming language. The teaching process includes interactive lectures, classroom discussions, problem-solving activities, case studies, coding demonstrations, individual and group assignments, laboratory exercises, project-based learning, and independent study. Students are encouraged to analyze algorithms, compare different approaches, optimize program performance, and apply appropriate data structures to solve real-world computational problems. Modern educational technologies, multimedia presentations, online learning platforms, algorithm visualization tools, programming environments, and automated testing systems are used to enhance learning. Continuous feedback, collaborative learning, and active student participation are promoted throughout the course to strengthen analytical thinking, computational problem-solving, and software development skills.
Prerequisites	Computer Science and Information Technology (CSIT) Information Technology (IT) Operating Systems (OS)
Exams and Assessment	Student performance is assessed through continuous assessment (20%), midterm assessment (30%), and a final written and practical examination (50%). The continuous assessment (20%) evaluates classroom participation, laboratory performance, coding exercises, algorithm implementation tasks, quizzes, homework assignments, and the successful completion of independent study activities, including the implementation of data structures, algorithm analysis, complexity evaluation, debugging, and computational problem-solving. The midterm assessment (30%) consists of written tests, practical programming tasks, algorithm design exercises, data structure implementation, complexity analysis, and the evaluation of programming assignments that assess students' understanding of fundamental data structures, sorting and

	searching algorithms, graph algorithms, tree structures, hash tables, and their ability to develop efficient algorithmic solutions. The final assessment (50%) consists of a written examination and practical programming tasks designed to evaluate students' knowledge of data structures, algorithm design and analysis, computational complexity, searching and sorting techniques, graph and tree algorithms, hash tables, priority queues, and their ability to implement efficient solutions to real-world computational problems using the C++ programming language. Successful completion of all independent study assignments is required as an essential component of the module assessment.
Recommended Literature	1. R. Lafore, Data Structures and Algorithms in Java, 2nd edition, 2017. 2. N. Karumanchi: Data Structures and Algorithms Made Easy: Data Structures and Algorithmic Puzzles, Second edition, 2015. 3. M. A. Weiss, Data Structures & Algorithm Analysis in C++, 4th edition, 2014. 4. A. Aho, J. Hopcroft, D. Ullman. Data Structures and Algorithms. M. : Williams, 2010. 5. Bucknall J. / Бакнелл Дж. - The Tomes of Delphi™ Algorithms and Data Structures / Фундаментальные алгоритмы и структуры данных в Delphi™, 2003. 6. Nazarenko P. A. Algorithms and Data Structures. Samara: PGUTI, 2015. 196 pp. 7. O. R. Yusupov, I. Q. Ximmatov, E. Sh. Eshonqulov. Algorithms and Data Structures. A textbook for higher education institutions. – Samarkand: SamDU Publishing House. 2021, 205 pages 8. Mirziyoyev Sh. M. Together we will build a free, prosperous, and democratic state of Uzbekistan. Speech at the joint session of the chambers of the Oliy Majlis dedicated to the ceremonial inauguration of the President of the Republic of Uzbekistan / S. M. Mirziyoyev. - Tashkent : O'zbekiston, 2016. - 56 p. 9. D. Sh. Matros, G. B. Podnebesova. Theory of Algorithms — M. : BINOM. Laboratory of Knowledge, 2008. — 202 p. 10. Gagarin L. D. , Koldaev V. D. Algorithms and Data Structures. INFRA-M, 2009. -304 p. 11. V. A. Uspensky, A. L. Semyonov. Theory of Algorithms: Basic Discoveries and Applications. M. : Nauka, 1987. – 288 pp.
Responsible Lecturer	Mamatkabilov Abdilmajid Khujamuratovich D

2nd ACADEMIC YEAR

Field	Description
Module Title	DIFFERENTIAL EQUATIONS
Module Code	DIT1406
Credit Points (ECTS)	6
Workload	Total workload: 180 hours Contact hours: 90 hours (44 hours Lecture, 46 hours Practical lessons) Self-Assignments: 90 hours
Frequency/Duration	IV semester
Module objective	
Module Learning Outcomes	Knowledge . To have an understanding of the basic concepts, types, solutions of differential equations, and their applications in the applied sciences. . Knowledge of first- and higher-order ordinary differential equations, their solution methods, and the theory of the Cauchy problem. . Know the basic concepts of linear and nonlinear differential equations, systems of differential equations, and partial differential equations. . Understanding the application areas of differential equations in engineering, physics, information technology, and mathematical modeling. . Knowledge of the fundamental theorems on the existence, uniqueness, and stability of solutions of differential equations. Skills . Be able to solve first-order differential equations using various methods. . Solve higher-order and linear differential equations using analytical methods and analyze the results. . Solving systems of differential equations and applying them to model physical and technical processes. LO9. Constructing mathematical models based on differential equations and analyzing their solutions. Qualifications

	LO10. Be proficient in applying differential equations to practical problems, constructing mathematical models, and interpreting their results. LO11. Be proficient in solving differential equations and visually analyzing the results using modern mathematical software (Maple, MATLAB, MathCAD, Python, etc.). LO12. Be proficient in independently solving scientific and practical problems in differential equations, evaluating the results, and applying them effectively in professional activities.
Content	Lectures Introduction to differential equations. Students study the basic concepts, classifications, and applications of differential equations. The topic provides a foundation for mathematical modeling in science, engineering, and technology. Separable differential equations and those reducible to them. Students learn analytical methods for solving separable differential equations and equations reducible to separable form. The topic develops problem-solving skills through variable separation and substitution techniques. Differential equations reducible to a homogeneous differential equation. Students study methods for transforming differential equations into homogeneous form using suitable substitutions. The topic emphasizes systematic solution techniques for first-order equations. Linear differential equations. The method of variation of constants. Bernoulli and Riccati equations. Students examine first-order linear differential equations and specialized solution methods, including variation of constants, Bernoulli, and Riccati equations. The topic develops analytical techniques for solving a wide range of differential equations. Complete Differential Equations. Integrating Factor and Theorems on its Existence. Students study exact differential equations, integrating factors, and the conditions required for their existence. The topic provides methods for converting non-exact equations into exact forms for solution. Cauchy problem for a first-order equation. Theorem on the existence and uniqueness of a solution. Students investigate initial value problems and the existence and uniqueness theorem for first-order differential equations. The topic establishes the theoretical basis for obtaining unique solutions. Unsolved differential equations with respect to the derivative. Students study differential equations that are not

explicitly solved for the derivative. The topic introduces analytical methods for solving implicit differential equations.

Higher-order differential equations. Students study the classification and solution methods of higher-order differential equations. The topic introduces applications of higher-order models in science and engineering.

Linear Differential Equations of Higher Order. Students examine analytical methods for solving higher-order linear differential equations. The topic develops techniques for obtaining general and particular solutions.

Linear homogeneous differential equations with constant coefficients. Students study homogeneous linear differential equations with constant coefficients using characteristic equations. The topic develops methods for constructing complete solution sets.

Reducing the order of higher-order equations. Students learn techniques for reducing higher-order differential equations to lower-order forms. The topic focuses on simplifying complex equations through appropriate substitutions.

Integration of homogeneous and generalized homogeneous higher-order equations with respect to their variables Students study methods for integrating homogeneous and generalized homogeneous higher-order differential equations. The topic develops analytical skills through variable transformation techniques.

Homogeneous linear differential equations. Main properties of the solution. Students examine the fundamental properties of homogeneous linear differential equations and their solution spaces. The topic introduces principles such as linear independence and superposition.

The Wronskian determinant and its properties. The fundamental system of solutions. Students study the Wronskian determinant as a criterion for linear independence of solutions. The topic develops an understanding of fundamental solution systems for linear differential equations.

Non-homogeneous n -th order linear differential equations and finding their general and particular solutions Students study methods for solving non-homogeneous linear differential equations of order n . The topic focuses on constructing general solutions by combining complementary and particular solutions.

Linear differential equations with constant coefficients Students examine analytical methods for solving linear differential

equations with constant coefficients. The topic emphasizes characteristic equations and practical solution techniques.

Linear ordinary differential equations with constant coefficients. Students study ordinary linear differential equations with constant coefficients and their solution methods. The topic provides practical applications in mathematical modeling and engineering.

Equations reducible to constant coefficients. Euler equation Students study techniques for transforming differential equations into equations with constant coefficients. The topic introduces Euler equations and methods for obtaining their solutions.

Normal form of differential equations. Students examine methods for transforming differential equations into their normal form. The topic provides a standardized approach for simplifying analysis and solution.

Linear differential equation system with constant coefficients. Students study systems of linear differential equations with constant coefficients using matrix methods and eigenvalue techniques. The topic develops skills for analyzing dynamic systems.

Linear, heterogeneous differential equation system with constant coefficients. Students examine non-homogeneous systems of linear differential equations with constant coefficients. The topic focuses on methods for constructing complete vector solutions.

First-order ordinary differential equations. Students study the fundamental theory and analytical methods for solving first-order ordinary differential equations. The topic provides the foundation for advanced studies in differential equations and mathematical modeling.

Practics lessons

Introduction. Concept of differential equations. Some physical and geometric problems expressed by ordinary differential equations. Students study the fundamental concepts, classifications, and applications of ordinary differential equations. The topic introduces mathematical models for physical, geometric, and engineering problems.

Separable and homogeneous differential equations Students learn analytical methods for solving separable and homogeneous differential equations. The topic develops solution techniques based on variable separation and substitution methods.

Differential equations reducible to a homogeneous differential equation. Students study methods for transforming differential equations into homogeneous form through suitable substitutions. The topic emphasizes systematic analytical approaches to solving first-order equations.

Linear differential equations. The method of variation of constants. Bernoulli and Riccati equations. Students examine first-order linear differential equations and advanced solution techniques, including variation of constants, Bernoulli, and Riccati equations. The topic develops analytical problem-solving skills for various equation types.

Complete differential equations. Integrating factor and theorems on its existence. Students study exact differential equations, integrating factors, and the conditions required for their existence. The topic introduces methods for converting non-exact equations into exact forms for solution.

Cauchy problem for a first-order equation. Theorem on the existence and uniqueness of a solution. Students investigate initial value problems and the existence and uniqueness theorem for first-order differential equations. The topic establishes the theoretical foundation for obtaining unique solutions.

Undetermined differential equations with respect to the derivative. Students study differential equations that are not explicitly solved with respect to the derivative. The topic introduces analytical methods for solving implicit differential equations.

Higher-order differential equations. Students examine the classification, properties, and solution methods of higher-order differential equations. The topic introduces applications of higher-order models in science and engineering.

Higher-order linear differential equations. Students study the theory and analytical solution methods for higher-order linear differential equations. The topic develops techniques for constructing complete solutions.

Linear homogeneous differential equations with constant coefficients. Students learn solution methods for homogeneous linear differential equations with constant coefficients using characteristic equations. The topic emphasizes the relationship between characteristic roots and solution forms.

Reducing the order of higher-order equations Students study techniques for reducing higher-order differential equations to lower-order forms. The topic develops analytical skills through substitution and order-reduction methods.

Integration of homogeneous and generalized homogeneous higher-order ordinary differential equations. Students examine methods for integrating homogeneous and generalized homogeneous higher-order ordinary differential equations. The topic focuses on variable transformations and systematic solution procedures.

Homogeneous linear differential equations. Main properties of the solution. Linearly dependent and linearly independent functions. Students study the properties of homogeneous linear differential equations and the concepts of linear dependence and independence of functions. The topic provides the theoretical basis for constructing general solutions.

The Wronskian determinant and its properties. Fundamental system of solutions. Ostrogradsky–Liouville formula. Students investigate the Wronskian determinant, the fundamental system of solutions, and the Ostrogradsky–Liouville formula. The topic develops methods for analyzing the independence and behavior of solution sets.

Non-homogeneous nth-order linear differential equations and finding their general and particular solutions Students study methods for solving non-homogeneous nth-order linear differential equations. The topic focuses on determining complementary and particular solutions to construct complete solutions.

Higher-order linear differential equations with constant coefficients. Students examine analytical methods for solving higher-order linear differential equations with constant coefficients. The topic emphasizes characteristic equations and applications in mathematical modeling.

Linear ordinary differential equations of the first order with constant coefficients. Students study first-order linear ordinary differential equations with constant coefficients and their analytical solution methods. The topic provides practical applications in engineering and applied mathematics.

Linear ordinary differential equations with constant coefficients. Students examine ordinary linear differential equations with constant coefficients using standard analytical techniques. The topic develops problem-solving skills for mathematical and engineering applications.

Equations reducible to constant coefficients. Euler equation Students study methods for transforming variable-coefficient equations into equations with constant coefficients. The topic

introduces Euler equations and analytical techniques for obtaining their solutions.

Normal form of differential equations

Students learn methods for transforming differential equations into their normal form. The topic provides a systematic framework for simplifying analysis and solution procedures.

Linear differential equation system with constant coefficients. Students study systems of linear differential equations with constant coefficients using matrix methods and eigenvalue analysis. The topic develops techniques for modeling and solving dynamic systems.

A system of linear, first-order, inhomogeneous differential equations with constant coefficients. Students examine first-order inhomogeneous systems of linear differential equations with constant coefficients. The topic focuses on analytical methods for constructing complete vector solutions.

First-order differential equations with constant coefficients

Students study analytical methods for solving first-order differential equations with constant coefficients. The topic provides a foundation for advanced studies in differential equations and mathematical modeling.

Student Independent Work(SIW)

Homogeneous and generalized homogeneous differential equations. Students study analytical methods for solving homogeneous and generalized homogeneous differential equations. The topic develops solution techniques based on variable transformations and substitution methods.

Linear differential equations. Properties of the solution

Students examine the fundamental properties of solutions to linear differential equations. The topic emphasizes linearity, superposition, and the structure of general solutions.

Theorem on the existence of an integrating factor

Students study the theorem on the existence of an integrating factor for differential equations. The topic develops methods for transforming non-exact equations into exact equations for solution.

Integration of equations by parametric substitution

Students learn methods for solving differential equations using parametric substitutions. The topic focuses on simplifying complex equations through appropriate variable transformations.

Lagrange and Clairaut's equations

Students study Lagrange and Clairaut differential equations together with their analytical solution methods. The topic

introduces singular and general solutions and their mathematical properties.

Calculation of the exponential matrix. Students examine methods for computing the matrix exponential and its properties. The topic provides the foundation for solving systems of linear differential equations.

Integration of matrix differential equations. Students study analytical techniques for solving matrix differential equations. The topic develops methods for modeling and analyzing multidimensional dynamic systems.

State space of autonomous systems. Students examine the state-space representation of autonomous dynamical systems. The topic introduces state variables, trajectories, and qualitative system behavior.

Constructing the Green's function for boundary value problems. Students study the construction and application of Green's functions for solving boundary value problems. The topic develops analytical methods for obtaining particular solutions of differential equations.

The Sturm–Liouville problem. Eigenvalues and eigenfunctions. Students investigate Sturm–Liouville boundary value problems and the concepts of eigenvalues and eigenfunctions. The topic provides a theoretical foundation for solving mathematical physics problems.

Methods for integrating matrix differential equations Students study advanced methods for integrating matrix differential equations using analytical and matrix-based techniques. The topic emphasizes applications to engineering and control systems.

Continuity of the solution. Domain of solutions Students examine the continuity of differential equation solutions and the domains in which they exist. The topic develops an understanding of solution behavior under different conditions.

Theorem on the continuous dependence of the solution on the initial data and on the integral. Students study the theorem describing the continuous dependence of solutions on initial conditions and integral terms. The topic highlights the stability and sensitivity of differential equation models.

Stability of the solution in the Lyapunov sense Students investigate Lyapunov stability theory for differential equations and dynamical systems. The topic introduces methods for analyzing the stability of equilibrium solutions.

	Types of equilibrium states for a system of linear equations Students study different types of equilibrium states in systems of linear differential equations. The topic develops techniques for classifying equilibrium points and analyzing system behavior near them.
Teaching Methods	The module is delivered through interactive lectures, inquiry-based and problem-based learning, guided problem-solving sessions, computer-based mathematical modeling, seminars, group discussions, and project-based activities. Students solve analytical and numerical differential equation problems using specialized mathematical software, develop mathematical modeling and computational skills, and apply differential equations to real-world scientific and engineering problems. Independent learning and digital educational resources further promote critical thinking, mathematical reasoning, scientific communication, and the application of modern analytical and numerical methods for differential equations.
Prerequisites	Higher Mathematics (OMTB110) Linear Algebra (CHA1306) Mathematical Analysis (MABB120) Analytical Geometry (AGEB120)
Exams and Assessment	Type of examination (written, oral, project) and its weight Student performance is assessed through continuous assessment (20%), midterm assessment (30%), and a final written examination (50%). The continuous assessment (20%) evaluates students' participation in lectures and practical classes, problem-solving activities, and completion of independent study assignments, including presentations, analytical reports, mathematical modeling tasks, computational exercises, literature reviews, and mini-projects related to first- and higher-order differential equations, linear differential equations, systems of differential equations, and their practical applications. The midterm assessment (30%) consists of a written test and the evaluation of independent study projects, such as reports, presentations, analytical papers, and practical assignments covering differential equation solution methods, mathematical modeling, linear differential equations, systems of differential equations, and software-based analysis. The final assessment (50%) is a written examination consisting of theoretical and practical questions designed to assess students' understanding of differential equations, mathematical modeling, analytical problem-solving, and the application of differential

	equation methods to engineering, information technology, and scientific problems. Successful completion of all independent study assignments is required as an essential component of the module assessment.
Recommended Literature	1. Denisov A. M. , Rudin A. V. Ordinary Differential Equations. M. : MAX-Press, 2009. 2. Dmitriyev V. I. Lectures on Ordinary Differential Equations. Moscow: KDU, 2007. 3. M. S. Salohiddinov, G'. Y. Nasriddinov. Ordinary Differential Equations. Tashkent: "O'zbekiston", 1994. 4. Stepanov V. V. Course of Differential Equations. Moscow: Gosfizmatgiz, 1958. 5. El'sgol'ts L. E. Differential Equations and Variational Calculus. Moscow: Nauka, 1965. 6. Y. P. Ortiqov, N. Turg'unov, L. A. G'affarov. Collection of Examples and Problems from Ordinary Differential Equations. Tashkent, "Fan va texnologiya", 2009. 7. R. Turg'unboyev, Sh. Ismoilov, O. Abdullayev. Collection of Examples and Problems from the Course of Differential Equations (textbook). Tashkent, 2007. 8. Filippov A. F. Collection of Problems on Differential Equations.
Responsible Lecturer	J. Bozorov – Head of the Department of Mathematical Analysis, Doctor of Philosophy in Physics and Mathematics, (PhD).
Field	Description
Module Title	LINEAR ALGEBRA
Module Code	CHA1306
Credit Points (ECTS)	6-credits
Workload	Total workload: 180 hours Contact hours: 90 hours (46 hours Lecture, 44 hours Practical lessons) Self-Assignments: 90 hours
Frequency/Duration	III semester
Module objective	It consists of introducing students to the essential body of knowledge in mathematics (concepts, theorems and their proofs, methods for solving practical problems, etc.). At the same time, it helps students develop logical thinking, draw correct conclusions, and enhance their mathematical culture.
Module Learning Outcomes	Knowledge LO1. To have an understanding of the basic concepts of linear algebra: matrices, determinants, vector spaces, and linear transformations.

	LO2. Knowledge of matrix algebra, properties of determinants, inverse matrices, and methods for solving systems of linear equations. LO3. Understand vector spaces, bases, dimension, linear dependence and linearly independent vectors, matrix rank, and the Kronecker–Capelli theorem. LO4. Knowledge of the theoretical foundations of complex numbers, polynomials, Euclidean spaces, quadratic forms, and linear transformations. LO5. Understanding the application areas of linear algebra methods in mathematics, computer science, engineering, and computer science. Skills LO6. Be proficient in performing operations on matrices, calculating determinants, and finding inverse matrices. LO7. Develop skills in solving systems of linear equations using the Gauss, Cramer, and matrix methods. LO8. Acquire skills in solving practical problems related to vector spaces, matrix rank, complex numbers, and polynomials. LO9. Be able to put quadratic forms into canonical form, determine the eigenvalues and eigenvectors of linear transformations, and analyze the results. Qualifications LO10. Be proficient in the effective use of linear algebra methods to solve mathematical and engineering problems. LO11. Develop the ability to apply elements of linear algebra in mathematical modeling, computer calculations, and algorithm development. LO12. Be proficient in independently analyzing linear algebra problems, verifying solutions using modern software tools, and scientifically interpreting the results.
Content	Lectures: Matrices and operations on them. Students study the fundamental concepts of matrices and the basic operations performed on them. The topic provides the foundation for solving algebraic problems and mathematical modeling. Small determinants. Students learn methods for calculating second- and third-order determinants. The topic develops computational skills required for solving systems of linear equations. nth-order determinants and their properties. Students study determinants of arbitrary order and their fundamental properties.

The topic emphasizes efficient techniques for determinant evaluation and applications.

Algebraic complements and minors. Laplace theorem. Students examine the concepts of minors, cofactors, and Laplace expansion. The topic develops methods for calculating determinants and solving matrix-related problems.

Additional properties of determinants and matrices. Inverse matrix. Students study advanced properties of determinants and matrices together with methods for finding inverse matrices. The topic emphasizes applications in solving linear systems.

Gauss elimination for solving a system of linear equations. Students learn the Gaussian elimination method for solving systems of linear equations. The topic develops systematic techniques for row reduction and solution analysis.

The Matrix Method for Solving a System of Linear Equations Students study matrix-based methods for solving systems of linear equations. The topic introduces matrix representations and computational solution techniques.

Cramer's Rule and Inverse Matrix Methods for Solving Systems of Linear Equations. Students examine Cramer's Rule and inverse matrix methods for solving linear systems. The topic compares these approaches and their practical applications.

n-dimensional vector space. Linear dependence. Basis of a vector space. Students study the structure of n-dimensional vector spaces, linear dependence, and vector space bases. The topic provides the theoretical foundation for linear algebra.

Linear subspaces. Linear span. Sum and intersection of subspaces. Students examine the concepts of linear subspaces, spans, sums, and intersections. The topic develops the ability to analyze relationships among vector spaces.

Matrix rank. Kronecker–Capelli theorem. Students study the concept of matrix rank and the Kronecker–Capelli theorem. The topic establishes criteria for determining the consistency of systems of linear equations.

Homogeneous system of equations. Fundamental solution. Students investigate homogeneous systems of linear equations and their fundamental solution sets. The topic develops methods for constructing general solutions.

Complex numbers and operations on them. Moivre's formula Students study complex numbers, their algebraic and trigonometric forms, and operations involving them. The topic introduces Moivre's formula for powers and roots of complex numbers.

Extracting a root from a complex number. Students learn methods for finding roots of complex numbers using algebraic and trigonometric techniques. The topic emphasizes practical computation and geometric interpretation.

Polynomials with one unknown. Horner's scheme. Bézout's theorem. Students study polynomial operations, Horner's scheme, and Bézout's theorem. The topic develops efficient methods for polynomial evaluation and factorization.

Remainder property. GCD of polynomials. Irreducible polynomials. Students examine the remainder theorem, greatest common divisors of polynomials, and irreducible polynomials. The topic provides methods for simplifying polynomial expressions.

Rational fractions and their simplification to lowest terms. Students study rational algebraic fractions and techniques for reducing them to their simplest form. The topic develops skills in polynomial factorization and algebraic simplification.

Euclidean space. Cauchy–Bunyakovsky inequality. The process of orthogonalization. Students investigate Euclidean spaces, the Cauchy–Bunyakovsky inequality, and orthogonalization methods. The topic introduces geometric concepts essential to linear algebra.

Linear, bilinear, and quadratic forms. Students study linear, bilinear, and quadratic forms together with their algebraic properties. The topic provides the foundation for applications in geometry and optimization.

Methods for putting a quadratic form into canonical form. Students examine methods for transforming quadratic forms into canonical form. The topic develops techniques for simplifying quadratic expressions through coordinate transformations.

Positively definite quadratic forms. The law of inertia. Students study positive definite quadratic forms and Sylvester's law of inertia. The topic emphasizes criteria for classifying quadratic forms and their applications.

Linear transformations and their matrices. Students investigate linear transformations and their matrix representations. The topic develops methods for analyzing mappings between vector spaces.

Image, kernel, invariant subspaces, eigenvalues, and eigenvectors of a linear transformation. Students study the image, kernel, invariant subspaces, eigenvalues, and eigenvectors associated with linear transformations. The topic

provides the theoretical foundation for advanced linear algebra and its applications in science and engineering.

Practices:

Matrices and operations on them. Students perform fundamental matrix operations, including addition, subtraction, multiplication, and transposition. The practical session develops computational skills for solving linear algebra problems.

Minor determinants. Students calculate second- and third-order determinants using standard computational techniques. The practical session strengthens accuracy and efficiency in determinant evaluation.

n th-order determinants and their properties. Students compute determinants of higher order and verify their fundamental properties through practical exercises. The session develops techniques for simplifying determinant calculations.

Algebraic complement and minors. Laplace theorem. Students determine minors and cofactors and apply Laplace's expansion theorem to evaluate determinants. The practical session reinforces systematic determinant computation methods.

Additional properties of determinants and matrices. Inverse matrix. Students investigate determinant properties and compute inverse matrices using analytical methods. The session develops practical skills in matrix manipulation and verification.

Gauss elimination method for solving a system of linear equations. Students solve systems of linear equations using the Gaussian elimination method. The practical session emphasizes row operations and solution verification.

The Matrix Method for Solving Linear Equation Systems. Students represent systems of linear equations in matrix form and solve them using matrix techniques. The session develops computational proficiency in linear algebra.

Cramer's and inverse matrix methods for solving a system of linear equations. Students solve systems of linear equations using Cramer's Rule and the inverse matrix method. The practical session compares the efficiency and applicability of both techniques.

n -dimensional vector space. Linear dependence. Basis of a vector space. Students determine linear dependence, independence, and bases of vector spaces through practical examples. The session develops an understanding of vector space structure.

Linear subspaces. Linear span. Sum and intersection of subspaces. Students identify linear subspaces and compute

their spans, sums, and intersections. The practical session strengthens analytical reasoning in vector space analysis.

Matrix rank. Kronecker–Capelli theorem. Students determine matrix rank and apply the Kronecker–Capelli theorem to analyze systems of linear equations. The session develops skills in testing system consistency.

Homogeneous system of equations. Fundamental solution. Students solve homogeneous systems of linear equations and construct their fundamental solution sets. The practical session emphasizes solution space analysis.

Complex numbers and operations on them. Moivre's formula. Students perform arithmetic operations with complex numbers and apply Moivre's formula to compute powers and roots. The session develops computational skills in complex analysis.

Taking the square root of a complex number. Students calculate square roots of complex numbers using algebraic and trigonometric methods. The practical session reinforces geometric interpretation and computational techniques.

Polynomials with one unknown. Horner's scheme. Bézout's theorem. Students evaluate and factor polynomials using Horner's scheme and Bézout's theorem. The practical session develops efficient polynomial computation methods.

Remainder property. GCD of polynomials. Irreducible polynomials. Students apply the remainder theorem, determine the greatest common divisor of polynomials, and identify irreducible polynomials. The session develops polynomial simplification techniques.

Rational fractions and reducing them to their simplest form Students simplify rational algebraic fractions using polynomial factorization methods. The practical session strengthens algebraic manipulation skills.

Euclidean space. Cauchy–Bunyakovsky inequality. Orthogonalization process. Students solve problems involving Euclidean spaces, the Cauchy–Bunyakovsky inequality, and orthogonalization. The practical session develops geometric and computational reasoning.

Linear, bilinear, and quadratic forms. Students analyze and compute linear, bilinear, and quadratic forms through practical exercises. The session develops the ability to interpret algebraic structures and their applications.

Methods for putting a quadratic form into canonical form Students transform quadratic forms into canonical form using

appropriate matrix methods. The practical session develops skills in simplifying quadratic expressions.

Positively definite quadratic forms. Law of inertia

Students determine whether quadratic forms are positive definite and apply the law of inertia. The practical session develops classification and analytical skills for quadratic forms.

Linear transformations and their matrices. Students construct matrix representations of linear transformations and analyze their properties. The practical session develops practical skills in applying linear transformations to vector spaces.

Student Independent Work

Axiomatic construction of the theory of determinants

Students study the axiomatic foundations of determinant theory and the properties that uniquely define determinants. The topic develops a rigorous understanding of determinant functions and their applications.

Fundamental Theorem of Algebra. Students examine the Fundamental Theorem of Algebra and its significance in polynomial theory. The topic establishes the existence of roots for polynomial equations over the complex numbers.

Third- and fourth-order equations. Students study analytical methods for solving cubic and quartic equations. The topic introduces classical solution techniques and their applications in algebra.

Sturm's theorem and its applications. Students investigate Sturm's theorem for determining the number of real roots of polynomial equations. The topic emphasizes practical methods for root analysis and localization.

Polynomials with rational coefficients. Students study the properties of polynomials with rational coefficients and techniques for analyzing their roots. The topic develops methods for factorization and algebraic manipulation.

Field of algebraic numbers. Students examine the concept of algebraic number fields and their algebraic structure. The topic introduces extensions of the rational number field and their mathematical properties.

Orthogonal complement and its properties. Students study orthogonal complements in Euclidean vector spaces and their fundamental properties. The topic develops methods for analyzing orthogonal subspaces.

Methods for finding a basis in an orthogonal complement space. Students learn techniques for constructing a basis of an

orthogonal complement. The topic emphasizes computational methods and applications in linear algebra.

Calculation of orthogonal complements and orthogonal projections. Students compute orthogonal complements and orthogonal projections using vector and matrix methods. The topic develops practical skills for solving geometric and algebraic problems.

Conjugate and self-adjoint linear transformations. Their matrices. Students study conjugate and self-adjoint linear transformations together with their matrix representations. The topic introduces their algebraic properties and applications.

Finding the characteristic roots of symmetric linear transformations. Students examine methods for determining the eigenvalues of symmetric linear transformations. The topic highlights the relationship between symmetry and spectral properties.

Geometric method for finding the Jordan normal form of a matrix. Students study geometric approaches for constructing the Jordan normal form of a matrix. The topic develops an understanding of generalized eigenvectors and canonical representations.

Diagonalization of linear transformations in complex linear spaces. Students investigate conditions for diagonalizing linear transformations in complex vector spaces. The topic develops techniques for simplifying matrix representations through eigenvalue analysis.

Complex numbers and operations on them Students study the algebraic and geometric representations of complex numbers and perform arithmetic operations with them. The topic provides the foundation for advanced algebraic concepts.

Taking roots of complex numbers. Roots of unity Students learn methods for finding roots of complex numbers and studying roots of unity. The topic develops computational techniques using trigonometric and exponential forms.

Linear spaces. Linear subspace. Sum and intersection of subspaces. Students examine linear spaces, subspaces, and operations involving their sums and intersections. The topic develops a deeper understanding of vector space structure.

Euclidean space. The Cauchy–Bunyakovsky inequality. The orthogonalization process. Students study Euclidean spaces, the Cauchy–Bunyakovsky inequality, and orthogonalization

	methods. The topic provides essential tools for geometric and algebraic analysis. Orthogonal complement and orthogonal projection Students investigate orthogonal complements and orthogonal projections in Euclidean spaces. The topic develops methods for decomposing vectors and solving approximation problems. Linear, bilinear, and quadratic forms. Students study the properties and applications of linear, bilinear, and quadratic forms. The topic establishes the theoretical basis for geometry, optimization, and matrix analysis. Methods for bringing a quadratic form to canonical form Students examine methods for transforming quadratic forms into canonical form. The topic develops analytical techniques for simplifying quadratic expressions. Positively definite quadratic forms. The law of inertia Students study positive definite quadratic forms and Sylvester's law of inertia. The topic develops methods for classifying quadratic forms and analyzing their properties. Linear transformations and their matrices Students investigate linear transformations and their corresponding matrix representations. The topic develops methods for analyzing mappings between vector spaces. Image and kernel of a linear transformation Students study the image and kernel of linear transformations and their role in vector space theory. The topic develops techniques for analyzing the structure and properties of linear mappings.
Teaching Methods	The module is delivered through interactive lectures, inquiry-based and problem-based learning, guided problem-solving sessions, computer-assisted matrix computations, seminars, group discussions, and project-based activities. Students investigate vector spaces, matrices, linear transformations, and systems of linear equations using specialized mathematical software, while developing analytical reasoning, computational proficiency, and mathematical modeling skills. Independent learning and digital educational resources further promote critical thinking, mathematical communication, and the application of linear algebra concepts to science, engineering, data analysis, and modern computational technologies.
Prerequisites	Higher Mathematics Mathematical analysis Analytic Geometry Computer Science and Information Technology

Exams and Assessment	Type of examination (written, oral, project) and its weight Student performance is assessed through continuous assessment (20%), midterm assessment (30%), and a final written examination (50%). The continuous assessment (20%) evaluates students' participation in lectures and practical classes, problem-solving activities, and completion of independent study assignments, including presentations, analytical reports, computational exercises, literature reviews, and mini-projects related to matrices, determinants, systems of linear equations, vector spaces, complex numbers, quadratic forms, and linear transformations. The midterm assessment (30%) consists of a written test and the evaluation of independent study projects, such as reports, presentations, analytical papers, and practical assignments covering matrix operations, determinants, systems of linear equations, vector spaces, complex numbers, polynomials, and linear transformations. The final assessment (50%) is a written examination consisting of theoretical and practical questions designed to assess students' understanding of linear algebra concepts, mathematical reasoning, problem-solving skills, and their ability to apply linear algebra methods to mathematical, engineering, and computer science problems. Successful completion of all independent study assignments is required as an essential component of the module assessment.
Recommended Literature	1. Ayupov A. Sh. , Omirov B. A. , Xudoyberdiyev A. X. Linear Algebra (textbook). Muxr-press Publishing House. Tashkent, 2023. – 392 pages. 2. Allakov I. Algebra Part I, "Surxon-nashriyoti". Termiz, 2024. – 332 pages. 3. Poskuryakov I. I. Collection of Problems in Linear Algebra. Nauka, 2005. 4. Xojiyev J. X. , Faynleyb A. S. Course of Algebra and Number Theory, Tashkent, "Uzbekiston", 2001. 5. Faddeev D. K. Lectures on Algebra, M. , "Nauka", 1984. 6. Kenneth Kuttler. Elementary Linear Algebra. 2012, Ventus Publishing ApS, ISBN 978-87-403-0018-5. 7. Kostrikin A. I. "Introduction to Algebra", M. , 1977. 8. Problems and Exercises from Algebra and Number Theory. Part II, textbook by Prof. U. Kh. Narzullayev, A. S. Soleev, and Kh. Kh. Ro'zimurodov. Samarkand, 2021. – 201 pages.

	9. Gelfand I. M. Lectures on Linear Algebra. "Higher and Secondary School," 1962. 10. Faddeev D. K. , Sominsky I. S. Problems in Higher Algebra. Saint Petersburg, 1999. 11. Kostrikin A. I. et al. Collection of Problems in Algebra. Nauka, 1986. 12. Allakov. Examples and Problems from Number Theory. – T. : Innovatsion Rivojlanish Publishing House, 2020. – 348 pp. 13. Kletnik D. V. Collection of Problems in Analytic Geometry. M. , FIZMATLIT, 2016. 14. Izu Vaisman. Analytical Geometry. World Scientific, USA, 2007
Responsible Lecturer	Xo'jamqulov Bekzod Turdaliyevich – Teacher of the " Algebra and Geometry " department at Termez State University.
Field	Description
Module Title	DATABASE
Module Code	MAB1306
Credit Points (ECTS)	6 Credits
Workload	Total workload: 180 hours Contact hours: 90 hours (44 hours Lecture, 46 hours Practical lessons) Self-Assignments: 90 hours
Frequency/Duration	III semester
Module objective	Data Base 60610600– Designed for students in the Software Engineering program, it teaches them, based on modern requirements, how to create databases in the digital world and find solutions to the problems they encounter.
Module Learning Outcomes	Knowledge LO1. Information, its storage methods, processing and transmission, the mathematical and software foundations of computing systems, their characteristics in scientific fields, manufacturing, and education, computer software, and an understanding of program types and features; LO2. Be proficient in developing algorithms for solving applied problems, building mathematical (computer) models and creating their software, generating linear lists, aggregating data in a structure, and working with indicators. LO3. Understanding of data, skill in creating a database LO4. SQL Server environments; LO5. SQL and NoSQL databases;

	Skills LO6. Working with SQL applications, writing code, designing and shaping databases LO7. Develop the skill of creating and shaping a database in the SQL query language LO8. Writing, editing, and optimizing SQL code; LO9. Establishing connections between programs Qualifications LO10. Knowledge of SQL commands; LO11. Develop skills to implement various connections between databases. LO12. Creating, designing, and implementing database projects.
Content	Lectures Introduction to the Relational Model. Students study the fundamental concepts of the relational database model, including relations, attributes, tuples, and keys. The topic provides the theoretical foundation for designing and managing relational databases. Introduction to SQL. Students learn the basic concepts and syntax of Structured Query Language (SQL). The topic introduces essential commands for interacting with relational databases. Setting up and configuring the SQL environment Students install, configure, and manage an SQL database environment. The topic develops practical skills for creating and administering database systems. Database Design and Normalization. Students study database design principles and normalization techniques to eliminate redundancy and improve data integrity. The topic emphasizes the development of efficient relational database schemas. Data types in SQL. Managing a database and tables in SQL Students examine SQL data types and learn how to create, modify, and manage databases and tables. The topic develops practical database administration skills. INSERT and SELECT operators. Students learn to insert records into database tables and retrieve data using SQL queries. The topic develops essential data manipulation and retrieval skills. DELETE and UPDATE commands in SQL. Students study SQL commands for modifying and removing data from database tables. The topic emphasizes safe and efficient data management practices.

The WHERE clause in SQL. The AND, OR, and NOT clauses in SQL. Students learn to filter database records using logical operators within SQL queries. The topic develops skills for constructing precise and efficient search conditions.

Filtering and sorting data in SQL. Students study methods for filtering, ordering, and organizing query results. The topic develops techniques for efficient data retrieval and presentation.

Aggregate functions in SQL and working with them Students examine aggregate functions such as COUNT, SUM, AVG, MIN, and MAX. The topic develops skills for summarizing and analyzing database information.

LIKE, IN, BETWEEN commands in SQL. Students study SQL operators for pattern matching, range searching, and value comparison. The topic develops flexible query-writing techniques for database applications.

Relationships in SQL. Working with the JOIN command Students learn how to establish relationships between database tables using JOIN operations. The topic develops skills for retrieving related information from multiple tables.

Working with grouping functions in SQL. Students study GROUP BY and HAVING clauses for grouping and filtering aggregated data. The topic develops techniques for generating meaningful statistical summaries.

Indexes and Performance. Students examine database indexing techniques and their impact on query performance. The topic introduces optimization strategies for improving database efficiency.

Transaction Management. Students study database transactions and the principles of maintaining data consistency and reliability. The topic emphasizes transaction control, commit, rollback, and concurrency management.

In SQL, the ALL, SOME, and ANY commands and their usage.

Subqueries. Students learn advanced SQL operators and subquery techniques for solving complex database queries. The topic develops analytical skills for advanced data retrieval.

The CASE and NULL functions in SQL. Students study conditional expressions and NULL value handling in SQL. The topic develops methods for writing flexible and robust database queries.

Working with comments in SQL. Operators in SQL. Single-line comments. Multi-line comments. Students learn to document SQL code using comments and apply arithmetic,

comparison, and logical operators. The topic promotes readable, maintainable, and efficient SQL programming practices.

Procedures and Constraints in SQL. Students study stored procedures and database constraints for maintaining data integrity and automating database operations. The topic develops practical database programming skills.

Query Processing. Query Optimization. Students examine how SQL queries are processed and optimized by database management systems. The topic introduces techniques for improving query execution efficiency.

Intermediate SQL. Advanced SQL. Students explore intermediate and advanced SQL concepts, including complex queries, views, stored procedures, and advanced data manipulation techniques. The topic prepares students for professional database development.

Final Project: Database Project. Students design, implement, and document a complete relational database application that integrates database design, SQL programming, optimization, and transaction management. The project develops practical problem-solving, database development, and technical documentation skills.

Practices

The Relational Database Model: Introduction
Students explore the fundamental concepts of the relational database model and its components. The practical session develops an understanding of tables, relationships, and relational database structures.

Writing simple SQL queries. Students write basic SQL queries to retrieve information from database tables. The practical session develops fundamental skills in querying relational databases.

Installing a local server and creating a database. Students install a local database server and create a relational database environment. The practical session develops practical skills in database setup and configuration.

Normalization practice. Students apply normalization techniques to improve database design and eliminate data redundancy. The practical session strengthens database modeling and design skills.

Creating a table with data types in SQL. Students create database tables and define appropriate SQL data types for different attributes. The practical session develops practical database schema design skills.

Inserting and selecting data in SQL. Students insert records into database tables and retrieve information using SQL queries. The practical session develops essential data manipulation and retrieval techniques.

Updating and deleting data in SQL. Students modify and remove database records using SQL commands. The practical session emphasizes maintaining data accuracy and integrity.

Queries with conditional filters in SQL. Students construct SQL queries using conditional filtering with logical operators. The practical session develops precise data selection and query-writing skills.

Sorting and limiting data in SQL. Students organize query results using sorting and limiting clauses. The practical session develops efficient techniques for presenting and retrieving data.

Statistical calculations in SQL. Students perform statistical calculations using SQL aggregate functions. The practical session develops skills in summarizing and analyzing database information.

Text and interval filters in SQL. Students use text matching and range filtering operators to retrieve specific records. The practical session develops advanced data filtering techniques.

Joins between tables in SQL. Students retrieve related data from multiple tables using different types of JOIN operations. The practical session develops practical skills in relational data retrieval.

Grouping and conditional grouping in SQL. Students group data and apply conditions to aggregated results using SQL grouping functions. The practical session develops techniques for producing meaningful reports.

Optimization with indexes. Students create and evaluate database indexes to improve query performance. The practical session develops skills in database optimization and performance tuning.

Transaction management. Students perform transaction control operations to ensure database consistency and reliability. The practical session develops practical experience with COMMIT, ROLLBACK, and transaction handling.

Subqueries and comparisons in SQL. Students write SQL queries using subqueries and comparison operators to solve complex problems. The practical session develops advanced query construction skills.

Working with CASE and NULL functions in SQL. Students apply CASE expressions and NULL-handling functions to create

flexible SQL queries. The practical session develops techniques for conditional data processing.

Writing high-quality code with comments in SQL. Students document SQL scripts using comments and apply coding best practices. The practical session promotes readable, maintainable, and professional SQL programming.

Procedures and constraints in SQL. Students create stored procedures and implement database constraints to ensure data integrity. The practical session develops practical database programming skills.

Query optimization in SQL. Students analyze and optimize SQL queries to improve execution efficiency. The practical session develops performance analysis and optimization techniques.

Modern SQL. Students apply advanced SQL features and modern database techniques to solve practical problems. The practical session develops professional database programming skills.

Mini-project: Small database system. Students design and implement a small relational database system using SQL. The practical session integrates database design, programming, and data management skills.

Final Project: Complete Database System. Students develop a complete relational database application that integrates database design, SQL programming, optimization, and documentation. The practical session demonstrates the ability to build a functional and efficient database system from start to finish.

Student Independent Work

University infrastructure. Students design and implement a database for managing university infrastructure, including buildings, classrooms, laboratories, staff, and facilities. The project develops skills in relational database design, normalization, and SQL implementation.

Notary system. Students develop a database system for managing clients, legal documents, contracts, and notarial services. The project emphasizes data integrity, security, and transaction management.

Supermarket database. Students design a database for managing products, customers, suppliers, sales, and inventory. The project develops practical skills in retail database management and reporting.

Online store database. Students implement a database for an e-commerce platform, including products, orders, customers, and payments. The project focuses on relational modeling and efficient transaction processing.

Restaurant database. Students create a database system for managing menus, reservations, customers, orders, and employees. The project develops practical skills in hospitality database applications.

Pharmacy database. Students design a database for managing medicines, prescriptions, suppliers, customers, and inventory. The project emphasizes data consistency and healthcare information management.

Hospital database. Students develop a database system for managing patients, doctors, appointments, medical records, and treatments. The project focuses on secure and efficient healthcare data management.

Air flight database. Students implement a database for managing flights, passengers, airports, aircraft, bookings, and schedules. The project develops skills in designing complex transportation database systems.

Hotel database. Students design a database for managing rooms, reservations, guests, employees, and billing information. The project develops practical experience in hospitality management systems.

Bus station database. Students create a database system for managing routes, buses, schedules, tickets, passengers, and drivers. The project emphasizes efficient transportation data management.

Travel Agency. Students develop a database for managing travel packages, customers, bookings, hotels, flights, and tour guides. The project integrates multiple related entities into a comprehensive information system.

Bank database. Students design a banking database for managing customers, accounts, transactions, loans, and financial services. The project emphasizes security, consistency, and transaction management.

World Cup database. Students implement a database for managing teams, players, matches, stadiums, referees, and tournament statistics. The project develops skills in sports data management and relational database design.

Movie and TV show database. Students create a database for managing movies, television shows, actors, directors, genres,

	ratings, and streaming information. The project focuses on efficient organization and retrieval of multimedia data. Automotive repair shop database. Students design a database for managing customers, vehicles, repair services, technicians, spare parts, and invoices. The project develops practical skills in service-oriented database design and management.
Teaching Methods	The module is delivered through interactive lectures, inquiry-based and problem-based learning, hands-on laboratory sessions, database design workshops, seminars, group discussions, and project-based activities. Students design, implement, and manage relational databases using modern database management systems, develop SQL programming and database modeling skills, and apply normalization, transaction management, and data security principles to real-world applications. Independent learning and digital educational resources further promote critical thinking, technical communication, and the application of contemporary database technologies in information systems development.
Prerequisites	Fundamentals of Programming (DASB113) Linear Algebra (CHA1306) Algorithms and Data Structures (ABSB210) Algorithm Design (ALL1506) Database (BBZB209)
Exams and Assessment	Type of examination (written, oral, project) and its weight Student performance is assessed through continuous assessment (20%), midterm assessment (30%), and a final written examination (50%). The continuous assessment (20%) evaluates students' participation in lectures and practical classes, laboratory activities, and completion of independent study assignments, including presentations, technical reports, SQL programming exercises, database design tasks, query optimization, and mini-projects related to relational databases, SQL, normalization, transaction management, and database systems. The midterm assessment (30%) consists of a written test and the evaluation of independent study projects, such as reports, presentations, analytical papers, and practical assignments covering SQL programming, database design, normalization, data analysis, query optimization, and transaction management. The final assessment (50%) is a written examination consisting of theoretical and practical questions designed to assess students' understanding of database concepts, SQL programming, relational database design, data management, query

	optimization, and their ability to develop and implement database solutions. Successful completion of all independent study assignments is required as an essential component of the module assessment.
Recommended Literature	1.Sh. Nazirov, A. Ne'matov, R. Qobulov, N. Mardonova "DATABASE". Textbook. Tashkent – 2007. Main Editorial Office of SHARQ Publishing-Printing Joint-Stock Company. 2007, 200 pp. 2.M. K. Djurayev, "DATABASE". Textbook. Termiz-2024. "Surhon Ilm Nashri Publishing House". 2024, 436 p. 3.S. B. Atajonova, B. P. Abdullayev, "CREATING A DATABASE SYSTEM. " Textbook. Andijan–2024. "Omadbek Print Number One" LLC Publishing House. 2024, 266 pp. 4.F. X. Xazratov, A. A. Ergashev, X. U. Xayatov "Data Base Management Systems" Part 1. Textbook. Bukhara-2024. "KAMOLOT" Publishing House. 2024, 184 pp. 5.M. X. Hakimov, S. M. Gaynazarov "Database Management Systems" Part 1. Textbook. Tashkent-2013. "Fan and Technology" Publishing House. 2013, 648 p. 6.M. X. Hakimov, S. M. Gaynazarov "Database Management Systems" Part 2. Textbook. Tashkent-2015. "Fan and Technology" Publishing House. 2015, 648 pp. 7.Allen Taylor, "SQL for Dummies," 9th edition. Moscow – Saint Petersburg – 2020. "Pervaya Obraztsovaya Tipografiya," 546 pp. 8.Sh. Nazirov, A. Ne'matov, R. Qobulov "Advanced Course in Database Programming. " Textbook. Tashkent – 2007. "SHARQ" Publishing-Printing Joint-Stock Company, Main Editorial Office. 2007, 136 p. 9.R. V. Qobulov, B. B. Akbaraliyev, J. T. Usmonov, O. B. Ro'zibayev. "Database. " Textbook. Tashkent – "Aloqachi" – 2021, 142 p. 10. J. D. Saidov, "Database. " Textbook. Guliston – 2022. "Sirdaryo Print" LLC, 165 pp. 11. B. T. Ergashev, K. Z. Abidov. "Database Management Systems". Textbook. Tashkent, 2020. ILM-ZIYO Publishing House, 257 pp.
Responsible Lecturer	Abdullayev Bakhtiyor Panjiyevich Senior Lecturer
Field	Description
Module Title	CYBERSECURITY FUNDAMENTALS
Module Code	KIA1406

Credit Points (ECTS)	6 Credits
Workload	Total workload: 180 hours Contact hours: 90 hours (46 hours Lecture, 44 hours Practical lessons) Self-Assignments: 90 hours
Frequency/Duration	IV semester
Module objective	"Cybersecurity Fundamentals" Course 60610400– Designed for students in the Software Engineering program, it aims to develop the knowledge, skills, and competencies needed to address cybersecurity issues related to information systems and information resources in their professional activities.
Module Learning Outcomes	Knowledge LO1. Have an understanding of the basic concepts of cybersecurity, the goals and objectives of information security, and the principles of information confidentiality, integrity, and availability; LO2. Knowledge of methods for identifying and assessing cybersecurity threats, vulnerabilities, attacks, and risks; LO3. Understanding the basic concepts of cryptography, symmetric and asymmetric encryption algorithms, and electronic digital signature technologies; LO4. Knowledge of network security, network protocols, firewalls, IDS/IPS systems, and VPN technologies; LO5. Knowledge of the principles of ensuring information security in operating systems, databases, web applications, and cloud technologies; Skills LO6. Have the skill to identify and analyze threats and vulnerabilities in information systems and networks; LO7. Develop skills in using cryptographic tools, encrypting data, and applying electronic digital signatures; LO8. Acquire skills in configuring and managing security parameters on operating systems and network devices; LO9. Have skills to solve practical problems in risk assessment, developing security policies, and ensuring information security; Qualifications LO10. Be proficient in using cybersecurity tools and technologies; LO11. Developing the ability to detect, log, and respond to security incidents in information systems and networks;

	LO12. Be competent in developing and implementing practical projects for protecting information resources, software tools, and networks;
Content	Lectures Basic Concepts of Cybersecurity. Students study the fundamental principles, objectives, and terminology of cybersecurity. The topic provides the foundation for protecting information systems, networks, and digital assets. Risk Management. Students examine methods for identifying, assessing, and mitigating cybersecurity risks. The topic develops strategies for reducing threats and improving organizational security. Cybercrime, Cyberlaw, Cybernetics, and Forensics. Students study cybercrime, legal and ethical issues, digital forensics, and the role of cybernetics in information security. The topic develops an understanding of cybersecurity regulations and investigative techniques. Cybersecurity Architecture, Strategy, and Policy. Students examine cybersecurity architectures, organizational security strategies, and policy development. The topic emphasizes designing secure systems and implementing effective governance practices. Basic Concepts of Cryptography. Students study the principles of cryptography, encryption, decryption, and secure communication. The topic provides the theoretical foundation for protecting sensitive information. Symmetric Cryptographic Algorithms. Students examine symmetric encryption algorithms and their applications in secure data transmission. The topic develops an understanding of key management and encryption techniques. Public-key cryptosystems. Students study public-key cryptography, key exchange mechanisms, and digital encryption techniques. The topic introduces secure communication methods using asymmetric cryptographic algorithms. Methods for ensuring data integrity. Students examine cryptographic techniques for verifying data integrity and authenticity. The topic introduces hashing algorithms, message authentication codes, and digital signatures. Cryptographic Applications. Students study practical applications of cryptography in secure communication, authentication, and data protection. The topic emphasizes real-world cybersecurity implementations.

Identification and authentication tools. Students examine methods and technologies used for user identification and authentication. The topic introduces passwords, biometrics, multi-factor authentication, and identity management systems.

Logical control of data access. Students study logical access control mechanisms for protecting information systems. The topic develops an understanding of authorization models and access management techniques.

Multi-level security models. Students examine multi-level security models used to protect classified and sensitive information. The topic introduces mandatory, discretionary, and role-based access control concepts.

Physical Security of Data. Students study physical security measures for protecting computing infrastructure and information assets. The topic emphasizes environmental, hardware, and facility security controls.

Computer Networks and Network Security Issues
Students examine computer network architectures, common network threats, and security vulnerabilities. The topic develops an understanding of secure network design and protection mechanisms.

Tools for Ensuring Network Security. Students study security tools used to monitor, detect, and prevent network attacks. The topic introduces firewalls, intrusion detection systems, VPNs, and network monitoring technologies.

Wireless Network Security. Students examine security mechanisms for wireless communication networks. The topic develops techniques for protecting Wi-Fi networks against unauthorized access and attacks.

Cloud computing systems and IoT (Internet of Things) security. Students study security challenges and protection mechanisms for cloud computing and Internet of Things environments. The topic emphasizes secure deployment, data privacy, and access control.

The concept of usability. Students examine the relationship between usability and cybersecurity. The topic emphasizes designing secure systems that remain efficient and user-friendly.

Security in Software Tools. Students study secure software development practices and methods for protecting software applications. The topic introduces security testing, vulnerability assessment, and secure coding principles.

Protection from Malicious Software. Students examine different types of malware and techniques for detecting,

preventing, and removing malicious software. The topic develops practical knowledge of endpoint protection strategies.

Protecting log entries and defense against social engineering. Students study methods for securing system logs and defending against social engineering attacks. The topic develops awareness of human-related security threats and incident monitoring techniques.

Artificial Intelligence and Cybersecurity. Students examine the applications of artificial intelligence in cybersecurity, including threat detection, intrusion prevention, and automated response. The topic explores both the opportunities and challenges of AI-driven security.

Modern Cybersecurity Concepts: Zero Trust and Cyber Resilience. Students study modern cybersecurity frameworks, including the Zero Trust security model and cyber resilience strategies. The topic emphasizes continuous verification, adaptive defense, and rapid recovery from cyber incidents.

Practics

Learning Risk Assessment in Cybersecurity. Students perform cybersecurity risk assessments by identifying assets, threats, vulnerabilities, and potential impacts. The practical session develops skills in evaluating and mitigating security risks.

Learning the working procedure of classical encryption algorithms. Students implement and analyze classical encryption algorithms to understand their principles and applications. The practical session develops foundational cryptographic skills.

Study the working procedure of classical encryption algorithms. Students examine the operation of traditional cryptographic algorithms through practical exercises. The practical session strengthens understanding of encryption and decryption processes.

Learn data encryption using the TrueCrypt program. Students encrypt and protect data using TrueCrypt or equivalent disk encryption software. The practical session develops practical skills in data confidentiality and secure storage.

Learn to install and configure a password-based authentication mechanism in the operating system (Windows OS). Students install and configure password-based authentication mechanisms in the Windows operating system. The practical session develops secure authentication and account management skills.

Learn how to carry out an intelligence attack. Students study intelligence-gathering techniques in a controlled cybersecurity laboratory environment. The practical session develops an understanding of information gathering and security assessment methods.

Learning to carry out an intelligence attack. Students perform authorized reconnaissance and intelligence collection using standard cybersecurity tools. The practical session emphasizes ethical security testing and information analysis.

Building network protection using a firewall tool. Students configure firewall rules to protect computer networks from unauthorized access. The practical session develops practical skills in network security and traffic filtering.

Building network protection using the firewall tool. Students implement network protection policies using firewall software and security rules. The practical session develops experience in securing network infrastructure.

Set up a secure Wi-Fi wireless network. Students configure a secure wireless network using modern authentication and encryption protocols. The practical session develops practical skills in wireless network protection.

Learn data recovery using specialized software tools. Students recover deleted or damaged data using professional recovery software. The practical session develops practical knowledge of data restoration and digital storage management.

Installing antivirus protection on personal computers. Students install, configure, and manage antivirus software on personal computers. The practical session develops skills in malware prevention and endpoint protection.

Learn to manage the use of passwords Students implement password management policies and secure credential handling techniques. The practical session develops best practices for authentication security.

Learn to collect data from social networks Students collect publicly available information from social networking platforms using ethical OSINT techniques. The practical session develops open-source intelligence and information analysis skills.

Learning to collect data from social networks Students apply open-source intelligence tools to gather and organize publicly accessible social media information. The practical session emphasizes ethical data collection and cybersecurity investigations.

Learn to analyze and monitor network traffic. Students capture, analyze, and monitor network traffic using network analysis tools. The practical session develops practical skills in detecting abnormal network behavior and security threats.

Learn to test web application security. Students assess web applications for common security vulnerabilities using authorized testing techniques. The practical session develops skills in web security assessment and vulnerability identification.

Learn the fundamentals of digital forensics. Students perform basic digital forensic investigations by collecting, preserving, and analyzing digital evidence. The practical session introduces standard forensic tools and procedures.

Learning to ensure information security in cloud services Students implement security controls for cloud-based systems and services. The practical session develops practical skills in cloud security configuration and data protection.

Learn to install and configure multi-factor authentication (MFA) systems. Students deploy and configure multi-factor authentication mechanisms to strengthen user authentication. The practical session develops practical experience in identity and access management.

Learn to detect and respond to cybersecurity incidents Students identify cybersecurity incidents and perform appropriate response procedures using incident management techniques. The practical session develops practical skills in cybersecurity operations.

Learn to detect cybersecurity threats using artificial intelligence tools. Students use artificial intelligence–based security tools to detect and analyze cybersecurity threats. The practical session develops experience in applying AI techniques to modern cyber defense.

Student Independent Work

Analysis of national and international regulatory and legal documents on cybersecurity. Students examine national and international cybersecurity laws, standards, and regulatory frameworks. The topic develops an understanding of legal compliance, information security policies, and global cybersecurity governance.

Cryptographic methods for information protection. Students study modern cryptographic techniques used to protect information confidentiality, integrity, and authenticity. The topic introduces practical applications of encryption in secure communication systems.

The Enigma cipher machine and its resilience. Students investigate the design, operation, and historical significance of the Enigma cipher machine. The topic explores its cryptographic strengths, weaknesses, and impact on the development of modern cryptography.

The role of antivirus, IDS, IPS, and TE tools in ensuring information usability. Students examine the functions of antivirus software, intrusion detection systems (IDS), intrusion prevention systems (IPS), and traffic evaluation tools in protecting information systems. The topic highlights their contribution to maintaining system availability and security.

Classification of malicious software and protection methods Students study different categories of malicious software and methods for preventing, detecting, and mitigating malware attacks. The topic develops practical knowledge of endpoint security.

Threats to information and communication technology security. Students analyze common threats affecting information and communication technology infrastructures. The topic develops an understanding of vulnerabilities, attack vectors, and security countermeasures.

Types of malicious software. Students examine the characteristics and behavior of viruses, worms, trojans, ransomware, spyware, and other forms of malicious software. The topic emphasizes methods for identifying and mitigating malware threats.

Computer viruses and virus protection methods. Students study the operation of computer viruses and techniques for preventing, detecting, and removing them. The topic develops practical knowledge of antivirus technologies and secure computing practices.

Analysis of foreign countries' digital signature algorithms Students examine digital signature algorithms adopted in different countries and compare their security characteristics. The topic develops an understanding of international cryptographic standards and authentication methods.

Concepts and tasks of identification and authentication Students study the principles of user identification and authentication in information systems. The topic introduces authentication methods and identity management techniques used in modern cybersecurity.

Methodology for detecting unauthorized access to information systems and resources. Students examine

methods for detecting unauthorized access using monitoring, logging, and intrusion detection techniques. The topic develops skills in identifying and responding to security violations.

Protecting information resources in wireless communication systems. Students study security mechanisms for protecting information transmitted through wireless communication networks. The topic introduces encryption protocols, authentication methods, and wireless network security practices.

Protecting information from unauthorized access Students examine access control mechanisms and security policies for preventing unauthorized access to information resources. The topic emphasizes authentication, authorization, and data protection strategies.

Analysis of social engineering attacks. Students investigate different forms of social engineering attacks and the techniques used by attackers to manipulate users. The topic develops awareness and preventive strategies for reducing human-related security risks.

Modern antivirus software tools and their capabilities Students study the architecture, features, and effectiveness of modern antivirus solutions. The topic evaluates malware detection, real-time protection, and endpoint security technologies.

Cybercrime and Cyberlaw. Students examine the nature of cybercrime together with the legal frameworks governing cybersecurity and digital investigations. The topic develops an understanding of cyber offenses, legal responsibilities, and regulatory compliance.

The role of cryptography in protecting information transmitted over a network. Students study how cryptographic techniques secure information during network transmission. The topic emphasizes encryption, key management, and secure communication protocols.

Malicious software and protection against it (for example, using Malwarebytes). Students examine methods for detecting and removing malicious software using modern security tools such as Malwarebytes. The topic develops practical skills in malware analysis and system protection.

Cybercrime and the reasons for its prevalence. Students analyze the technological, economic, and social factors contributing to the growth of cybercrime. The topic develops an understanding of current cyber threats and preventive measures.

	Facial recognition authentication method and its characteristics. Students study facial recognition as a biometric authentication method and evaluate its strengths, limitations, and security implications. The topic explores the application of biometric technologies in modern identity management systems.
Teaching Methods	The module is delivered through interactive lectures, inquiry-based and problem-based learning, hands-on laboratory exercises, cybersecurity simulations, seminars, group discussions, and project-based activities. Students analyze cyber threats, identify system vulnerabilities, and apply fundamental security principles using industry-standard tools and virtual laboratory environments. They develop practical skills in risk assessment, network security, cryptography, access control, and incident response while strengthening analytical and ethical decision-making abilities. Independent learning and digital educational resources further promote critical thinking, technical communication, and the application of cybersecurity best practices to protect modern information systems.
Prerequisites	Programming (DAS1110) Linear Algebra (CHA1306) Algorithms and Data Structures (ABSB210) Algorithm Design (ALL1506) Database (BBZB209) Computer Networks (KOT1306) Operating Systems (OPT1506) Discrete Structures (DIT1306) Web Programming Probability and Statistics (EVS1406)
Exams and Assessment	Type of examination (written, oral, project) and its weight Student performance is assessed through continuous assessment (20%), midterm assessment (30%), and a final written examination (50%). The continuous assessment (20%) evaluates students' participation in lectures and practical classes, laboratory activities, and completion of independent study assignments, including presentations, analytical reports, literature reviews, security analysis, practical exercises, and mini-projects related to cybersecurity, cryptography, network security, digital forensics, risk management, and information protection. The midterm assessment (30%) consists of a written test and the evaluation of independent study projects, such as reports, presentations, analytical papers, and practical assignments covering cybersecurity fundamentals, cryptography,

	authentication, access control, network security, malware protection, and digital forensics. The final assessment (50%) is a written examination consisting of theoretical and practical questions designed to assess students' understanding of cybersecurity principles, cryptographic techniques, network and system security, risk assessment, incident response, and their ability to apply modern cybersecurity technologies in practical environments. Successful completion of all independent study assignments is required as an essential component of the module assessment.
Recommended Literature	1. Robin Sharp. Introduction to Cybersecurity: A Multidisciplinary Challenge. Textbook. - Springer Cham, 2023. – 452 p. 2. S. K. Ganiyev, A. A. Ganiyev, Z. T. Xudoyqulov. Fundamentals of Cybersecurity: A Textbook. - T. : "Nihol print" LLC, 2021. – 224 p. 3. S. K. Ganiyev, Z. T. Khudoykulov, N. B. Nasrullayev. Fundamentals of Cybersecurity: A Textbook. - T. : "Mahalla va oila nashriyoti", 2021. – 240 pp. 4. Tahirov Behzod Nasriddinovich. Fundamentals of Information Security: A Textbook. – Bukhara State University. : "FAN VA TA'LIM" Publishing House, 2022. – 156 pages. 5. Ganiev Salim Karimovich, Karimov Madjit Malikovich, Tashev Komil Akhmatovich. Information Security: Textbook. – Tashkent. : "TATU" Publishing House, 2016. – 366 pages. 6. Tairakawa Kousho. Securing the Digital Frontier: An Introduction to Cybersecurity for Beginners. Independently published, 2023. — 139 p. 7. V. F. Shankin. Information Security of Computer Systems and Networks: A Textbook. - M. : FORUM : INFRA-M, 2019. - 416 pp. 8. Ian Neil. (2018). CompTIA security+ certification guide: master IT security essentials and exam topics for CompTIA security+ SY0-501 certification. Birmingham: Packt Publishing. (online access from PolyU Library) (No. of words required: 10,000 words). 9. Wu, C. (2021). IoT Network Layer Security. In Internet of Things Security (Advances in Computer Science and Technology, pp. 107-123). Singapore: Springer Singapore. (online access from PolyU Library). ISBN: 97816113710 ISBN: 9789811613715, DOI: 10. 1007/978-981-16-1372-2_7.
Responsible Lecturer	Abdullayev Baxtiyor Panjiyevich Senior Lecturer
Field	Description

Module Title	ELECTRONICS AND CIRCUITS 1
Module Code	EVS1306
Credit Points (ECTS)	6 Credits
Workload	Total workload: 180 hours Contact hours: 60 hours (30 hours Lecture, 30 hours Practical lessons) Self-Assignments: 120 hours
Frequency/Duration	III semester
Module objective	the types and characteristics of circuits and electronic devices used in information and communication technologies, the study of the types, characteristics, and technological and schematic features of circuits and electronic devices used in information and communication technologies, their structure, operating mechanisms, and the complex devices created with them, as well as the development of skills for their practical application.
Module Learning Outcomes	Knowledge: LO1. The structure and operating principles of electronic components. LO2. Methods for calculating electrical circuits. LO3. Fundamentals of analog and digital circuits. LO4. Practical application of electronic devices. Skills LO5. Reading and analyzing electrical circuits. LO6. Use of multimeters and measuring instruments. LO7. Assembling and testing electronic circuits. LO8. Modeling circuits in simulation software. Qualifications LO9. Designing electronic devices. LO10. Fault detection and troubleshooting. LO11. Development of modern electronic systems.
Content	Lectures Introduction to the course "Electronics and Circuits. " Course objectives and tasks. Students study the fundamental concepts, objectives, and scope of electronics and circuit analysis. The topic introduces the role of electronic systems in modern engineering and technology. Electrical quantities, methods and instruments for measuring resistance. Students examine basic electrical quantities and learn methods for measuring resistance using standard electrical instruments. The topic develops practical skills in electrical measurement and circuit testing.

DC electrical circuits and their analysis. Students study the principles of direct current (DC) circuits and methods for analyzing electrical networks. The topic develops skills in applying fundamental circuit laws to practical problems.

Calculation of DC electrical circuits. Students learn analytical techniques for calculating voltages, currents, and power in DC electrical circuits. The topic emphasizes the application of Ohm's law and Kirchhoff's laws.

Characteristics of electrical circuits with sinusoidal signals Students examine the behavior of electrical circuits under sinusoidal excitation. The topic introduces alternating current (AC) concepts, impedance, phase relationships, and frequency response.

Mutually Inductive Circuits: Magnetic driving force, magnetically coupled circuits; Kirchhoff's laws for magnetic circuits. Students study magnetic circuits, mutual inductance, magnetic driving force, and magnetically coupled circuits. The topic develops methods for analyzing magnetic systems using Kirchhoff's laws for magnetic circuits.

Physical principles of semiconductor device operation Students examine the physical principles governing semiconductor materials and electronic devices. The topic provides the theoretical foundation for modern electronic components.

Contact Phenomena in Semiconductors. Students study contact phenomena in semiconductor materials, including p–n junctions and metal–semiconductor interfaces. The topic explains the operating principles of semiconductor devices.

Semiconductor Diodes. Students investigate the structure, characteristics, and applications of semiconductor diodes. The topic develops an understanding of rectification, switching, and voltage regulation.

Bipolar transistors. Students study the operating principles, configurations, and characteristics of bipolar junction transistors (BJTs). The topic emphasizes transistor applications in amplification and switching circuits.

Multilayer semiconductor devices. Students examine the structure and operation of multilayer semiconductor devices, including thyristors and related components. The topic introduces their applications in power electronics and control systems.

Field-effect transistors (FET). Students study the structure, operating principles, and characteristics of field-effect transistors.

The topic develops an understanding of FET applications in analog and digital electronic circuits.

Integrated Circuits. Students examine the design, classification, and applications of integrated circuits. The topic introduces the role of integrated circuits in modern electronic systems.

Analog electronics. Students study the principles and components of analog electronic circuits. The topic develops an understanding of signal processing, amplification, and analog circuit design.

Operational amplifiers. Analog signal converters based on operational amplifiers. Students examine the operating principles and applications of operational amplifiers in analog electronic systems. The topic develops skills in designing amplifier circuits and analog signal conditioning circuits.

Practics

Study of diode operation. Students investigate the operating principles and voltage-current characteristics of semiconductor diodes through laboratory experiments. The practical session develops skills in analyzing diode behavior under different operating conditions.

AC Voltage Rectification Using a Bridge Circuit. Students construct and analyze a bridge rectifier circuit for converting alternating current (AC) into direct current (DC). The practical session develops practical skills in power supply and rectification circuits.

Study of the operation of a bipolar transistor. Students examine the operating principles and characteristics of bipolar junction transistors through experimental measurements. The practical session develops an understanding of transistor switching and amplification.

Study of the operation of a field-effect transistor
Students investigate the operating characteristics of field-effect transistors (FETs) using laboratory experiments. The practical session develops practical skills in analyzing FET-based electronic circuits.

Study of Amplifiers Based on Field-Effect Transistors
Students design and analyze amplifier circuits based on field-effect transistors. The practical session develops skills in signal amplification and electronic circuit performance evaluation.

Study of a transistor amplifier. Students construct and test transistor amplifier circuits under different operating conditions. The practical session develops practical knowledge of amplifier gain, biasing, and signal amplification.

Study of the transistor switch. Students investigate the operation of transistors used as electronic switches. The practical session develops practical skills in designing and analyzing digital switching circuits.

Studying the transistor as a functional generator. Students examine the application of transistors in signal generation circuits. The practical session develops an understanding of waveform generation and oscillator design.

Investigation of the characteristics and parameters of the stabilatron. Students investigate the voltage-current characteristics and operating parameters of Zener diodes (stabilitrons). The practical session develops practical skills in voltage stabilization circuits.

Research on the optron. Students study the structure, operation, and applications of optocouplers (optrons) in electronic circuits. The practical session develops an understanding of electrical isolation and optical signal transmission.

Investigating the static V–I characteristics of BJT in the common-emitter (CE) connection scheme. Students measure and analyze the static voltage-current characteristics of a bipolar transistor in the common-emitter configuration. The practical session develops practical skills in transistor parameter evaluation.

Investigation of the static V–I characteristics of BJT in the common-base (CB) connection scheme. Students investigate the static voltage-current characteristics of a bipolar transistor in the common-base configuration. The practical session develops an understanding of transistor behavior in different circuit configurations.

Investigation of MOSFET static characteristics. Students investigate the static characteristics of metal-oxide-semiconductor field-effect transistors (MOSFETs) through laboratory measurements. The practical session develops skills in evaluating transistor performance.

Investigation of the characteristics of an induced metal–dielectric–semiconductor (MOS) channel. Students examine the electrical characteristics of an induced metal–oxide–semiconductor (MOS) channel. The practical session develops an understanding of charge transport and channel formation in semiconductor devices.

Research on the operational amplifier
Students investigate the operating characteristics and practical applications of operational amplifiers through laboratory

experiments. The practical session develops skills in designing and analyzing analog electronic circuits using operational amplifiers.

Student Independent Work

Innovation and project management. Students study the fundamental concepts of innovation and project management and their role in organizational development. The topic introduces project planning, execution, and innovation management principles.

Types of innovation. Students examine different types of innovation, including product, process, organizational, and marketing innovations. The topic develops an understanding of innovation strategies across various industries.

The role of innovations in projects. Students study how innovation contributes to project success and sustainable development. The topic emphasizes integrating innovative solutions into project planning and implementation.

Types of projects and their essence. Students examine different categories of projects and their characteristics across various sectors. The topic develops an understanding of project objectives, life cycles, and management approaches.

Foreign projects and their impact on the national economy
Students analyze international project experiences and their contribution to national economic development. The topic examines technology transfer, investment, and economic growth.

International projects, the process of formalizing documents related to international projects. Students study the structure and documentation requirements of international projects. The topic develops skills in preparing and managing project proposals, agreements, and related documents.

State support and management of projects. Students examine government policies and mechanisms for supporting and managing projects. The topic introduces public funding, regulatory frameworks, and project governance.

Stages of project formation. Students study the main stages of project development, from idea generation to project completion. The topic emphasizes planning, implementation, monitoring, and evaluation processes.

Attracting investments and their effectiveness. Students examine methods for attracting investment and evaluating investment effectiveness. The topic introduces financial analysis techniques for project decision-making.

	Investment attractiveness of projects. Students study the factors influencing the investment attractiveness of projects. The topic develops methods for assessing project feasibility and investment potential. Attraction of investments into projects. Students examine strategies and financial mechanisms for securing investments in projects. The topic emphasizes investor relations, funding sources, and financial planning. Effective and ineffective investments. Students analyze the characteristics of effective and ineffective investments using financial and economic indicators. The topic develops skills for evaluating investment performance. Innovation implementation in projects and identification of risk in investments. Students study methods for implementing innovations in projects while identifying and managing investment risks. The topic emphasizes risk assessment and innovation management strategies. The scale of income fluctuations and the level of risk in investments. Students examine the relationship between investment returns and associated risks. The topic introduces methods for analyzing income variability and investment risk management. Commercialization of innovations. Students study the process of transforming innovative ideas into marketable products and services. The topic develops an understanding of technology transfer, intellectual property, and commercialization strategies.
Teaching Methods	The module is delivered through interactive lectures, inquiry-based and problem-based learning, hands-on laboratory experiments, circuit design and simulation activities, seminars, group discussions, and project-based assignments. Students analyze and design basic electronic circuits, perform experimental measurements using laboratory instruments, and develop practical skills in circuit analysis, electronic components, and analog electronics through simulation software and hardware implementation. Independent learning and digital educational resources further promote critical thinking, technical communication, and the application of fundamental electronic principles to real-world engineering problems.
Prerequisites	Algebra, Probability Theory, Mathematical Logic (DMMB111) Basic knowledge of programming languages such as Python, Java, or C++ (DASB113) English (XJTB106)

Exams and Assessment	Type of examination (written, oral, project) and its weight Student performance is assessed through continuous assessment (20%), midterm assessment (30%), and a final written examination (50%). The continuous assessment (20%) evaluates students' participation in lectures and practical classes, laboratory work, and completion of independent study assignments, including presentations, technical reports, literature reviews, circuit analysis, simulation exercises, and mini-projects related to electronic components, analog and digital circuits, semiconductor devices, and operational amplifiers. The midterm assessment (30%) consists of a written test and the evaluation of independent study projects, such as reports, presentations, analytical papers, and practical assignments covering electrical circuits, semiconductor devices, analog and digital electronics, circuit analysis, and electronic system design. The final assessment (50%) is a written examination consisting of theoretical and practical questions designed to assess students' understanding of electronics, circuit analysis, semiconductor devices, analog and digital circuits, electronic measurements, and their ability to apply electronic design principles in practical applications. Successful completion of all independent study assignments is required as an essential component of the module assessment.
Recommended Literature	1.A. A. Tulyaganov, S. S. Parsiev, V. A. Tulyaganova, U. M. Abdullayev. Theory of Electrical Circuits. (textbook). T. : Aloqachi, 2018, 144 pages. 2.X. K. Aripov, A. M. Abdullayev, N. B. Alimova, J. T. Maxsudov, A. A. Tulyaganov, Sh. T. Toshmatov. Electronics and Schematic Design (textbook). Tashkent: "Aloqachi", 2017, 376 p. 3.Aripov X. K. , Abdullaev A. M. , Alimova N. B. , Bustanov X. X. , Toshmatov Sh. T. "Schemotechnika" (textbook), Tashkent. "Tafakkur bo'stoni", 2013, 447 p. 4.V. T. Yerenko, A. A. Rabochiy, A. P. Fysun, I. I. Nevrov, A. V. Tyutyakin, A. E. Georgievsky. Fundamentals of Electrical Engineering and Electronics. Textbook. Orel, 2012. 5. X. K. Aripov, A. M. Abdullayev, N. B. Alimova, X. X. Bustanov, Sh. T. Toshmatov. Design of Digital Logic Devices (textbook). Tashkent: "Aloqachi," 2017, 396 pp. 6.
Responsible Lecturer	Amirkulov Chori Jumayevich teacher
Field	Description
Module Title	DISCRETE STRUCTURES

Module Code	DIT1306
Credit Points (ECTS)	6 Credits
Workload	Total workload: 180 hours Contact hours: 90 hours (46 hours Lecture, 44 hours Practical lessons) Self-Assignments: 90 hours
Frequency/Duration	III semester
Module objective	Discrete Structures, 60610400 – Software Engineering, 60610300 – Computer Engineering, intended for students in the educational program, is designed to provide students with theoretical knowledge of discrete mathematical objects, logical operations, of discrete structures, problem modeling, algorithm analysis, and the mathematical foundations of software solutions. Within the course, students acquire the competencies necessary for modeling problems using discrete structures, analyzing algorithms, building the mathematical foundations for software solutions, and designing modern software systems.
Module Learning Outcomes	Knowledge LO1. Have an understanding of the basic concepts of discrete mathematics, set theory, logical operations, and mathematical reasoning; LO2. Knowledge of relations, functions, equivalence and ordering relations, and their properties; LO3. Understanding the elements of combinatorics, counting methods, recurrence relations, and mathematical induction; LO4. Knowledge of the basic concepts of graph theory, types of graphs, trees, and paths, and their practical applications; LO5. Knowledge of the mathematical foundations of algorithms, Boolean algebra, and the role and importance of discrete structures in software engineering; Skills LO6. Have the ability to solve problems related to sets, relations, and functions, and to apply them to practical situations; LO7. Develop skills in constructing, simplifying, and using Boolean algebra methods; LO8. Acquire skills in solving combinatorial problems, mathematical induction, and using recurrence relations; LO9. Be able to model, analyze, and develop algorithmic solutions to problems based on graphs and trees; Qualification LO10. Be proficient in creating mathematical models of programming problems using discrete structures methods;

	LO11. Develop the ability to use discrete mathematical methods to analyze algorithms and evaluate their efficiency; LO12. Be proficient in applying discrete structures, graph theory, and logical modeling tools to solve practical problems in software engineering and information technology.
Content	Lectures Sets and operations on them. Students study the basic concepts of sets and fundamental set operations. The topic provides the foundation for discrete mathematics. Relationships. Binary relationships. Students examine relations and binary relations between sets. The topic introduces their properties and applications. Basic elements of combinatorics. Basic combinations. Students study counting principles and basic combinatorial methods. The topic develops problem-solving skills in discrete mathematics. Repetitive Combinations in Combinatorics. Students examine combinations with repetition and their applications. The topic develops counting techniques for practical problems. Logic. Operations on Logic. The Concept of a Formula Students study propositional logic, logical operations, and logical formulas. The topic introduces formal reasoning methods. Equivalence of formulas. Basic equivalences of the logic algebra. Students examine logical equivalence and basic Boolean algebra identities. The topic develops logical simplification techniques. Normal forms. Complete disjunctive and conjunctive normal forms. Students study normal forms of logical expressions. The topic develops methods for representing logical formulas in standard form. Elementary Boolean functions. Methods of their definition. Important and spurious variables. Students examine Boolean functions and methods for defining them. The topic introduces essential and redundant variables. Binary functions. Mapping a Boolean function onto the Zhegalkin polynomial. Monotonic functions. Students study binary Boolean functions, Zhegalkin polynomials, and monotonic functions. The topic develops methods for Boolean function analysis. Functional Closed Classes. Post Theorem. Students examine functional completeness and Post's theorem. The topic introduces the classification of Boolean function classes.

Axioms of the calculus of reasoning. The deduction theorem. Gödel's completeness theorem . Students study formal proof systems, the deduction theorem, and Gödel's completeness theorem. The topic develops logical reasoning skills.

The concept of a predicate. Logical operations on predicates. The quantifiers of universality and existence Students study predicate logic, logical operations, and quantifiers. The topic extends formal logic to mathematical reasoning.

Formulas of predicate algebra. Equivalence of predicate algebra formulas . Students examine predicate logic formulas and their equivalence. The topic develops techniques for logical transformation.

Normal canonical forms of formulas. Students study canonical representations of logical formulas. The topic develops methods for formula simplification.

Applications of Predicate Logic. Students examine practical applications of predicate logic in mathematics and computer science. The topic emphasizes formal reasoning and knowledge representation.

Terms and formulas. The deduction theorem. Students study logical terms, formulas, and the deduction theorem. The topic develops formal proof techniques.

Functional elements. Relay contact schemes. Students examine logical functional elements and relay contact circuits. The topic introduces the relationship between logic and digital systems.

Graphs and their representations. Types of graphs. Students study graph structures, representations, and classifications. The topic provides the foundation for graph theory.

Connectivity of graphs. Path, chain, cycle. Eulerian and Hamiltonian graphs. Students examine graph connectivity and special graph types. The topic develops methods for analyzing graph structures.

Graph metrics. Trees, networks. Students study graph metrics, trees, and network models. The topic introduces applications of graph theory in computing.

Algorithms. Recursive and recursively enumerable sets. Students examine algorithms and recursive sets. The topic introduces computability concepts in discrete mathematics.

Partially recursive and total recursive functions. Students study recursive function theory and computability. The topic develops an understanding of recursive algorithms.

Turing machine. Algorithmically unsolvable problems
Students study Turing machines and the limits of computation. The topic introduces undecidable problems and computability theory.

Practics

Operations on sets. Students perform fundamental operations on sets, including union, intersection, difference, and complement. The practical session develops skills in solving set-theoretic problems.

Relationships. Binary relationships. Students analyze binary relations and their properties through practical examples. The practical session develops skills in representing and interpreting relationships between sets.

Basic elements of combinatorics. Basic combinations. Students solve counting problems using fundamental combinatorial principles. The practical session develops analytical and problem-solving skills.

Repetitive combinations in combinatorics. Students solve problems involving combinations with repetition. The practical session develops techniques for counting arrangements in practical applications.

Logic. Operations on logics. The concept of a formula
Students construct and evaluate logical expressions using basic logical operations. The practical session develops formal reasoning skills.

Equivalences of formulas. Basic equivalences of the algebra of logic. Students simplify logical expressions using equivalence laws. The practical session develops techniques for transforming logical formulas.

Normal forms. Complete conjunctive normal forms
Students convert logical formulas into complete conjunctive normal form (CCNF). The practical session develops skills in logical expression standardization.

Complete disjunctive normal forms. Students transform logical expressions into complete disjunctive normal form (CDNF). The practical session develops methods for Boolean function representation.

Elementary Boolean functions. Methods of their definition. Important and spurious variables. Students construct Boolean functions and identify essential and redundant variables. The practical session develops Boolean function analysis skills.

Binary functions. Students analyze and construct binary Boolean functions using truth tables. The practical session develops practical skills in Boolean logic.

Extending this function to the Zhegalkin polynomial.

Monotonic functions. Students represent Boolean functions using Zhegalkin polynomials and analyze monotonic functions. The practical session develops algebraic methods for Boolean functions.

Functional closed classes: Post's theorem. Students classify Boolean functions using Post's theorem. The practical session develops an understanding of functional completeness.

Axioms of the theory of considerations. Deductive theorem Students apply formal axioms and the deduction theorem to logical proofs. The practical session develops mathematical reasoning skills.

The concept of a predicate. Logical operations on predicates. The quantifiers of universality and existence Students construct predicate logic expressions using universal and existential quantifiers. The practical session develops formal logical analysis skills.

Formulas of predicate algebra. Equivalence of predicate algebra formulas. Students transform and simplify predicate logic formulas using equivalence rules. The practical session develops symbolic reasoning techniques.

Normal canonical forms of formulas. Applications of predicate logic. Students convert predicate logic formulas into canonical forms and solve practical logic problems. The practical session develops applications of formal logic.

Functional elements. Relay-contact circuits. Students design and analyze relay-contact circuits based on Boolean logic. The practical session develops digital logic implementation skills.

Graphs and their representations. Types of graphs. Students represent graphs using matrices and lists and classify different graph types. The practical session develops graph modeling skills.

Connectivity of graphs. Path, chain, cycle. Eulerian and Hamiltonian graphs. Students solve problems involving graph connectivity, paths, cycles, and special graph types. The practical session develops graph analysis techniques.

Graph metrics. Trees, networks. Students calculate graph metrics and analyze trees and network structures. The practical session develops practical graph theory skills.

Algorithms. Recursive and recursively enumerable sets. Partially recursive and total recursive functions
 Students solve problems involving recursive algorithms and recursive functions. The practical session develops an understanding of computability concepts.

Turing machine. Implementing an algorithm on a Turing machine. Students design and simulate algorithms using a Turing machine model. The practical session develops practical skills in algorithm implementation and computability analysis.

Student Independent Work.

Rational and irrational numbers, real numbers, properties
 Students study the properties of rational, irrational, and real numbers. The topic provides the foundation for mathematical reasoning.

Equivalence relations. Operations on relations. Students examine equivalence relations and operations on relations. The topic develops skills in relation analysis.

Multinomial formula in combinatorics. Students study the multinomial theorem and its applications in combinatorial problems. The topic develops counting techniques.

Fibonacci numbers. Catalan numbers. Students examine Fibonacci and Catalan sequences and their combinatorial applications. The topic introduces important mathematical sequences.

Types of formulas, executable and non-executable formulas
 Students study different types of logical formulas and their properties. The topic develops formal reasoning skills.

Basic identities of the algebra of logic. Students examine the fundamental identities of Boolean algebra. The topic develops techniques for simplifying logical expressions.

Truth and falsity tables for propositional algebra formulas
 Students construct truth tables to evaluate propositional logic formulas. The topic develops logical analysis skills.

Boolean algebra and its axioms. Students study the axioms and fundamental principles of Boolean algebra. The topic provides the basis for digital logic.

Important closed classes and their lemmas
 Students examine important closed classes of Boolean functions and their properties. The topic introduces key theoretical concepts.

Applications of the Post theorem. Students study practical applications of Post's theorem in Boolean function analysis. The topic develops an understanding of functional completeness.

	Theory, completeness of the theory, consistency, inconsistency. Students examine the concepts of completeness, consistency, and inconsistency in formal theories. The topic develops logical reasoning skills. Unary and n-ary predicates. Students study unary and n-ary predicates and their applications. The topic extends the concepts of predicate logic. Equivalent formulas in predicate algebra, basic equivalences. Students examine equivalence rules for predicate logic formulas. The topic develops methods for logical transformation. Application of predicate logic to mathematics. Students study the use of predicate logic in mathematical proofs and reasoning. The topic develops formal problem-solving skills. Mathematical theories in algebra, geometry, and mathematical analysis. Students examine the role of formal logic in algebra, geometry, and mathematical analysis. The topic highlights applications of mathematical theories. Multi-tacial schemes. Students study multi-stage logical and switching schemes. The topic introduces their applications in digital systems. Operations on graphs. Students examine basic graph operations and transformations. The topic develops graph manipulation skills. Cyclic graphs, Eulerian and Hamiltonian graphs. Students study cyclic, Eulerian, and Hamiltonian graphs. The topic develops techniques for graph analysis. König and Berge graphs. Students examine the properties of König and Berge graphs. The topic introduces advanced concepts in graph theory. Maximum flow problem. Students study algorithms for solving the maximum flow problem in networks. The topic develops optimization and network analysis skills.
Teaching Methods	The module is delivered through interactive lectures, inquiry-based and problem-based learning, guided problem-solving sessions, computer-assisted exercises, seminars, group discussions, and project-based activities. Students explore discrete mathematical structures, including logic, sets, relations, functions, graphs, trees, combinatorics, and discrete probability, while developing analytical reasoning, mathematical proof techniques, and algorithmic thinking. Independent learning and digital educational resources further promote critical thinking, mathematical communication, and the application of discrete

	structures to computer science, software engineering, and information technology.
Prerequisites	Programming (DAS1110) Linear Algebra (CHA1306) Discrete Structures (DIT1306) Probability and Statistics (EVS1406)
Exams and Assessment	Type of examination (written, oral, project) and its weight Student performance is assessed through continuous assessment (20%), midterm assessment (30%), and a final written examination (50%). The continuous assessment (20%) evaluates students' participation in lectures and practical classes, problem-solving activities, and completion of independent study assignments, including presentations, analytical reports, literature reviews, logical proofs, graph analysis, algorithm modeling, and mini-projects related to sets, Boolean algebra, predicate logic, graph theory, and computational theory. The midterm assessment (30%) consists of a written test and the evaluation of independent study projects, such as reports, presentations, analytical papers, and practical assignments covering discrete mathematics, combinatorics, Boolean algebra, predicate logic, graph theory, algorithms, and Turing machines. The final assessment (50%) is a written examination consisting of theoretical and practical questions designed to assess students' understanding of discrete structures, logical reasoning, graph theory, algorithm analysis, computational theory, and their ability to apply discrete mathematical methods to software engineering problems. Successful completion of all independent study assignments is required as an essential component of the module assessment.
Recommended Literature	1. Du D. -Z. , Ko K. -I. Theory of computational complexity. – John Wiley & Sons, 2000. 2. K. Zhetpisov. Mathematical Logic and Discrete Mathematics. – Almaty, 2011. 3. Du D. -Z. , Ko K. -I. Theory of computational complexity. – John Wiley & Sons, 2000¹⁰. 4. Sipser M. Introduction to the theory of computation. – Thomson Course Technology, 2nd ed. , 2006¹¹. 5. Lavrov, Igor Andreevich. Mathematical Logic: A Textbook for Students of Higher Education Institutions Studying Technical and Natural Sciences / I. A. Lavrov; ed. by L. L. Maksimova. – Moscow: Akademiya, 2006. 39 p. , [1] p. : ill. ; 22 cm.

	(University Textbook. Applied Mathematics and Informatics). ISBN 5-7695-2735-8¹². 6. Mendelson, E. Introduction to Mathematical Logic / E. Mendelson; ed. by S. I. Allen; trans. from English by F. A. Kabakova. – Moscow: Nauka, 1971 - 320 pp. : ill. ; Same [Electronic resource]. - URL: http://biblioclub.ru/index.php?page=book&id=458257¹³. 7. D. Sh. Djurayeva. , Discrete Mathematics and Mathematical Logic. – Termiz 2023. 8. H. T. To'rayev, I. Azizov. Mathematical Logic and Discrete Mathematics. Volumes 1 and 2. Tashkent. 2011. 9. N. Kh. Kasymov, Dadajanov R. N. , F. N. Ibragimov, Fundamentals of Discrete Mathematics and Mathematical Logic, Tashkent 2019. 115 pages. 10. Turaev H. , Azizov I. , Otaqulov S. , Combinatorics and Graph Theory, Tashkent, 2009. 11. Yablonsky S. V. , Introduction to Discrete Mathematics. – M. : Vyshaya Shkola, 2003. 12. Igoshin V. I. , Mathematical Logic and Algorithm Theory. – M. : Academy, 2008. 13. Yunusov A. S. Elements of Mathematical Logic and Algorithm Theory, T. , 2008. 14. Xudoyberdiyev A. X. , Combinatorics and Graph Theory, Tashkent, 2017.
Responsible Lecturer	Djurayeva D. Sh. – Associate Professor, Department of Algebra and Geometry, Termez State University; Doctor of Philosophy (PhD) in Pedagogical Sciences
Field	Description
Module Title	COMPUTER NETWORKS
Module Code	KOT1306
Credit Points (ECTS)	6 Credits
Workload	Total workload: 180 hours Contact hours: 60 hours (30 hours Lecture, 30 hours Practical lessons) Self-Assignments: 120 hours
Frequency/Duration	III semester
Module objective	The "Computer Networks" course is designed for students in the 60610400 Software Engineering program, and the course objective is to cover the architecture and components of networks and information transmission systems; the principles of organization and operation of networks and information transmission systems; the structure and functions of the technical

	and software components of local and global computer networks and information transmission systems; the data transmission technology in information transmission systems; computer network protocols; and the characteristics of using networks and information systems in accordance with information security requirements.
Module Learning Outcomes	Knowledge LO1. The architecture and components of networks and information transmission systems; the principles of organization and operation of networks and information transmission systems; the structure and function of the technical and software components of local and global computer networks and information transmission systems; data transmission technology in information transmission systems; computer network protocols; the specifics of using networks and information systems in accordance with information security requirements. LO2. Be proficient in creating a local area network, selecting the necessary equipment, configuring the network, working on the network, and creating and using shared resources on it; LO3. Knowledge of the OSI reference model for open systems interconnection and its division into layers, the relationships between layers, the local, regional, and global types of networks, their similarities and differences, Must have competencies in TCP/IP tools for diagnosing local network operation and methods for troubleshooting. LO4. The architecture and elements of networks and information transmission systems; the principles of organization and operation of networks and information transmission systems; the structure and function of the technical and software components of local and global computer networks and information transmission systems; data transmission technology in information transmission systems; computer network protocols; and the characteristics of using networks and information systems in accordance with information security requirements. Skills LO5. Develop the ability to work proficiently with computer networks. This enables the student to quickly learn how to work with computer networks. LO6. Developing skills in working with computer networks. Qualifications LO7. The student will master computer networks, analyze the results, and be able to draw scientific conclusions.

	LO8. Can effectively apply the theoretical foundations and methods of computer networks in laboratory and scientific-practical work. LO9. Students develop hands-on skills in working directly with modern computer networks.
Content	Lectures Introduction to Computer Networks. Students study the basic concepts, components, and functions of computer networks. The topic provides the foundation for network communication and architecture. Basic Principles of Network Design. Students examine the principles of designing reliable and efficient computer networks. The topic introduces network planning and topology selection. Ethernet and Fast Ethernet network architecture. Students study the architecture and operation of Ethernet and Fast Ethernet technologies. The topic emphasizes data transmission and network performance. Token-Ring network. Students examine the structure and operation of Token-Ring networks. The topic introduces token-based communication methods. CAN and ArcNET networks. Students study the architecture and applications of CAN and ArcNET networks. The topic highlights their use in industrial and embedded systems. Transport protocols and IPX/SPX protocols. Students examine transport layer protocols and the IPX/SPX protocol suite. The topic develops an understanding of network communication services. AppleTalk, NetBIOS, WINS protocols. Students study AppleTalk, NetBIOS, and WINS protocols and their networking functions. The topic introduces legacy network communication protocols. Brief description of the history, structure, and protocols of the TCP/IP stack. Students examine the development, architecture, and protocols of the TCP/IP model. The topic provides the foundation for Internet communication. IP inter-network communication protocol. Students study the Internet Protocol (IP) and its role in inter-network communication. The topic introduces packet delivery and addressing concepts. Addressing in an IP network. Students examine MAC addresses, IP addresses, and DNS names used in network communication. The topic develops an understanding of network addressing.

Mapping of address designations in IP addresses: DHCP and DNS. Students study DHCP and DNS services for automatic IP address assignment and name resolution. The topic introduces essential network configuration services.

IP inter-network communication protocol, management of segmentation. Students examine IP packet segmentation and reassembly mechanisms. The topic develops an understanding of efficient data transmission.

Routing using IP addresses. Students study IP routing principles and packet forwarding between network devices. The topic introduces routing processes and communication examples.

Communication channels: Cable communication channels, fiber-optic cables. Students examine wired communication media, including copper and fiber-optic cables. The topic compares their characteristics and applications.

Wireless communication channels. Structured cabling for network installation. Students study wireless communication technologies and structured cabling systems. The topic introduces modern network installation and infrastructure principles.

Practics

Network devices and their functions. Network adapters, repeaters, bridges, hubs. Students study the functions of common network devices and their roles in data communication. The topic introduces essential networking hardware.

Network emulators and their use. Students examine network emulators for designing and testing network configurations. The topic develops practical network simulation skills.

Data encoding methods. Students study common data encoding techniques used in digital communication. The topic introduces methods for reliable data transmission.

Local area network technologies: Ethernet, Fast Ethernet. Analyzing packets using network sniffers. Students examine Ethernet technologies and analyze network traffic using packet sniffing tools. The topic develops practical network analysis skills.

Wireless information transmission technologies. Configuring a Wi-Fi network. Students study wireless communication technologies and Wi-Fi network configuration. The topic develops practical wireless networking skills.

IP addresses. Class addresses. Addressing using masks. Economical use of IP addresses. Students examine IP addressing, subnet masks, and efficient address allocation. The topic develops practical network addressing skills.

Router Configuration. Students configure routers for network communication and routing. The topic develops practical skills in network device management.

Creating a network application using sockets. Students develop basic network applications using socket programming. The topic introduces client-server communication principles.

Installing and initial configuration of the Windows Server operating system. Students install and configure Windows Server for network services. The topic develops basic server administration skills.

Installing and configuring network services (DHCP, DNS) on the Windows Server operating system. Students configure DHCP and DNS services on Windows Server. The topic develops practical network management skills.

Managing users and their permissions in the Windows operating system. Students create user accounts and manage access permissions in Windows. The topic introduces user and security administration.

Creating and using shared network resources. Students configure shared folders and network resources for collaborative access. The topic develops resource management skills.

Configuring the firewall in the Windows operating system. Using the VPN service. Students configure Windows Firewall and establish VPN connections. The topic develops practical network security skills.

Detecting network issues, using operating system utilities. Students diagnose network problems using built-in operating system tools. The topic develops troubleshooting and maintenance skills.

Layers of network architecture. Students study the layers of network architecture and their functions. The topic provides the foundation for understanding network communication.

Student Independent Work. Classification of computer networks. Students study the classification of computer networks based on size, topology, and purpose. The topic introduces the fundamental types of network architectures.

Layers of network architecture: physical, data link, network, transport, session, application. Students examine the functions of the OSI network layers and their roles in data communication. The topic provides the foundation for understanding network operations.

Transmission media. Channel bandwidth. Students study wired and wireless transmission media and the concept of

channel bandwidth. The topic explains factors affecting network performance.

Local and global networks. The Internet. Students examine local area networks, wide area networks, and the Internet. The topic introduces modern network communication environments.

Ethernet, Fast Ethernet, Token Ring technologies. Students study the architecture and characteristics of Ethernet, Fast Ethernet, and Token Ring technologies. The topic compares their performance and applications.

Local and global network protocols and characteristics. Students examine communication protocols used in local and global networks. The topic develops an understanding of protocol functions and network interoperability.

OSI/ISO standards, TCP/IP protocols. Students study the OSI/ISO reference model and the TCP/IP protocol suite. The topic provides the foundation for modern network communication.

Information transmission in local computer networks. Students examine the principles of data transmission in local computer networks. The topic introduces communication methods and network efficiency.

Core set of network protocols and services for a sample corporate network. Students study the essential protocols and services used in corporate networks. The topic emphasizes practical network infrastructure design.

Typical network administration tasks. Students examine common network administration activities, including configuration, monitoring, and maintenance. The topic introduces network management practices.

Principles of inter-network communication. Students study the principles of communication between interconnected networks. The topic introduces routing and packet forwarding concepts.

Network Operating Systems (using the Windows Server OS family as an example). Students examine the features and functions of network operating systems using Windows Server as an example. The topic develops an understanding of server-based network management.

Novell network products. Microsoft Network operating system. Students compare Novell and Microsoft network operating systems and their networking services. The topic highlights their architecture and practical applications.

Modern concepts and technologies for operating system design. Students study modern operating system concepts and

	design technologies. The topic introduces current trends in operating system development. Requirements for 21st-century operating systems. Students examine the functional, security, and performance requirements of modern operating systems. The topic emphasizes reliability, scalability, and cybersecurity.
Teaching Methods	The module is delivered through interactive lectures, inquiry-based and problem-based learning, hands-on laboratory exercises, network simulation and configuration activities, seminars, group discussions, and project-based assignments. Students design, configure, and analyze computer networks using industry-standard networking tools and simulation software, while developing practical skills in network architecture, routing, switching, network protocols, wireless communication, and network troubleshooting. Independent learning and digital educational resources further promote critical thinking, technical communication, and the application of modern networking technologies to real-world computing environments.
Prerequisites	Operating systems Algorithms and Data Structures English Language (XJTB106)
Exams and Assessment	Type of examination (written, oral, project) and its weight Student performance is assessed through continuous assessment (20%), midterm assessment (30%), and a final written examination (50%). The continuous assessment (20%) evaluates students' participation in lectures and practical classes, laboratory activities, and completion of independent study assignments, including presentations, technical reports, network configuration exercises, literature reviews, mini-projects, and practical tasks related to computer network design, IP addressing, routing, network protocols, and network services. The midterm assessment (30%) consists of a written test and the evaluation of independent study projects, such as reports, presentations, analytical papers, and practical assignments covering network architectures, TCP/IP protocols, IP addressing, routing, network services, and network administration. The final assessment (50%) is a written examination consisting of theoretical and practical questions designed to assess students' understanding of computer network architecture, communication protocols, network configuration, routing, network management, and their ability to apply networking concepts in practical environments. Successful completion of all independent study

	assignments is required as an essential component of the module assessment.
Recommended Literature	1. E. Tanenbaum, D. Wetherall. Computer Networks. 5th ed. St. Petersburg: Piter, 2012. 960 pp. 2. V. Olifer, N. Olifer. Computer Networks: Principles, Technologies, Protocols: Textbook for Higher Education Institutions. 5th ed. 992 pp. 2016 3. Limoncelli T. , Hogan Ch. , Chalup S. The Practice of System and Network Administration. Pearson Education, Inc. , 2007, p. 1024. 4. X. Zayniddinov, S. Orinboyev, A. Beletskiy. Advanced Course in Computer Networks. "Sharq Publishing House", Tashkent, 2007, p. 164. 5. Mirziyoyev, Sh. M. The Rule of Law and Ensuring Human Interests – the Guaranty of National Development and People's Welfare. Tashkent, Uzbekistan Publishing House, 2017. 6. Broydo V. L. Computational Systems, Networks, and Telecommunications: A Textbook for Universities. 2nd ed. - St. Petersburg: Piter, 2004. - 703 pp. : ill. 7. Stepanov A. N. Architecture of Computer Systems and Computer Networks. Textbook. Piter, 2007. - 512 p. 8. Kulgin M. Technologies of Corporate Networks. Encyclopedia. — St. Petersburg: "Piter", 2008, 704 p.
Responsible Lecturer	Choriyev Ilhom Kenjayevich, Senior Lecturer, Department of Mathematical Modeling and Computer Sciences
Field	Description
Module Title	ARTIFICIAL INTELLIGENCE FUNDAMENTALS
Module Code	SIA1406
Credit Points (ECTS)	6 Credits
Workload	Total workload: 180 hours Contact hours: 60 hours (30 hours Lecture, 30 hours Practical lessons) Self-Assignments: 120 hours
Frequency/Duration	IV semester
Module objective	To develop in students a deep knowledge and skills in the theoretical and practical foundations of artificial intelligence (AI) technologies, their stages of development, and their applications in modern fields.
Module Learning Outcomes	Knowledge LO1. Can explain the concept of artificial intelligence, its development history, and its application areas; distinguish knowledge representation models (logical, semantic, frame, etc.

	); and explain the operating principles and mathematical foundations of neural networks. LO2. Has the skills to design and train simple artificial neural networks and can implement AI-based algorithms in a programming language, as well as apply deductive and non-deductive logical inference methods; LO3. Able to analyze intelligent systems and their potential applications in real-world domains, and to have knowledge of the main methods of machine learning (supervised, unsupervised, and reinforcement learning); LO4. Will have initial experience in using artificial intelligence technologies in practical projects and be prepared to independently study scientific and technical literature and apply new technologies. Skills LO5. analyze complex problems and approach solving them using artificial intelligence methods. LO6. application of intelligent systems in real-world domains (healthcare, education, economics, etc.). LO7. Independently select and apply SI approaches. LO8. develop new ideas and apply artificial intelligence technologies in an innovative way. Qualifications LO9. Knowledge of the fundamental concepts of artificial intelligence, its directions (expert systems, machine learning, neural networks, etc.), and their application areas LO10. create simple artificial intelligence algorithms, apply machine learning models, and solve practical problems using Python libraries. LO11. Analyzing real-world problems and developing AI-based solutions, participating in team projects, and implementing new technologies.
Content	Lectures The concept of artificial intelligence and its development history. Students study the fundamental concepts of artificial intelligence and its historical development. The topic introduces the evolution of AI technologies and applications. Modern applications of artificial intelligence and future trends. Students examine current AI applications across various industries and emerging trends. The topic highlights the future directions of artificial intelligence. The concept of an intelligent agent, its types, and operating principles. Students study intelligent agents, their classifications,

and operating principles. The topic introduces autonomous decision-making systems.

Intelligent analysis of data. Data collection and preparation
Students examine techniques for data collection, preprocessing, and intelligent data analysis. The topic emphasizes the importance of high-quality data for AI systems.

Data cleansing, compression, and search algorithms
Students study methods for data cleaning, compression, and efficient search algorithms. The topic develops data processing skills for AI applications.

Knowledge bases and knowledge representation models
Students examine knowledge representation techniques and the structure of knowledge bases. The topic introduces symbolic reasoning in artificial intelligence.

Fuzzy knowledge, Bayesian networks, and expert systems
Students study fuzzy logic, Bayesian networks, and expert systems for intelligent decision-making. The topic explores uncertainty handling in AI.

Artificial neural networks. Perceptron and multilayer neural networks. Students examine the structure and operation of artificial neural networks, including perceptrons and multilayer networks. The topic introduces the foundations of deep learning.

The training process in neural networks. Students study neural network training methods, learning algorithms, and model optimization. The topic develops an understanding of supervised learning processes.

Introduction to Artificial Intelligence. Main Types of Artificial Intelligence. Students study the basic concepts and major categories of artificial intelligence. The topic provides an overview of AI systems and their capabilities.

Supervised learning: regression and classification algorithms. Students examine supervised learning techniques, including regression and classification algorithms. The topic introduces predictive modeling methods.

Unsupervised and reinforced learning. Students study unsupervised learning and reinforcement learning techniques. The topic develops an understanding of pattern discovery and decision-making.

The concept of Deep Learning. Students examine the principles and architectures of deep learning models. The topic introduces advanced neural network techniques for complex tasks.

Fundamentals of Natural Language Processing (NLP). Students study the basic concepts and methods of natural

language processing. The topic introduces techniques for understanding and processing human language.

Problems and Future Directions of NLP in Uzbek. Students examine the challenges and future opportunities of Natural Language Processing for the Uzbek language. The topic highlights research directions and practical applications of Uzbek NLP.

Practics

Analyzing modern application areas of artificial intelligence technologies. Students analyze real-world applications of artificial intelligence across different domains. The practical session develops skills in evaluating AI solutions.

Designing models of intelligent agents and their operating principles. Students design simple intelligent agent models and analyze their behavior. The practical session develops problem-solving and AI modeling skills.

Working with tools used in data collection and preparation processes. Students use software tools to collect and prepare datasets for AI applications. The practical session develops data preprocessing skills.

Applying data cleansing, normalization, and transformation methods. Students clean, normalize, and transform datasets for machine learning tasks. The practical session develops data preparation techniques.

Using data mining and analysis algorithms. Students apply data mining algorithms to discover patterns in datasets. The practical session develops analytical and data exploration skills.

Creating a knowledge base and building knowledge representation models. Students develop simple knowledge bases and implement knowledge representation methods. The practical session introduces symbolic AI techniques.

Creating simple models for expert systems and Bayes networks. Students build basic expert systems and Bayesian network models. The practical session develops knowledge-based reasoning skills.

Create a simple neural network based on the perceptron model. Students implement a simple perceptron neural network for classification tasks. The practical session introduces the fundamentals of neural networks.

Building and training multi-layer neural networks. Students construct and train multilayer neural networks using sample datasets. The practical session develops deep learning fundamentals.

Working with an artificial intelligence environment (Python, Scikit-learn). Students develop AI applications using Python and the Scikit-learn library. The practical session builds practical machine learning skills.

Applying regression and classification algorithms to practical problems. Students implement regression and classification models on real-world datasets. The practical session develops predictive modeling skills.

Testing clustering and reinforced learning algorithms Students apply clustering and reinforcement learning algorithms to practical problems. The practical session develops experience with unsupervised and reinforcement learning.

Create a Deep Learning model using TensorFlow or PyTorch. Students build and train deep learning models using TensorFlow or PyTorch. The practical session develops practical deep learning skills.

Text processing and working with NLP libraries. Students process text data using modern Natural Language Processing libraries. The practical session develops practical NLP skills.

Develop an NLP project for Uzbek-language texts (tokenization, classification, or sentiment analysis). Students develop a simple NLP application for Uzbek-language text processing. The practical session applies NLP techniques to real-world language tasks.

Student Independent Work

The concept of artificial intelligence and its main directions Students study the fundamental concepts of artificial intelligence and its major research areas. The topic introduces the core branches of AI.

Expert systems and their practical applications. Students examine expert systems and their use in solving real-world problems. The topic introduces knowledge-based decision-making systems.

Logical and production model: modeling data using the frame model. Students study logical, production, and frame-based knowledge representation models. The topic develops an understanding of symbolic AI.

Fuzzy logic and decision making. Students examine fuzzy logic techniques for decision-making under uncertainty. The topic introduces intelligent reasoning methods.

Machine learning and deep learning algorithms and their types. Students study machine learning and deep learning

algorithms and their classifications. The topic provides an overview of modern AI methods.

Comparison of biological neurons and artificial neurons

Students compare biological neurons with artificial neural network models. The topic explains the inspiration behind neural computing.

ANN training algorithms: supervised and unsupervised

Students study supervised and unsupervised learning algorithms for artificial neural networks. The topic introduces different neural network training approaches.

Gradient descent method in the weight coefficient space

Students examine the gradient descent optimization method for training neural networks. The topic explains parameter optimization techniques.

Hebb's learning method and rule-based learning. Students study Hebbian learning and rule-based learning methods. The topic introduces fundamental learning principles in AI.

Kolmogorov-Arnold theorem and its application to AI systems. Students examine the Kolmogorov-Arnold theorem and its applications in neural network design. The topic highlights its significance in AI theory.

Backpropagation

algorithm

Students study the backpropagation algorithm for training multilayer neural networks. The topic introduces error minimization techniques.

Kohonen neural networks and self-organizing maps.

Students examine Kohonen neural networks and self-organizing maps for unsupervised learning. The topic develops clustering and visualization concepts.

Radial basis functions (RBF) and networks. Students study radial basis function networks and their applications in pattern recognition. The topic introduces RBF-based learning models.

Convolutional neural networks and image recognition

Students examine convolutional neural networks (CNNs) for image classification and computer vision tasks. The topic introduces deep learning for visual data.

Enhanced training algorithms. Students study advanced optimization and training algorithms for neural networks. The topic emphasizes improving learning efficiency and accuracy.

Hopfield neural networks and associative memory

Students examine Hopfield neural networks and their role in associative memory. The topic introduces recurrent neural network concepts.

	Hopfield neural networks and their training Students study training methods and applications of Hopfield neural networks. The topic develops an understanding of memory-based neural models. Elman recurrent networks and time series analysis Students examine Elman recurrent neural networks for sequential and time-series data analysis. The topic introduces recurrent learning techniques. Jordan recurrent networks and their training Students study Jordan recurrent neural networks and their training process. The topic develops an understanding of recurrent architectures for sequence modeling. Generative Artificial Intelligence (Generative AI) and Large Language Models (LLMs). Students examine Generative AI and large language models, including their architectures, capabilities, and applications. The topic introduces modern generative technologies. Computer Vision and Object Detection Technologies Students study computer vision techniques and object detection algorithms. The topic introduces AI methods for image analysis and visual recognition. Ethical issues, safety, and responsible AI systems in artificial intelligence. Students examine ethical principles, safety challenges, and responsible AI development. The topic emphasizes fairness, transparency, and trustworthy AI systems.
Teaching Methods	The module is delivered through interactive lectures, inquiry-based and problem-based learning, hands-on laboratory exercises, AI programming and model development activities, seminars, group discussions, and project-based assignments. Students explore fundamental concepts of artificial intelligence, including intelligent agents, search algorithms, knowledge representation, machine learning, and ethical considerations, while developing practical skills using modern AI frameworks and programming tools. Independent learning and digital educational resources further promote critical thinking, computational reasoning, scientific communication, and the application of artificial intelligence techniques to real-world problems.
Prerequisites	Mathematical Logic (DMMB111) Basic knowledge of programming languages such as Python, Java, or C++ (DASB113) Data Structures (BBZB209) English Language (XJTB106)

Exams and Assessment	Type of examination (written, oral, project) and its weight Student performance is assessed through continuous assessment (20%), midterm assessment (30%), and a final written examination (50%). The continuous assessment (20%) evaluates students' participation in lectures and practical classes, programming exercises, and completion of independent study assignments, including presentations, analytical reports, literature reviews, mini-projects, and practical implementations using machine learning, neural networks, deep learning, and natural language processing tools. The midterm assessment (30%) consists of a written test and the evaluation of independent study projects, such as reports, presentations, analytical papers, and practical assignments covering artificial intelligence concepts, knowledge representation, machine learning, neural networks, deep learning, and NLP applications. The final assessment (50%) is a written examination consisting of theoretical and practical questions designed to assess students' understanding of artificial intelligence principles, intelligent systems, machine learning algorithms, neural networks, deep learning, and their ability to apply AI techniques to real-world problems. Successful completion of all independent study assignments is required as an essential component of the module assessment.
Recommended Literature	1. Russell, S. , & Norvig, P. Artificial Intelligence: A Modern Approach. Pearson, 2021. – 1152 pages. 2. Goodfellow, I. , Bengio, Y. , & Courville, A. Deep Learning. MIT Press, 2016. – 800 pages. 3. Jain, L. C. , Mao, J. , & Mohiuddin, K. M. Artificial Neural Networks: A Practical Course. Springer, 1996. – 320 pages 4. Zadeh, L. A. , & Klir, G. J. Fuzzy Logic: Systems, Control, and Information. Prentice Hall, 1996. – 500 pages 5. Bishop, C. M. Pattern Recognition and Machine Learning. Springer, 2006. – 738 pages. 6. Haykin, S. Neural Networks and Learning Machines. Pearson, 2009. – 842 pages. 7. Kosko, B. Neural Networks and Fuzzy Systems: A Dynamical Systems Approach to Machine Intelligence. Prentice Hall, 1992. – 460 pages.
Responsible Lecturer	Amirkulov Chori Jumayevich teacher.
Field	Description

Module Title	PROBABILITY AND STATISTICS
Module Code	EVS1406
Credit Points (ECTS)	6 Credits
Workload	Total workload: 180 hours Contact hours: 90 hours (44 hours Lecture, 46 hours Practical lessons) Self-Assignments: 90 hours
Frequency/Duration	IV semester
Module objective	The "Probability and Statistics" course is designed for students in the 60610400 – Software Engineering and 60610300 – Computer Engineering educational programs, and it aims to develop students' ability to mathematically model random events and processes, to develop theoretical knowledge, practical skills, and competencies in the collection, processing, analysis, and scientific interpretation of data. Within the course, students will develop theoretical knowledge, practical skills, and competencies in mathematical modeling of random phenomena and processes, data collection, processing, analysis, and the scientific interpretation of results. statistical data description methods, samples and populations, statistical estimation, hypothesis testing, correlation and regression analysis, and data visualization methods. Additionally, students will learn to analyze real-world processes using probabilistic and statistical methods, making decisions based on data, assessing uncertainties in software and information systems, and acquiring the competencies required in the fields of artificial intelligence, machine learning, and data analysis.
Module Learning Outcomes	Knowledge LO1. Have an understanding of the basic concepts of probability theory, random events, and the classical and statistical definitions of probability; LO2. Knowledge of random variables, their distribution laws, the mathematical expectation, variance, and other numerical characteristics; LO3. Understanding the properties of discrete and continuous probability distributions, the Bernoulli scheme, binomial, Poisson, and normal distributions; LO4. Knowledge of the basic concepts of mathematical statistics, the population and the sample, statistical series, and statistical indicators;

	LO5. Understand the nature and practical importance of statistical estimation, confidence intervals, testing of statistical hypotheses, and correlation and regression analysis. Skills LO6. Ability to calculate the probabilities of random events and solve problems related to probability theory; LO7. Develop skills in determining the numerical characteristics of random variables and working with probability distributions; LO8. Acquire skills in collecting, grouping, analyzing, and graphically representing statistical data; LO9. Have the skills to identify relationships between data using statistical methods and to conduct correlation and regression analyses. Qualifications LO10. Be proficient in creating mathematical models of real processes and phenomena using probabilistic and statistical methods; LO11. Develop the ability to draw statistical inferences, estimate parameters, and test hypotheses based on sample data; LO12. Be proficient in applying probabilistic and statistical methods to solve practical problems in software engineering, computer engineering, artificial intelligence, and data analysis.
Content	Lectures Space of elementary events. The concept of the probability of an event and its classical, geometric, and statistical definitions. Students study the fundamentals of probability theory and different approaches to defining probability. The topic provides the foundation for probabilistic analysis. Algebra of events. Kolmogorov axioms. Properties of probability. Students examine event algebra, Kolmogorov's axioms, and the basic properties of probability. The topic develops formal probability reasoning. Conditional probability. Dependence of events. Total probability and Bayes' formulas. Students study conditional probability, event dependence, the law of total probability, and Bayes' theorem. The topic develops probabilistic inference skills. Sequence of independent trials. Bernoulli scheme and formula. Properties of the binomial distribution. Students examine Bernoulli trials, the binomial distribution, and their applications. The topic introduces discrete probability models. De Moivre–Laplace local and integral limit theorems. Poisson's theorem. Students study approximation theorems for probability distributions, including the De Moivre–Laplace and

Poisson theorems. The topic develops methods for probability estimation.

Random variable and its distribution function. Random vector and its distribution. Students examine random variables, distribution functions, and random vectors. The topic introduces probabilistic modeling techniques.

Mathematical expectation of a random variable. Students study the concept and properties of mathematical expectation. The topic develops methods for calculating expected values.

The variance of a random variable. Students examine variance as a measure of data dispersion. The topic introduces methods for measuring variability.

Some important distributions. Students study commonly used probability distributions and their properties. The topic introduces models for real-world random phenomena.

Multivariate distributions. Students examine probability distributions involving multiple random variables. The topic develops methods for multivariable statistical analysis.

Distributions of functions of random variables. Composite formulas. Students study the distributions of transformed random variables and composite probability formulas. The topic develops advanced probabilistic analysis techniques.

Higher-order moments. Correlation coefficient and its properties. Students examine higher-order moments and correlation coefficients. The topic introduces measures of distribution shape and variable relationships.

Chebyshev's inequality and theorem. Law of large numbers Students study Chebyshev's inequality and the law of large numbers. The topic explains the behavior of random variables in repeated experiments.

Basic problems of mathematical statistics. Students examine the objectives and methods of mathematical statistics. The topic introduces statistical data analysis concepts.

Empirical distribution function and its properties. Statistical estimation. Students study empirical distribution functions and statistical estimation methods. The topic develops data analysis and inference skills.

Point estimates and methods for constructing estimates Students examine point estimation techniques and estimation methods. The topic introduces parameter estimation in statistics.

A reliable interval method for estimating unknown parameters. Students study confidence intervals for estimating

unknown population parameters. The topic develops statistical inference skills.

Distributions related to normal distributions: chi-square, Student's t , and Fisher's distributions. Students examine chi-square, Student's t , and Fisher distributions. The topic introduces their applications in statistical analysis.

Linear correlation. Students study linear correlation and methods for measuring relationships between variables. The topic develops statistical interpretation skills.

Statistical hypotheses and their types. Type I and Type II errors. Students examine statistical hypothesis testing and decision errors. The topic introduces hypothesis testing principles.

Testing statistical hypotheses about the mathematical expectation and variances. Students study statistical tests for population means and variances. The topic develops practical hypothesis testing skills.

Pearson's chi-square statistic. Students examine Pearson's chi-square test for goodness-of-fit and independence analysis. The topic introduces one of the fundamental methods of statistical hypothesis testing.

Practics

The fundamental principle of combinatorics and some combinatorial formulas. Students study the basic counting principles and fundamental combinatorial formulas. The topic develops techniques for solving counting problems.

The limitations of the classical, geometric, and statistical methods for calculating probabilities. The relationship between the continuous and countable additivity axioms Students examine different probability definitions and their limitations. The topic introduces the theoretical foundations of probability axioms.

Properties of probability derived from the A. N. Kolmogorov axioms. Students study the main properties of probability based on Kolmogorov's axioms. The topic develops formal probabilistic reasoning.

Conditional probability. Independence of events and probability of the sum of independent events. Total probability and Bayes' formulas. Students examine conditional probability, event independence, and Bayes' theorem. The topic develops methods for probabilistic inference.

Relationship between independence in the sample of events and pairwise independence. Bernstein's example

Students study the concepts of event independence and pairwise independence using Bernstein's example. The topic highlights important probability concepts.

Poisson scheme for a sequence of independent experiments. Students examine the Poisson model for independent random experiments. The topic introduces practical applications of the Poisson distribution.

Generating functions. Students study generating functions and their applications in probability and combinatorics. The topic develops analytical problem-solving techniques.

Study of some important distributions encountered in practice. Students examine commonly used probability distributions and their applications. The topic introduces practical statistical models.

Construct examples of random variables that do not have a mathematical expectation or variance. Students analyze random variables with undefined expectation or variance. The topic develops an understanding of exceptional probability distributions.

Practical application of the correlation coefficient. Students study the use of correlation coefficients for measuring relationships between variables. The topic develops practical data analysis skills.

Characteristic function and its properties. Students examine characteristic functions and their role in probability theory. The topic introduces advanced methods for analyzing distributions.

Law of large numbers. Chebyshev theorem and inequality Students study the law of large numbers and Chebyshev's theorem. The topic explains convergence properties of random variables.

Central limit theorem. Lyapunov theorem. Applications of the central limit theorem. Proof of the central limit theorem under the Lindberg condition. Students examine the central limit theorem, Lyapunov theorem, and their applications. The topic introduces fundamental limit theorems in probability.

Basic problems of mathematical statistics. Population and sample collections. Students study the concepts of populations, samples, and statistical analysis. The topic introduces the foundations of mathematical statistics.

Empirical distribution function and its properties. Empirical measures and their calculation. Students examine empirical distribution functions and methods for calculating empirical statistics. The topic develops statistical data analysis skills.

The concept of a statistical estimate. Point estimates and methods for constructing estimates. Students study statistical estimation and point estimation methods. The topic introduces parameter estimation techniques.

Confidence interval method for estimating unknown parameters. Important distributions. Students examine confidence intervals and probability distributions used in statistical estimation. The topic develops statistical inference skills.

Point estimates for some important distribution parameters Students study methods for estimating the parameters of common probability distributions. The topic develops practical estimation techniques.

Empirical distribution function. Empirical measures and their calculation. Glivenko theorem. Students examine empirical distributions and the Glivenko theorem. The topic introduces convergence properties of empirical data.

Distributions related to the normal distribution: the chi-square, Student's t , and Fisher's distributions. Students study the chi-square, Student's t , and Fisher distributions. The topic introduces their applications in statistical testing.

Confidence interval method for estimating unknown parameters. Students study confidence interval methods for statistical parameter estimation. The topic develops reliable statistical inference techniques.

Confidence intervals for estimating the mathematical expectation of the normal distribution, both when its variance is known and when it is unknown Students examine confidence intervals for estimating the mean of a normal distribution. The topic covers both known and unknown variance cases.

Confidence intervals for estimating the mean squared deviation of the normal distribution. Students study confidence interval estimation for the standard deviation of a normal distribution. The topic develops methods for estimating population variability.

Student Independent Work

The fundamental principle of combinatorics and some combinatorial formulas. Students study the basic counting principles and combinatorial formulas. The topic develops techniques for solving counting problems.

The limitations of the classical, geometric, and statistical methods of probability calculation. The relationship

between the continuous and countable additivity axioms Students examine different probability calculation methods and their limitations. The topic introduces the theoretical foundations of probability.

Properties of probability derived from A. N. Kolmogorov's axioms. Conditional probability. Independence of events and the probability of the union of independent events. Total probability and Bayes' formulas. Students study the fundamental properties of probability, conditional probability, and Bayes' theorem. The topic develops probabilistic reasoning and inference skills.

Poisson scheme of a sequence of independent experiments. Sequence of independent experiments Students examine independent random experiments and the Poisson scheme. The topic introduces models for repeated probabilistic events.

Generating functions. Polynomial distribution of probabilities. Students study generating functions and polynomial probability distributions. The topic develops analytical methods for probability analysis.

Stieltjes integral. Students examine the Stieltjes integral and its applications in probability theory. The topic introduces advanced integration concepts.

Practical application of the correlation coefficient. Students study the use of the correlation coefficient for measuring relationships between variables. The topic develops practical statistical analysis skills.

Characteristic function and its properties. Students examine characteristic functions and their applications in probability distributions. The topic introduces advanced tools for probability analysis.

Proof of the central limit theorem under the Lindberg condition. Students study the Lindberg condition and its role in proving the central limit theorem. The topic develops an understanding of probability convergence.

Representativeness of a sample and methods for its construction. Students examine sampling methods and the concept of representative samples. The topic develops reliable statistical data collection techniques.

Methods for estimating unknown parameters. Properties of the resulting estimates. Students study statistical parameter estimation methods and the properties of estimators. The topic introduces accuracy and efficiency in estimation.

	Point estimates for the parameters of some important distributions. Students examine point estimation methods for common probability distributions. The topic develops practical skills in statistical parameter estimation.
Teaching Methods	The module is delivered through interactive lectures, inquiry-based and problem-based learning, statistical computing laboratories, data analysis workshops, seminars, group discussions, and project-based activities. Students apply probability theory and statistical methods to analyze real-world data using statistical software and programming tools, while developing skills in data collection, descriptive and inferential statistics, probability distributions, hypothesis testing, regression analysis, and data interpretation. Independent learning and digital educational resources further promote critical thinking, quantitative reasoning, scientific communication, and evidence-based decision-making through modern statistical techniques.
Prerequisites	Programming (DAS1110) Linear Algebra (CHA1306) Discrete Structures (DIT1306) Probability and Statistics (EVS1406)
Exams and Assessment	Type of examination (written, oral, project) and its weight Student performance is assessed through continuous assessment (20%), midterm assessment (30%), and a final written examination (50%). The continuous assessment (20%) evaluates students' participation in lectures and practical classes, problem-solving activities, and completion of independent study assignments, including presentations, analytical reports, statistical data analysis, probability problem solving, literature reviews, and mini-projects related to probability theory, mathematical statistics, and data analysis. The midterm assessment (30%) consists of a written test and the evaluation of independent study projects, such as reports, analytical papers, presentations, and practical assignments covering probability theory, random variables, probability distributions, statistical estimation, hypothesis testing, and regression analysis. The final assessment (50%) is a written examination consisting of theoretical and practical questions designed to assess students' understanding of probability theory, mathematical statistics, statistical modeling, data analysis, and their ability to apply probabilistic and statistical methods to real-world problems.

	Successful completion of all independent study assignments is required as an essential component of the module assessment.
Recommended Literature	1.Sh. Q. Farmonov, "Probability Theory," Tashkent, "Universitet," 2014. 2.A. A. Abdushukurov, T. M. Zuparov. "Probability Theory and Mathematical Statistics," Tafakkur Bo'stoni, 2015. 3.Borovkov A. A. "Theory of Probability," Moscow, "Lan," 2010. 4.Ivchenko G. I. , Medvedev Yu. I. Introduction to Mathematical Statistics, M. : LKI, 2010. 5.Gmurlan V. Ye. "Guide to Solving Problems in Probability Theory and Mathematical Statistics," Tashkent, "O'qituvchi," 1980. 6.Ross Sheldon M. A First Course in Probability, Pearson Education Inc. , 2010. 7.Robert B. Basic Probability Theory. Dover Publications, Inc. , 2008. 8.Jun Shao. Mathematical Statistics, Springer, 2007. 9.A. A. Abdushukurov, T. A. Azlarov, A. A. Jamirzayev. "Collection of Examples and Problems from Probability Theory and Mathematical Statistics," Tashkent, "Universitet," 2003. 10. Sh. Mirziyoyev. We will build our great future together with our brave and noble people. Tashkent, "Uzbekiston", 2017. 488 pages. 11. A. N. Shiryaev. Probability. In 2 vols. 3rd ed. , Moscow, MCNMO, 2004. 12. T. X. Adirov, E. N. Mamurov. "Problems and Instructions for Solving Them in Probability Theory and Mathematical Statistics," Tashkent, "Iqtisod-moliya," 2007. 13. A. V. Prokhorov, V. G. Ushakov, N. G. Ushakov. Problems in Probability Theory, Moscow, Nauka, 1986. 14. A. M. Zubkov, B. A. Sevastyanov, V. P. Chistyakov. Collection of Problems in Probability Theory, Moscow, Nauka, 1989. 15. Abdushukurov A. A. Probability Theory and Mathematical Statistics, Universitet, 2010, 164 pages. 16. S. X. Sirojiddinov, M. Mamatov. "Probability Theory and Mathematical Statistics," Tashkent, "O'qituvchi," 1980. 17. Sevastyanov B. A. "Course in Probability Theory and Mathematical Statistics," Moscow, "Nauka," 1982.
Responsible Lecturer	Djurayeva D. Sh. – Associate Professor, Department of Algebra and Geometry, Termez State University; Doctor of Philosophy (PhD) in Pedagogical Sciences
Field	Description

Module Title	PROFESSIONAL SPEECH CULTURE
Module Code	KNM303
Credit Points (ECTS)	3 Credits
Workload	Total workload: 90 hours Contact hours: 30 hours (14 hours Lecture, 16 hours Practical lessons) Self-Assignments: 60 hours
Frequency/Duration	IV semester
Module objective	It is to develop the written and oral speech culture, stylistics, and communication skills necessary for students to communicate successfully in their professional lives. This course not only enhances the knowledge acquired during the learning process but also lays the groundwork for students to find their place in professional activities and become successful specialists.
Module Learning Outcomes	Knowledge LO1. Theoretical foundations of professional speech culture: The student must know the content, purpose, and tasks of professional speech culture, the specific features of its oral and written forms (), and the basic principles of professional communication. For example, understanding the structure of formal and informal communication, professional correspondence, and presentations. LO2. Rules and Etiquette of Professional Communication: The student must know the principles of professional communication, communication etiquette, professional etiquette, conducting formal and informal correspondence, and adhering to ethical standards in professional activities. LO3. Written and Oral Speech Style: The student must know how to prepare formal documents, emails, reports, memoranda, business letters, and presentations, as well as the rules for expressing ideas clearly, fluently, and logically. LO4. Digital and intercultural communication: The student must have knowledge of effective use of digital communication tools, online communication etiquette, principles of intercultural communication, and proper use of professional terminology. Skills LO5. Effective Management of Professional Communication: The ability to properly organize oral and written communication in various professional situations, to express ideas clearly, fluently, and logically, and to conduct effective communication. LO6. Preparation of professional documents and correspondence: Skill in preparing and formalizing official

	documents, service letters, emails, reports, memoranda, and other professional papers in accordance with current stylistic requirements. LO7. Presentation and Public Speaking Skills: Ability to prepare presentations on professional topics, speak confidently in front of an audience, use communication techniques, and establish effective rapport with listeners. LO8. Digital and intercultural communication: The ability to use electronic communication tools effectively, adhere to online communication etiquette, and conduct effective professional communication with representatives of different cultures. Qualifications LO9. Resolving professional communication problems: The ability to analyze communication issues that arise during professional activities, find solutions, constructively resolve conflict situations, and apply effective communication strategies. LO10. Application of professional speech culture in practical activities: The ability to independently and effectively apply professional etiquette, speech culture, rules of written and oral communication, and professional terminology in practical activities.
Content	Lectures Theoretical Foundations of Professional Speech Culture Students study the principles and importance of professional speech culture. The topic develops effective communication skills in professional environments. Theoretical Foundations of Professional Speech Culture Students examine the theoretical concepts and standards of professional communication. The topic emphasizes clear, accurate, and ethical language use. Professional Etiquette: Formal and Informal Communication. Students study the rules of professional etiquette in formal and informal communication. The topic develops appropriate interpersonal communication skills. Ethical Foundations of Speech Culture in Professional Communication. Students examine ethical principles and communication standards in professional settings. The topic promotes respectful and responsible communication. Analysis of the characteristics and types of written and spoken discourse. Students study different forms of written and spoken discourse. The topic develops skills in analyzing communication styles and purposes.

Professional Writing: Documents and Their Written Structure. Students examine the structure and preparation of professional documents. The topic develops effective business and academic writing skills.

Digital Technologies and Professional Speech Students study the use of digital technologies in professional communication. The topic develops effective communication skills in digital environments.

Practices

Developing skills in writing official documents. Students prepare official documents using standard formats and professional writing conventions. The practical session develops formal documentation skills.

Email Writing and Communication Ethics. Students write professional emails while applying communication etiquette and ethical principles. The practical session develops effective digital communication skills.

The Speaker and the Distinctive Characteristics of Oratory Students practice public speaking techniques and analyze the qualities of effective speakers. The practical session develops presentation and oratory skills.

Characteristics of digital tools. Students explore digital communication tools and their professional applications. The practical session develops effective use of modern communication technologies.

Exercises for expressing ideas clearly and effectively Students practice organizing and presenting ideas in a clear and logical manner. The practical session develops oral and written communication skills.

Analyzing and solving communication problems Students analyze common communication challenges and propose appropriate solutions. The practical session develops critical thinking and interpersonal communication skills.

Developing listening skills in professional communication Students practice active listening techniques in professional situations. The practical session improves comprehension and effective communication.

Role-plays to enhance communication culture. Students participate in role-playing activities to improve professional communication and etiquette. The practical session develops confidence and interpersonal communication skills.

Student Independent Work

Studying the style and rules of professional communication and analyzing them based on practical examples. Students analyze professional communication styles using practical examples. The practical session develops effective workplace communication skills.

Written speech style: studying the rules for drafting and editing formal documents. Students prepare and edit formal documents following professional writing standards. The practical session develops official writing skills.

Preparing an effective oral presentation and delivering it to an audience. Students prepare and deliver professional presentations. The practical session develops public speaking and presentation skills.

Learning methods for effective use of digital communication tools in professional activities
Students use digital communication tools in professional scenarios. The practical session develops digital communication skills.

Analyze the basic principles of professional etiquette and communication culture
Students examine professional etiquette through practical communication exercises. The practical session develops appropriate workplace behavior.

Study the specific characteristics of professional communication with representatives of different cultures
Students analyze intercultural communication in professional settings. The practical session develops cross-cultural communication skills.

Proper use of professional terminology and analysis of its importance in speech culture
Students practice using professional terminology correctly in oral and written communication. The practical session develops professional language skills.

Study of technical communication tools and their role in professional activity. Students examine technical communication tools and their workplace applications. The practical session develops practical communication skills.

Analyzing the factors for success in professional communication. Students evaluate the key factors influencing successful professional communication. The practical session develops effective communication strategies.

Practical session on preparing and delivering an effective presentation. Students prepare and present a professional

	presentation to an audience. The practical session improves presentation and communication skills. Learning methods for developing conversations and applying constructive criticism. Students practice effective conversations and constructive feedback techniques. The practical session develops interpersonal communication skills. Analyzing ways to foster sincere communication and positive relationships. Students explore methods for building trust and positive professional relationships. The practical session develops teamwork and communication skills. Learn question-and-answer techniques and methods for effective audience engagement. Students practice questioning and audience interaction techniques. The practical session develops active communication and engagement skills. Prepare a presentation on strategies for capturing and managing attention in communication. Students prepare a presentation on attention management techniques in communication. The practical session develops persuasive speaking skills. Analyzing the importance of digital communication and modern communication technologies in professional activity. Students examine the role of digital communication technologies in professional environments. The practical session develops effective use of modern communication tools.
Teaching Methods	The module is delivered through interactive lectures, inquiry-based and practice-oriented learning, communication workshops, seminars, group discussions, role-playing activities, and project-based assignments. Students develop professional speaking and communication skills by practicing effective oral presentations, public speaking, professional dialogue, negotiation, and interpersonal communication in academic and workplace settings. Independent learning and digital educational resources further promote critical thinking, effective verbal communication, ethical professional conduct, and the application of speech culture principles in diverse professional environments.
Prerequisites	Modern Uzbek literary language Stylistics Culture of speech Official Language Proficiency
Exams and Assessment	Type of examination (written, oral, project) and its weight Student performance is assessed through continuous assessment (20%), midterm assessment (30%), and a final assessment (50%).

	The continuous assessment (20%) evaluates students' participation in lectures and practical classes, classroom discussions, and the successful completion of independent study assignments, including presentations, reports, academic writing exercises, document preparation, literature reviews, and mini-projects related to the application of the Uzbek language in professional and scientific communication. The midterm assessment (30%) consists of a written test and the evaluation of independent study projects, such as analytical reports, presentations, academic papers, and practical assignments covering scientific writing, professional communication, terminology, document preparation, and official writing. The final assessment (50%) is a written examination consisting of theoretical and practical questions designed to assess students' knowledge of academic and professional Uzbek, scientific writing, official documentation, communication skills, and the application of language norms in professional contexts. Successful completion of all independent study assignments is required as an essential component of the module assessment.
Recommended Literature	1. Hasanov B. Culture of Communication. Textbook for students of higher education institutions / Ministry of Higher and Secondary Specialized Education of the Republic of Uzbekistan. National University of Uzbekistan. – T. : "Ta'lim Nashriyoti", 2009. – 156 p. 2. Jumaniyozov R. The Art of Speech. T. : Adolat, 2005. 3. Safarova R. , G'ulomova L. , Kenjayeva X. , Qodirova M. , Berdiyeva G. Professional Speech Culture. – Tashkent: "Fan va texnologiya", 2021. – 164 p. 4. Mirziyoyev, Sh. M. We will build our great future together with our brave and noble people. – Tashkent: "Uzbekiston" NMIU, 2017. – 488 p. 5. Decree of the President of the Republic of Uzbekistan No. PF-4947 of February 7, 2017, "On the 'Action Strategy for the Further Development of the Republic of Uzbekistan . " 6. Decree of the President of the Republic of Uzbekistan No. PF-5850 of October 21, 2019, "On Measures to Fundamentally Enhance the Prestige and Status of the Uzbek Language as the State Language" (National Legal Information Portal, 7. Mamajonov B. , Kenjayev Z. , Qo'chqorov F. Professional Communication and Speech Culture. – Tashkent: "Yangi asr avlodi", 2022. – 185 p.

Responsible Lecturer	Zaynura Kadirova, PhD, Associate Professor of Philology.
Field	Description
Module Title	USE OF THE UZBEK LANGUAGE IN PROFESSIONAL CONTEXTS(ELECTIVE)
Module Code	O'SQ203
Credit Points (ECTS)	3 Credits
Workload	Total workload: 90 hours Contact hours: 30 hours (14 hours Lecture, 16 hours Practical lessons) Self-Assignments: 60 hours
Frequency/Duration	IV semester
Module objective	To enhance students' speech competence in the field in Uzbek; to develop skills in the effective use of specialized terminology in both oral and written professional communication, and to improve students' abilities to express their acquired knowledge of the field in written and oral Uzbek; to ensure the standardization of terminology, key concepts, and the formatting of documents used in the field; to develop skills in discourse styles, particularly the use and practical application of the scientific and official styles within the field. To improve students' written Uzbek language skills and enhance their literacy.
Module Learning Outcomes	Knowledge LO1. Basic Principles and Structures of Speech: The student must know the purposes, structure, and types of professional speech. For example, understanding various communication formats such as presentations, lectures, and written letters. LO2. Grammatical and stylistic rules: Knowledge of proper grammatical structure, parts of speech, and the use of language style in a professional setting. LO3. Rhetorical Devices and Strategies: Studying the rhetorical techniques and arguments needed for persuasive and effective expression in speech. LO4. Communication Theory: Knowledge of various communication theories, their application in professional discourse, and their effectiveness. Skills LO5. Effective Speech Preparation and Delivery: Skill in preparing a speech in a timely manner and presenting the content in a systematic and engaging way.

	LO6. Proper Pronunciation and Vocal Techniques: The ability to use correct pronunciation, vocal management, and physical expressions effectively in speech. LO7. Oral Mimicry: The skill of adapting and effectively using facial expressions, gestures, and physical expression during speech. LO8. Application of Communication Strategies: The ability to apply appropriate communication strategies in various communicative situations (e. g. , discussion, negotiation, decision-making). Competence LO9. Audience Analysis: Knowledge of how to understand and adapt to the needs, interests, and communication styles of different audience groups. LO10. Conflict Resolution: Skill in applying effective speech and communication methods to resolve conflicts and problems that arise in a professional environment.
Content	Lectures Objectives and Tasks of Teaching the Course "Application of Uzbek Language in the Field" Students study the objectives, scope, and significance of using the Uzbek language in professional fields. The topic develops effective communication skills in academic and workplace environments. The development of language and its evolutionary stages Students examine the historical development and evolutionary stages of language. The topic introduces the factors influencing language change and growth. The emergence and development of writing. Students study the origin and historical development of writing systems. The topic explains the role of writing in communication and culture. Lexical layers of the Uzbek language. Students examine the lexical structure of the Uzbek language and its vocabulary layers. The topic develops an understanding of word origins and usage. Language and Terminology. Formation of Terms. Students study the principles of terminology and methods of term formation. The topic emphasizes the use of professional vocabulary in different fields. Text and its types. Microtext and microtext. Students examine the structure, types, and organization of texts. The topic develops skills in analyzing and producing coherent written texts.

Literary speech and its styles. Students study the styles of literary speech and their characteristics. The topic develops effective written and oral communication skills in formal contexts.

Practics

Organization of speech. The beginning and end of speech.

The communicative qualities of speech. Students study the structure of effective speech and its communicative qualities. The topic develops clear and professional oral communication skills.

Speech and literary norms. Credible speech and television show speech. Students examine literary language norms and different forms of public speech. The topic develops effective and accurate language use.

Writing an initial scientific research paper. Avoiding plagiarism, plagiarism, citation, and referencing

Students study the principles of academic writing, citation, referencing, and plagiarism prevention. The topic develops ethical research and writing skills.

Official-business style and its stylistic features

Students examine the characteristics of the official-business writing style. The topic develops professional document writing skills.

Information and data documents. Students study the preparation and formatting of information and data documents. The topic develops practical documentation skills.

Organizational documents. Students examine the structure and purpose of organizational documents. The topic introduces standard practices in official documentation.

Order documents. Students study the preparation and use of official order documents. The topic develops skills in drafting administrative documents.

Service correspondence. Documents related to employee activities. Students examine official correspondence and employee-related documents. The topic develops professional communication and document management skills.

Student Independent Work

The Decree of the President of the Republic of Uzbekistan, Sh. M. Mirziyoyev, on measures to fundamentally enhance the prestige and status of the Uzbek language as the state language

Familiarize yourself with articles by linguists and literary scholars that present their thoughts and reflections on the practical implementation of the tasks set forth in the Decree "On Measures to Fundamentally Enhance the Prestige and Status of the Uzbek Language as the State

	Language. Students analyze the Presidential Decree and review scholarly articles on its implementation. The practical session develops understanding of state language policy. Spelling rules of the Uzbek language and the literacy of field specialists. Creating a creative text on the topic "History of Biology" Students apply Uzbek spelling rules and prepare a creative text on a professional topic. The practical session develops writing accuracy and language skills. Sources of enrichment for the Uzbek language's lexicon. Classification of native and borrowed words specific to the field. Students identify native and borrowed terms used in their field. The practical session develops terminology analysis skills. Uzbek lexicography. Familiarization with monolingual and bilingual dictionaries. Students use Uzbek dictionaries to analyze the meanings of field-specific terms. The practical session develops lexicographic research skills. Working on a scientific article on current issues in the field Students analyze scientific articles and identify key terms and concepts. The practical session develops academic reading and terminology skills. Analysis of a literary work using the "Mind Map" method Students analyze a literary work using Mind Map technology. The practical session develops critical thinking and text analysis skills. Journalistic discourse style. Writing an essay on "My Chosen Field and the Media". Students analyze journalistic texts and write an essay on their professional field. The practical session develops analytical and writing skills. Preparing a video report on Termiz State University Students prepare a short video report about the university. The practical session develops presentation and communication skills. Write a review of one scientific article related to the field Students prepare a critical review of a scientific article. The practical session develops academic evaluation and writing skills. Prepare an annotation for a scientific work related to the field. Students write an annotation for a selected scientific publication. The practical session develops summarizing and academic writing skills. Prepare an illustrated presentation on "Uzbekistan's Biologists in the Global Perspective". Students prepare and deliver an illustrated presentation on the selected topic. The practical session develops research and presentation skills.
--	--

	Working with sample professional documents Students prepare and analyze professional documents used in their field. The practical session develops practical documentation skills. Research on the topic and the processes of conducting it Students carry out a small research task and document the research process. The practical session develops research methodology skills. Preparing a presentation on the field's innovative developments. Students prepare a presentation on recent innovations in their field. The practical session develops research and public speaking skills. Proper use of language capabilities in writing an initial scientific research. Students apply academic language and writing standards when preparing a research paper. The practical session develops scientific writing and communication skills.
Teaching Methods	The module is delivered through interactive lectures, inquiry-based and task-based learning, practical language workshops, seminars, group discussions, and project-based activities. Students develop professional communication skills by applying the Uzbek language in academic and workplace contexts, preparing official documents, technical reports, presentations, and professional correspondence. Independent learning and digital educational resources further promote critical thinking, effective written and oral communication, linguistic accuracy, and the appropriate use of Uzbek language norms in professional and interdisciplinary environments.
Prerequisites	Modern Uzbek literary language Stylistics Culture of speech Official Language Proficiency
Exams and Assessment	Type of examination (written, oral, project) and its weight Student performance is assessed through continuous assessment (20%), midterm assessment (30%), and a final assessment (50%). The continuous assessment (20%) evaluates students' participation in lectures and practical classes, classroom discussions, and the successful completion of independent study assignments, including presentations, reports, academic writing exercises, document preparation, literature reviews, and mini-projects related to the application of the Uzbek language in professional and scientific communication.

	The midterm assessment (30%) consists of a written test and the evaluation of independent study projects, such as analytical reports, presentations, academic papers, and practical assignments covering scientific writing, professional communication, terminology, document preparation, and official writing. The final assessment (50%) is a written examination consisting of theoretical and practical questions designed to assess students' knowledge of academic and professional Uzbek, scientific writing, official documentation, communication skills, and the application of language norms in professional contexts. Successful completion of all independent study assignments is required as an essential component of the module assessment.
Recommended Literature	1. Kadirova Z. Z. Application of the Uzbek Language in the Field. A Textbook. – Termez: "Globe Edit", 2024. – 110 p. 2. Korotkina I. B. Academic Writing: Process, Product. A textbook for universities. Moscow: Yurayt, 2021 3. Kuvshinskaya Y. M. , Zevakhina N. A. , Akhapkina Y. E. , Gordienko E. N. Academic Writing: From Research to Text. Textbook and Workbook for Universities. Moscow: Yurayt, 2022 4. Korotkina I. B. A Model for Teaching Academic Writing. A Textbook for Universities. Moscow: Yurayt, 2021 5. N. Jiyanova, O. Mo'ydinova. Speech Culture. T. : "Ilm ziyo zakovat", 2019. 221 p. 6. Qo'ng'urov R. , Begmatov E. , Tojiyev Yo. Fundamentals of Speech Culture and Stylistics, T. : "Fan", 1992, 254 pp. 7. Husanov, N. , Rasulov, Mo'ydinov, Q. The Culture of Speech and the Art of Oratory. T. : "IQTISOD-MOLIYA", 2006. 8. M. Akhmedova. Speech Culture and Professional Public Speaking. TATU, 2017, 114 p.
Responsible Lecturer	Zaynura Kadirova, PhD in Philology, Associate Professor.
Field	Description
Module Title	PHYSICAL CULTURE
Module Code	JMA203
Credit Points (ECTS)	3 Credits
Workload	Total workload: 90 hours Contact hours: 30 hours (30 hours Practical lessons) Self-Assignments: 60 hours
Frequency/Duration	IV semester

Module objective	To impart contemporary theoretical knowledge and practical skills related to physical culture and sports in Uzbekistan, including physical education, physical fitness, three-tier sports competitions (Umid nihollari, Barkamol avlod, Universiada, etc.), as well as methods and tools that promote the enhancement of specialized theoretical knowledge; to develop skills for self-directed physical improvement and the independent organization of mass health-improvement activities.
Module Learning Outcomes	Knowledge LO1. To provide students with the comprehensive knowledge to conduct training sessions for various sports, organize and analyze sports games, movement and ball-handling techniques in volleyball and handball, must have theoretical knowledge of offensive and defensive techniques and tactics; LO2. Must know methods for teaching and developing sports games, methods for organizing competitions, and ways to develop and train physical qualities; LO3. Knowledge acquired through oral responses and discussions during class must be applied in practice; LO4. Must know and be able to use the system of competition load assignments, and know how to organize and conduct the lesson and training process based on modern requirements; Skills LO5. To provide students with comprehensive knowledge for conducting training sessions in various sports, organizing and analyzing sports games, movement and ball-handling techniques in volleyball and handball, must have theoretical knowledge of offensive and defensive techniques and tactics; LO6. Must know methods for teaching and developing sports games, methods for organizing competitions, and ways to develop and train physical qualities; Qualifications LO7. Conducts work on selection in collaboration with specialists from disciplines closely related to other sports (physiology, anatomy, sports medicine, massage, psychology, etc.); LO8. Understands how to obtain rapid information and manage it effectively using modern technology during the training process;
Content	Practical lessons Assessment of physical fitness level. Current Norm (CN): 60 m Sprint. This practical session evaluates students' physical fitness through the Current Norm (CN) 60 m sprint test. Students develop speed, coordination, and running efficiency while learning standardized assessment procedures, performance

evaluation techniques, and the importance of physical fitness monitoring.

Track and Field Athletics. Teaching the standing long jump technique. This practical session develops fundamental athletics skills through instruction in the standing long jump technique. Students learn proper body positioning, coordinated take-off and landing mechanics, and effective movement patterns to improve lower-body power, balance, and overall athletic performance.

Teaching short-distance running technique. This practical session focuses on the correct execution of short-distance running techniques. Students practice starting positions, acceleration, stride mechanics, and finishing techniques while improving speed, coordination, and movement efficiency through systematic technical training and performance analysis.

Middle-distance running technique. Teaching 500-1000 m running. This practical session introduces the technical and physiological principles of middle-distance running. Students develop efficient running mechanics, pacing strategies, breathing techniques, and endurance while performing 500–1000 m running exercises to enhance aerobic capacity and overall physical performance.

Team Sports. Volleyball: rules of the game; teaching specialized ball exercises. This practical session introduces the fundamental rules of volleyball and develops essential technical skills through specialized ball exercises. Students practice passing, serving, setting, and coordinated team play while improving motor skills, communication, tactical awareness, and cooperation in competitive situations.

Basketball: rules of the game; teaching specialized ball exercises. This practical session introduces the fundamental rules of basketball and develops essential technical skills through specialized ball exercises. Students practice dribbling, passing, shooting, and defensive techniques while enhancing coordination, teamwork, decision-making, and game-related tactical awareness.

Handball: rules of the game; teaching specialized ball exercises. This practical session focuses on the basic rules of handball and the development of technical skills through specialized ball exercises. Students improve passing, catching, dribbling, shooting accuracy, and tactical cooperation while strengthening physical fitness and effective team communication.

Football: rules of the game; teaching specialized ball exercises. This practical session familiarizes students with the

fundamental rules of football and develops essential playing techniques through specialized ball exercises. Students practice passing, dribbling, ball control, shooting, and tactical movement, while improving teamwork, coordination, agility, and physical endurance.

Badminton: rules of the game; teaching specialized shuttlecock exercises. This practical session introduces the basic rules of badminton and develops technical proficiency through specialized shuttlecock exercises. Students practice racket grip, serving, footwork, stroke techniques, and court positioning while improving agility, reaction speed, coordination, and precision.

Table Tennis: rules of the game; teaching specialized ball exercises. This practical session develops students' understanding of the rules of table tennis while strengthening technical skills through specialized ball exercises. Students practice grip techniques, serving, forehand and backhand strokes, movement patterns, and tactical play to enhance coordination, concentration, and reaction time.

Teaching active games. This practical session introduces active games as an effective means of promoting physical fitness, teamwork, and recreational participation. Students learn the organization, rules, and instructional methods of various active games while developing agility, coordination, communication, and leadership skills.

Gymnastics. Teaching formation drills. This practical session focuses on teaching formation drills as a fundamental component of gymnastics. Students develop correct posture, body alignment, discipline, synchronization, and movement precision while performing individual and group formations in accordance with established instructional standards.

Teaching general developmental exercises. This practical session develops students' ability to perform and instruct general developmental exercises designed to improve overall physical fitness. Emphasis is placed on flexibility, strength, endurance, coordination, and the systematic organization of exercises for different age groups and fitness levels.

Teaching basic acrobatic exercises. This practical session introduces the fundamental techniques of basic acrobatic exercises. Students practice safe execution of rolls, balances, and elementary acrobatic movements while improving body control, flexibility, coordination, spatial awareness, and adherence to safety principles during physical activity.

	Teaching strength-development exercises on gymnastic apparatus. This practical session develops students' practical skills in performing strength-development exercises using gymnastic apparatus. Emphasis is placed on correct exercise techniques, safety requirements, muscular strength development, physical conditioning, and the effective use of gymnastic equipment in physical education classes. Self-Assignments: Preparation for practical sessions and homework. This independent study enables students to prepare systematically for practical sessions by reviewing theoretical concepts and completing assigned tasks. It strengthens understanding of physical education principles, enhances practical readiness, and promotes responsible learning habits and continuous self-improvement. Independent study of topics using educational literature This independent study encourages students to explore physical education topics through textbooks, scientific literature, and other academic resources. It develops independent learning skills, broadens theoretical knowledge, and strengthens the ability to interpret and apply evidence-based information in professional practice. Studying video materials on topics. This independent study utilizes educational video materials to reinforce theoretical knowledge and practical skills in physical education. Students analyze movement techniques, instructional strategies, and sport-specific activities, thereby improving observational skills, technical understanding, and reflective learning abilities. Preparing test questions. This independent study develops students' understanding of key concepts by designing test questions covering theoretical and practical aspects of physical education. The activity promotes critical thinking, knowledge organization, self-assessment, and the ability to evaluate learning outcomes effectively. Preparing presentations on assigned topics. This independent study requires students to prepare and deliver presentations on assigned physical education topics. It enhances research competence, communication skills, academic presentation techniques, and the ability to explain theoretical concepts and practical applications in a clear and professional manner.
Teaching Methods	The module is delivered through interactive practical sessions, practical demonstrations, Problem-Based Learning (PBL),

	Collaborative Learning, sports games, physical exercises, small-group activities, seminars, and individual and team-based learning tasks. Students develop practical skills in performing physical exercises and sports techniques, improve physical fitness and motor skills, and promote healthy lifestyle practices. Independent learning and digital educational resources further enhance theoretical knowledge, critical thinking, communication skills, and the ability to apply modern pedagogical approaches in physical education and sports.
Prerequisites	General Biochemistry and Sports Biochemistry (BI02. 06) Athletics, gymnastics (BI02. 08) National and Sports Games (BI02. 09)
Exams and Assessment	Type of Assessment (Practical Examination) and Its Weight Student performance is assessed through Continuous Assessment (50%) and a Final Practical Examination (50%). The Continuous Assessment (50%) is conducted throughout the semester. Of the allocated 50 points, 20 points are awarded for attendance, active participation, and performance in practical classes, while the remaining 30 points are awarded for the successful completion of Independent Study assignments. Performance in practical classes is assessed based on students' attendance, active participation, correct execution of physical exercises, demonstration of sports techniques and motor skills, compliance with safety requirements, physical fitness development, and achievement of the prescribed performance standards. The Independent Study component consists of five assignments, each worth 6 points, for a total of 30 points. Students are evaluated based on the quality of their presentations, understanding of theoretical concepts, completion of assigned learning activities, and timely submission of assignments. Assignments submitted after the specified deadline may not be accepted. The Final Practical Examination (50%) is conducted on the sports ground or in the sports hall and is designed to assess students' practical competence in physical education. Students demonstrate the correct performance of physical exercises, sports techniques, movement skills, and physical fitness standards covered during the module. Assessment is based on technical accuracy, physical performance, compliance with safety requirements, and the effective application of practical skills acquired throughout the course. Successful completion of all

	Independent Study assignments is required as an essential component of the module assessment.
Recommended Literature	1. B. K. Amanturdiyev. The Game of Football. A Textbook. Termiz, "Surxon Nashr", 2021. 167 pages 2. Q. Q. Xaqnazarov. Basketball: Sportivnaya podgotovka. O'quv qo'llanma. Termiz, "Surxon nashr" 2021 y. 132 bet 3. B. K. Amanturdiyev. The Game of Football. Textbook. Termez, "Surhon Nashr", 2021. 167 pp. 4. Mirziyoyev, Sh. M. Strategy of the New Uzbekistan. – Tashkent: "Uzbekiston", 2021 5. Law of the Republic of Uzbekistan "On Physical Culture and Sport" of September 4, 2015
Responsible Lecturer	Sh.M. Xudoyberdiyev - Dean, Faculty of Physical Culture, TerSPI; Professor
Field	Description
Module Title	PHYSICAL TRAINING
Module Code	JTA203
Credit Points (ECTS)	3 Credits
Workload	Total workload: 90 hours Contact hours: 30 hours (30 hours Practical lessons) Self-Assignments: 60 hours
Frequency/Duration	IV semester
Module objective	Physical culture of Uzbekistan, physical education, Uzbekistan sports, physical training, children's sports, the physical culture system, physical development, physical preparation, sports training, physical qualities, the three-stage sports competition "Hope Sprouts," "Barkamol avlod," "Universiada," and (etc.), modern theoretical knowledge, practical skills, and tools and methods that help develop specialized theoretical knowledge and enhance practical skills, self-physical improvement, organizing mass health promotion events, and teaching them for independent use
Module Learning Outcomes	Knowledge LO1. To provide students with the comprehensive knowledge to conduct training sessions in various sports, organize and analyze sports games, movement and ball-handling techniques in volleyball and handball, must have theoretical knowledge of offensive and defensive techniques and tactics; LO2. Must know methods for teaching and developing sports games, methods for organizing competitions, and ways to develop and train physical qualities;

	LO3. Knowledge acquired through oral responses and discussions during class must be applied in practice; LO4. Must know and be able to use the system of competition load assignments, and know how to organize and conduct the lesson and training process based on modern requirements; Skills LO5. To provide students with comprehensive knowledge for conducting training sessions in various sports, organizing and analyzing sports games, and the techniques of movement and ball control in volleyball and handball, must have theoretical knowledge of offensive and defensive techniques and tactics; LO6. Must know methods for teaching and developing sports games, methods for organizing competitions, and ways to develop and train physical qualities; Qualifications LO7. The selection process can be carried out in collaboration with specialists from other disciplines closely related to sports (physiology, anatomy, sports medicine, massage, psychology, etc.); LO8. Is able to obtain rapid information and manage it effectively using modern technology during the training process;
Content	Practical lessons Assessment of physical fitness level. Current Norm (CN): 60 m Sprint. This practical session evaluates students' physical fitness through the Current Norm (CN) 60 m sprint test. Students develop speed, coordination, and running efficiency while learning standardized assessment procedures, performance evaluation techniques, and the importance of physical fitness monitoring. Track and Field Athletics. Teaching the standing long jump technique. This practical session develops fundamental athletics skills through instruction in the standing long jump technique. Students learn proper body positioning, coordinated take-off and landing mechanics, and effective movement patterns to improve lower-body power, balance, and overall athletic performance. Teaching short-distance running technique. This practical session focuses on the correct execution of short-distance running techniques. Students practice starting positions, acceleration, stride mechanics, and finishing techniques while improving speed, coordination, and movement efficiency through systematic technical training and performance analysis. Middle-distance running technique. Teaching 500-1000 m running. This practical session introduces the technical and

physiological principles of middle-distance running. Students develop efficient running mechanics, pacing strategies, breathing techniques, and endurance while performing 500–1000 m running exercises to enhance aerobic capacity and overall physical performance.

Team Sports. Volleyball: rules of the game; teaching specialized ball exercises. This practical session introduces the fundamental rules of volleyball and develops essential technical skills through specialized ball exercises. Students practice passing, serving, setting, and coordinated team play while improving motor skills, communication, tactical awareness, and cooperation in competitive situations.

Basketball: rules of the game; teaching specialized ball exercises. This practical session introduces the fundamental rules of basketball and develops essential technical skills through specialized ball exercises. Students practice dribbling, passing, shooting, and defensive techniques while enhancing coordination, teamwork, decision-making, and game-related tactical awareness.

Handball: rules of the game; teaching specialized ball exercises. This practical session focuses on the basic rules of handball and the development of technical skills through specialized ball exercises. Students improve passing, catching, dribbling, shooting accuracy, and tactical cooperation while strengthening physical fitness and effective team communication.

Football: rules of the game; teaching specialized ball exercises. This practical session familiarizes students with the fundamental rules of football and develops essential playing techniques through specialized ball exercises. Students practice passing, dribbling, ball control, shooting, and tactical movement, while improving teamwork, coordination, agility, and physical endurance.

Badminton: rules of the game; teaching specialized shuttlecock exercises. This practical session introduces the basic rules of badminton and develops technical proficiency through specialized shuttlecock exercises. Students practice racket grip, serving, footwork, stroke techniques, and court positioning while improving agility, reaction speed, coordination, and precision.

Table Tennis: rules of the game; teaching specialized ball exercises. This practical session develops students' understanding of the rules of table tennis while strengthening technical skills through specialized ball exercises. Students

practice grip techniques, serving, forehand and backhand strokes, movement patterns, and tactical play to enhance coordination, concentration, and reaction time.

Teaching active games. This practical session introduces active games as an effective means of promoting physical fitness, teamwork, and recreational participation. Students learn the organization, rules, and instructional methods of various active games while developing agility, coordination, communication, and leadership skills.

Gymnastics. Teaching formation drills. This practical session focuses on teaching formation drills as a fundamental component of gymnastics. Students develop correct posture, body alignment, discipline, synchronization, and movement precision while performing individual and group formations in accordance with established instructional standards.

Teaching general developmental exercises. This practical session develops students' ability to perform and instruct general developmental exercises designed to improve overall physical fitness. Emphasis is placed on flexibility, strength, endurance, coordination, and the systematic organization of exercises for different age groups and fitness levels.

Teaching basic acrobatic exercises. This practical session introduces the fundamental techniques of basic acrobatic exercises. Students practice safe execution of rolls, balances, and elementary acrobatic movements while improving body control, flexibility, coordination, spatial awareness, and adherence to safety principles during physical activity.

Teaching strength-development exercises on gymnastic apparatus. This practical session develops students' practical skills in performing strength-development exercises using gymnastic apparatus. Emphasis is placed on correct exercise techniques, safety requirements, muscular strength development, physical conditioning, and the effective use of gymnastic equipment in physical education classes.

Self-Assignments:

Preparation for practical sessions and homework. This independent study enables students to prepare systematically for practical sessions by reviewing theoretical concepts and completing assigned tasks. It strengthens understanding of physical education principles, enhances practical readiness, and promotes responsible learning habits and continuous self-improvement.

Independent study of topics using educational literature

	This independent study encourages students to explore physical education topics through textbooks, scientific literature, and other academic resources. It develops independent learning skills, broadens theoretical knowledge, and strengthens the ability to interpret and apply evidence-based information in professional practice. Studying video materials on topics, This independent study utilizes educational video materials to reinforce theoretical knowledge and practical skills in physical education. Students analyze movement techniques, instructional strategies, and sport-specific activities, thereby improving observational skills, technical understanding, and reflective learning abilities. Preparing test questions. This independent study develops students' understanding of key concepts by designing test questions covering theoretical and practical aspects of physical education. The activity promotes critical thinking, knowledge organization, self-assessment, and the ability to evaluate learning outcomes effectively. Preparing presentations on assigned topics. This independent study requires students to prepare and deliver presentations on assigned physical education topics. It enhances research competence, communication skills, academic presentation techniques, and the ability to explain theoretical concepts and practical applications in a clear and professional manner.
Teaching Methods	The module is delivered through interactive practical sessions, practical demonstrations, Problem-Based Learning (PBL), Collaborative Learning, sports games, physical exercises, small-group activities, seminars, and individual and team-based learning tasks. Students develop practical skills in performing physical exercises and sports techniques, improve physical fitness and motor skills, and promote healthy lifestyle practices. Independent learning and digital educational resources further enhance theoretical knowledge, critical thinking, communication skills, and the ability to apply modern pedagogical approaches in physical education and sports.
Prerequisites	General Psychology Physiology and Sports Physiology
Exams and Assessment	Type of Assessment (Practical Examination) and Its Weight Student performance is assessed through Continuous Assessment (50%) and a Final Practical Examination (50%). The Continuous Assessment (50%) is conducted throughout the semester. Of the allocated 50 points, 20 points are awarded for

	attendance, active participation, and performance in practical classes, while the remaining 30 points are awarded for the successful completion of Independent Study assignments. Performance in practical classes is assessed based on students' attendance, active participation, correct execution of physical exercises, demonstration of sports techniques and motor skills, compliance with safety requirements, physical fitness development, and achievement of the prescribed performance standards. The Independent Study component consists of five assignments, each worth 6 points, for a total of 30 points. Students are evaluated based on the quality of their presentations, understanding of theoretical concepts, completion of assigned learning activities, and timely submission of assignments. Assignments submitted after the specified deadline may not be accepted. The Final Practical Examination (50%) is conducted on the sports ground or in the sports hall and is designed to assess students' practical competence in physical education. Students demonstrate the correct performance of physical exercises, sports techniques, movement skills, and physical fitness standards covered during the module. Assessment is based on technical accuracy, physical performance, compliance with safety requirements, and the effective application of practical skills acquired throughout the course. Successful completion of all Independent Study assignments is required as an essential component of the module assessment.
Recommended Literature	6. B. K. Amanturdiyev. The Game of Football. A Textbook. Termiz, "Surxon Nashr", 2021. 167 pages 7. Q. Q. Xaqnazarov. Basketball: Sportivnaya podgotovka. O'quv qo'llanma. Termiz, "Surxon nashr" 2021 y. 132 bet 8. B. K. Amanturdiyev. The Game of Football. Textbook. Termez, "Surhon Nashr", 2021. 167 pp. 9. Mirziyoyev, Sh. M. Strategy of the New Uzbekistan. – Tashkent: "Uzbekiston", 2021 10. Law of the Republic of Uzbekistan "On Physical Culture and Sport" of September 4, 2015
Responsible Lecturer	A.X. Sharipov - Department of Sports Activities and Management, Termez State University; Associate Professor

3rd ACADEMIC YEAR

Field	Description
Module Title	ALGORITHM DESIGN
Module Code	ALL1506
Credit Points (ECTS)	6 Credits
Workload	Total workload: 180 hours Contact hours: 60 hours (30 hours Lecture, 30 hours Practical lessons) Self-Assignments: 120 hours
Frequency/Duration	V semester
Module objective	To identify the most efficient algorithm from among those that can be applied to solve the main problem of the "Algorithm Design" course, acquiring knowledge and skills on a range of theoretical and practical issues, such as evaluating the complexity metrics of newly developed or existing algorithms. The objectives of this course are to develop in students the following knowledge.
Module Learning Outcomes	Knowledge: LO 1. Must have knowledge of the relationships between algorithm design and the interactions among computing machine components, as well as the design stages. LO 2. Knowledge and ability to use the logical structure of the components and tools that constitute algorithm design. LO 3. Must have practical application development skills, such as writing small program modules in a low-level programming language (e. g. , assembler). LO 4. General Information: The Software Engineering major aims to prepare a new generation of specialists in the field of digital technologies. The program's goal is to provide students with the knowledge and skills necessary to become leaders in the field of software engineering. In terms of skills LO 5. Become proficient in developing algorithms for various problems and expressing them in flowchart and pseudocode. LO 6. Will be able to analyze the time and memory complexity of algorithms and evaluate their efficiency. LO 7. Will have the ability to apply sorting, searching, recursion, dynamic programming, and graph algorithms to practical problems. LO 8. Skilled in programming, testing, and optimizing algorithms using modern programming languages.

	Competencies: LO 9. Will have the competence to develop efficient algorithms for solving complex practical and scientific problems and to implement them using software tools. LO 10. Will be able to analyze the efficiency of algorithms, compare different solutions, and select the optimal algorithmic approach. LO 11. Will be competent in the judicious use of data structures and algorithms in software projects, and in testing and optimizing algorithms.
Content	Lectures Introduction to Algorithm Design. Evaluating algorithms in terms of time and space. Horner's scheme for evaluating polynomials. Introduces the fundamental concepts and goals of algorithm design. Explains criteria for evaluating algorithms based on time and space complexity. Demonstrates Horner's scheme as an efficient method for polynomial evaluation. Linear algorithms. Cycles. Approximate methods for calculating integrals, their efficiency. Multiplying matrices. Calculating determinants. Algorithm executor. Reviews the structure and characteristics of linear algorithms and loop constructs. Discusses approximate numerical integration methods and their efficiency trade-offs. Covers algorithms for matrix multiplication and determinant calculation. Branching algorithms. Methods for approximate solution of algebraic and transcendental equations. Evaluation of algorithm efficiency. Iterative loops. Explains the structure and logic of branching algorithms. Reviews approximate methods for solving algebraic and transcendental equations. Discusses criteria and techniques for evaluating iterative algorithm efficiency. The secant method. The Newton method. Explains the theoretical basis and iterative procedure of the secant method. Introduces Newton's method for root-finding and its convergence properties. Compares the efficiency and applicability of both methods. Simple iteration algorithms, programs. Reviews the principles underlying simple iteration methods for equation solving. Discusses convergence conditions required for iterative methods to succeed. Demonstrates the implementation of simple iteration algorithms as working programs. Mathematical models of linear programming problems, economic analysis. Objective function. Dual problem.

Introduces the formulation of linear programming problems and their economic applications. Explains the role and structure of the objective function. Discusses the concept of the dual problem and its relationship to the primal problem.

The simplex method algorithm for solving linear programming problems and its analysis. Explains the step-by-step procedure of the simplex method algorithm. Discusses how the method navigates feasible solution vertices to reach optimality. Reviews the computational efficiency and limitations of the simplex approach.

Economic analysis of the solution obtained by the simplex method. Conclusion and recommendations. Explains how to interpret simplex method solutions in economic terms. Discusses sensitivity analysis and its role in decision-making. Reviews approaches for formulating practical conclusions and recommendations from optimization results.

Number of Baseline Solutions and Construction Method of ChDM. Explains the concept and significance of baseline (basic feasible) solutions in linear programming. Discusses methods for determining the number of possible baseline solutions. Introduces the construction method for building such solutions systematically.

Transport problem. Introduces the formulation and structure of the classical transportation problem. Discusses methods for finding an initial feasible solution, such as the northwest corner method. Reviews optimization techniques for minimizing transportation costs.

Game Theory. Introduces fundamental concepts of game theory, including players, strategies, and payoffs. Discusses classifications of games such as zero-sum and non-zero-sum games. Reviews solution concepts including Nash equilibrium and optimal strategies.

Fourier row-based digital information spreading algorithm. Reliability assessment. Explains the use of Fourier series in digital information spreading algorithms. Discusses methods for assessing the reliability of such algorithms. Reviews practical considerations in implementing Fourier-based signal processing techniques.

Algorithm for separating the signal's dominant harmonics. Spectral analysis. Introduces techniques for identifying and isolating dominant harmonics within a signal. Explains the principles of spectral analysis applied to signal processing.

Reviews practical applications of harmonic separation in engineering contexts.

Least squares method in statistical modeling. Dynamic programming. Explains the least squares method as a technique for statistical model fitting. Introduces the principles and applications of dynamic programming. Discusses how both methods contribute to solving optimization and estimation problems.

Algorithm and program for building linear and quadratic models. Reviews algorithms for constructing linear and quadratic mathematical models from data. Discusses implementation of such algorithms as working programs. Explores practical applications of linear and quadratic modeling in analysis and prediction.

Practical lessons

Designing algorithms. Evaluating the correctness and efficiency of algorithms. Algorithm for finding the roots of a quadratic equation. Heron's formula for the area of a triangle. Practices the process of designing algorithms from problem statement to implementation. Applies methods for verifying algorithm correctness and assessing efficiency. Implements the quadratic root-finding algorithm and Heron's formula as working programs.

Sorting array elements. Algorithm for finding the maximum and minimum elements in a matrix. Matrix multiplication program. Evaluating the order of operations. Implements sorting algorithms and analyzes their step-by-step execution. Codes an algorithm to locate maximum and minimum elements within a matrix. Writes and evaluates a matrix multiplication program in terms of operation count.

Approximate integration methods. Choosing a step size that ensures the required accuracy. Implements approximate numerical integration techniques such as the trapezoidal or Simpson's rule. Analyzes how step size selection affects computational accuracy. Tests different step sizes to balance accuracy and computational cost.

In solving algebraic and transcendental equations, bisection and iteration methods. Roots and Newton's methods for solving equations. Rate of convergence. Implements the bisection and simple iteration methods for equation solving. Codes Newton's method and compares its

performance to other techniques. Analyzes the rate of convergence across the different methods tested.

Linear programming problem. Mathematical model of the problem. Solution for a linear programming problem, optimal solution, geometric method for finding it. Formulates a linear programming problem as a mathematical model. Solves the problem using the geometric method for two-variable cases. Interprets the optimal solution in the context of the original problem.

Dual problem for a linear programming problem, its economic analysis, the duality gap condition. Simplex method algorithm and its program for solving linear programming problems. Constructs the dual of a given linear programming problem and analyzes its economic meaning. Examines the duality gap condition between primal and dual solutions. Implements the simplex method as a working program to solve linear programming problems.

Developing conclusions and economic recommendations based on the solution of a linear programming problem. Examples of formulating an economic problem based on a mathematical model. Interprets simplex method results in terms of practical economic outcomes. Formulates evidence-based conclusions and recommendations from the solution. Practices translating real economic scenarios into linear programming models.

Expansion of a table function into a Fourier series. Calculation of the Fourier coefficients. Selection of the number of series terms. Selection of an approximate integration formula and its accuracy assessment. Determination of the dominant harmonics of digital signals based on the Fourier series. Computes Fourier coefficients for a given tabulated function.

Division of an information stream into segments. Dynamic programming. Linear model. Application of the least squares method in building the mathematical model of a process. Applies dynamic programming techniques to segment an information stream efficiently.

Quadratic and inverse proportional relationship models. Constructs quadratic mathematical models to represent nonlinear relationships in data. Builds inverse proportional models for appropriate datasets.

Designing algorithms that operate according to the "divide and conquer" principle. An algorithm for sorting a set of

elements by a certain key. Applies the divide-and-conquer strategy to design an efficient algorithm. Implements a sorting algorithm based on a specified key.

Paths in connected graphs and their evaluation by cost (distance). Greedy algorithms. Algorithm for determining the shortest path. Estimating its size in terms of the number of alternatives. Analyzes paths within connected graphs and evaluates them by associated cost or distance.

Kruskal's algorithm. Prim's algorithm. Hoffman's algorithm. Implements Kruskal's algorithm to construct a minimum spanning tree. Implements Prim's algorithm and compares its performance to Kruskal's approach. .

NP-complete problems. Reduction methods for NP-complete problems. Reviews the definition and characteristics of NP-complete problems. Practices reduction techniques for transforming one NP-complete problem into another.

Graph edge-deletion problem. The traveling salesman problem. Calculating the sum of rows. Calculating the sum of subsets. Analyzes the graph edge-deletion problem and its computational characteristics. Implements approaches to the traveling salesman problem for small instances.

Independent Study

Static and dynamic complexity measures of algorithms. Time and space complexity challenges. Reviews the distinction between static and dynamic complexity measures. Discusses common challenges in accurately assessing time and space complexity.

Evaluating algorithms in worst-case and average cases. Explains the difference between worst-case and average-case complexity analysis. Discusses scenarios where each type of analysis is most informative.

Flat and logarithmic benchmarks for evaluating the time and space complexity of algorithms. Introduces the use of flat and logarithmic growth benchmarks in complexity analysis.

Ways of representing sequences, sets, trees, and graphs. Reviews common data structure representations for sequences, sets, trees, and graphs. Discusses the advantages and limitations of each representation method.

Comparison of approximate integration methods in terms of accuracy and computational complexity. Compares common approximate integration methods based on accuracy. Discusses the computational complexity trade-offs among these methods.

Evaluating methods for approximating solutions of algebraic and transcendental equations based on convergence rate. Reviews convergence rate as a criterion for evaluating equation-solving methods. Compares convergence behavior across bisection, iteration, and Newton's methods.

Approximate solution methods for systems of linear algebraic equations. Convergence conditions. Reviews iterative methods for solving systems of linear algebraic equations. Discusses the mathematical conditions required for convergence.

Canonical form of linear programming problems. Simplex method. Explains the process of converting a linear programming problem into canonical form. Reviews the role of canonical form in enabling the simplex method.

The Fourier method in digital information processing. Spectral analysis. Reviews the application of Fourier methods in digital information processing. Discusses the principles underlying spectral analysis of signals. Explores practical use cases where Fourier-based analysis is essential.

Dynamic programming methods in statistical data processing and forecasting. Reviews how dynamic programming techniques apply to statistical data processing. Discusses their use in forecasting models and trend analysis.

Rotating graphs horizontally and vertically (checking). Reviews methods for transforming graph representations through rotation. Discusses verification techniques for confirming transformation correctness.

Kruskal's greedy algorithm for constructing the cheapest spanning tree of graphs. Explains the greedy strategy underlying Kruskal's algorithm. Reviews the step-by-step process for constructing a minimum spanning tree. Discusses the algorithm's efficiency and correctness guarantees.

Prim-Dijkstra algorithm. Time complexity analysis. Reviews the operating principles of the Prim and Dijkstra algorithms. Compares their respective time complexity characteristics. Discusses scenarios where each algorithm is most appropriately applied.

"Divide and Conquer" algorithms. Reviews the general strategy underlying divide-and-conquer algorithm design. Discusses common examples such as merge sort and quicksort. Explores the efficiency benefits this strategy provides for suitable problems.

P and NP classes, the concept of NP-complete problems. Introduces the classification of problems into P and NP complexity classes. Explains the defining characteristics of NP-complete problems. Discusses the significance of this classification for algorithm design and computational theory.

Criteria for evaluating algorithms. Examples of time and space complexity evaluations. Reviews standard criteria used to evaluate algorithm performance. Provides examples illustrating time and space complexity assessment. Discusses how these criteria inform algorithm comparison and selection.

Newton–Cotes Formulas for Approximate Integration. Concept and Order of Error. Introduces the family of Newton-Cotes formulas for numerical integration. Discusses the concept of error order associated with each formula. Reviews practical considerations for selecting an appropriate formula.

Gaussian formulas for the approximate evaluation of integrals. Concept and order of error. Efficiency. Introduces Gaussian quadrature methods for numerical integration. Discusses the error order and accuracy advantages of these formulas. Reviews their computational efficiency compared to other integration methods.

Abbreviations in sets. Examples of them and their practical applications. Reviews common notational abbreviations used in set theory. Provides examples illustrating their use in mathematical and computational contexts. Discusses practical applications where such notation simplifies problem representation.

Comparing the efficiency of bisection and root-finding methods for approximating solutions of algebraic and transcendental equations. Compares the computational efficiency of the bisection method against other root-finding techniques. Discusses trade-offs between reliability and convergence speed. Reviews scenarios favoring one method over another.

Comparing the effectiveness of the iterative and Newton methods for approximating solutions to algebraic and transcendental equations. Compares convergence speed and reliability between iterative and Newton's methods. Discusses conditions under which each method performs best. Reviews practical examples illustrating their comparative effectiveness.

Simple Iteration in Approximate Solution of Algebraic and Transcendent Equations. Recommendations for constructing the simple iteration method and its efficient variants for

approximating solutions of algebraic and transcendental equations. Reviews the theoretical basis of the simple iteration method. Discusses recommendations for constructing efficient iteration formulas. Explores variants designed to improve convergence speed and reliability.

Matrix norm and its determination methods. Introduces the concept of matrix norms and their mathematical definitions.

Reviews common methods for calculating different types of matrix norms. Discusses the role of matrix norms in numerical analysis and error estimation.

Simple iteration and Zeydel methods for solving systems of linear algebraic equations, and their convergence conditions. Reviews the simple iteration and Seidel (Gauss-Seidel) methods for solving linear systems. Discusses the mathematical conditions required for convergence in each method. Compares their relative efficiency and reliability.

The concept of a reference solution for a linear programming problem and methods for determining them.

Introduces the concept of a reference (basic feasible) solution in linear programming. Reviews methods for systematically determining such solutions. Discusses their role in the broader simplex method process.

Duality problem for the linear programming problem, its formulation, and analysis of its economic meaning.

Reviews the formulation process for constructing a dual linear programming problem. Discusses methods for analyzing the dual's economic interpretation. Explores the relationship between primal and dual solution values.

Spectral analysis of digital information and its application to solving practical problems.

Reviews the principles of spectral analysis applied to digital information. Discusses practical problems that benefit from spectral analysis techniques. Explores examples of spectral methods applied in engineering and data processing.

Methods for constructing a graph tree and assessing complexity levels.

Reviews methods for constructing tree structures from graph data. Discusses techniques for assessing the complexity level of resulting trees. Explores practical applications of tree construction in algorithmic problems.

Estimating the number of operations required to compute determinants.

Reviews methods for calculating matrix determinants and their associated computational cost. Discusses how operation count scales with matrix size. Explores

	strategies for reducing computational overhead in determinant calculation. The number of operations required to solve systems of linear algebraic equations exactly. Estimating the number of operations. Reviews exact solution methods for systems of linear algebraic equations. Discusses how the number of required operations scales with system size. Explores the computational implications of this scaling for large systems.
Teaching Methods	The module is delivered through interactive lectures, problem-based and inquiry-based learning, computational laboratory sessions, algorithm design workshops, computer simulations, seminars, group discussions, and project-based activities. Students design, implement, and analyze algorithms for numerical methods, optimization, and graph problems using specialized software, and develop analytical, computational, and problem-solving skills. Independent learning and digital educational resources further promote critical thinking, scientific communication, and the application of modern algorithm design concepts.
Prerequisites	Algebra, Probability Theory, Mathematical Logic Basic knowledge of programming languages such as Python, Java, or C++ Data Structures English Language
Exams and Assessment	Student performance is assessed through continuous assessment (20%), midterm assessment (30%), and a final written examination (50%). The continuous assessment (20%) evaluates classroom participation, laboratory performance, algorithm implementation exercises, seminar presentations, and the successful completion of independent study assignments, including algorithm complexity analysis reports, convergence rate comparisons, spanning tree construction exercises, and NP-completeness case studies. The midterm assessment (30%) consists of written assignments, short in-class quizzes, and the evaluation of independent study projects, such as a report comparing numerical method efficiency, a linear programming duality analysis, a presentation on Fourier-based spectral analysis, and an essay on divide-and-conquer algorithm design. The final assessment (50%) is a written examination consisting of theoretical questions and problem-solving tasks designed to assess students' understanding of algorithm design, analytical

	skills, and their ability to apply computational methods to practical problems. Successful completion of all independent study assignments is required as an essential component of the module assessment.
Recommended Literature	1. Thomas Cormen, Charles Leiserson, Ronald Rivest, Clifford Stein. Algorithm Design and Analysis. Moscow–St. Petersburg–Kyiv. Izd. dom "Vilyams," 2005. 1293 pp. 2. Levetan, Anany. Introduction to the Design & Analysis of Algorithms. 3rd ed. Villanova University. New Jersey. 2012. 93 pages. 3. Rod Stevens. Ready-made Algorithms. M. : DMK Press. Piter 2014. 384 pp. 4. Computer Algorithms by Horowitz E. , Sahni. , Rajasekaran S. , Galgotia Publications, 2001. 5. Introduction to the Design and Analysis of Algorithms A Strategic Approach, R. C. T. Lee, S. S. 6. Tseng, R. C. Chang & Y. T. Tsai, TMH, 2005.
Responsible Lecturer	Mamatkabilov Abdilmajid Khujamuratovich Doctor of Philosophy (PhD) in Physical and Mathematical Sciences
Field	Description
Module Title	SOFTWARE SYSTEM DESIGN
Module Code	DTT1706
Credit Points (ECTS)	5 Credits
Workload	Total workload: 180 hours Contact hours: 90 hours (46 hours Lecture, 44 hours Practical lessons) Self-Assignments: 90 hours
Frequency/Duration	VI semester
Module objective	The goal of the "Software Systems Design" course is to develop students' theoretical knowledge and practical skills in the analysis, design, and development of modern software systems. During the course, students will study the software life cycle, architectural models, modeling techniques (UML, ERD), design patterns, and quality and security criteria.
Module Learning Outcomes	Knowledge: LO1. The student will learn the basic concepts, principles, and modern approaches of software system design (SDLC, architecture, modeling). LO 2. The student will learn the stages of the software life cycle (SDLC), their tasks, and the differences between various models (waterfall, spiral, agile).

	LO3. The student will study the theoretical foundations of UML diagrams, the ER model, and visual modeling tools, as well as their areas of application. LO4. The student acquires fundamental knowledge of software quality criteria, design patterns, and security fundamentals. Skills & Application LO5. The student can analyze requirements for software systems and distinguish between functional and non-functional requirements. LO 6. The student will be able to independently create UML diagrams (Use Case, Sequence, Activity, Class) and ER diagrams. LO 7. The student will be able to create modular and architectural designs for software systems (MVC, Layered, Pipe & Filter). LO 8. The student can assess software quality, calculate metrics, and perform basic code analysis (static analysis). Professional Competencies: LO 9. The student acquires the skills to design a software system for real-world projects, select its architecture, and develop it in stages. LO 10. The student will acquire the skills to apply design patterns in software systems to create an efficient, extensible, and reusable code structure. LO 11. The student will be able to analyze software quality and security, justify design decisions, and develop optimized solutions.
Content	Lectures Introduction to Software Systems. Introduces the fundamental concepts and terminology of software systems. Discusses the scope and purpose of studying software system design. Reviews examples of common software systems across different domains. Software Development Process. Explains the typical stages involved in developing software from concept to deployment. Discusses roles and responsibilities within a software development team. Reviews common process models used to organize development activities. Software Development Life Cycle (SDLC). Introduces the phases of the SDLC, including requirements, design, implementation, testing, and maintenance. Discusses various SDLC models such as waterfall, iterative, and agile. Reviews criteria for selecting an appropriate SDLC model for a given project.

Evolution of Software System Modules. Traces how software modules evolve over successive versions and releases. Discusses factors driving module evolution, such as changing requirements and technology. Reviews strategies for managing module evolution while maintaining system stability.

Software System Design Criteria. Reviews key criteria used to guide software system design decisions. Discusses trade-offs between criteria such as performance, maintainability, and scalability. Explores how design criteria influence architectural choices.

Fundamentals of Software Evaluation. Introduces methods for evaluating software quality and performance. Discusses common evaluation criteria such as reliability, usability, and efficiency. Reviews approaches for conducting systematic software evaluation.

Software System Security. Reviews fundamental security concepts relevant to software system design. Discusses common vulnerabilities and threats facing software systems. Explores practices for incorporating security considerations into the design process.

Good software characteristics. Reviews the defining characteristics of well-designed software. Discusses qualities such as reliability, maintainability, and usability. Explores how these characteristics guide design and evaluation decisions.

Software System Design Models. Introduces common modeling approaches used in software system design. Discusses the purpose and benefits of using models throughout development. Reviews examples of design models applied to real software projects.

Concepts of Visual Modeling. Explains the role of visual modeling in representing software system structure and behavior. Discusses common visual modeling notations and tools.

Concepts of Decomposition and Modularity. Introduces the principles of decomposing complex systems into manageable modules. Discusses the benefits of modularity for maintainability and reuse.

Problems in analyzing the subject area. Discusses common challenges encountered when analyzing a software system's subject area. Reviews techniques for gathering and clarifying domain requirements. Explores strategies for resolving ambiguity in subject area analysis.

ER diagrams and data modeling operations. Introduces entity-relationship diagrams as a tool for representing data

structure. Discusses common data modeling operations, including normalization. Reviews practical examples of constructing ER diagrams for real systems.

UML Feature Diagrams. Introduces UML notation for representing software system features. Discusses how feature diagrams support requirements analysis and communication. Reviews examples illustrating feature diagram construction.

Architectural Pattern Structure. **Reviews common architectural patterns used in software system design.** Discusses the structural characteristics defining each pattern. Explores criteria for selecting an appropriate architectural pattern for a given system.

Visual modeling process. Explains the step-by-step process of developing visual models for a software system. Discusses how visual modeling integrates with broader design activities. Reviews best practices for maintaining consistency across visual models.

Structural Design Patterns. Introduces common structural design patterns and their intended use cases. Discusses how these patterns organize relationships between classes and objects. Reviews examples illustrating structural pattern implementation.

View templates and template properties. Reviews the concept of view templates within software system design. Discusses key properties and configuration options associated with templates. Explores practical applications of view templates in system development.

Main and additional templates. Distinguishes between main and additional templates within a software design context. Discusses how these template types are used together in system construction. Reviews examples illustrating their combined application.

Evaluation of Software System Decisions. Reviews methods for evaluating design decisions made throughout software development. Discusses criteria for assessing the impact of such decisions on system quality. Explores techniques for documenting and justifying design choices.

Understanding Software Metrics. Introduces common software metrics used to quantify system quality and complexity. Discusses how metrics support objective evaluation of software artifacts. Reviews examples of applying metrics to real software projects.

An Introduction to Algorithmic Modeling. Introduces the concept of representing algorithms through formal modeling techniques. Discusses the benefits of algorithmic modeling for design clarity and verification. Reviews examples of algorithmic models applied to software components.

Performing static code analysis and testing. Explains the purpose and techniques of static code analysis. Discusses common testing approaches used alongside static analysis. Reviews how these practices contribute to identifying defects early in development.

Practical lessons

Software Systems Analysis. Analyzes the structure and requirements of a sample software system. Identifies key stakeholders and their functional needs. Documents findings as a foundation for subsequent design work.

Software Life Cycle Modeling. Models the stages of a software project using a selected SDLC approach. Compares how different lifecycle models apply to the same project scenario. Discusses the practical implications of lifecycle model selection.

Requirements Gathering and Analysis. Practices techniques for eliciting requirements from stakeholders. Organizes and categorizes gathered requirements for clarity. Analyzes requirements for completeness, consistency, and feasibility.

Comparison of monolithic and modular architecture. Examines the structural characteristics of monolithic and modular software architectures. Compares their respective advantages and limitations for a given project. Discusses scenarios favoring one architectural approach over the other.

Evaluation of DTT design criteria. Applies established design criteria to evaluate a sample software design. Identifies strengths and weaknesses relative to the criteria. Proposes improvements based on the evaluation results.

Conduct risk identification and assessment. Identifies potential risks associated with a software development project. Assesses the likelihood and impact of identified risks. Proposes mitigation strategies for the most significant risks.

Working with the DTT security model. Applies a security model framework to a sample software system. Identifies potential vulnerabilities using the model's criteria. Proposes security measures to address identified weaknesses.

UML Basics and Working with Use Case Diagrams. Reviews the core notation elements of UML. Constructs use case

diagrams to represent system functionality. Analyzes actor-system interactions depicted in the diagrams.

System Design Based on Decomposition. Decomposes a complex system into smaller, manageable modules. Evaluates the resulting decomposition for maintainability and clarity.

Creating an ER diagram. Constructs an entity-relationship diagram for a sample data domain. Identifies entities, attributes, and relationships accurately. Refines the diagram to eliminate redundancy and ambiguity.

Working with UML sequence and activity diagrams. Constructs UML sequence diagrams to represent object interactions over time. Builds activity diagrams to model workflow and control flow. Compares the two diagram types for representing different system aspects.

Designing the MVC architecture. Designs a software component following the Model-View-Controller pattern. Implements clear separation of concerns across the three layers. Evaluates the design for maintainability and scalability benefits.

Working with the Layered Architecture Model. Designs a system following a layered architectural approach. Defines responsibilities and interactions between architectural layers. Evaluates the design for modularity and separation of concerns.

Working with Pipe and Filter Models. Designs a system using the pipe-and-filter architectural pattern. Implements data processing stages as independent filter components. Evaluates the design for flexibility and reusability.

Working with Creational Patterns. Implements common creational design patterns in a sample project. Analyzes how each pattern addresses object creation challenges. Compares patterns to determine suitability for different scenarios.

Working with the Builder and Prototype patterns. Implements the Builder pattern to construct complex objects step by step. Implements the Prototype pattern for object cloning scenarios. Compares the two patterns in terms of use cases and benefits.

Working with Feature Templates. Applies feature template concepts to represent system functionality. Constructs a feature diagram based on defined templates. Analyzes how templates support requirements traceability.

Working with additional templates. Explores supplementary template types used in software design. Applies these templates to extend or refine an existing design. Evaluates their contribution to overall design clarity.

Performing software metrics calculations. Calculates common software metrics for a sample codebase. Interprets metric results to assess code quality and complexity. Proposes improvements based on metric findings.

Study of the algorithmic modeling process. Constructs an algorithmic model representing a software component's logic. Verifies the model's correctness against expected behavior. Discusses how algorithmic modeling supports the design process.

Perform static code analysis. Applies static analysis tools to a sample codebase. Identifies potential defects, code smells, and vulnerabilities. Proposes corrections based on analysis findings.

Learning comprehensive evaluation of a software system. Conducts a comprehensive evaluation of a software system using multiple criteria. Synthesizes findings from design, security, and metric analyses. Presents an overall assessment with recommendations for improvement.

:

Software Systems Analysis. Reviews theoretical frameworks used to analyze existing software systems. Studies real-world case examples of system analysis reports. Prepares a written summary of analytical findings for review.

Software Life Cycle Modeling. Studies alternative SDLC models beyond those covered in lectures. Compares their suitability for different project types and scales. Prepares a comparative summary of lifecycle model characteristics.

Requirements Gathering and Analysis. Reviews additional techniques for eliciting and documenting requirements. Studies examples of requirements specifications from real projects. Prepares a critique of a sample requirements document.

Comparison of monolithic and modular architecture. Researches case studies contrasting monolithic and modular system implementations. Analyzes long-term maintenance outcomes associated with each approach. Prepares a written comparison supported by researched examples.

Evaluation of DTT design criteria. Studies supplementary design criteria frameworks beyond those introduced in class. Applies these criteria to a self-selected software example. Prepares an evaluation report summarizing the findings.

Conduct risk identification and assessment. Researches additional risk assessment methodologies used in industry. Applies a chosen methodology to a hypothetical project scenario.

Prepares a risk register documenting identified risks and mitigation plans.

Working with the DTT security model. Studies the theoretical foundations of the security model in greater depth. Researches real-world security incidents relevant to the model's concerns. Prepares a summary connecting theory to practical security lessons.

UML Basics and Working with Use Case Diagrams. Reviews additional UML notation elements not covered in class. Studies example use case diagrams from published software documentation. Prepares an original use case diagram for a self-chosen system.

System Design Based on Decomposition. Researches decomposition strategies used in large-scale software projects. Studies trade-offs between different decomposition granularities. Prepares a written analysis of a decomposition case study.

Creating an ER diagram. Studies advanced ER modeling concepts such as weak entities and inheritance. Reviews examples of complex ER diagrams from real database systems. Prepares an original ER diagram for a self-selected data domain.

Working with UML sequence and activity diagrams. Reviews additional notation conventions for sequence and activity diagrams. Studies examples illustrating complex object interactions. Prepares original diagrams for a self-chosen system scenario.

Designing the MVC architecture. Researches variations of the MVC pattern used in modern frameworks. Studies example implementations across different programming languages. Prepares a written analysis comparing MVC variants.

Working with the Layered Architecture Model. Studies real-world examples of layered architecture in enterprise systems. Researches common pitfalls encountered when applying this model. Prepares a summary of best practices for layered design.

Working with Pipe and Filter Models. Researches applications of the pipe-and-filter pattern in data processing systems. Studies examples from stream processing and compiler design. Prepares a written analysis of the pattern's strengths and limitations.

Working with Creational Patterns. Studies additional creational patterns beyond those covered in class. Reviews example implementations from open-source projects. Prepares a summary comparing pattern use cases.

	Working with the Builder and Prototype patterns. Researches advanced use cases for the Builder and Prototype patterns. Studies example implementations in different programming contexts. Prepares a written comparison of the two patterns' practical applications. Working with Feature Templates. Studies the theoretical basis of feature-oriented design. Reviews examples of feature templates used in product-line engineering. Prepares a summary connecting feature templates to requirements traceability. Working with additional templates. Researches supplementary template types used in advanced software design. Studies their application in real development projects. Prepares a written summary of their practical benefits. Performing software metrics calculations. Studies additional software metrics not covered in class. Researches how these metrics are used in industry quality assurance. Prepares a report applying selected metrics to a sample codebase. Study of the algorithmic modeling process. Reviews theoretical foundations of algorithmic modeling in greater depth. Studies examples of algorithmic models from published research. Prepares a summary connecting algorithmic modeling to software design. Perform static code analysis. Researches additional static analysis tools and their capabilities. Studies example reports generated by such tools. Prepares a written summary comparing tool features and outputs. Learning comprehensive evaluation of a software system. Studies frameworks for conducting comprehensive software system evaluations. Reviews example evaluation reports from real projects. Prepares a summary outlining a comprehensive evaluation methodology.
Teaching Methods	The module is delivered through interactive lectures, problem-based and inquiry-based learning, laboratory practicals, hands-on design modeling exercises, computer simulations, seminars, group discussions, and project-based activities. Students design, model, and evaluate software systems using UML diagrams, architectural patterns, and design pattern implementations, and develop analytical, design, and problem-solving skills. Independent learning and digital educational resources further promote critical thinking, scientific communication, and the application of modern software design concepts.
Prerequisites	Fundamentals of Programming

	Algorithms and Data Structures Computer systems and operating systems Computer Networks Fundamentals
Exams and Assessment	Student performance is assessed through continuous assessment (20%), midterm assessment (30%), and a final written examination (50%). The continuous assessment (20%) evaluates classroom participation, laboratory performance, UML diagram construction exercises, seminar presentations, and the successful completion of independent study assignments, including ER diagram design, design pattern implementation reports, software metrics calculations, and architectural comparison analyses. The midterm assessment (30%) consists of written assignments, short in-class quizzes, and the evaluation of independent study projects, such as an MVC architecture design report, a comparative analysis of monolithic and modular systems, a presentation on structural design patterns, and an essay on static code analysis practices. The final assessment (50%) is a written examination consisting of theoretical questions and problem-solving tasks designed to assess students' understanding of software systems design, analytical skills, and their ability to apply modeling and design principles to practical problems. Successful completion of all independent study assignments is required as an essential component of the module assessment.
Recommended Literature	1. Roger S. Pressman, Bruce R. Maxim. Software Engineering: A Practitioner's Approach. McGraw-Hill Education, 2019 (9th Edition). 2. Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994. 3. Grady Booch, James Rumbaugh, Ivar Jacobson. The Unified Modeling Language User Guide. Addison-Wesley, 2005. 4. Martin Fowler. UML Distilled: A Brief Guide to the Standard Object Modeling Language. Addison-Wesley, 2018 (3rd Edition). 5. Craig Larman. Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design. Pearson, 2004.
Responsible Lecturer	Sh. M. Yuldashev – Head of the Department of Mathematical Modeling and Computer Sciences, TDU, DSc. , PhD.
Field	Description
Module Title	INTRODUCTION TO SOFTWARE ENGINEERING
Module Code	DIK1506

Credit Points (ECTS)	6 Credits
Workload	Total workload: 180 hours Contact hours: 60 hours (30 hours Lecture, 30 hours Practical lessons) S
Frequency/Duration	V semester
Module objective	The main goal of the "Introduction to Software Engineering" course is to provide students with fundamental knowledge of software development processes, software engineering principles, software project management, requirements analysis, the design, programming, testing, and maintenance processes.
Module Learning Outcomes	Knowledge: LO1 Knowledge of the general concepts of the Introduction to Software Engineering course, namely the fundamental properties of data structures and algorithms used to solve practical problems in professional activities. LO2 Knowledge of the basic principles of developing and applying effective mathematical models using modern information and communication technologies, including specialized mathematical software systems; LO3 Ability to use modern programming tools to process, visualize, and analyze research results in various areas of programming and its applications; LO4 Must be equipped with modern tools to solve practical problems in professional activities. Skills: LO5 In today's "Information" age, it is not only about increasing young people's computer literacy, but also about exploring opportunities to work with programming technologies; LO6 Studies the theoretical foundations of using personal computers in software engineering processes and the methods for applying them to various data structure domains; LO7 Studies methods for the effective use of software engineering software systems in management systems; LO8 Learns skills for working with Internet systems and data structure technologies; Competencies: LO9 Studies the development directions of software engineering and their descriptions; LO10 It consists of forming concepts about when and for what purposes to use software engineering's software systems, and providing knowledge on how to generate data;

	LO11 They will apply methods to ensure the security of software engineering's software systems and become familiar with their functional capabilities.
Content	Lectures Concept of Software Engineering and its Development History. Introduces the definition and scope of software engineering as a discipline. Traces the historical development of software engineering practices. Discusses the factors that drove its emergence as a distinct field. Software and its types. Reviews the concept of software and its role within computing systems. Discusses classifications of software, such as system, application, and embedded software. Explores examples illustrating each software type. Principles and standards of software engineering. Introduces fundamental principles guiding sound software engineering practice. Reviews common industry standards relevant to software development. Discusses how adherence to these principles and standards improves software quality. Software Development Life Cycle (SDLC). Introduces the phases comprising the software development life cycle. Discusses the purpose and activities associated with each phase. Reviews how the SDLC provides structure to the development process. Software Development Models: Waterfall Model. Explains the sequential structure of the Waterfall development model. Discusses the advantages and limitations of this approach. Reviews scenarios where the Waterfall model remains an appropriate choice. Iterative and Incremental Development Models. Explains the principles underlying iterative and incremental development approaches. Discusses how these models differ from the traditional Waterfall model. Reviews benefits such as early feedback and reduced risk. Agile methodology and Scrum technology. Introduces the core values and principles of Agile methodology. Explains the Scrum framework, including roles, ceremonies, and artifacts. Discusses how Agile and Scrum support flexible, iterative development. Fundamentals of Software Project Management. Reviews key responsibilities involved in managing software projects. Discusses planning, scheduling, and resource allocation techniques. Explores common challenges faced in software project management.

Requirements for Software Products. Introduces the concept of software requirements and their categories. Discusses functional and non-functional requirement types. Reviews the importance of clear requirements for project success.

Analysis and documentation of requirements. Explains techniques for analyzing gathered software requirements. Discusses standard approaches for documenting requirements clearly. Reviews how requirement documentation supports subsequent design and development.

Fundamentals of Software Systems Design. Introduces core principles guiding software system design. Discusses the relationship between design decisions and system quality. Reviews common design approaches used across software projects.

UML Modeling Language and Diagrams. Introduces the Unified Modeling Language (UML) and its purpose. Reviews common UML diagram types used in software design. Discusses how UML supports communication among development stakeholders.

The Concept of Software Architecture. Introduces the concept and significance of software architecture. Discusses common architectural styles and their characteristics. Reviews how architectural decisions influence long-term system maintainability.

Software Testing Processes. Reviews the purpose and stages of software testing. Discusses common testing types, including unit, integration, and system testing. Explores how testing processes contribute to overall software quality.

Software Product Quality and Future Development Directions. Reviews key dimensions of software product quality. Discusses methods for assessing and ensuring quality throughout development. Explores emerging trends shaping the future of software engineering.

Practical lessons

Practical Assignments on Software Engineering Concepts. Applies core software engineering concepts to a set of practical exercises. Reinforces theoretical understanding through hands-on problem-solving. Discusses how foundational concepts translate into real development tasks.

Analysis of Software Types. Analyzes examples of different software types encountered in practice. Identifies distinguishing characteristics of system, application, and embedded software.

Discusses appropriate development approaches for each software type.

Developing SDLC stages. Practices working through each stage of the software development life cycle for a sample project. Documents deliverables associated with each SDLC phase. Discusses how stage-by-stage progression supports project structure.

Creating a project based on the Waterfall model. Plans and structures a sample software project following the Waterfall model. Documents sequential progression through each development phase. Evaluates the model's fit for the chosen project scenario.

Modeling Agile and Scrum Processes. Simulates an Agile development workflow for a sample project. Practices applying Scrum processes such as sprint cycles and backlog management. Discusses observed benefits and challenges of the Agile approach.

Scrum Roles and Sprint Planning. Assigns and practices Scrum roles such as Product Owner, Scrum Master, and Development Team. Plans a sample sprint including backlog prioritization and task estimation. Discusses coordination challenges within the Scrum framework.

Preparing a technical specification for a software project. Drafts a technical specification document for a sample software project. Ensures the specification captures functional and technical requirements clearly. Reviews the specification for completeness and consistency.

Defining Requirements and Creating a Use Case. Defines functional requirements for a sample software system. Constructs use case diagrams representing user-system interactions. Validates use cases against the defined requirements.

Working with UML Diagrams. Constructs various UML diagrams to represent a sample system's structure and behavior. Practices selecting appropriate diagram types for different design aspects. Reviews diagrams for clarity and consistency.

Designing Software System Architecture. Designs an architectural structure for a sample software system. Selects and justifies an appropriate architectural style. Evaluates the design for scalability and maintainability.

Introduction to the Git Version Control System. Practices basic Git commands for version control, including commit, branch, and merge. Sets up a sample repository and tracks

changes over time. Discusses the role of version control in collaborative development.

Software Code Quality Assessment. Applies code quality assessment techniques to a sample codebase. Identifies issues related to readability, maintainability, and structure. Proposes improvements based on the assessment findings.

Development of test plans. Drafts a test plan outlining testing objectives, scope, and approach. Defines test cases covering key functional scenarios. Reviews the plan for completeness and testing coverage.

Determining software product quality indicators. Identifies key indicators used to assess software product quality. Applies these indicators to evaluate a sample software product. Discusses how quality indicators inform improvement decisions.

Final Software Project Presentation. Prepares a comprehensive presentation summarizing a completed software project. Demonstrates the application of concepts covered throughout the module. Reflects on lessons learned and areas for future improvement.

Independent Study

Scrum Roles and Sprint Planning. Studies the responsibilities and interactions among Scrum roles in greater depth. Researches real-world examples of sprint planning practices. Prepares a summary comparing Scrum role implementations across organizations.

Preparing a technical specification for the software project. Researches standard formats and templates for technical specifications. Studies examples of well-documented specifications from industry projects. Prepares a critique of a sample technical specification.

Requirements Identification and Use Case Creation. Studies additional techniques for identifying software requirements. Researches examples of use case documentation from real systems. Prepares an original use case analysis for a self-selected scenario.

Working with UML Diagrams. Reviews additional UML diagram types beyond those covered in class. Studies example diagrams from published software documentation. Prepares original diagrams illustrating a self-chosen system.

Designing Software System Architecture. Researches architectural styles not covered in depth during lectures. Studies real-world examples of architectural design decisions. Prepares a written analysis of an architecture case study.

Study the history of software engineering. Researches key milestones and figures in the history of software engineering. Studies how historical developments shaped current practices. Prepares a summary timeline of significant events.

Analysis of modern software development methodologies. Studies contemporary development methodologies beyond Agile and Waterfall. Compares their principles, strengths, and typical use cases. Prepares a comparative summary of modern methodologies.

Advantages of the Agile methodology. Researches the specific benefits Agile methodology offers over traditional approaches. Studies case studies of successful Agile adoption. Prepares a written summary of Agile's practical advantages.

Creating UML diagrams. Practices constructing UML diagrams for a self-selected software scenario. Reviews diagrams for accuracy and completeness. Prepares a portfolio of original UML diagrams.

Preparation of software project documentation. Studies standard practices for comprehensive software project documentation. Researches documentation templates used in industry settings. Prepares a sample documentation set for a chosen project.

Learning software testing tools. Researches popular software testing tools and their capabilities. Studies how these tools integrate into development workflows. Prepares a summary comparing tool features and use cases.

Study of software quality standards. Researches established software quality standards, such as ISO/IEC 25010. Studies how these standards are applied in industry practice. Prepares a summary connecting standards to quality assessment criteria.

Learning software testing processes. Studies testing process models beyond those covered in class. Researches how testing processes integrate with development life cycles. Prepares a summary of effective testing process practices.

Software code quality assessment. Researches additional code quality assessment techniques and metrics. Studies example code quality reports from real projects. Prepares a written summary comparing assessment approaches.

Development of test plans. Studies advanced test planning techniques and documentation standards. Researches examples of comprehensive test plans from industry. Prepares an original test plan for a self-selected project scenario.

Teaching Methods	The module is delivered through interactive lectures, problem-based and inquiry-based learning, laboratory practicals, hands-on project development exercises, computer simulations, seminars, group discussions, and project-based activities. Students design, document, and develop software projects using Agile/Scrum practices, UML modeling, and version control tools such as Git, and develop technical, analytical, and problem-solving skills. Independent learning and digital educational resources further promote critical thinking, scientific communication, and the application of modern software engineering concepts.
Prerequisites	Introduction to Programming Data Structures and Algorithms Discrete Mathematics
Exams and Assessment	Student performance is assessed through continuous assessment (20%), midterm assessment (30%), and a final written examination (50%). The continuous assessment (20%) evaluates classroom participation, laboratory performance, project development exercises, seminar presentations, and the successful completion of independent study assignments, including UML diagram creation, technical specification preparation, software testing tool reports, and code quality assessment tasks. The midterm assessment (30%) consists of written assignments, short in-class quizzes, and the evaluation of independent study projects, such as a Scrum sprint planning report, a software project documentation set, a presentation on Agile methodology advantages, and an essay on software quality standards. The final assessment (50%) is a written examination consisting of theoretical questions and problem-solving tasks designed to assess students' understanding of software engineering, analytical skills, and their ability to apply development and project management principles to practical problems. Successful completion of all independent study assignments is required as an essential component of the module assessment.
Recommended Literature	1. Ian Sommerville. Software Engineering. 10th Edition. Pearson Education, 2016. 2. Roger S. Pressman, Bruce R. Maxim. Software Engineering: A Practitioner's Approach. 9th Edition. McGraw-Hill Education, 2020. 3. Pankaj Jalote. An Integrated Approach to Software Engineering. 4. Martin Fowler. UML Distilled.

	5. Steve McConnell. Code Complete 6. Approach. 9th Edition. McGraw-Hill Education, 2020. 7. Shari Lawrence Pfleeger, Joanne M. Atlee. Software Engineering: Theory and Practice. Pearson, 2010 8. Karl E. Wieggers, Joy Beatty. Software Requirements. Microsoft Press, 2013. 9. Jalolov T. , Aripov M. Fundamentals of Information Technology and Software. Tashkent, 2021. 10. Methodological recommendations for the discipline of Software Engineering for higher education institutions of the Republic of Uzbekistan
Responsible Lecturer	Mamatkabilov Abdilmajid Khujamuratovich Doctor of Philosophy (PhD) in Physical and Mathematical Sciences.
Field	Description
Module Title	OPERATING SYSTEMS
Module Code	OPT1506
Credit Points (ECTS)	6 Credits
Workload	Total workload: 180 hours Contact hours: 60 hours (30 hours Lecture, 30 hours Practical lessons) Self-Assignments: 120 hours
Frequency/Duration	V semester
Module objective	The "Operating Systems" course is designed for students in the 60610400-Software Engineering educational track. The course objective is to provide students with theoretical knowledge and practical lessons on modern operating systems, file system installation, process and memory management, and practical lessons to acquire theoretical knowledge of modern operating systems, file system installation, process and memory management, bootloaders, and network operation, and to apply this knowledge in solving practical problems.
Module Learning Outcomes	Knowledge: LO1. Have an understanding of the types and functions of operating systems, their classification, and descriptions of their functional components. Information, its storage, and methods of processing processing and transmission, the mathematical and software foundations of computing systems, their applications in scientific fields, manufacturing, and education, and an understanding of computer software, types of programs, and their characteristics;

	LO 2. Creating and working with processes and threads, managing input/output devices, the structure and allocation of main memory, to know and be able to use the file system and its organization methods; to have skills in studying object-oriented programming concepts, encapsulation, inheritance, and polymorphism, and in solving practical problems using them; LO3. Must have skills in selecting an operating system based on application areas, hardware, and the intended purpose, as well as installing and configuring it. Be proficient in developing an algorithm to solve applied problems, constructing a mathematical (computer) model and creating its software, generating linear lists, aggregating data in a structured format, and working with indicators. Skills & Application LO 4. Develop the ability to work proficiently with the computer's operating system. This enables the student to quickly learn how to use operating systems. LO 5. Develop the skill to work with different versions of operating systems. Professional Competencies LO 6. understand the concepts of processes and threads in operating systems, their life cycle, scheduling algorithms (FCFS, SJF, Round-Robin, etc.) and apply synchronization mechanisms (semaphore, mutex), and analyze deadlock issues. LO 7. explain methods such as virtual memory, paging, segmentation, swapping, and demand paging, and be able to apply memory allocation and protection mechanisms in practice. LO 8. understands the structure of file systems, directories, file placement methods (contiguous, linked, indexed), and disk scheduling algorithms (FCFS, SSTF, SCAN), and can analyze them.
Content	Lectures Introduction to operating systems. Basic computer organization. Abstraction of the operating system. Introduces the fundamental concepts and role of an operating system within a computer. Reviews basic computer organization relevant to understanding OS functions. Explains the abstraction layer the OS provides between hardware and applications. Definition of operating systems and their structure. Defines the operating system and its core responsibilities. Reviews the general structural components comprising an OS. Discusses how these components interact to manage system resources.

Functions of the operating system and its development stages. Reviews the primary functions performed by an operating system. Traces the historical development stages of operating systems. Discusses how OS functionality has evolved to meet changing computing needs.

Operating system design principles. Two-mode operation. Protection of the CPU, memory, input/output, and storage. Introduces key design principles guiding operating system architecture. Explains the distinction between kernel mode and user mode operation. Discusses mechanisms for protecting CPU, memory, I/O, and storage resources.

Classification of operating systems, their architecture, and trends in operating system development. Reviews common classifications of operating systems by type and use case. Discusses different architectural approaches, such as monolithic and microkernel designs. Explores current trends shaping the future direction of operating system development.

Classification, architecture, and directions of operating systems. Revisits the classification of operating systems in greater depth. Discusses architectural variations across different OS families. Reviews emerging directions influencing modern operating system design.

The concept of resources and the classification of resources. Introduces the concept of system resources managed by the operating system. Reviews common classifications of resources, such as hardware and software resources. Discusses how resource classification informs management strategies.

The concept of a process and processes in memory. Defines the concept of a process within an operating system context. Explains how processes are represented and managed in memory. Discusses the relationship between processes and system resource allocation.

Paging: internal fragmentation, page tables. Introduces paging as a memory management technique. Explains the causes and effects of internal fragmentation. Discusses the structure and function of page tables in address translation.

Page replacement strategies: FIFO, OPT, LRU, NRU, LFU, MFU, MRU. Reviews the purpose of page replacement algorithms in memory management. Compares the operating principles of FIFO, OPT, LRU, NRU, LFU, MFU, and MRU strategies. Discusses trade-offs in performance and implementation complexity among these strategies.

File system concept in operating systems. Introduces the role and purpose of file systems within operating systems. Discusses core file system operations such as storage, retrieval, and organization. Reviews basic file system structures and terminology.

FAT, VFAT, FAT32, HPFS and NTFS file systems. Reviews the structure and characteristics of the FAT, VFAT, and FAT32 file systems. Introduces HPFS and NTFS as more advanced file system designs. Compares these file systems in terms of features and performance.

Network operating systems: Unix and Linux operating systems. Introduces the concept of network operating systems and their distinguishing features. Reviews the architecture and design philosophy of Unix and Linux. Discusses their significance in networked and server environments.

Unix file system: directories, mount points, abstractions. Explains the hierarchical structure of the Unix file system. Discusses the role of directories and mount points in file system organization. Reviews key abstractions underlying Unix file system design.

The Concept of Information Security in Operating Systems. Introduces fundamental information security concepts relevant to operating systems. Discusses common security threats and vulnerabilities at the OS level. Reviews mechanisms operating systems use to protect data and resources.

Practical lessons

User interface for working with operating systems. Explores common user interface elements found in modern operating systems. Practices navigating and configuring interface settings. Compares interface features across different operating systems.

Working in BIOS, Command-line Interpreter in Operating Systems. Accesses and navigates BIOS/UEFI settings on a sample system. Practices basic command-line interpreter usage and common commands. Discusses the relationship between BIOS configuration and OS startup.

Working with the registry in the MS Windows operating system. Explores the structure and organization of the Windows registry. Practices viewing and safely modifying registry entries. Discusses the role of the registry in system configuration.

Command-line shell in operating systems. Creating a bootable USB flash drive. Practices advanced command-line shell operations across different platforms. Creates a bootable

USB flash drive using appropriate tools. Discusses practical uses of bootable media for system installation and recovery.

Symmetric multiprocessing in operating systems, working with processors. Operating system processor management. Explores how operating systems manage multiple processors in symmetric multiprocessing configurations. Practices monitoring processor usage and process assignment. Discusses OS strategies for balancing workload across processors.

Memory Management in Operating Systems. Practices monitoring memory usage and allocation on a sample system. Explores tools for analyzing memory management behavior. Discusses observed memory management patterns and their implications.

Working in the Windows operating system. Practices core administrative and configuration tasks within Windows. Explores system settings relevant to performance and usability. Discusses practical troubleshooting approaches for common issues.

Launching the help system and standard programs. Practices accessing and navigating the operating system's help resources. Explores commonly bundled standard programs and their functions. Discusses how these tools support everyday system use.

Working with the file system in operating systems. Managing data input-output in operating systems. Practices file system operations such as creating, organizing, and managing files. Explores mechanisms for managing data input-output processes. Discusses practical considerations for efficient file system use.

Configuring general permissions and user rights. Practices configuring user accounts and access permissions. Explores how permission settings affect file and system access. Discusses best practices for managing user rights securely.

Working in the Linux Terminal. Practices essential Linux terminal commands and navigation. Explores file manipulation and system administration tasks via the terminal. Discusses the advantages of terminal-based system interaction.

Configuring security components of operating systems. Practices configuring built-in security features of an operating system. Explores settings related to firewalls, permissions, and access control. Discusses the impact of these configurations on overall system security.

Simple memory management techniques. Virtual memory management techniques. Practices implementing basic memory management strategies. Explores virtual memory configuration and monitoring tools. Discusses how virtual memory techniques improve system performance.

File Concept. Catalog and Storage Services. File Names and Metadata. Explores the fundamental concept of files within an operating system. Practices working with catalog and storage service functions. Discusses the role of file names and metadata in file organization.

Using Windows operating system protection tools. Practices using built-in Windows protection tools such as Windows Defender. Explores configuration options for enhancing system protection. Discusses practical scenarios where these tools mitigate security threats.

Independent Study

User interface for working with operating systems. Studies design principles behind effective user interfaces in operating systems. Compares interface approaches across different OS platforms. Prepares a summary of findings on interface usability.

Working in BIOS, Command-line Interpreter in Operating Systems. Researches advanced BIOS/UEFI configuration options beyond those covered in class. Studies additional command-line interpreter features and scripting basics. Prepares a summary of practical command-line techniques.

Working with the registry in the MS Windows operating system. Studies the broader structure and function of the Windows registry in greater depth. Researches common registry-related troubleshooting scenarios. Prepares a summary connecting registry structure to system behavior.

Command-line shell in operating systems. Creating a bootable USB flash drive. Researches shell scripting basics across different operating systems. Studies alternative tools and methods for creating bootable media. Prepares a written comparison of bootable media creation approaches.

Symmetric multiprocessing in operating systems, working with processors. Operating system processor management. Studies theoretical models underlying symmetric multiprocessing systems. Researches processor scheduling strategies used in modern operating systems. Prepares a summary of processor management approaches.

Memory Management in Operating Systems. Researches advanced memory management techniques beyond those

covered in class. Studies real-world examples of memory management challenges and solutions. Prepares a written report summarizing key findings.

Working in the Windows operating system. Studies advanced administrative features of the Windows operating system. Researches best practices for system configuration and maintenance. Prepares a summary of practical administrative techniques.

Launching the help system and standard programs. Researches the broader ecosystem of standard programs bundled with operating systems. Studies how help systems are structured to support user needs. Prepares a summary evaluating help system usability.

Working with the file system in operating systems. Managing data input-output in operating systems. Studies file system organization strategies used in different operating systems. Researches data input-output management techniques in greater depth. Prepares a written summary connecting file systems to I/O management.

Configuring general permissions and user rights. Researches permission and access control models used across different operating systems. Studies best practices for secure user rights configuration. Prepares a summary comparing permission models.

Working in the Linux Terminal. Studies advanced Linux terminal commands and scripting techniques. Researches practical use cases for terminal-based system administration. Prepares a summary of useful terminal commands and their applications.

Configuring security components of operating systems. Researches additional security configuration options beyond those covered in class. Studies real-world security incidents related to OS misconfiguration. Prepares a written report on effective security configuration practices.

Simple memory management techniques. Virtual memory management techniques. Studies theoretical foundations of virtual memory management in greater depth. Researches performance implications of different virtual memory strategies. Prepares a summary comparing memory management technique outcomes.

File Concept. Catalog and Storage Services. File Names and Metadata. Researches the theoretical basis of file concepts across different operating systems. Studies how catalog and

	storage services are implemented in practice. Prepares a summary connecting file naming and metadata to system organization. Using Windows operating system protection tools. Studies the broader range of Windows protection tools beyond Windows Defender. Researches best practices for configuring comprehensive system protection. Prepares a summary evaluating the effectiveness of different protection tools.
Teaching Methods	The module is delivered through interactive lectures, problem-based and inquiry-based learning, laboratory practicals, hands-on system configuration exercises, computer simulations, seminars, group discussions, and project-based activities. Students work directly with Windows and Linux operating systems to configure memory, file systems, and security settings using tools such as the command-line shell, registry editor, and terminal, and develop technical, analytical, and problem-solving skills. Independent learning and digital educational resources further promote critical thinking, scientific communication, and the application of modern operating system concepts.
Prerequisites	Operating Systems Computer networks Computer Hardware Programming
Exams and Assessment	Student performance is assessed through continuous assessment (20%), midterm assessment (30%), and a final written examination (50%). The continuous assessment (20%) evaluates classroom participation, laboratory performance, hands-on system configuration exercises, seminar presentations, and the successful completion of independent study assignments, including registry and command-line configuration reports, memory management analysis, file system comparison exercises, and Linux terminal task reports. The midterm assessment (30%) consists of written assignments, short in-class quizzes, and the evaluation of independent study projects, such as a report on page replacement strategy comparison, a security configuration case study, a presentation on network operating systems, and an essay on virtual memory management. The final assessment (50%) is a written examination consisting of theoretical questions and problem-solving tasks designed to assess students' understanding of operating systems, analytical skills, and their ability to apply system management principles to

	practical problems. Successful completion of all independent study assignments is required as an essential component of the module assessment.
Recommended Literature	1. Operating Systems (textbook) – U. R. Khammadamov, D. J. B. Sultonov, S. S. Parsiyev, U. M. Abdullayev. Tashkent, 2021. 436 pp. 2. Van Renesse R. Concurrent Programming with Harmony. - Ithaca, NY: Cornell University, 2021-230 p. –Electronic source: http://harmony.cs.cornell.edu/book.pdf Molchanov A. Y. System Software. Laboratory Practicum. – St. Petersburg: Piter, 2005. – 284 p. : ill. 3. Stollings V. Operating Systems. St. Petersburg: 2004. 4. Mashechkin I. V. , Petrovskiy M. I. , Skulachev P. D. , Terexin A. N. System Software: File Systems of Unix and Windows NT. M. : Dialog-Moscow. 1997.
Responsible Lecturer	Choriyev Ilhom Kenjayevich, Senior Lecturer, Department of Operating Systems and Computer Science.
Field	Description
Module Title	ECOLOGY AND ENVIRONMENTAL PROTECTION
Module Code	<i>EAM202</i>
Credit Points (ECTS)	2 Credits
Workload	Total workload: 60 hours Contact hours: 30 hours (14 hours Lecture, 16 hours Practical lessons) Self-Assignments: 30 hours
Frequency/Duration	V semester
Module objective	To teach students the scientific and theoretical foundations of ecology and environmental protection, the environmental problems arising from energy production and the factors causing them, their spatial and temporal changes, environmental assessment, nature conservation, and the ecological basis of the rational use of natural resources.
Module Learning Outcomes	Knowledge: LO1. Shall have an understanding of the fundamentals of the subject of Ecology and Environmental Protection, and of phenomena in the environment and ways of preventing them. LO2. The reforms being carried out in environmental protection and green development. LO3. The role of society in environmental protection and green development.

	LO4. Ecology and Environmental Protection – shall possess the skills to describe the methods of applying theoretical issues in the field on the basis of ecological laws. Skills: L O . shall possess skills such as protecting and preserving the environment for the purposes of ecology and environmental protection, and the skill of identifying and observing physical factors affecting the environment and analyzing their impact on the ecological condition. LO7. shall master the skills of collecting and processing data related to energy-saving technologies and renewable energy sources, and applying them in practical activities.. LO8. shall acquire the skill of analyzing statistical and scientific data on environmental problems, comparing results and drawing well-founded conclusions. Competencies: LO9. shall acquire the competence to analyze the principles of environmental protection and green development in connection with the theoretical and practical foundations of physics, and to develop scientific solutions to environmental problems. LO10. shall be able to explain the physical foundations of the use of energy-saving technologies, renewable energy sources and environmentally safe technologies, and effectively integrate them into the educational process. LO11. shall acquire the competence to assess physical processes affecting the state of the environment, identify ecological risks, and apply scientific approaches serving the goals of sustainable development. LO12. shall be able to promote ecological culture, rational use of resources and ideas of green development in their professional activity, and to foster in learners a responsible attitude towards nature and ecological thinking.
Content	Lectures The subject, objectives, purpose and history of Ecology and Environmental Protection. The subject, tasks, and methods of the discipline "Ecology and Environmental Protection." Branches of general ecology, the history of the development of ecology, environmental problems, and ways to prevent them. Scientific and theoretical foundations of nature conservation The concept of environment in ecology. The adaptation of living organisms to their habitats and its types. The classification of

ecological factors and their impact on living organisms. The interrelationship between nature and society. The impact of the scientific-technological revolution on the environment. Population growth and environmental protection.

The concept of the biosphere. V.I. Vernadsky's doctrine of the biosphere. The concept of the biosphere. V.I. Vernadsky's doctrine, the boundaries of the biosphere, the concept of the noosphere, the cycling of matter in the biosphere. The circulation of energy in the biosphere.

Classification of natural resources, their rational use, and protection of atmospheric air; the dispersion of pollutants in the atmosphere is explained by the laws of diffusion and gas motion. Natural resources and their classification. Strategies for the rational use of natural resources. Alternative energy types and their environmental safety. The composition of the atmosphere and its sources of pollution. The problem of climate change, causes of ozone layer depletion. Acid rain and measures to combat it, the harmful effects of photochemical smog. Radioactive contamination and its impact on the environment. The impact of noise on living organisms.

Water and land resources and their protection. The distribution of water in nature and its significance for the life of living organisms. The problem of freshwater, sources of water pollution. Water purification methods. The ecological condition of Uzbekistan's water bodies, the environmental problems of the Aral Sea and the Aral Sea region, and ways to address them. The "Yashil Makon" (Green Space) national movement. Humus content in soil and its significance, the role of soil in the cycling of matter, the current ecological condition of Uzbekistan's soils. Soil contamination by chemical compounds and measures to combat it. Salinization and its impact on soil fertility. The process of desertification and its causes.

Conservation of biological diversity. The diversity of plants, endemic plants and their protection, the ecology of medicinal plants, forest resources and their protection. The diversity of the animal kingdom, methods of protecting rare animal species. The "Red Book" and its significance in environmental protection. Specially protected natural areas in the Republic of Uzbekistan: game reserves (zakazniks), nature reserves (zapovedniks), national parks, and natural monuments.

Issues of sustainable development and environmental education. Sustainable Development Goals. Issues of developing environmental education in the Republic of

Uzbekistan. Environmental protection, ecological culture, environmental education and upbringing, ecological enlightenment. Mechanisms for implementing ecological outreach. Forming and developing ecological knowledge, awareness, and culture among the rising younger generation, effectively organizing the process of environmental education and upbringing, and introducing innovative technologies into the field.

Practical lessons

Current environmental problems. This practical seminar is devoted to a comprehensive examination of the most pressing environmental problems facing the world today, at both global and regional levels. The session is organized around two main components. The first component addresses global environmental problems, analyzing issues such as climate change, ozone layer depletion, biodiversity loss, deforestation, and the depletion and pollution of freshwater resources, along with their interconnected causes and consequences. The second component focuses on environmental problems specific to Uzbekistan and Central Asia, discussing the ecological crisis of the Aral Sea, land salinization and desertification, water scarcity, air pollution in major cities, and waste management challenges, as well as the policy responses and institutional mechanisms currently in place to address them. Students analyze case studies of both global and regional environmental problems, compare the effectiveness of different response strategies, and develop proposals for addressing a specific environmental problem relevant to Uzbekistan.

The expected learning outcome of the seminar is for students to develop a systematic understanding of current environmental problems at multiple scales, strengthen their ability to critically evaluate policy responses, and enhance their skills in analysis and problem-solving related to environmental challenges.

Interactions between nature and society

This practical seminar is devoted to examining the historical and contemporary forms of interaction between human society and the natural environment, and the theoretical frameworks used to understand this relationship. The session is organized around two main components. The first component addresses the theoretical foundations of nature-society interaction, analyzing the evolution of human impact on nature across different historical stages of civilization (from hunter-gatherer societies to the agricultural, industrial, and post-industrial eras), and key

conceptual approaches such as anthropocentrism, ecocentrism, and the idea of co-evolution between nature and society. The second component focuses on contemporary forms and consequences of this interaction, discussing the impact of economic activity, urbanization, and technological development on natural systems, along with emerging models for reconciling human development with ecological limits, such as sustainable development and the green economy. Students analyze case studies illustrating both destructive and constructive patterns of interaction between society and nature, and develop proposals for improving this relationship within a specific sector relevant to Uzbekistan.

The expected learning outcome of the seminar is for students to understand the historical evolution and theoretical frameworks of nature-society interaction, strengthen their ability to critically evaluate different models of human-environment relations, and develop practical proposals for fos

Cycling of matter and energy in the biosphere. This practical seminar is devoted to examining the fundamental processes by which matter and energy circulate within the biosphere, sustaining the functioning of ecosystems and life on Earth. The session is organized around two main components. The first component addresses the biogeochemical cycles of matter, analyzing the carbon, nitrogen, oxygen, water, and phosphorus cycles, the role of living organisms in driving these cycles, and the ways in which human activity disrupts their natural balance. The second component focuses on the flow of energy through the biosphere, discussing the capture of solar energy through photosynthesis, the transfer of energy along trophic levels and food chains, the concept of ecological efficiency and energy loss between trophic levels, and the fundamental distinction between the cyclical nature of matter and the unidirectional flow of energy in ecosystems. Students perform practical exercises modeling a specific biogeochemical cycle or energy flow within a selected ecosystem, and analyze the consequences of human-induced disruptions to these processes, such as the enhanced greenhouse effect resulting from disturbances to the carbon cycle.

The expected learning outcome of the seminar is for students to understand the fundamental mechanisms governing the circulation of matter and energy in the biosphere, strengthen their ability to analyze the ecological consequences of disruptions to

these cycles, and develop practical skills in ecosystem-level analysis.

Alternative energy sources and their environmental safety.

This practical seminar is devoted to examining the main types of alternative energy sources and evaluating their environmental safety in comparison with conventional fossil fuel-based energy production. The session is organized around two main components. The first component addresses the main types of alternative energy sources, analyzing solar, wind, hydropower, geothermal, biomass, and hydrogen energy, along with their respective technological principles, advantages, and limitations. The second component focuses on the environmental safety of these energy sources, discussing their comparative life-cycle environmental impacts, including land and resource use, emissions during manufacturing and disposal, effects on local ecosystems and wildlife, and the extent to which each source reduces greenhouse gas emissions relative to fossil fuels. Students analyze case studies comparing the environmental performance of different alternative energy technologies and assess the prospects and environmental considerations relevant to expanding a specific alternative energy source in Uzbekistan. The expected learning outcome of the seminar is for students to understand the technological principles and comparative environmental impacts of alternative energy sources, strengthen their ability to critically evaluate energy technologies from an environmental safety perspective, and develop practical proposals for advancing environmentally sound energy development in Uzbekistan.

Environmental problems of the Aral Sea and the Aral Sea region and ways of preventing them

This practical seminar is devoted to examining the causes, consequences, and mitigation strategies related to one of the most significant ecological crises in Central Asia — the desiccation of the Aral Sea. The session is organized around two main components. The first component addresses the causes and consequences of the Aral Sea crisis, analyzing the historical decisions regarding water diversion for irrigated agriculture that led to the sea's shrinkage, the resulting ecological consequences such as the exposure of the Aralkum desert, increased salt and dust storms, biodiversity loss, and the impact on regional climate, as well as the socioeconomic and public health consequences for the population of the Aral Sea region. The second component focuses on measures and strategies for preventing and

mitigating the crisis, discussing afforestation and vegetation-cover restoration projects on the exposed seabed, water conservation and efficient irrigation initiatives, regional and international cooperation frameworks (including the International Fund for Saving the Aral Sea and UN Multi-Partner Human Security Trust Fund), and national programs aimed at improving the socioeconomic conditions and public health of the Aral Sea region. Students analyze case studies of specific mitigation projects implemented in the region and develop proposals for strengthening a chosen area of intervention.

The expected learning outcome of the seminar is for students to understand the causes and multidimensional consequences of the Aral Sea crisis, strengthen their ability to critically evaluate mitigation and prevention strategies, and develop practical proposals for addressing environmental crises of similar scale and complexity.

Specially protected natural areas of the Republic of Uzbekistan and their importance in nature conservation.

This practical seminar is devoted to examining the system of specially protected natural areas in Uzbekistan and evaluating their contribution to biodiversity conservation and ecological sustainability. The session is organized around two main components. The first component addresses the classification and structure of protected areas, analyzing the distinct categories established in national legislation — state nature reserves (zapovedniks), national parks, game reserves (zakazniks), and natural monuments — along with their respective conservation objectives, legal regimes, and permitted levels of human activity. The second component focuses on the practical role and effectiveness of these areas, discussing their contribution to preserving representative and unique ecosystems, protecting habitats for endangered and endemic species, maintaining ecological corridors, and supporting scientific research and ecotourism, as well as the main challenges they face, such as insufficient funding, pressure from surrounding land use, and gaps in ecosystem coverage. Students analyze case studies of specific protected areas in Uzbekistan, evaluate their conservation effectiveness using available data, and develop proposals for strengthening the management of a chosen protected area.

The expected learning outcome of the seminar is for students to understand the structure and legal framework of Uzbekistan's protected area system, strengthen their ability to critically

evaluate conservation effectiveness, and develop practical proposals for improving the protection and management of natural areas.

Towards the Sustainable Development Goals.

This practical seminar is devoted to examining the global pathways, instruments, and challenges involved in progressing toward the achievement of the Sustainable Development Goals (SDGs). The session is organized around two main components. The first component addresses the mechanisms driving global progress toward the SDGs, analyzing the role of international financing instruments, public-private partnerships, technology transfer, and capacity-building initiatives, as well as the mid-term global assessment of progress and the identification of goals at greatest risk of falling behind schedule. The second component focuses on pathways and strategies for accelerating progress, discussing integrated policy approaches that address multiple goals simultaneously, the role of data and monitoring systems in tracking progress, the importance of multi-stakeholder partnerships involving government, business, and civil society, and lessons learned from countries that have made notable advances toward specific goals. Students analyze case studies of successful acceleration strategies from different countries and develop proposals for a pathway toward achieving a specific SDG within a chosen national or sectoral context.

The expected learning outcome of the seminar is for students to understand the global mechanisms and strategies for advancing the SDGs, strengthen their ability to critically evaluate acceleration pathways, and develop practical proposals for translating global sustainability goals into actionable strategies.

Issues of environmental protection, ecological culture, environmental education and ecological enlightenment.

This practical seminar is devoted to examining the interconnected roles of environmental protection policy, ecological culture, environmental education, and public ecological enlightenment in fostering a sustainable relationship between society and nature. The session is organized around two main components. The first component addresses the conceptual foundations of these interrelated notions, analyzing environmental protection as a policy and institutional framework, ecological culture as a system of values and behavioral norms guiding individual and collective conduct toward nature, and the distinction between formal environmental education and broader ecological enlightenment aimed at raising public awareness. The

second component focuses on practical mechanisms for strengthening ecological culture and awareness, discussing the role of educational institutions at all levels, the involvement of mass media and civil society organizations in ecological outreach, the use of public campaigns and community initiatives, and relevant national policy frameworks such as the concept of raising the population's ecological culture up to 2030. Students analyze case studies of successful ecological education and awareness campaigns from Uzbekistan and other countries, and develop proposals for strengthening ecological culture within a specific community, institution, or sector.

The expected learning outcome of the seminar is for students to understand the conceptual relationship between environmental protection, ecological culture, and education, strengthen their ability to critically evaluate awareness-raising mechanisms, and develop practical proposals for fostering ecological culture among the population.

Self-Assignments

Development of the science of ecology in Uzbekistan. This independent work assignment is designed to deepen students' understanding of the historical formation and institutional development of ecological science in Uzbekistan, as a complement to the theoretical material covered in lectures. Students are required to independently study the following: the historical stages of the development of ecology as a scientific discipline in Uzbekistan, from its origins within natural science research during the Soviet period to its establishment as an independent field of study; the contribution of leading national scientists and research institutions (such as the Academy of Sciences of Uzbekistan and specialized ecological research centers) to the development of ecological knowledge; the evolution of environmental research priorities in response to national ecological challenges, including the Aral Sea crisis, desertification, and biodiversity conservation; the current structure of ecological education and research in the country, including relevant university departments, research institutes, and international research collaborations; and recent trends and directions in ecological science, such as the integration of ecology with economics, climate science, and sustainable development studies.

Ecological problems and ways to prevent them. This independent work assignment is designed to deepen students' understanding of the diversity of ecological problems and the

range of preventive and mitigating measures available at local, national, and global levels, building on the theoretical foundations covered in lectures. Students are required to independently study the following: the classification of ecological problems by scale and origin (local, regional, and global, as well as natural versus anthropogenic causes); the main categories of environmental degradation, including air, water, and soil pollution, deforestation, biodiversity loss, and climate change; the underlying economic and social drivers of environmental degradation, such as unsustainable production and consumption patterns, population growth, and inadequate environmental governance; preventive approaches and instruments, including environmental impact assessment, cleaner production technologies, economic incentives (taxes, subsidies, and tradable permits), and legal and regulatory frameworks for environmental protection; and international and national experience in successfully preventing or mitigating specific ecological problems, drawn from case studies of particular relevance to Uzbekistan or the Central Asian region.

Habitats of living organisms and their adaptation to them.

This independent work assignment is designed to deepen students' understanding of the ecological relationships between living organisms and their habitats, complementing the theoretical foundations of ecology covered in lectures. Students are required to independently study the following: the concept and classification of habitats (terrestrial, aquatic, and transitional environments), and the main abiotic and biotic factors shaping them, such as temperature, light, moisture, soil composition, and interspecies interactions; the concept of ecological niche and the distinction between habitat and niche; the principal types and mechanisms of organismal adaptation, including morphological, physiological, and behavioral adaptations, as well as adaptations to extreme environmental conditions (arid, high-altitude, or cold habitats); the role of natural selection in shaping adaptive traits over time; and case examples of species adapted to habitats characteristic of Uzbekistan and Central Asia, such as desert, steppe, and mountain ecosystems.

Mechanisms by which various ecological factors affect living organisms. This independent work assignment is designed to deepen students' understanding of how different categories of ecological factors influence the physiology, behavior, and distribution of living organisms, complementing the theoretical material covered in lectures. Students are required to

independently study the following: the classification of ecological factors into abiotic (temperature, light, moisture, salinity, and soil composition), biotic (interspecies and intraspecies interactions such as competition, predation, and symbiosis), and anthropogenic factors (human-induced environmental changes); the concept of the limiting factor and the law of the minimum, explaining how a single factor in short supply can restrict the growth or distribution of an organism; the concept of the range of tolerance and the distinction between eurybiont and stenobiont organisms based on their tolerance to environmental variation; the specific physiological and behavioral mechanisms through which organisms respond to and are affected by ecological factors, such as thermoregulation, osmoregulation, and photoperiodic responses; and the combined and interactive effects of multiple ecological factors acting simultaneously on an organism or ecosystem.

Human demography and environmental protection. This independent work assignment is designed to deepen students' understanding of the relationship between population dynamics and environmental sustainability, complementing the theoretical material covered in lectures. Students are required to independently study the following: the main indicators and trends of human demography, including population growth rates, fertility and mortality patterns, urbanization, and migration, both globally and in Uzbekistan; the theoretical frameworks explaining the relationship between population growth and environmental pressure, such as the IPAT model ($\text{Impact} = \text{Population} \times \text{Affluence} \times \text{Technology}$) and the concept of ecological footprint and carrying capacity; the specific environmental consequences of demographic processes, including increased demand for land, water, and energy resources, growing waste generation, and pressure on biodiversity and ecosystems, particularly in the context of rapid urbanization; the role of demographic policy and family planning in environmental strategies, along with the ethical and social dimensions of linking population policy to environmental goals; and the specific demographic and environmental context of Uzbekistan, including population growth trends, urbanization patterns, and their implications for natural resource management and sustainable development planning.

The concept of the noosphere and environmental protection. This independent work assignment is designed to deepen students' understanding of the philosophical and

scientific concept of the noosphere and its relevance to contemporary environmental thought, complementing the theoretical material covered in lectures. Students are required to independently study the following: the origins and development of the noosphere concept, including the contributions of Vladimir Vernadsky and other founding thinkers, and its place within the broader evolution of ideas about the biosphere; the core theoretical premises of the noosphere concept, namely the idea that human reason and scientific activity constitute a new, transformative stage in the evolution of the biosphere, along with the conditions under which this transformation can occur in harmony with, rather than in destruction of, natural systems; the relationship between the noosphere concept and modern principles of sustainable development, including points of convergence and points of critique or divergence between the two frameworks; the practical and ethical implications of the noosphere concept for environmental policy, science, and education, particularly regarding the responsible use of scientific and technological knowledge in managing human impact on nature; and the relevance and application of noospheric thinking to contemporary environmental challenges, such as climate change and biodiversity loss.

Mining, chemical, and metallurgical industries and environmental protection. This independent work assignment is designed to deepen students' understanding of the environmental impacts of extractive and heavy industrial sectors and the mechanisms available for mitigating them, complementing the theoretical material covered in lectures. Students are required to independently study the following: the main environmental impacts associated with the mining industry, including land disturbance, soil and water contamination from tailings and waste rock, and the risks associated with abandoned or poorly managed mining sites; the environmental hazards specific to the chemical industry, such as air and water pollution from hazardous chemical emissions and effluents, and the risks of soil and groundwater contamination from improper storage or disposal of chemical waste; the environmental burden of the metallurgical industry, including air pollution from smelting processes, high energy and water consumption, and the generation of solid industrial waste and slag; the regulatory, technological, and economic mechanisms available for reducing environmental impact in these sectors, such as environmental impact assessment, cleaner production technologies, pollution-

control equipment, and economic instruments (environmental taxes, fees, and permits); and the current state and environmental challenges of the mining, chemical, and metallurgical industries in Uzbekistan, along with relevant national regulations and modernization initiatives.

Climate change and ecological adaptation. This independent work assignment is designed to deepen students' understanding of the scientific basis of climate change and the range of ecological adaptation strategies available in response to it, complementing the theoretical material covered in lectures. Students are required to independently study the following: the physical mechanisms driving climate change, including the enhanced greenhouse effect, rising atmospheric concentrations of greenhouse gases, and their sources in human activity; the observed and projected impacts of climate change on ecosystems, such as shifts in species distribution and phenology, changes in precipitation patterns, glacier retreat, and increased frequency of extreme weather events; the distinction between climate change mitigation and adaptation, and the specific role of ecological adaptation in reducing the vulnerability of natural and human systems; the main approaches to ecological adaptation, including ecosystem-based adaptation, the conservation and restoration of natural buffers (such as wetlands and forests), and the development of climate-resilient agricultural and water management practices; and the specific climate vulnerabilities and adaptation priorities of Uzbekistan and the Central Asian region, including glacier melt in the Pamir-Alay mountains, water scarcity, and the vulnerability of agriculture to changing climate conditions.

Current state of Uzbekistan's water bodies. Water problems in the region and ways to prevent them. This independent work assignment is designed to deepen students' understanding of the condition of water resources in Uzbekistan and the broader Central Asian region, and the measures available to address the associated challenges, complementing the theoretical material covered in lectures. Students are required to independently study the following: the main types and current condition of water bodies in Uzbekistan, including rivers (the Amu Darya and Syr Darya basins), lakes, reservoirs, and groundwater sources, along with available data on water quality, quantity, and seasonal variability; the historical and ongoing decline of the Aral Sea and its ecological, economic, and public health consequences for the surrounding region; the main regional water problems, including

water scarcity driven by climate change and glacier retreat, inefficient irrigation practices, pollution from agricultural runoff and industrial discharge, and challenges of transboundary water allocation among Central Asian countries; the institutional and legal mechanisms for water resource management in Uzbekistan and the region, including national water legislation and regional cooperation frameworks; and preventive and mitigating measures, such as water-saving irrigation technologies, wastewater treatment, integrated water resource management approaches, and international cooperation initiatives.

Problems of salinization and desertification and environmental protection. This independent work assignment is designed to deepen students' understanding of the causes, mechanisms, and consequences of soil salinization and desertification, and the environmental protection measures available to address them, complementing the theoretical material covered in lectures. Students are required to independently study the following: the concept and classification of soil salinization, including primary (natural) and secondary (human-induced) salinization, and its main causes, such as improper irrigation practices, poor drainage systems, and rising groundwater levels; the concept and driving factors of desertification, including climate change, overgrazing, deforestation, and unsustainable agricultural practices, along with its main indicators and stages; the environmental, agricultural, and socioeconomic consequences of salinization and desertification, such as declining soil fertility and crop yields, loss of vegetation cover, increased dust storms, and rural livelihood insecurity; the specific scale and drivers of salinization and desertification in Uzbekistan and the Aral Sea region, including the role of the drying Aral Sea in accelerating desertification in the surrounding areas; and the main preventive and remediation approaches, including improved irrigation and drainage management, soil reclamation techniques, afforestation and reforestation programs, and sustainable land management practices.

The role of plants and animals in the biosphere. The flora and fauna of Uzbekistan and their protection. This independent work assignment is designed to deepen students' understanding of the ecological functions performed by plants and animals within the biosphere, and of the specific conservation challenges facing the flora and fauna of Uzbekistan, complementing the theoretical material covered in

lectures. Students are required to independently study the following: the fundamental ecological roles of plants and animals in the biosphere, including the role of plants as primary producers in energy flow and biogeochemical cycles (carbon, oxygen, and nitrogen cycles), and the role of animals in nutrient cycling, pollination, seed dispersal, and the regulation of ecosystem balance through trophic relationships; the concept of biodiversity and its significance for ecosystem stability and resilience; the composition and characteristics of Uzbekistan's flora and fauna, including the diversity of ecosystems represented (desert, steppe, mountain, and wetland habitats) and species endemic to or characteristic of the region; the main threats to flora and fauna in Uzbekistan, such as habitat loss, overexploitation, poaching, and the impacts of climate change and desertification; and the national and international mechanisms for the protection of flora and fauna, including protected areas and nature reserves, the Red Book of Uzbekistan, and relevant national legislation and international conventions (such as CITES and the Convention on Biological Diversity).

The “Red Book” of the Republic of Uzbekistan and its role in environmental protection. This independent work assignment is designed to deepen students' understanding of the Red Book of the Republic of Uzbekistan as a scientific and legal instrument for biodiversity conservation, complementing the theoretical material covered in lectures. Students are required to independently study the following: the concept, history, and purpose of the Red Book as an internationally recognized mechanism for documenting species under threat of extinction, and the historical development of the Red Book of Uzbekistan since its establishment; the structure and classification system of the Red Book, including the categories of conservation status (such as extinct, critically endangered, vulnerable, and rare species) and the criteria used to assign species to each category; the composition of the Red Book of Uzbekistan, including representative examples of endangered plant and animal species and the ecosystems in which they are found; the legal status and practical significance of the Red Book, including its role in shaping national environmental legislation, guiding conservation planning, and regulating activities that may affect listed species; and the main challenges and measures involved in the conservation of Red Book species in Uzbekistan, such as habitat protection, breeding and reintroduction programs, and public awareness initiatives.

Specially protected natural areas in the Republic of Uzbekistan and their role in environmental protection.

This independent work assignment is designed to deepen students' understanding of the system of specially protected natural areas in Uzbekistan and their contribution to biodiversity conservation and ecological sustainability, complementing the theoretical material covered in lectures. Students are required to independently study the following: the concept and classification of specially protected natural areas, including categories such as state nature reserves (zapovedniks), national parks, state nature reserves for wildlife (zakazniks), nature monuments, and biosphere reserves, along with the distinct conservation objectives and legal regimes associated with each category; the historical development and current network of specially protected natural areas in Uzbekistan, including their geographic distribution across different ecosystem types (desert, mountain, wetland, and riparian habitats); the ecological functions of protected areas, such as preserving representative and unique ecosystems, safeguarding habitats for endangered and endemic species, maintaining ecological corridors, and supporting scientific research and environmental monitoring; the institutional and legal framework governing protected areas in Uzbekistan, including relevant national legislation and the government bodies responsible for their management; and the main challenges facing the protected area system, such as insufficient funding, pressure from adjacent land use, and the need for expanded coverage, along with international experience and cooperation relevant to strengthening the system.

Achieving the Sustainable Development Goals. This independent work assignment is designed to deepen students' understanding of the mechanisms, indicators, and challenges involved in achieving the United Nations Sustainable Development Goals (SDGs), complementing the theoretical material covered in lectures. Students are required to independently study the following: the structure of the SDG framework, including the 17 goals, their associated targets, and the global indicator system used to measure progress; the methods and institutions responsible for monitoring and reporting SDG progress at the global level (such as the UN's annual SDG reports) and at the national level; the concept of SDG interlinkages, including synergies and trade-offs among different goals (for example, between economic growth and environmental sustainability, or between energy access and

	climate action); the main global and regional challenges hindering progress toward the SDGs, such as financing gaps, data limitations, conflict, and the impacts of climate change; and the specific progress, achievements, and remaining gaps in Uzbekistan's efforts to achieve the SDGs, including national reporting mechanisms and priority areas requiring accelerated action. Issues of developing environmental education in the Republic of Uzbekistan. This independent work assignment is designed to deepen students' understanding of the current state, challenges, and development priorities of environmental education in Uzbekistan, complementing the theoretical material covered in lectures. Students are required to independently study the following: the concept and objectives of environmental education, including its role in shaping ecological awareness, values, and behavior across different age groups and levels of society; the current structure of environmental education in Uzbekistan, from preschool and general secondary education through vocational and higher education, including relevant curricula, subjects, and specialized programs; the role of non-formal environmental education, including public awareness campaigns, the involvement of civil society organizations, mass media, and international partners in promoting ecological literacy; the main challenges hindering the development of environmental education in Uzbekistan, such as limited integration across educational levels, insufficient teacher training, a shortage of localized educational materials, and weak links between environmental education and practical civic engagement; and relevant national policy documents and international experience related to strengthening environmental education, including the concept of raising the population's ecological culture up to 2030.
Teaching Methods	The course is delivered through interactive lectures, seminar discussions, case studies, problem-based learning, group work, classroom debates, multimedia presentations, and independent study. Practical learning is supported through the analysis of environmental issues, field observations where appropriate, project-based assignments, presentations, and the evaluation of real-world ecological case studies. Students are encouraged to participate actively in discussions, develop critical thinking skills, and apply sustainable environmental principles to practical situations.
Prerequisites	Ekalogiya va atrof-muhit muhafazasi (AMYAR202)

Exams and Assessment	Student performance is assessed through continuous assessment (20%), midterm assessment (30%), and a final examination (50%). The continuous assessment (20%) evaluates classroom participation, attendance, seminar activities, quizzes, presentations, independent study tasks, case study analyses, and group assignments related to environmental issues. The midterm assessment (30%) consists of written tests, short-answer and analytical questions, practical assignments, and the evaluation of students' understanding of ecological concepts, environmental protection principles, and sustainable development. The final examination (50%) is a comprehensive written examination that assesses students' knowledge of ecology, biodiversity conservation, natural resource management, environmental legislation, sustainable development, and their ability to analyze and solve environmental problems using scientific approaches.
Recommended Literature	1. X. X. Zokirov, M. B. Abramov, et al. "Ecology and Environmental Protection" Textbook. 2025. 285 pp. 2. M. Abramov, X. Zokirov "Ecology and Nature Conservation" // Textbook. – Tashkent, PUBLISHING HIGH FUTURE LLC, 2024. - 219 pp. 3. Ministry of Ecology, Environmental Protection and Climate Change of the Republic of Uzbekistan. (2023). National Report on the State of the Environment: Uzbekistan. International Institute for Sustainable Development. 4. Mirziyoyev, S. M. "We will build our great future together with our brave and noble people. " Uzbekistan, 2017. 5. Michael Begon, Colin R. Townsend, John L. Harper / Ecology: from individuals to ecosystems – 4th ed. P 759. (2006). 6. Voluntary NATIONAL REPORT of Uzbekistan on the implementation of the national goals and objectives on sustainable development for 2030. Tashkent, 2023.
Responsible Lecturer	Mukhamadi Abramov, Candidate of Biological Sciences, Associate Professor Mukhriddin Qodirov, Teacher
Field	Description
Module Title	PROTECTION OF THE ENVIRONMENT AND GREEN DEVELOPMENT
Module Code	AMYAR302

Credit Points (ECTS)	Credits
Workload	Total workload: 60 hours Contact hours: 30 hours (14 hours Lecture, 16 hours Practical lessons) S
Frequency/Duration	V semester
Module objective	Students with the scientific and theoretical foundations of environmental protection and green development, The course aims to teach students the scientific and theoretical foundations of environmental protection and green development, environmental problems and the factors that cause them, their changes over time and space, environmental assessment, and the ecological principles of nature conservation and the rational use of natural resources.
Module Learning Outcomes	Knowledge: LO1. Fundamentals of the "Environmental Protection and Green Development" course, and an understanding of environmental phenomena and their prevention methods. LO2. Reforms being carried out in environmental protection and green development. LO3. The role of society in environmental protection and green development. LO4. Must have the skills to describe the application methods of theoretical issues in environmental protection and green development based on environmental laws. In terms of skills. Skills: LO5. For environmental protection and green development, they should have skills such as conserving and caring for the environment, and preserving, protecting, and increasing the biological resources that are important for the people in the environment. Competencies: LO6. They will have the competence to independently draw conclusions and make decisions on environmental issues. LO7. They will acquire the competence to carry out practical activities for environmental protection and to promote ecological culture.
Content	Lectures Subject, purpose, and objectives of the discipline of environmental protection and green development.

Introduces the scope and goals of the discipline within broader environmental studies. Explains the relationship between environmental protection and sustainable green development. Discusses the practical relevance of the discipline to current global and national challenges.

The nature, consequences, and solutions of ecological problems. Ecological culture. Reviews the origins and characteristics of major ecological problems. Discusses the social, economic, and environmental consequences of ecological degradation. Introduces the concept of ecological culture as a foundation for sustainable behavior.

Ways to adapt to the effects of climate change and to use natural resources wisely. Explains key adaptation strategies for coping with climate change impacts. Discusses principles of sustainable and efficient natural resource use. Reviews examples of successful adaptation and resource management practices.

Conservation of biodiversity. Introduces the concept and importance of biodiversity conservation. Discusses major threats facing ecosystems and species diversity. Reviews strategies and policies aimed at protecting biological diversity.

Sustainable Development Goals. Introduces the framework and purpose of the Sustainable Development Goals (SDGs). Discusses the environmental dimensions among the broader set of goals. Reviews how the SDGs guide national and international environmental policy.

"Green energy" as a factor of sustainable development. Explains the role of renewable and green energy sources in sustainable development. Discusses the environmental and economic benefits of transitioning to green energy. Reviews challenges and opportunities in expanding green energy adoption.

Environmental protection issues in implementing the "Uzbekistan-2030" strategy. Introduces the environmental protection components of the "Uzbekistan-2030" strategy. Discusses specific goals and initiatives related to environmental sustainability within the strategy. Reviews the expected impact of the strategy on the country's environmental future.

Practical lessons

Transition to a green economy: theory, practice, and opportunities in Uzbekistan. Reviews the theoretical foundations underlying the transition to a green economy. Discusses practical steps and policy measures taken in

Uzbekistan toward this transition. Analyzes opportunities and challenges specific to the national context.

Climate Change and Ecological Adaptation. Examines the observed and projected impacts of climate change on local and global ecosystems. Discusses adaptation strategies employed to reduce climate-related risks. Analyzes case studies illustrating successful ecological adaptation measures.

Concept for enhancing the ecological culture of the population in Uzbekistan for the period up to 2030. Reviews the goals and framework of the national ecological culture enhancement concept. Discusses planned initiatives and their expected outcomes through 2030. Analyzes the role of public education in achieving these goals.

Effective use of water resources. Discusses principles of sustainable and efficient water resource management. Analyzes current challenges related to water scarcity and usage practices. Reviews practical measures for improving water use efficiency.

Waste problem, recycling and zero-waste technologies. Examines the scope and impact of waste generation and management challenges. Discusses recycling methods and their role in reducing environmental harm. Reviews zero-waste technologies and their potential for broader adoption.

Sustainable Development Goals in Uzbekistan. Reviews the specific Sustainable Development Goals prioritized within Uzbekistan's national agenda. Discusses progress and challenges in achieving these goals. Analyzes the role of policy and institutional frameworks in supporting the SDGs.

Green energy, development and prospects of renewable energy sources. Discusses the current state and development of renewable energy sources. Analyzes the economic and environmental prospects of expanding green energy use. Reviews policy measures supporting renewable energy adoption.

International cooperation in environmental protection. Examines frameworks and agreements supporting international environmental cooperation. Discusses Uzbekistan's participation in global environmental initiatives. Analyzes the benefits and challenges of cross-border environmental collaboration.

Independent study

Development of Ecology in Uzbekistan. Traces the historical development of ecological science and practice within Uzbekistan. Discusses key institutions and milestones shaping

	the field. Reviews current directions in national ecological research and policy. Environmental problems and ways to prevent them. Reviews major environmental problems affecting local and global communities. Discusses common prevention and mitigation strategies. Explores the role of policy and individual action in addressing these problems. Habitats of living organisms and the adaptations of organisms to them. Explains the concept of habitat and its role in supporting biological life. Discusses how organisms adapt structurally and behaviorally to their environments. Reviews examples of adaptation across different habitat types. Mechanisms of various environmental factors affecting living organisms. Reviews key environmental factors influencing living organisms, such as temperature, light, and water availability. Discusses the mechanisms through which these factors affect organism physiology and behavior. Explores examples of organism responses to environmental stress. Human Demographics and Environmental Protection. Discusses the relationship between population growth and environmental pressures. Reviews demographic trends and their implications for resource use. Explores strategies for balancing demographic needs with environmental protection. The Concept of the Noosphere and Environmental Protection. Introduces the concept of the noosphere as a sphere of human thought and influence on the planet. Discusses its relevance to modern environmental protection theory. Reviews the implications of this concept for sustainable human development. Mining, Chemistry, Metallurgy Industry and Environmental Protection. Reviews the environmental impacts associated with mining, chemical, and metallurgical industries. Discusses regulatory and technological measures for reducing industrial environmental harm. Explores case studies of environmental management in these sectors. Climate Change and Ecological Adaptation. Reviews the causes and observed effects of climate change on ecosystems. Discusses adaptation strategies employed to reduce ecological vulnerability. Explores examples of successful climate adaptation initiatives.
Teaching Methods	The module is delivered through interactive lectures, inquiry-based and problem-based learning, field observations, laboratory analyses, computer simulations, seminars, group

	discussions, and project-based activities. Students analyze environmental data and case studies related to green development and develop observational, analytical, and research skills. Independent learning and digital educational resources further promote critical thinking, ecological awareness, and the application of modern environmental protection concepts.
Prerequisites	Environmental Protection and Green Development
Exams and Assessment	Student performance is assessed through continuous assessment (20%), midterm assessment (30%), and a final written examination (50%). The continuous assessment (20%) evaluates classroom participation, laboratory performance, field observation activities, seminar presentations, and the successful completion of independent study assignments, including analysis of local environmental problems, biodiversity conservation case studies, water resource use reports, and assessments of green energy initiatives. The midterm assessment (30%) consists of written assignments, short in-class quizzes, and the evaluation of independent study projects, such as a report on the transition to a green economy in Uzbekistan, a climate change adaptation case study, a presentation on the "Uzbekistan-2030" environmental strategy, and an essay on sustainable development goals. The final assessment (50%) is a written examination consisting of theoretical questions and problem-solving tasks designed to assess students' understanding of environmental protection and green development, analytical skills, and their ability to apply ecological principles to practical problems. Successful completion of all independent study assignments is required as an essential component of the module assessment. .
Recommended Literature	1. M. Abramotov, X. Zokirov. Ecology and Nature Conservation: A Textbook. Tashkent: PUBLISHING HIGH FUTURE LLC, 2024. 219 pp. 2. S. N. Khashimova. "Green Economy". Tashkent. "Ma'rifat". 2024. Textbook. 3. A. V. Vakhobov, Sh. X. Khajibakiyev. "Green Economy" Tashk. : "Universitet", 2020. -262 p. 4. Nigmatov, Theoretical Foundations of Ecology, Tashkent - 2014. 260 pages. 5. Odum Y. Fundamentals of Ecology. Translation of the English edition. M. ; Mir, 1975, 740 p.

	6. Mustafojev S. , O'roqov S. , Suvonov P. General Ecology. Tashkent -2006. Tursunov X. T. , Rahimova T. U. Ecology: A Textbook. Tashkent, 2006.
Responsible Lecturer	Mukhamadi Abramotov, Candidate of Biological Sciences, Associate Professor.
Field	Description
Module Title	MATHEMATICAL PROGRAMMING
Module Code	MD305
Credit Points (ECTS)	5 Credits
Workload	Total workload: 150 hours Contact hours: 60 hours (42 hours Lecture, 34 hours Practical lessons) Self-Assignments: 74 hours
Frequency/Duration	V semester
Module objective	Mathematical Programming is the science of solving optimization problems, which aims to optimally allocate resources, make decisions, and improve systems using mathematical models and algorithms. Problems of finding an optimal solution to maximize or minimize a given objective function arise in many fields, such as economics, industry, logistics, marketing, and others. In manufacturing processes, mathematical programming methods are used to solve problems such as maximizing profit, reducing transportation costs, or allocating resources efficiently. In mathematical programming, mathematical methods are used to solve problems related to the constraints of the problem being solved, as well as the number, size, and available capabilities of resources. optimal allocation of resources (financial, time, material, etc.), optimizing the distribution of raw materials or production capacity in manufacturing processes, and effectively directing marketing and advertising expenses. Mathematical programming methods are used to analyze various data and make optimal decisions when choosing a marketing strategy, evaluating investment projects, or planning production.
Module Learning Outcomes	Knowledge: LO1. Knowledge of the general concepts of the discipline of mathematical programming, namely the basic computational methods of algebra and mathematical analysis used to solve practical problems in professional activity. LO 2. Knowledge of the basic principles of developing and applying effective mathematical models using modern

	information and communication technologies, including specialized mathematical software systems; LO3. Ability to use modern computational tools to process, visualize, and analyze research results in various areas of mathematics and its applications; LO4. Must be equipped with modern tools to solve practical problems in professional activities. Skills: LO5. In today's "Information" age, not only will young people's computer literacy be increased, but they will also learn the opportunities of working with Mathematical Programming; LO 6. Studies the theoretical foundations of using personal computers in mathematical programming processes and the methods for applying them to various data structure domains; LO 7. Studies methods for the effective use of mathematical programming software systems in control systems; LO 8. Learns the skills to work with Internet systems and data structure technologies. Competencies: Studies the development trends of mathematical programming and their descriptions; It involves forming concepts about when and for what purposes to use mathematical programming software systems, and providing knowledge on how to generate data; They will apply methods to ensure safety when working with mathematical programming software systems and become familiar with their functional capabilities.
Content	Lectures Linear programming models. The subject matter of linear programming. Economic problems solvable by linear programming methods and their mathematical model. Graphical method. General formulation of a linear programming problem and its expression in various forms. Geometric interpretation of a linear programming problem. Graphical method. Solving an economic problem by the graphical method. Solving linear programming problems by algebraic methods. The basic solution of a linear programming problem and methods for finding it. The optimal condition of the basic solution. The finite optimal solution. The infeasibility condition. The rule for switching to a new basic solution. Solution of linear systems. The augmented basis method. The special linear programming problem. Geometric representation.

Substitution by the Jordan method. General concepts of the Jordan method. Duality theory in linear programming.

Simplex method. Analysis of solution of economic problems. Simplex method for a dual problem.

Basic concepts of duality theory. Complementary problems and their economic interpretation. Symmetric and non-symmetric complementary problems.

Twin problems and their solution. Formulation of a dual problem and derivation of its optimal solutions. Formulation and types of parametric linear programming problems.

Sensitivity problem and its analysis. Canonical form. Problem with a function dependent on a parameter. Problem with a free term dependent on a parameter (dual parametric programming problem). Parametric optimization of the objective function.

General concepts of inductive inference and sensitivity analysis. Transportation problem. Mathematical model and properties of the transportation problem. Methods for finding the initial basis solution of the transportation problem. The potential method for finding the optimal solution of the transportation problem.

Simplex method for solving the theory of duality and sensitivity analysis. The special transportation problem and its ϵ -correction method. The open-model transportation problem. The differential rents method (Brudno method).

Integer linear programming. Economic problems. Formulation, types, and geometric interpretation of an integer linear programming problem. The Gomory method for solving an integer programming problem.

Nonlinear programming problems. Formulation and types of nonlinear programming problems. Geometric interpretation of nonlinear programming problems. The graphical method.

Constraint-free optimization problem. Constrained extremum problem with equalities as constraints and the Lagrange method for its solution. Convex programming. Convex set. Convex functions. Extremum of a convex function. Convex programming.

Convex programming. Convex set. Convex functions. Extremum of a convex function. Convex programming.

Quadratic forms and their canonical form. Kuhn–Tucker conditions for a quadratic programming problem.

Barankin–Dorfman method for solving the quadratic programming problem. Bil method for solving the quadratic programming problem.

Gradient methods. The concept of a function gradient. Possible directions. Determining an unconditional extremum of a function with the gradient method.

Gradient methods for solving a convex programming problem. The steepest descent method. Solving a quadratic programming problem with the gradient method. Zoutendijk's feasible direction method.

Basic concepts of dynamic programming. The principle of optimality. Economic problems solved by the dynamic programming method.

General formulation of the dynamic programming problem. Bellman's functional equations. The dynamic programming method. Solving the problem of optimal investment allocation using the dynamic programming method.

Practical lessons

Economic problems that can be solved using linear programming methods and their mathematical models. This topic introduces economic problems that can be solved using linear programming techniques. Students learn how to formulate mathematical models by defining decision variables, objective functions, and system constraints to represent real-world optimization problems.

Optimality condition of the base solution. Constrained optimal solution. This topic explains the conditions under which a basic feasible solution becomes optimal. Students study optimality criteria, feasibility conditions, and methods for identifying constrained optimal solutions in linear programming.

Cycling and its elimination method (E-method). This topic examines the cycling phenomenon that may occur during the simplex algorithm. Students learn the causes of cycling and apply the E-method to prevent repeated iterations and ensure convergence to the optimal solution.

Compound problems and their economic interpretation. This topic covers the formulation and solution of compound optimization problems. Students analyze their mathematical structures and interpret the results in terms of resource allocation, production planning, and economic decision-making.

Analysis of economic problem solutions. The dual simplex method. This topic focuses on analyzing optimal solutions of economic optimization problems using sensitivity analysis and duality theory. Students also learn the dual simplex method for solving linear programming problems efficiently.

Formulation and types of parametric linear programming problems. This topic introduces parametric linear programming, where one or more parameters vary within the optimization model. Students study different types of parametric problems and analyze how parameter changes affect optimal solutions.

A problem where the objective function is a function of the parameter. This topic examines optimization problems in which the objective function depends on a parameter. Students investigate the impact of parameter variations on the objective value and determine the corresponding optimal solutions.

Methods for finding the initial base solution of a transportation problem. This topic presents methods for obtaining an initial basic feasible solution for transportation problems. Students apply techniques such as the North-West Corner Rule, Least Cost Method, and Vogel's Approximation Method.

Potential methods for finding the optimal solution of the transportation problem. This topic introduces the potential (MODI) method for optimizing transportation problems. Students learn to evaluate and improve transportation plans until an optimal solution with minimum transportation cost is obtained.

Open-model transport problem. Differential rent method (Brudno method). This topic studies transportation problems with unbalanced supply and demand. Students learn balancing techniques and apply the Brudno differential rent method to determine economically efficient transportation plans.

Formulation, types, and geometric interpretation of the integer linear programming problem. This topic covers the mathematical formulation of integer programming problems, their classification, and geometric interpretation. Students understand how integer constraints influence feasible regions and optimal solutions.

Gomori method for solving the integer programming problem. This topic introduces Gomory's cutting-plane method for solving integer programming problems. Students learn the theoretical foundations of the algorithm and apply it to obtain integer optimal solutions.

Geometric interpretation of nonlinear programming problems. The graphical method. This topic explains the geometric interpretation of nonlinear optimization problems. Students use graphical methods to visualize feasible regions and identify optimal solutions for problems involving two decision variables.

Extremum of a convex function. This topic examines the properties of convex functions and their importance in optimization. Students study conditions for local and global extrema and understand why convex optimization guarantees globally optimal solutions.

Kun–Tucker conditions for a quadratic programming problem. This topic introduces the Karush–Kuhn–Tucker (KKT) optimality conditions for constrained quadratic programming problems. Students learn how these conditions are applied to determine optimal solutions under equality and inequality constraints.

Finding an unconstrained extremum of a function by the gradient method. This topic focuses on unconstrained optimization using the gradient method. Students study iterative optimization algorithms for locating minimum or maximum points of differentiable functions.

General formulation of the dynamic programming problem.

Bellman's functional equations. This topic presents the principles of dynamic programming and Bellman's optimality principle. Students formulate multistage decision problems and solve them using Bellman's functional equations to obtain optimal strategies.

Independent study.

Economic problems that can be solved using linear programming methods and their mathematical models. This topic introduces the formulation of economic optimization problems using linear programming techniques. Students learn to develop mathematical models by defining decision variables, objective functions, and constraints for efficient resource allocation.

Optimality condition of the base solution. Constrained optimal solution. This topic explains the conditions required for a basic feasible solution to become optimal. Students study optimality criteria and methods for determining constrained optimal solutions in linear programming problems.

Cycling and its elimination method (E-method). This topic examines the cycling phenomenon that may occur during simplex

iterations. Students learn the causes of cycling and apply the E-method to eliminate repeated iterations and ensure convergence to an optimal solution.

Compound problems and their economic interpretation. This topic focuses on compound optimization problems and their mathematical formulation. Students analyze solution methods and interpret the results in terms of economic efficiency and decision-making.

Analysis of economic problem solutions. The dual simplex method. This topic covers the interpretation of optimization results and introduces the dual simplex method for solving linear programming problems. Students evaluate the economic significance of optimal solutions and resource utilization.

Formulation and types of parametric linear programming problems. This topic presents parametric linear programming models in which problem parameters vary. Students study different types of parametric problems and analyze the influence of parameter changes on optimal solutions.

A problem where the objective function is a function of the parameter. This topic explores optimization problems in which the objective function depends on one or more parameters. Students investigate how changes in parameter values affect the objective function and optimal solution.

Methods for finding the initial base solution of a transportation problem. This topic introduces methods for constructing an initial feasible solution for transportation problems. Students apply the North-West Corner Rule, Least Cost Method, and Vogel's Approximation Method.

Potential methods for finding the optimal solution of the transportation problem. This topic focuses on the potential (MODI) method for improving transportation plans. Students learn to test optimality and identify cost-effective transportation solutions.

Open-model transport problem. Differential rent method (Brudno method). This topic examines transportation problems with unbalanced supply and demand. Students study balancing procedures and apply the Brudno differential rent method to determine optimal transportation plans.

Formulation, types, and geometric interpretation of the integer linear programming problem. This topic introduces the formulation and classification of integer programming problems. Students analyze their geometric interpretation and understand the impact of integer constraints on feasible solutions.

	Gomori method for solving the integer programming problem. This topic presents Gomory's cutting-plane algorithm for solving integer programming problems. Students apply iterative cutting procedures to obtain optimal integer solutions. Geometric interpretation of nonlinear programming problems. The graphical method. This topic explains the graphical representation of nonlinear programming problems. Students identify feasible regions and determine optimal solutions using graphical analysis for problems with two variables. Extremum of a convex function. This topic introduces the properties of convex functions and their role in optimization. Students study the conditions for local and global extrema and their applications in mathematical programming. Kun–Tucker conditions for a quadratic programming problem. This topic covers the Karush–Kuhn–Tucker optimality conditions for constrained quadratic programming problems. Students apply these conditions to identify optimal solutions under equality and inequality constraints. Finding an unconstrained extremum of a function by the gradient method. This topic introduces the gradient method for solving unconstrained optimization problems. Students use iterative procedures to locate minimum or maximum points of differentiable functions. General formulation of the dynamic programming problem. Bellman's functional equations. This topic presents the principles of dynamic programming and Bellman's optimality principle. Students formulate multistage decision problems and solve them using Bellman's functional equations to obtain optimal strategies.
Teaching Methods	The course is delivered through a combination of interactive lectures, practical exercises, and problem-based learning activities. Lectures introduce the theoretical foundations of mathematical programming, optimization techniques, and their applications in engineering, economics, and management. Practical sessions emphasize the formulation and solution of optimization problems using analytical methods and specialized software tools. Teaching methods include classroom discussions, case studies, mathematical modeling, individual and group problem-solving activities, computational laboratory work, presentations, and independent learning tasks. Students are encouraged to analyze real-world optimization problems, develop mathematical models,

	interpret computational results, and apply optimization algorithms to practical decision-making scenarios. Independent study consists of literature review, solving optimization exercises, preparing assignments, implementing mathematical programming algorithms, analyzing case studies, and completing project-based tasks that strengthen analytical thinking, computational skills, and the practical application of optimization methods.
Prerequisites	Calculus Probability and Statistics Physics Programming
Exams and Assessment	Student performance is assessed through continuous assessment (20%), a midterm assessment (30%), and a final written and practical examination (50%). The continuous assessment (20%) evaluates classroom participation, attendance, problem-solving activities, homework assignments, quizzes, practical exercises, laboratory performance, mathematical modeling tasks, and the successful completion of independent learning activities related to optimization techniques and mathematical programming. The midterm assessment (30%) consists of written tests, analytical problem-solving tasks, computational exercises, and practical assignments that assess students' understanding of linear programming, transportation models, integer programming, nonlinear programming, and dynamic programming methods. The final assessment (50%) includes a comprehensive written and practical examination covering all major topics of the course. Students are required to formulate mathematical models, solve optimization problems using appropriate algorithms, interpret computational results, and demonstrate their ability to apply mathematical programming techniques to real-world engineering and economic decision-making problems.
Recommended Literature	1. Jemilev N. I. Collection of Problems in Mathematical Programming: A Textbook / N. I. Jemilev, M. I. Eldelnant. – T. : O'qituvchi, 1990. – 224 p. 2. Yu. N. Kuznesov, V. I. Kuzubov, A. B. Voloshenko. Mathematical Programming: A Textbook / Yu. N. Kuznesov, V. I. Kuzubov, A. B. Voloshenko. - 2nd ed. , rev. and supplemented. - M. : Vysshaya shkola, 1980. - 300 pp. 3. I. L. Akulich. Mathematical Programming in Examples and Problems: A Textbook for Economic Specialties in Higher Education / I. L. Akulich. - M. : Higher School, 1986. - 317 pp.

	4. M. Raisov. Mathematical Programming: A Textbook / M. Raisov. - T. : "Voriz nashriyot" LLC, 2009. - 176 p. - (Ministry of Higher and Secondary Specialized Education of the Republic of Uzbekistan). - Bibliogr. : p. 174. 5. M. Raisov. Mathematical Programming: Textbook / M. Raisov. - T. : Cho'lpon, 2013. - 208 pp. - (Ministry of Higher and Secondary Specialized Education of the Republic of Uzbekistan). - Bibliography: p. 204. 6. Watson T. J. , Parana K. Quantitative Methods in Finance. M. "Finansi". 1999.
Responsible Lecturer	Choriyev Ilhom Kenjayevich, Senior Lecturer, Department of Mathematical Programming and Computer Science
Field	Description
Module Title	MATHEMATICAL MODELING
Module Code	MM305
Credit Points (ECTS)	5 Credits
Workload	Total workload: 180 hours Contact hours: 60 hours (42 hours Lecture, 34 hours Practical lessons) Self-Assignments: 74 hours
Frequency/Duration	V semester
Module objective	The "Mathematical Modeling" course is designed for students in the 60610600–Software Engineering educational track, The course is designed to provide students with the knowledge, based on modern requirements, on using mathematical modeling to control computers in the digital world and on finding solutions to the problems they encounter.
Module Learning Outcomes	Knowledge: LO1. To provide information about mathematical modeling, demonstrate the role of information technologies in each person's life and in society's development, and reveal the possibilities of mathematical modeling. LO 2. Know the basic principles of developing and applying effective mathematical models using modern information and communication technologies, including specialized mathematical software systems; LO3. Be able to use modern computational tools to process, visualize, and analyze research results in various areas of mathematics and its applications; LO4. Must be equipped with modern tools to solve practical problems in professional activities. Skills:

	LO5. In today's "Information" age, not only will young people's computer literacy be increased, but they will also learn about opportunities to work with Mathematical Modeling; LO 6. Studies the theoretical foundations of using personal computers in mathematical modeling processes and the methods for applying them to various data structure domains; LO 7. Studies methods for the effective use of mathematical modeling software systems in control systems; LO 8. Learns the skills to work with Internet systems and data structure technologies; Competencies: LO 9. Studies the development trends of mathematical modeling and their descriptions; LO 10. It consists of forming concepts about when and for what purposes to use software systems for mathematical modeling, and providing knowledge about how to generate data; LO 11. They will apply methods to ensure safety when working with software systems for mathematical modeling and become familiar with their functional capabilities.
Content	Lectures The concept of a model and modeling. Stages of modeling. A generalized mathematical model. Objectives of model building. Introduces the fundamental concepts of a model and the modeling process. Reviews the typical stages involved in constructing a model. Discusses the objectives that guide the model-building process. Types of models. The concept of a mathematical model. Mathematical modeling. Types of modeling. Properties of models. Presentation forms of models. Reviews various types of models used across different disciplines. Introduces the concept and defining properties of mathematical models. Discusses common forms for presenting and communicating models. Classification of economic and mathematical models. Basic principles of mathematical modeling. Degree of fit of a mathematical model. Stages of mathematical modeling. Reviews classifications of economic and mathematical models. Discusses fundamental principles guiding sound mathematical modeling practice. Explains how the degree of fit is assessed throughout the modeling stages. Methods for constructing and studying mathematical models. Basic requirements for mathematical models. Methods for building mathematical models. Introduces

methods for constructing and analyzing mathematical models. Discusses essential requirements a well-formed mathematical model must satisfy. Reviews systematic approaches for building models from real-world observations.

Correspondence between a mathematical model and its real-world object. Theoretical and practical research of mathematical models, their adequacy. Explains how a mathematical model's correspondence to its real-world object is evaluated. Discusses theoretical and practical approaches for researching model behavior. Reviews criteria for assessing model adequacy.

Dimensional analysis. Dimensional scaling. Scale symmetries. Introduces the principles of dimensional analysis in mathematical modeling. Discusses techniques for dimensional scaling of variables and parameters. Reviews the concept of scale symmetries and their modeling implications.

Particular solutions of differential equations. Reviews methods for identifying particular solutions to differential equations. Discusses how particular solutions relate to general solution forms. Explores examples illustrating particular solution derivation.

Using conservation laws and other methods in process modeling. Explains how conservation laws inform the construction of process models. Discusses additional modeling methods complementing conservation-based approaches. Reviews practical examples of conservation-law-based modeling.

The analogy method in mathematical modeling. Building mathematical models using the hierarchy principle. Introduces the analogy method as a modeling technique. Discusses the hierarchy principle for structuring model construction. Reviews examples illustrating both approaches in practice.

Building a mathematical model using the law of conservation of momentum. Explains how the law of conservation of momentum informs model construction. Discusses the derivation process for momentum-based models. Reviews practical examples applying this conservation law.

Asymptotic analysis. Regular perturbation problems. Introduces the principles of asymptotic analysis in mathematical modeling. Discusses the structure and solution approach for regular perturbation problems. Reviews examples illustrating asymptotic methods.

Singular perturbation problems. Boundary layer analysis. Introduces singular perturbation problems and their distinguishing characteristics. Discusses boundary layer analysis as a solution technique. Reviews examples demonstrating singular perturbation methods.

Analogy and Variational Principles in Mathematical Modeling. Reviews the use of analogy as a tool for constructing new models. Introduces variational principles and their role in modeling. Discusses examples connecting analogy-based and variational approaches.

Use of analogies in model building. Explores further applications of the analogy method in model construction. Discusses criteria for identifying suitable analogies across domains. Reviews examples illustrating analogy-based model derivation.

A Hierarchical Approach to Deriving Models. Reviews the hierarchical approach to systematically deriving mathematical models. Discusses how this approach organizes complexity across modeling stages. Explores examples demonstrating hierarchical model derivation.

Some models of competition. Biological models. A model of the predator-prey system interaction. Introduces mathematical models representing competitive interactions. Reviews the structure of biological models in this context. Discusses the predator-prey model as a classic example of system interaction.

Biological population model. Volterra-Lotka model. Model of microorganism diffusion, biosynthesis models. Reviews the structure and assumptions of biological population models. Introduces the Volterra-Lotka model in greater depth. Discusses models representing microorganism diffusion and biosynthesis processes.

Modeling the number of interacting populations. Explains modeling approaches for systems involving multiple interacting populations. Discusses how interaction terms are incorporated into such models. Reviews examples illustrating multi-population dynamics.

Model of microorganism diffusion, biosynthesis models. Revisits diffusion and biosynthesis models in greater analytical depth. Discusses mathematical formulations describing these biological processes. Reviews practical applications of such models.

Calculus of variations. Euler-Lagrange equations. Hamiltonian dynamics. Introduces the calculus of variations as a mathematical modeling tool. Explains the derivation and use of the Euler-Lagrange equations. Reviews the fundamentals of Hamiltonian dynamics in this context.

Formulation and mathematical model of the transportation problem. Theorems on the properties of solutions to the transportation problem. Open and closed model transportation problems. Methods for converting an open model into a closed model. Introduces the formulation of the transportation problem as a mathematical model. Reviews key theorems concerning properties of transportation problem solutions. Discusses the distinction between open and closed models and methods for converting between them.

Practical lessons:

The concept of modeling. Building simple mathematical models. Reviews the fundamental concept of modeling through simple examples. Constructs basic mathematical models representing everyday phenomena. Analyzes the constructed models for accuracy and simplicity.

Problems in the theoretical and applied research of mathematical models. Solves problems related to the theoretical analysis of mathematical models. Applies research techniques to assess model behavior. Discusses findings in the context of applied modeling scenarios.

Problems on constructing mathematical models using the laws of nature. Constructs mathematical models grounded in fundamental physical laws. Applies conservation and other natural laws to derive model equations. Verifies the constructed models against expected physical behavior.

Dimensional Analysis. Dimensionalization and Dimensional Parameters. Applies dimensional analysis techniques to sample modeling problems. Practices dimensionalizing variables and identifying dimensional parameters. Verifies dimensional consistency across constructed models.

Problems on constructing mathematical models using the variational principle. Applies variational principles to construct sample mathematical models. Practices deriving governing equations from variational formulations. Analyzes the resulting models for correctness.

Problems on building mathematical models using the hierarchical principle. Applies the hierarchical principle to construct layered mathematical models. Practices organizing

model components according to complexity. Analyzes the resulting hierarchical model structure.

Practical problems and steps for solving them on a computer. Implements a sample mathematical model computationally. Practices structuring a step-by-step computational solution approach. Verifies computational results against expected outcomes.

Quasi-steady state and the Michaelis-Menten model. Applies the quasi-steady state assumption to a sample kinetic model. Constructs and analyzes the Michaelis-Menten model equations. Interprets model behavior in the context of reaction kinetics.

Prey-predator interaction model. Constructs a mathematical model representing prey-predator interactions. Analyzes the model's qualitative behavior under different parameters. Interprets simulation results in ecological terms.

Volterra-Lotka model. Modeling population numbers. Implements the Volterra-Lotka model to represent population dynamics. Analyzes population trends produced by the model. Compares model behavior across varying initial conditions.

Euler-Lagrange equation. Hamiltonian dynamics. Derives the Euler-Lagrange equation for a sample system. Applies Hamiltonian dynamics principles to analyze system behavior. Verifies derived equations against known physical systems.

Waves. The Turing problem. Turing stability. Explores mathematical models representing wave phenomena. Analyzes the Turing problem and conditions for pattern formation. Discusses Turing stability in the context of reaction-diffusion systems.

Transport Problem and its Mathematical Model. Formulates the transportation problem as a mathematical model. Applies solution methods to a sample transportation scenario. Analyzes the resulting solution for optimality.

Modeling the number of interacting populations. Constructs models representing multiple interacting populations. Analyzes how interaction terms affect population dynamics. Interprets simulation results for practical insights.

Model of microorganism diffusion, biosynthesis models. Constructs diffusion and biosynthesis models for microorganism systems. Analyzes model behavior under varying parameter conditions. Interprets results in the context of biological processes.

Calculus of variations. Euler-Lagrange equations. Hamiltonian dynamics. Applies calculus of variations

techniques to a sample optimization problem. Derives and solves the corresponding Euler-Lagrange equations. Analyzes the resulting dynamics using Hamiltonian formulation.

Formulation and mathematical model of the transportation problem. Theorems on the properties of solutions to the transportation problem. Open and closed model transportation problems. Methods for converting an open model to a closed model. Formulates open and closed transportation problem models for sample scenarios. Applies relevant theorems to verify solution properties. Practices converting an open model into a closed model form.

Independent Study Topics

The concept of modeling. Building simple mathematical models. Researches additional simple modeling examples beyond those covered in class. Studies alternative approaches to constructing basic mathematical models. Prepares extra practice examples demonstrating model-building fundamentals.

Problems in the theoretical and applied research of mathematical models. Studies theoretical research methods for analyzing model behavior in greater depth. Researches applied case studies illustrating model research techniques. Prepares a written summary connecting theory to application.

Problems on constructing mathematical models using the laws of nature. Researches additional physical laws applicable to model construction. Studies examples of models derived from natural laws in various domains. Prepares extra worked examples demonstrating this construction approach.

Dimensional Analysis. Dimensionalization and Dimensional Parameters. Studies advanced dimensional analysis techniques beyond those covered in class. Researches practical applications where dimensional analysis simplifies complex problems. Prepares a summary of dimensional analysis best practices.

Problems on constructing mathematical models using the variational principle. Researches additional applications of the variational principle in modeling. Studies examples from physics and engineering employing this principle.

Problems on building mathematical models using the hierarchical principle. Studies the hierarchical principle's application across different modeling contexts. Researches examples illustrating multi-level model construction.

Practical problems and steps for solving them on a computer. Researches computational tools and approaches for

implementing mathematical models. Studies best practices for structuring computational solution workflows.

Quasi-steady state and the Michaelis-Menten model. Studies the theoretical assumptions underlying the quasi-steady state approximation. Researches additional applications of the Michaelis-Menten model in biochemistry.

Prey-predator interaction model. Researches variations and extensions of the classic prey-predator model. Studies real-world ecological examples illustrating model applicability.

Volter-Lotka model. Modeling population numbers. Studies the mathematical properties of the Volterra-Lotka model in greater depth. Researches historical and modern applications of the model.

Euler-Lagrange equation. Hamiltonian dynamics. Researches additional applications of the Euler-Lagrange equation across physical systems. Studies the relationship between Lagrangian and Hamiltonian formulations. Prepares a written summary comparing the two approaches.

Waves. The Turing problem. Turing stability. Studies the mathematical conditions underlying Turing instability in greater depth. Researches examples of pattern formation described by the Turing model.

Transport Problem and its Mathematical Model. Researches additional formulations and solution methods for the transportation problem. Studies real-world logistics examples modeled as transportation problems. Prepares a written summary of practical transportation problem applications.

Modeling the number of interacting populations. Studies extended models involving three or more interacting populations. Researches examples illustrating complex multi-population dynamics. Prepares a summary of findings on extended population modeling.

Model of microorganism diffusion, biosynthesis models. Researches additional diffusion and biosynthesis modeling approaches. Studies real-world biological examples employing such models. Prepares a written summary connecting model structure to biological processes.

Calculus of variations. Euler-Lagrange equations. Hamiltonian dynamics. Studies advanced applications of the calculus of variations in optimization problems. Researches examples connecting variational methods to Hamiltonian dynamics. Prepares a summary of key theoretical and practical connections.

	Formulation and mathematical model of the transportation problem. Theorems on the properties of solutions to the transportation problem. Studies the mathematical proofs underlying key transportation problem theorems. Researches examples illustrating theorem application in solution verification.
Teaching Methods	The module is delivered through interactive lectures, problem-based and inquiry-based learning, computational modeling laboratories, hands-on model construction exercises, computer simulations, seminars, group discussions, and project-based activities. Students construct, analyze, and simulate mathematical models of physical, biological, and economic processes using computational tools, and develop analytical, computational, and research skills. Independent learning and digital educational resources further promote critical thinking, scientific communication, and the application of modern mathematical modeling concepts.
Prerequisites	Calculus Probability and Statistics Physics Programming
Exams and Assessment	Student performance is assessed through continuous assessment (20%), midterm assessment (30%), and a final written examination (50%). The continuous assessment (20%) evaluates classroom participation, laboratory performance, model construction exercises, seminar presentations, and the successful completion of independent study assignments, including dimensional analysis reports, prey-predator model simulations, Michaelis-Menten model exercises, and transportation problem case studies. The midterm assessment (30%) consists of written assignments, short in-class quizzes, and the evaluation of independent study projects, such as a Volterra-Lotka model report, a Euler-Lagrange and Hamiltonian dynamics case study, a presentation on the Turing stability problem, and an essay on hierarchical model construction. The final assessment (50%) is a written examination consisting of theoretical questions and problem-solving tasks designed to assess students' understanding of mathematical modeling, analytical skills, and their ability to apply modeling principles to practical problems. Successful completion of all independent study assignments is required as an essential component of the module assessment.

Recommended Literature	1. Alder, M. D. An Introduction to Mathematical Modelling. This Edition. Michael D. Alder, 2001. -209 p 2. Sh. Mo'minov, Mathematical Models and Methods. Tashkent, Turon-Iqbol, 2006. -272 p. 3. Kublanov M. S. Mathematical Modeling: Methodology and Methods for Developing Mathematical Models of Mechanical Systems and Processes. Part I: Modeling of Systems and Processes. Third edition, revised and supplemented: Uchebbie-M. : MGTU GA, 2004. -108 p. 4. Mirziyoyev Sh. M. We will jointly build a free, prosperous, and democratic state of Uzbekistan. Speech at the joint session of the chambers of the Oliy Majlis dedicated to the inauguration ceremony of the President of the Republic of Uzbekistan / S. M. Mirziyoyev. - Tashkent: Uzbekistan, 2016. - 56 p. 5. Skirukhin V. I. "Mathematical Modeling" M. , 1989. 6. Edited by V. T. Shorin, "Economic-Mathematical Methods and Models for Planning and Management," Znaniye, Moscow, 1973. 7. Popov I. G. "Mathematical Methods in Planning of Industries and Enterprises. " Moscow, "Ekonomika," 1973.
Responsible Lecturer	Choriyev Ilhom Kenjayevich, Senior Lecturer, Department of Mathematical Modeling and Computer Sciences
Field	Description
Module Title	PARALLEL COMPUTER ARCHITECTURE AND PROGRAMMING
Module Code	PKAD305
Credit Points (ECTS)	5 Credits
Workload	Total workload: 150 hours Contact hours: 74 hours (42 hours Lecture, 32 hours Practical lessons) Self-Assignments: 76 hours
Frequency/Duration	V semester
Module objective	The course aims to develop theoretical knowledge and practical skills in the architecture, principles of operation, and parallel programming technologies of modern parallel computing systems. The course is aimed at developing software solutions that work effectively in parallel and distributed computing environments, and teaching methods for using the capabilities of multi-core processors, graphics processing units (GPUs), cluster systems, and high-performance computing platforms.

Module Learning Outcomes	Knowledge LO1. Have theoretical knowledge of parallel computer architectures, parallel computing systems, and Flynn classification. LO2. Explain the principles of operation of multi-core processors, GPU architecture, cluster systems, and high-performance computing (HPC) technologies. LO3. Have knowledge of parallel algorithms, synchronization mechanisms, principles of working with processes and threads, and parallel programming models. LO4. Know parallel programming technologies such as OpenMP, MPI, and CUDA, and methods for evaluating and optimizing the performance of parallel programs. Skills LO5. Acquire skills in creating multi-threaded programs, managing processes and flows, and using synchronization tools. LO6. Acquire skills in developing parallel algorithms and programs using OpenMP technology. LO7. Organize distributed computing and exchange information between processes using MPI. LO8. Acquire practical skills in organizing parallel computing using CUDA or other GPU technologies, analyzing and optimizing program performance. Qualifications LO9. Have the skills to develop effective software solutions for complex computational problems using parallel computing technologies. LO10. Have the skills to design, develop and implement parallel programs for multi-core, distributed and high-performance computing environments.
Content	Lectures Introduction to Parallel Computing Systems and History of Development Students study the fundamentals of parallel computing systems and their historical development. The topic emphasizes the evolution of parallel architectures and their role in solving computationally intensive problems. Parallel Computer Architectures and Flynn Classification Students examine different parallel computer architectures based on Flynn's classification. The topic emphasizes SISD, SIMD, MISD, and MIMD architectures and their practical applications. Multi-Core Processor Architecture Students study the architecture and operation of multi-core

processors. The topic emphasizes processor organization, parallel execution, and efficient utilization of computing resources.

Parallel Computing Models and Amdahl's Law

Students examine parallel computing models and the theoretical limits of parallel performance. The topic emphasizes Amdahl's Law and its application in evaluating parallel system scalability.

Processes and Threads

Students study the concepts of processes and threads in parallel computing environments. The topic emphasizes process management, thread creation, communication, and concurrent execution.

Principles of Parallel Algorithm Design

Students examine the techniques used for designing efficient parallel algorithms. The topic emphasizes task decomposition, workload balancing, and minimizing communication overhead.

Synchronization Mechanisms and Deadlock

Students study synchronization techniques for coordinating concurrent processes and threads. The topic emphasizes mutual exclusion, synchronization primitives, deadlock conditions, and prevention strategies.

Introduction to OpenMP Technology. Students examine the fundamentals of OpenMP for shared-memory parallel programming. The topic emphasizes compiler directives, execution models, and parallel program development.

Parallel Programming with OpenMP. Students study the implementation of parallel applications using OpenMP. The topic emphasizes parallel regions, work-sharing constructs, synchronization, and performance improvement.

Basics of MPI (Message Passing Interface). Students examine the principles of the Message Passing Interface for distributed-memory computing. The topic emphasizes communication models, message passing operations, and distributed application development.

Distributed Computing with MPI

Students study distributed computing techniques using MPI libraries. The topic emphasizes process communication, collective operations, and scalable distributed program design.

Basics of GPU Architecture and CUDA Programming

Students examine the architecture of graphics processing units and CUDA programming fundamentals. The topic emphasizes massively parallel processing and GPU-based application development.

Parallel Computing with GPU. Students study techniques for implementing parallel algorithms on graphics processing units. The topic emphasizes kernel programming, memory optimization, and computational acceleration.

Cluster Systems and High Performance Computing (HPC)

Students examine the architecture and operation of cluster computing systems and high-performance computing environments. The topic emphasizes distributed resources, scalability, and scientific computing applications.

Authentication and Authorization Systems

Students study authentication and authorization mechanisms used in distributed and parallel computing environments. The topic emphasizes secure access control, identity management, and resource protection.

Performance Evaluation of Parallel Programs

Students examine methods for measuring and evaluating the performance of parallel applications. The topic emphasizes speedup, efficiency, scalability, and benchmarking techniques.

Parallel Program Optimization Methods

Students study optimization techniques for improving the performance of parallel programs. The topic emphasizes load balancing, memory optimization, communication reduction, and execution efficiency.

Cloud Computing and Parallel Programming

Students examine cloud computing technologies that support parallel application execution. The topic emphasizes virtualized computing resources, distributed processing, and cloud-based scalability.

Big Data and Parallel Computing Technologies

Students study the application of parallel computing technologies for processing large-scale datasets. The topic emphasizes distributed data processing frameworks and high-performance analytics.

Parallel Computing in Artificial Intelligence and Machine Learning.

Students examine the role of parallel computing in artificial intelligence and machine learning applications. The topic emphasizes accelerating model training, large-scale data processing, and computational optimization.

Security and Reliability Issues in Parallel Systems

Students study security challenges and reliability requirements in parallel computing systems. The topic emphasizes fault tolerance, secure communication, data integrity, and system resilience.

Modern Development Trends in Parallel Computing Technologies. Students examine emerging technologies and current research directions in parallel computing. The topic emphasizes advances in hardware architectures, programming models, cloud platforms, and AI-driven high-performance computing.

Practics

Installing Parallel Programming Tools in a Linux Environment. Students install and configure the software tools required for parallel programming in a Linux environment. The topic emphasizes preparing development environments and verifying compiler and library configurations.

Working with POSIX Threads (Pthreads). Students implement multithreaded applications using the POSIX Threads library. The topic emphasizes thread creation, management, and concurrent execution in shared-memory systems.

Creating Multithreaded Programs
Students develop applications that execute multiple threads simultaneously. The topic emphasizes thread coordination, workload distribution, and improving computational efficiency.

Synchronization Using Mutex and Semaphore
Students implement synchronization mechanisms using mutexes and semaphores. The topic emphasizes preventing race conditions and ensuring safe access to shared resources.

Detecting and Resolving Deadlocks
Students analyze common deadlock scenarios in parallel programs. The topic emphasizes deadlock detection techniques, prevention strategies, and resource management.

Creating Parallel Loops Using OpenMP
Students implement parallel loops using OpenMP directives. The topic emphasizes loop parallelization, workload scheduling, and efficient utilization of processor cores.

Working with Shared and Private Memory in OpenMP
Students implement OpenMP programs using shared and private memory variables. The topic emphasizes memory management, variable scope, and data consistency in parallel execution.

Creating and Running MPI Programs
Students develop distributed applications using the Message Passing Interface (MPI). The topic emphasizes process creation, execution, and communication across multiple computing nodes.

Data Exchange Operations in MPI
Students implement point-to-point communication using MPI

message passing functions. The topic emphasizes efficient data exchange between distributed processes.

MPI Collective Communication Practice. Students perform collective communication operations using MPI libraries. The topic emphasizes broadcasting, gathering, scattering, and reduction techniques for distributed applications.

Working with the CUDA Environment. Students configure and use the CUDA development environment for GPU programming. The topic emphasizes compiling, executing, and debugging CUDA-based parallel applications.

Performing Vector and Matrix Operations on the GPU
Students implement parallel vector and matrix computations using CUDA. The topic emphasizes GPU acceleration, memory optimization, and high-performance numerical processing.

Performance Analysis of Parallel Algorithms
Students evaluate the performance of parallel algorithms using benchmarking techniques. The topic emphasizes measuring execution time, speedup, scalability, and computational efficiency.

Organization of Parallel Computing in a Cluster Environment
Students implement parallel applications within a cluster computing environment. The topic emphasizes distributed resource management, cluster configuration, and scalable execution.

Final Parallel Software Project. Students develop a complete parallel computing application by integrating the concepts learned throughout the course. The topic emphasizes software design, implementation, optimization, testing, and documentation.

Project Presentation and Defense. Students present and defend their completed parallel computing projects. The topic emphasizes demonstrating technical implementation, evaluating performance results, and communicating project outcomes effectively.

Student Independent Work

History of Parallel Computing Technologies Development
Students study the historical development of parallel computing technologies from early systems to modern high-performance platforms. The topic emphasizes major technological milestones and the evolution of parallel processing techniques.

Modern Interpretation of Flynn Classification
Students examine the modern interpretation of Flynn's classification of computer architectures. The topic emphasizes

contemporary implementations of SISD, SIMD, MISD, and MIMD systems in modern computing.

Comparison of NUMA and UMA Architectures. Students compare Non-Uniform Memory Access (NUMA) and Uniform Memory Access (UMA) architectures. The topic emphasizes memory organization, processor communication, and performance characteristics.

Gustafson's Law and Parallel Efficiency. Students study Gustafson's Law as a model for evaluating scalable parallel computing systems. The topic emphasizes parallel efficiency, workload scalability, and performance analysis.

Load Balancing Methods in Parallel Algorithms Students examine techniques for distributing computational workloads among processing units. The topic emphasizes static and dynamic load balancing methods to improve parallel program performance.

Race Condition Problem in Parallel Computing. Students study race conditions that occur in concurrent execution environments. The topic emphasizes identifying synchronization issues and applying techniques to ensure correct program behavior.

Lock-Free and Wait-Free Algorithms. Students examine lock-free and wait-free programming techniques for concurrent systems. The topic emphasizes non-blocking synchronization methods that improve scalability and reduce contention.

Hybrid Programming (MPI + OpenMP) Technology Students study hybrid parallel programming using MPI and OpenMP together. The topic emphasizes combining distributed-memory and shared-memory programming models for high-performance applications.

OpenACC Technology and Applications Students examine the OpenACC programming model for accelerator-based computing. The topic emphasizes compiler directives, GPU acceleration, and simplified parallel application development.

Comparison of NVIDIA CUDA and AMD ROCm Platforms Students compare the CUDA and ROCm platforms for heterogeneous computing. The topic emphasizes programming models, hardware support, performance, and software ecosystem differences.

Tensor Processing Unit (TPU) Architecture Students study the architecture and operational principles of Tensor Processing Units. The topic emphasizes hardware

	acceleration for artificial intelligence and machine learning workloads. Apache Hadoop and MapReduce Technologies. Students examine the Hadoop ecosystem and the MapReduce programming model for distributed data processing. The topic emphasizes large-scale data storage, parallel computation, and fault tolerance. Apache Spark and In-Memory Parallel Computing. Students study Apache Spark as a framework for high-performance distributed data processing. The topic emphasizes in-memory computation, parallel analytics, and scalable big data applications. Grid Computing Technologies. Students examine the principles and architecture of grid computing systems. The topic emphasizes distributed resource sharing, collaborative computing, and large-scale scientific applications. Edge and Fog Computing Architectures Students study edge and fog computing architectures for distributed data processing. The topic emphasizes low-latency computation, decentralized processing, and integration with cloud computing environments.
Teaching Methods	The module is delivered through interactive lectures, inquiry-based and problem-based learning, hands-on parallel programming laboratories, computer simulations, seminars, group discussions, and project-based activities. Students design, implement, analyze, and optimize parallel algorithms using modern parallel computing frameworks and programming models such as OpenMP, MPI, CUDA, and OpenACC. They develop skills in parallel system architecture, performance analysis, synchronization, load balancing, debugging, and scalability while solving computationally intensive problems. Independent learning and digital educational resources further promote critical thinking, collaborative software development, scientific communication, and the application of contemporary parallel computing technologies in scientific, engineering, and data-intensive applications.
Prerequisites	Programming (DAS1110) Database (MAB1306) Computer Networks (KOT1306)
Exams and Assessment	Type of examination (written, oral, project) and its weight Student performance is assessed through continuous assessment (20%), midterm assessment (30%), and a final written examination (50%).

	The continuous assessment (20%) evaluates students' participation in lectures and practical classes, parallel programming activities, and completion of independent study assignments, including OpenMP, MPI, and CUDA programming tasks, multithreaded programming exercises, performance analysis, cluster computing projects, presentations, technical reports, and practical assignments related to parallel algorithms, GPU computing, and high-performance computing (HPC). The midterm assessment (30%) consists of a written test and the evaluation of independent study projects, such as reports, presentations, analytical papers, and practical assignments covering parallel computer architectures, parallel algorithms, OpenMP, MPI, CUDA, synchronization mechanisms, distributed computing, cluster systems, and performance optimization. The final assessment (50%) is a written examination consisting of theoretical and practical questions designed to assess students' understanding of parallel computer architecture, parallel programming technologies, GPU computing, distributed and high-performance computing systems, performance optimization, and their ability to design, implement, and evaluate efficient parallel software solutions. Successful completion of all independent study assignments is required as an essential component of the module assessment.
Recommended Literature	1.Introduction to Parallel Programming — Subodh Kumar. – 2022. 2.Programming Massively Parallel Processors: A Hands-on Approach — Wen-mei W. Hwu, David B. Kirk, Izzat El Hajj. – 2022. 3.Parallel Programming: for Multicore and Cluster Systems — Thomas Rauber, Gudula Rünger. – 2023. 4.Programming Your GPU with OpenMP — Tom Deakin, Timothy G. Mattson. – 2023. 5.Computer Architecture: A Quantitative Approach (6th Edition) — John L. Hennessy, David A. Patterson. – 2021. 6.Heterogeneous Computing with OpenCL 2. 0 — David R. Kaeli, Perhaad Mistry, Dana Schaa, Dong Ping Zhang. – 2021. 7.High Performance Parallel Runtimes: Design and Implementation — Yves Robert, Frédéric Vivien. – 2021.
Responsible Lecturer	Bobomurodov Jasurbek Abdirasulovich teacher
Field	Description
Module Title	MOBILE APPLICATION DEVELOPMENT

Module Code	MII305
Credit Points (ECTS)	5 Credits
Workload	Total workload: 150 hours Contact hours: 74 hours (42 hours Lecture, 32 hours Practical lessons) Self-Assignments: 76 hours
Frequency/Duration	V semester
Module objective	To develop knowledge, skills, and competencies in the theoretical foundations of mobile application development, modern mobile platform architecture, design, development, testing, optimization, and deployment of mobile applications for Android and iOS platforms using the Flutter framework and the Dart programming language.
Module Learning Outcomes	Knowledge LO11. Have theoretical knowledge of mobile computing technologies, mobile platforms and mobile operating system architecture. LO12. Explain the basic principles of the Flutter framework, the Dart programming language and mobile application development methodologies. LO13. Know the principles of mobile application interface, navigation, data management and user experience design. LO14. Have knowledge of mobile application security, testing, optimization and application deployment technologies. Skills LO15. Gain skills in creating mobile applications and developing user interfaces in the Flutter environment. LO16. Integrate mobile applications with databases, REST APIs, and Firebase services. LO17. Use authentication, data storage, and state management mechanisms in mobile applications. LO18. Gain practical skills in testing, troubleshooting, and optimizing the performance of mobile applications. Qualifications LO19. Acquire the skills to independently develop mobile applications that solve real-world practical problems based on Flutter technology. LO20. Acquire the skills to prepare and deploy mobile applications on Google Play and App Store platforms.
Content	Lectures Introduction to Mobile Computing Technologies and the Evolution of Mobile Platforms. Students study the

fundamentals of mobile computing technologies and the historical evolution of mobile platforms. The topic emphasizes the development of mobile devices, operating systems, and their role in modern digital ecosystems.

Architecture of Mobile Operating Systems (Android and iOS). Students examine the architecture of Android and iOS operating systems. The topic emphasizes system components, application lifecycle, memory management, and platform-specific features.

Methodologies for Developing Mobile Applications Students study software development methodologies applied to mobile application projects. The topic emphasizes planning, implementation, testing, deployment, and maintenance using modern development practices.

Concept of Cross-Platform Programming and Flutter Technology. Students examine the principles of cross-platform mobile application development using Flutter. The topic emphasizes code reusability, development efficiency, and multi-platform deployment.

Fundamentals of the Dart Programming Language. Students study the core features of the Dart programming language used in Flutter development. The topic emphasizes syntax, object-oriented programming, and essential language constructs.

Architecture of the Flutter Framework. Students examine the architecture and internal structure of the Flutter framework. The topic emphasizes the widget tree, rendering engine, and interaction between Flutter and the Dart runtime.

Concept of Widgets and Principles of UI Creation. Students study widgets as the fundamental building blocks of Flutter user interfaces. The topic emphasizes reusable components, interface composition, and interactive application design.

Material Design and Cupertino Design Systems. Students examine the Material Design and Cupertino design systems for mobile applications. The topic emphasizes platform-specific interface components and consistent user experience design.

Layouts and Responsive Interface Design. Students study responsive layout techniques for mobile application interfaces. The topic emphasizes adaptive design, screen responsiveness, and flexible arrangement of interface elements.

Navigation and Routing Technologies. Students examine navigation mechanisms and routing strategies in mobile applications. The topic emphasizes screen transitions, route management, and data exchange between application pages.

Data Management in Mobile Applications (State Management). Students study techniques for managing application state in mobile software. The topic emphasizes efficient data sharing, state synchronization, and scalable application architecture.

Local Databases and Data Storage. Students examine local data storage technologies used in mobile applications. The topic emphasizes persistent data management, offline functionality, and efficient database operations.

Integration with REST API and Web Services. Students study methods for integrating mobile applications with REST APIs and web services. The topic emphasizes HTTP communication, data retrieval, and interaction with remote servers.

JSON Data Processing. Students examine techniques for processing JSON data within mobile applications. The topic emphasizes serialization, deserialization, and structured data exchange with web services.

Authentication and Authorization in Mobile Applications Students study authentication and authorization mechanisms for securing mobile applications. The topic emphasizes user identity verification, access control, and secure session management.

Using Firebase Services. Students examine the integration of Firebase services into mobile applications. The topic emphasizes authentication, cloud databases, storage, analytics, and backend service implementation.

Mobile Application Security. Students study the principles of mobile application security and secure software development. The topic emphasizes data protection, encryption, secure coding practices, and mitigation of common security threats.

Mobile Application Testing and Quality Control. Students examine testing techniques and quality assurance processes for mobile applications. The topic emphasizes unit testing, integration testing, usability evaluation, and software reliability.

Optimizing Mobile Application Performance. Students study methods for improving the performance and efficiency of mobile applications. The topic emphasizes resource optimization, memory management, and application responsiveness. .

Deploying Mobile Applications on Google Play and App Store Platforms. Students examine the process of publishing mobile applications on official application stores. The topic emphasizes application packaging, signing, deployment requirements, and release management.

Modern Mobile Technologies and the Development Prospects of the Flutter Ecosystem. Students study emerging technologies and future trends in mobile application development. The topic emphasizes innovations within the Flutter ecosystem and their impact on modern cross-platform software development.

Practices

Setting up the Flutter SDK and Visual Studio Code/Android Studio. Students configure the Flutter development environment using Flutter SDK, Visual Studio Code, or Android Studio. The topic emphasizes proper installation, project configuration, and emulator setup for mobile application development.

Building Your First Flutter App. Students create and run their first Flutter application to understand the basic development workflow. The topic emphasizes project structure, application lifecycle, and fundamental Flutter concepts.

Working with Stateless and Stateful Widgets. Students study the implementation of Stateless and Stateful widgets in Flutter applications. The topic emphasizes widget lifecycle, state management, and the creation of interactive user interfaces.

Building a User Interface Using Layout Widgets. Students develop responsive user interfaces using Flutter layout widgets. The topic emphasizes arranging interface components efficiently across different screen sizes and device orientations.

Accepting Form Elements and User Data. Students implement forms for collecting and validating user input in mobile applications. The topic emphasizes handling text fields, form validation, and secure data entry techniques.

Navigation and Data Transfer Between Pages. Students implement navigation between application screens and transfer data across routes. The topic emphasizes route management, parameter passing, and efficient navigation strategies.

Creating ListView, GridView and Dynamic Interfaces. Students develop dynamic user interfaces using ListView and GridView widgets. The topic emphasizes efficient presentation of collections, scrolling behavior, and dynamic data rendering.

State Management Using Provider. Students implement application state management using the Provider package. The topic emphasizes managing shared data efficiently while maintaining scalable and organized application architecture.

Working with SharedPreferences and SQLite. Students implement local data storage using SharedPreferences and

SQLite. The topic emphasizes persistent storage, data retrieval, and offline application functionality.

Integrating with REST API. Students integrate mobile applications with RESTful web services. The topic emphasizes sending HTTP requests, handling responses, and communicating with remote servers.

Receiving and Processing JSON Data
Students process JSON data received from web services within Flutter applications. The topic emphasizes serialization, deserialization, and efficient data handling techniques.

Navigation and Data Transfer Between Pages. Students extend their knowledge of multi-screen application development using advanced navigation techniques. The topic emphasizes managing complex navigation flows and passing structured data between pages.

Implementing Firebase Authentication. Students implement secure user authentication using Firebase Authentication services. The topic emphasizes user registration, login functionality, and authentication management.

Working with Cloud Firestore. Students develop cloud-based applications using Cloud Firestore for real-time data storage. The topic emphasizes database operations, synchronization, and scalable cloud data management.

Push Notification and Firebase Cloud Messaging
Students implement push notification services using Firebase Cloud Messaging. The topic emphasizes configuring notifications, message delivery, and improving user engagement.

Developing the Final Project. Students design and implement a complete Flutter-based mobile application by integrating the knowledge acquired throughout the course. The topic emphasizes software design, implementation, testing, optimization, and project presentation.

Student Independent Work

History of Development of Mobile Operating Systems
Students study the historical evolution of mobile operating systems from early platforms to modern ecosystems. The topic emphasizes key technological milestones, architectural changes, and their influence on contemporary mobile computing.

Modern Trends in the Mobile Application Development Market. Students examine current trends and emerging technologies in the mobile application industry. The topic emphasizes market demands, innovative development approaches, and future directions of mobile software.

Comparison of Native and Cross-Platform Technologies

Students compare native and cross-platform approaches to mobile application development. The topic emphasizes differences in performance, development efficiency, maintainability, and platform compatibility.

Analysis of Flutter and React Native Frameworks

Students analyze the architecture and development features of Flutter and React Native frameworks. The topic emphasizes their advantages, limitations, and practical applications in cross-platform development.

Principles of Clean Architecture in Mobile Applications

Students study the principles of Clean Architecture for designing scalable mobile applications. The topic emphasizes separation of concerns, maintainable code structure, and software flexibility.

SOLID Principles in Mobile Applications Students examine the SOLID principles of object-oriented software design in mobile development. The topic emphasizes writing modular, reusable, and maintainable application components.

Design Patterns (Singleton, Factory, Observer) in Mobile Applications. Students study common software design patterns used in mobile application development. The topic emphasizes improving code organization, flexibility, and maintainability through established design practices.

Accessibility Standards in Mobile Applications

Students examine accessibility standards for developing inclusive mobile applications. The topic emphasizes designing interfaces that are usable by people with diverse abilities and accessibility needs.

UX Research and Prototyping Tools for Mobile Applications

Students study user experience research methods and prototyping techniques for mobile applications. The topic emphasizes identifying user requirements and validating interface designs before implementation.

Designing Mobile Interfaces Using Figma. Students learn to design mobile application interfaces using Figma. The topic emphasizes creating wireframes, interactive prototypes, and visually consistent user interface designs.

Push Notification Technologies in Mobile Applications

Students study the technologies and mechanisms used for implementing push notifications in mobile applications. The topic emphasizes user engagement, message delivery, and notification management.

	GPS and Geolocation Services in Mobile Applications Students examine the implementation of GPS and geolocation services in mobile applications. The topic emphasizes location tracking, map integration, and the development of location-aware applications. Sensors of Mobile Devices and Their Use. Students study the functionality of mobile device sensors and their integration into applications. The topic emphasizes utilizing sensor data to create interactive and context-aware mobile solutions. Working with Multimedia Resources in Mobile Applications Students learn techniques for integrating multimedia resources into mobile applications. The topic emphasizes efficient handling of images, audio, video, and animations to enhance user experience. Offline-First Concept in Mobile Applications. Students study the Offline-First approach to mobile application development. The topic emphasizes local data storage, synchronization strategies, and reliable application functionality without continuous internet access.
Teaching Methods	The module is delivered through interactive lectures, inquiry-based and project-based learning, hands-on programming laboratories, mobile application development workshops, seminars, group discussions, and collaborative software development activities. Students design, implement, test, and deploy mobile applications using modern development frameworks, development environments, and software engineering practices. They develop programming, user interface design, debugging, testing, and problem-solving skills while creating cross-platform and native mobile applications for real-world scenarios. Independent learning and digital educational resources further promote critical thinking, technical communication, and the application of contemporary mobile application development technologies and industry best practices.
Prerequisites	Programming (DAS1110) Database (MAB1306) Computer Networks (KOT1306)
Exams and Assessment	Type of examination (written, Interview, project) and its weight Student performance is assessed through continuous assessment (20%), midterm assessment (30%), and a final written examination (50%). The continuous assessment (20%) evaluates students' participation in lectures and practical classes, mobile application

	development activities, and completion of independent study assignments, including Flutter programming tasks, Dart coding exercises, user interface design, REST API and Firebase integration, mobile application testing, presentations, technical reports, and practical projects related to Android and iOS application development. The midterm assessment (30%) consists of a written test and the evaluation of independent study projects, such as reports, presentations, analytical papers, and practical assignments covering Flutter architecture, Dart programming, mobile UI development, state management, local databases, REST APIs, Firebase services, authentication, mobile security, and application deployment. The final assessment (50%) is a written examination consisting of theoretical and practical questions designed to assess students' understanding of mobile application development, Flutter framework, Dart programming language, mobile application architecture, testing, optimization, security, and their ability to design, develop, and deploy cross-platform mobile applications for Android and iOS. Successful completion of all independent study assignments is required as an essential component of the module assessment.
Recommended Literature	8. Beginning-flutter_book. Marco L. Napoli – 2020 9. Dart Language Spec_5th edition-v2. 10 - 2021 10.Flutter-engineering_book. Majid Hajian - 2024 11.clean-architecture-a-comprehensive-beginners-guide-to-learn-the-realms-of-clean-architecture-from-a-z-2020. Elijah_Lewis 12.Flutter Documentation (Google) 13.Flutter_tutorialspoint_PDF-Copyright 2019 14.Mobile application development. (manual) Yuldoshev Sh. , Amirkulov Ch. – 2025
Responsible Lecturer	Bobomurodov Jasurbek Abdirasulovich teacher
Field	Description
Module Title	SOFTWARE REQUIREMENTS ANALYSIS
Module Code	DAS1110
Credit Points (ECTS)	4 Credits
Workload	Total workload: 120 hours Contact hours: 60 hours (34 hours Lecture, 26 hours Practical lessons) Self-Assignments: 60 hours

Frequency/Duration	VI semester
Module objective	The "Software Requirements Analysis" course is designed to study the theoretical and practical foundations of identifying, analyzing, modeling, documenting, and managing requirements during the software development process.
Module Learning Outcomes	Knowledge: LO1. The student knows and can explain the basic concepts, terminology, and theoretical foundations of the discipline of software requirements analysis; LO 2. Understands the operating principles of the main methods, algorithms, and technologies within the field of software requirements analysis; LO3. Knows the modern approaches and development trends in the field of study. LO4. Can distinguish between the main models and methods used to solve practical problems and understand their purpose. Skills: LO5. The student can effectively apply methods for gathering, analyzing, and documenting software requirements and use them in practical projects. LO 6. Thoroughly understands how to distinguish functional and non-functional requirements, formulate them correctly, and document them appropriately for the system. LO 7. They will have the skills to model system structure, visually represent and analyze requirements using UML diagrams. LO 8. They can develop a requirements document, review it, and apply and evaluate it in software projects. Competencies LO 9. The student can independently carry out the processes of analyzing software requirements, identifying system needs, and documenting them, and can apply them effectively in practical projects. LO 10. The student will have professional skills to understand system architecture based on requirements, create UML models, and evaluate and optimize software solutions. LO 11. The student can participate in identifying client requirements in real projects, converting them into technical requirements, and managing the software development process.
Content	Lectures Introduction to Software Engineering. Introduces the scope and objectives of software engineering as a discipline. Discusses its relevance to individual project work. Reviews the connection

between software engineering principles and project-based learning.

Concept of an individual project, project activity, and project culture. Defines the concept of an individual project within an educational context. Discusses the components of project activity and project management. Introduces the notion of project culture and its role in successful outcomes.

Methodology and technology of software development. Reviews common methodologies used in software development projects. Discusses supporting technologies that facilitate development activities. Explores how methodology choice influences project execution.

Designing software topics and problems. Explains approaches for identifying and framing suitable project topics. Discusses techniques for defining a clear problem statement. Reviews criteria for evaluating topic feasibility and relevance.

Programming concepts. Consistency – evidence and its justification. Reviews core programming concepts relevant to project development. Discusses the importance of logical consistency in software design. Explains how to justify design and implementation decisions with evidence.

Methods for defining software objectives and breaking them down into tasks, verifying specificity and topic relevance. Introduces techniques for defining clear, specific project objectives. Discusses methods for decomposing objectives into manageable tasks. Reviews criteria for verifying that tasks remain relevant to the overall topic.

Logic of actions and sequence of steps in planning an individual project. Explains the logical flow required for effective project planning. Discusses how to sequence project steps for efficient execution. Reviews common planning pitfalls and how to avoid them.

Systematization of the software design and development process. Reviews methods for organizing the design and development process systematically. Discusses documentation practices that support process systematization. Explores how systematic organization improves project outcomes.

Use of information technologies in research and project activities. Discusses the role of information technologies in supporting research and project work. Reviews common tools used for research, collaboration, and documentation. Explores practical examples of technology-supported project activities.

Organizational structure of projects. Reviews common organizational structures used to manage project teams and activities. Discusses roles and responsibilities within a typical project structure. Explores how structure influences project coordination and accountability.

Project Financing. Introduces basic concepts of project financing and budgeting. Discusses common funding sources and resource allocation methods. Reviews considerations for managing project costs effectively.

Communication barriers in publicly defending the results of a software project. Discusses common communication barriers encountered when presenting project results. Reviews strategies for overcoming these barriers effectively. Explores techniques for engaging and persuading an audience.

Presentation of the educational project results. Analysis of the project implementation, and drawing conclusions. Reviews best practices for presenting educational project outcomes. Discusses methods for analyzing project implementation success. Explains how to draw well-supported conclusions from project results.

Preparing possible formats for presenting the results. Reviews different formats suitable for presenting project results. Discusses criteria for selecting an appropriate presentation format. Explores examples of effective result presentation across formats.

Analysis of recommendations and identified errors. Discusses methods for analyzing feedback and recommendations received on a project. Reviews approaches for identifying and addressing project errors. Explores how error analysis contributes to project improvement.

Initial public presentation: topic, working hypothesis, relevance. Reviews the components of an initial public project presentation. Discusses how to clearly articulate a topic, working hypothesis, and its relevance. Explores techniques for engaging an audience from the outset.

Initial public presentation: research plan, expected results, project plan. Reviews how to present a research plan and expected project results. Discusses effective communication of an overall project plan. Explores best practices for structuring this portion of a presentation.

Practical lessons

Identifying and selecting problems related to the field of software engineering. Explores potential project topics within

the software engineering field. Applies criteria for assessing problem suitability and feasibility. Selects and justifies a chosen project problem for further development.

The role and importance of the design concept in developing project software (UI/UX): objectives, design tasks, and challenges. Reviews the objectives and tasks involved in UI/UX design for project software. Identifies common design challenges encountered during development. Discusses how design decisions align with overall project goals.

Methodology and technology of software operation. Methods for creating software design. Applies a selected development methodology to a sample project scenario. Explores supporting technologies relevant to the chosen methodology. Practices creating a preliminary software design based on project requirements.

Designing software topics and problems. Practices framing a clear, well-defined project topic. Applies techniques for articulating the associated problem statement. Reviews the topic and problem framing for feasibility and clarity.

Programming concept. Consistency – applying evidence and its justification in the project. Applies core programming concepts within the context of the individual project. Ensures logical consistency across project design and implementation choices. Documents justifications for key programming decisions made.

Defining the software's purpose and breaking it down into tasks, methods for verifying uniqueness and topic relevance. Collecting key materials on the topic. Defines the project's software purpose and decomposes it into actionable tasks. Applies methods for verifying the uniqueness and relevance of the chosen topic. Collects and organizes key reference materials supporting the project.

Logic of actions and sequence of steps in planning an individual project. Creating a calendar schedule for your work using the SCRUM methodology. Practices sequencing project actions in a logical order. Applies SCRUM methodology to construct a calendar-based work schedule. Reviews the schedule for feasibility and alignment with project milestones.

Systematization of the software design and development process. Culture of using information and information tools. Software development tools – methods, techniques, technologies. Organizes the software design and development process into a systematic workflow. .

Use of information technologies in research and project activities. Organizing work with scientific literature. Introduction to catalogs. Applies information technologies to support research activities related to the project. Explores the use of catalogs and databases for research purposes.

Use of information technologies in an individual project. Methods and forms of data presentation. Applies information technologies to support various stages of the individual project. Practices selecting appropriate methods and forms for presenting project data.

Presentation of the educational project results. Analysis of information, project implementation, and drawing conclusions. Prepares a presentation summarizing the educational project's results. Analyzes information gathered during project implementation.

Preparing possible formats for presenting the results. Explaining the obtained results. Explores different formats suitable for presenting the project's final results. Practices explaining obtained results clearly to an audience.

Analysis of recommendations and identified errors.

Correction of deficiencies. Searching for, comparing, and identifying the strengths and weaknesses of similar projects. Analyzes feedback and recommendations received on the project.

Independent Study Topics

Identifying and selecting problems related to the field of software engineering. Researches current problem areas within software engineering suitable for individual projects. Studies criteria used to evaluate problem significance and feasibility. Prepares a shortlist of potential project problems with justification.

The role and importance of the design concept in project software development (UI/UX): objectives, design tasks, and challenges. Studies UI/UX design principles relevant to project software development. Researches common design challenges faced in similar projects. Prepares a summary connecting design objectives to project goals.

Methodology and technology of software development. Methods for creating software designs. Researches development methodologies suitable for the chosen project. Studies design creation techniques applicable to the project's scope. Prepares a written comparison of methodology options.

Designing software topics and problems. Refines the project topic and associated problem statement independently. Researches similar projects to inform topic framing. Prepares a clear, justified topic and problem definition.

Programming concept. Validity – applying evidence and its justification in the project. Studies principles of logical validity as applied to programming decisions. Researches methods for supporting design choices with evidence.

Methods for defining the software's objectives and breaking them down into tasks, verifying uniqueness and topic relevance. Collecting key materials on the topic. Researches techniques for decomposing project objectives into tasks. Studies methods for verifying topic uniqueness and relevance. .

Logic of actions and sequence of steps in planning your personal project. Creating a calendar for your activities using the SCRUM methodology. Studies SCRUM methodology principles in greater depth. Researches best practices for sequencing project planning steps. Prepares a personal project calendar based on SCRUM scheduling techniques.

Systematization of the software design and development process. Culture of using information and information tools. Software development tools – methods, techniques, technologies. Researches tools and techniques for systematizing the development process. Studies best practices for effective use of information tools.

Using information technologies in research and project activities. Organizing work with scientific literature. Introduction to catalogs. Researches methods for organizing scientific literature relevant to the project. Studies the use of academic catalogs and databases for research.

Use of information technologies in the individual project. Methods and forms of presenting information. Studies various methods and forms for presenting project information. Researches information technology tools that support data presentation.

Presentation of the educational project results. Analysis of information, project implementation, and drawing conclusions. Analyzes information gathered throughout project implementation.

Prepare possible formats for presenting the results. Explain the obtained results. Researches alternative formats for presenting project results effectively. Studies techniques for clearly explaining technical findings to an audience.

	Analysis of recommendations and identified errors. Correction of deficiencies. Searching for, comparing, and identifying the strengths and weaknesses of similar projects. Reviews feedback and recommendations received on the project. Identifies and corrects deficiencies based on this feedback.
Teaching Methods	The module is delivered through interactive lectures, problem-based and inquiry-based learning, hands-on project planning workshops, SCRUM-based scheduling exercises, seminars, group discussions, and project-based activities. Students identify, design, and present individual software engineering projects using modern information technologies and presentation tools, and develop planning, analytical, and communication skills. Independent learning and digital educational resources further promote critical thinking, scientific communication, and the application of modern project development and defense concepts. .
Prerequisites	Algebra, Probability Theory, Mathematical Logic Basic knowledge of programming languages such as Python, Java, or C++ Data Structures English
Exams and Assessment	Student performance is assessed through continuous assessment (20%), midterm assessment (30%), and a final written examination (50%). The continuous assessment (20%) evaluates classroom participation, laboratory performance, project planning exercises, seminar presentations, and the successful completion of independent study assignments, including topic and problem selection reports, SCRUM-based calendar schedules, literature organization tasks, and UI/UX design analyses. The midterm assessment (30%) consists of written assignments, short in-class quizzes, and the evaluation of independent study projects, such as a project objective breakdown report, a data presentation format proposal, a presentation on information technology tools used in research, and an essay on software design methodology. The final assessment (50%) is a written examination consisting of theoretical questions and problem-solving tasks designed to assess students' understanding of software project planning and defense, analytical skills, and their ability to apply project management principles to practical scenarios. Successful

	completion of all independent study assignments is required as an essential component of the module assessment.
Recommended Literature	1. Sommerville, I. Software Engineering. Pearson Education, 10th Edition, 2015. 2. Pressman, R. S. , Maxim, B. R. Software Engineering: A Practitioner's Approach. McGraw-Hill, 8th Edition, 2015. 3. Wiegers, K. , Beatty, J. Software Requirements. Microsoft Press, 3rd Edition, 2013. 4. Robertson, S. , Robertson, J. Mastering the Requirements Process. Addison-Wesley, 2012. 5. Hull, E. , Jackson, K. , Dick, J. Requirements Engineering. Springer, 5th Edition, 2011. 6. Young, R. R. The Requirements Engineering Handbook. Artech House, 2004.
Responsible Lecturer	Sh. M. Yuldoshev - Head of the Department of Mathematical Modeling and Computer Sciences, TerDU, PhD.
Field	Description
Module Title	NUMERICAL METHODS AND ALGORITHMS
Module Code	SUA305
Credit Points (ECTS)	4 Credits
Workload	Total workload: 120 hours Contact hours: 60 hours (34 hours Lecture, 26 hours Practical lessons) Self-Assignments: 60 hours
Frequency/Duration	VI semester
Module objective	The main objectives of the Numerical Methods and Algorithms course are error analysis, approximation of functions, numerical integration, and methods for solving algebraic and transcendental equations (including systems of equations), The main objective of the course is to study error theory, approximation of functions, numerical integration, methods for solving algebraic and transcendental equations (including systems of equations), the conditions for the solvability of systems, numerical differentiation and integration, methods for approximating solutions to initial and boundary value problems for ordinary differential equations, and the development of algorithms and programs for the numerical solution of problems.
Module Learning Outcomes	<i>Knowledge:</i> LO1. Source and classification of errors, absolute and relative errors, errors in arithmetic operations, understanding the concepts of error theory and correct problem-solving.

LO 2. Know the numerical methods for solving systems of linear algebraic equations and their conditions for application, the convergence conditions of iterative methods, and the main steps for solving linear algebraic equations systems (LAES) using numerical methods.

LO3. Knowing how to factor the roots of nonlinear equations, the numerical methods for solving nonlinear equations, and the main steps for numerically solving a nonlinear equation and a system of equations.

LO4. Know the numerical methods for solving eigenvalue and eigenvector problems for matrices.

LO5. Know methods for approximate integration of functions and how to estimate their errors.

LO 6. Basic concepts used in numerical methods for ODE solutions (approximation error, finite differences, etc.), They know the classification of numerical solutions of the Cauchy problem for first-order ODE, and the approximate and numerical solution methods for boundary value problems for second-order ODE.

LO 7. Knowledge of the sources and classification of errors, absolute and relative errors, errors in arithmetic operations, and the concepts of error theory for correct problem solving.

Skills:

LO 8. Calculate the relative and absolute errors of an approximate value and calculate the relative and absolute errors of arithmetic operations.

LO 9. Apply numerical methods to solve given LAES, find solutions to LAES using numerical methods, and acquire the skills to solve them using computers.

LO 10. They will be able to factor nonlinear equations, apply numerical methods, and use a computer to obtain and analyze numerical results.

LO 11. They will acquire skills in the numerical solution of eigenvalue and eigenvector problems for matrices.

LO 12. They will acquire practical skills in the importance of the interpolation problem and in constructing interpolation polynomials for a precisely formulated problem.

LO 13. They will acquire skills in approximating definite integrals, evaluating their errors, and using computers to approximate integrals.

LO 14. They will acquire skills in applying numerical solution methods for ODE and in solving them on computers.

Competencies:

	LO 15. They will be qualified to independently apply numerical methods and approximation techniques to solve mathematical problems. LO 16. They will be able to assess the accuracy and errors of computational results, analyze the obtained results, and draw well-founded conclusions. LO 17. Will be able to use modern information technologies and computational tools to solve mathematical models numerically and to address practical problems.
Content	Lectures Main objectives of the course "Numerical Methods. " Error in problem solving. Error theory. Sources of errors. Absolute, relative, and limit relative error. Introduces the scope and objectives of studying numerical methods. Discusses common sources of error encountered in numerical problem solving. Methods for isolating and numerically solving the roots of nonlinear equations (simple iterations, bisection method). Reviews techniques for isolating root intervals of nonlinear equations. Explains the simple iteration method for approximating roots Numerical methods for solving nonlinear equations (the secant and Newton methods). Introduces the secant method and its iterative procedure. Explains Newton's method and its convergence characteristics. Iterative methods for solving linear algebraic systems. Gauss method. Reviews the Gaussian elimination method for solving linear systems. Discusses its computational structure and complexity. Approximate methods for solving a system of linear algebraic equations (Simple iteration, Zeidel method). Explains the simple iteration method applied to systems of linear equations. Introduces the Gauss-Seidel (Zeidel) method as an improved iterative approach. Computing the solution of a system of linear algebraic equations using software packages. Reviews software tools commonly used for solving linear systems numerically. Discusses practical considerations when implementing solutions in software. Simple iteration method for solving nonlinear systems of equations. Explains the extension of the simple iteration method to nonlinear systems. Discusses convergence requirements specific to systems of equations.

Newton's method for solving a system of nonlinear equations. Introduces the multivariable extension of Newton's method for systems. Explains the role of the Jacobian matrix in the solution process.

Computing the solution of a nonlinear system of equations using software packages. Reviews software tools used for solving nonlinear systems numerically. Discusses practical implementation considerations for such tools.

Numerical methods for finding eigenvalues and eigenvectors (Krylov method). Introduces the concept of eigenvalues and eigenvectors within numerical analysis. Explains the Krylov method for their approximate determination.

Methods of function approximation. Approximation by algebraic polynomials. Reviews the general purpose and principles of function approximation. Discusses polynomial approximation techniques in detail.

Lagrange interpolation formula. Introduces the Lagrange interpolation formula and its derivation. Discusses its application for constructing interpolating polynomials.

Newton's first and second interpolation formulae. Explains Newton's forward and backward interpolation formulas. Discusses scenarios where each formula is most appropriate

Methods for approximate calculation of exact integrals. Rectangular and trapezoidal methods. Reviews the rectangular and trapezoidal methods for numerical integration. Discusses their accuracy and computational simplicity.

Simpson's formula and its error in the approximation of exact integrals. Introduces Simpson's rule as a higher-accuracy integration method. Discusses the error characteristics associated with the formula.

Numerical methods for solving the initial value problem for ordinary differential equations. Successive approximation, Euler methods. Reviews the successive approximation method for solving initial value problems. Introduces Euler's method and its basic implementation.

The Runge-Kutta method for solving the initial value problem for ordinary differential equations. Introduces the Runge-Kutta method as an improved numerical technique for differential equations. Discusses its higher accuracy compared to Euler's method.

Practical lessons

Main objectives of the course "Numerical Methods. " Error in problem solutions. Error theory. Sources of errors.

Absolute, relative, and limit relative error. Calculates absolute, relative, and limit relative errors for sample numerical problems. Identifies sources of error in given computational scenarios.

Methods for isolating the roots of nonlinear equations and their numerical solution. Examples solved using simple iterations and the bisection method. Applies root isolation techniques to sample nonlinear equations. Implements the simple iteration method to approximate roots.

Numerical methods for solving nonlinear equations. Solving examples using the secant and Newton methods. Applies the secant method to solve sample nonlinear equations. Implements Newton's method and compares convergence speed with the secant method

Iterative methods for solving a system of linear algebraic equations. Finding solutions to a system of equations by the Gauss method. Applies the Gaussian elimination method to solve sample linear systems. Verifies solution accuracy through back-substitution.

Approximate methods for solving systems of linear algebraic equations. Application of simple iteration and the Zeidel methods. Implements the simple iteration method for solving linear systems. Applies the Gauss-Seidel (Zeidel) method to the same systems for comparison.

Solving examples and problems using the simple iteration method for solving nonlinear systems of equations. Applies the simple iteration method to sample nonlinear systems. Verifies convergence conditions before beginning iteration.

Newton's method for solving nonlinear systems of equations and its application. Implements Newton's method for solving sample nonlinear systems. Constructs and applies the Jacobian matrix within the solution process.

Numerical methods for finding eigenvalues and eigenvectors: the Krylov method and its practical application. Applies the Krylov method to determine eigenvalues and eigenvectors of sample matrices. Verifies computed results against known theoretical values.

Methods of function approximation. Approximation by algebraic polynomials. Solving examples and problems using the Lagrange interpolation formula. Constructs polynomial approximations for sample functions.

Newton's first and second interpolation methods. Applies Newton's forward interpolation formula to sample data sets. Applies Newton's backward interpolation formula for comparison
Approximate methods for exact integrals. Rectangle, trapezoidal, and Simpson methods. Implements the rectangle, trapezoidal, and Simpson's methods for numerical integration. Compares accuracy and computational cost across the three methods.

Numerical methods for solving the initial value problem for ordinary differential equations. Implements the successive approximation and Euler methods for solving initial value problems. Compares solution accuracy between the two approaches.

The Runge-Kutta method for solving the initial value problem for ordinary differential equations. Implements the Runge-Kutta method to solve sample initial value problems. Compares its accuracy and stability to the Euler method.

Independent Study

Main objectives of the course "Numerical Methods. " Error in problem solutions. Error theory. Sources of errors. Absolute, relative, and limit relative error. Researches additional sources of numerical error not covered in class, such as rounding and truncation error. Studies error propagation through multi-step calculations.

Methods for isolating the roots of nonlinear equations and their numerical solution. Examples solved using simple iterations and the bisection method. Studies additional root isolation techniques beyond those covered in class. Researches convergence guarantees for the bisection method.

Numerical methods for solving nonlinear equations. Solving examples using the secant and Newton methods. Researches convergence order comparisons between the secant and Newton methods. Studies conditions under which each method may fail to converge.

Iterative methods for solving linear algebraic systems. Finding solutions to systems of equations using the Gauss method. Studies computational complexity of the Gauss method for large systems. Researches variations such as Gauss-Jordan elimination.

Approximate methods for solving a system of linear algebraic equations. Simple iteration, application of the Zeidel methods. Researches convergence conditions in greater

	mathematical depth for both methods. Studies practical examples where one method converges faster than the other Solving a nonlinear system of equations using the simple iteration method: examples and problem solutions. Studies additional convergence criteria for the simple iteration method applied to systems. Researches practical examples from engineering or scientific contexts. Newton's method for solving nonlinear systems of equations and its application. Researches modified Newton's method variants for improved efficiency. Studies practical applications of Newton's method in engineering problems. Numerical methods for finding eigenvalues and eigenvectors: the Krillov method and its practical application. Studies the theoretical foundation of the Krylov method in greater depth. Researches alternative eigenvalue computation methods for comparison. Approximation methods for functions. Approximation using algebraic polynomials. Solving examples and problems using the Lagrange interpolation formula. Studies the error bounds associated with Lagrange interpolation. Researches practical scenarios where polynomial approximation is preferred. Newton's first and second interpolation methods. Studies the derivation of Newton's forward and backward interpolation formulas in greater depth. Researches scenarios favoring one formula over the other. Approximate methods for calculating exact integrals. Rectangle, trapezoidal, and Simpson methods. Studies error bounds associated with each integration method. Researches how step size selection affects accuracy for each method. Numerical methods for solving the initial value problem for ordinary differential equations. Researches additional numerical methods for solving initial value problems, such as the midpoint method. Studies stability considerations for different solution approaches. The Runge-Kutta method for solving the initial value problem for ordinary differential equations. Studies higher-order Runge-Kutta variants beyond the standard fourth-order method. Researches practical applications where Runge-Kutta methods are preferred.
Teaching Methods	The module is delivered through interactive lectures, problem-based and inquiry-based learning, computational laboratory sessions, numerical algorithm implementation exercises, computer simulations, seminars, group discussions, and project-

	based activities. Students implement and analyze numerical methods for solving algebraic, differential, and interpolation problems using specialized software packages, and develop analytical, computational, and problem-solving skills. Independent learning and digital educational resources further promote critical thinking, scientific communication, and the application of modern numerical methods concepts.
Prerequisites	Mathematical Analysis Fundamentals of Programming Mathematical Modeling
Exams and Assessment	Student performance is assessed through continuous assessment (20%), midterm assessment (30%), and a final written examination (50%). The continuous assessment (20%) evaluates classroom participation, laboratory performance, numerical method implementation exercises, seminar presentations, and the successful completion of independent study assignments, including root-finding method comparison reports, interpolation formula exercises, eigenvalue computation tasks, and numerical integration accuracy analyses. The midterm assessment (30%) consists of written assignments, short in-class quizzes, and the evaluation of independent study projects, such as a report comparing iterative methods for linear systems, a Newton's method application case study, a presentation on the Runge-Kutta method, and an essay on error theory in numerical computation. The final assessment (50%) is a written examination consisting of theoretical questions and problem-solving tasks designed to assess students' understanding of numerical methods and algorithms, analytical skills, and their ability to apply computational techniques to practical problems. Successful completion of all independent study assignments is required as an essential component of the module assessment.
Recommended Literature	1. Isralov M. I. Computational Methods. Part 1. – Tashkent: O'qituvchi, 2003. – 450 pp. 2. Israyelov M. I. Computational Methods. Part 2. – Tashkent: O'qituvchi, 2008. – 340 pp. 3. S. S. Irisqulov, K. D. Ismanova, M. Olimov, A. Imomov. Numerical Methods and algorithms. – N. : "Namangan" Publishing House, 2013, 244 p. 4. Kalitkin N. N. , Alshina E. A. Numerical Methods: in 2 vols. Numerical Analysis: textbook for students of higher professional

	education institutions. - M. : Publishing Center "Academy", 2013. Vol. 1. - 304 pp. Vol. 2. - 304 pp. 5. Richard L. Burden and J. Douglas Faires. Numerical Analysis. 9th Edition. Youngstown State University Press, 2011. – 895 p. 6. L. Ridgway Scott. Numerical Analysis. Princeton University Press, 2011. – 342 p. 7. Ismatullayev G. P. , Pulatov S. I. , Fayazov Q. S. Handbook of Numerical Methods. – Tashkent, University, 2006. – 140 p. 8. Bakvalov N. S. , Zhidkov N. P. , Kobelkov G. M. Numerical Methods. – Moscow: Bynom Publishing House. Laboratory of Knowledge, 2011. – 640 pp. 9. Samarsky A. A. , Vabischevich P. N. , Samarskaya E. A. Problems and Exercises in Numerical Methods. – M. : Libronom Publishing House, 2009. – 208 pp. 10. Samarsky A. A. Introduction to Numerical Methods. – M. : Lan' Publishing House, 2009. – 288 pp.
Responsible Lecturer	Yuldashev Shamsiddin Mamarajabovich, Head of the Department of Mathematical Modeling and Computer Sciences at TerDU
Field	Description
Module Title	COMPUTATIONAL THINKING AND INTRODUCTION TO PROGRAMMING
Module Code	HTD305
Credit Points (ECTS)	5 Credits
Workload	Total workload: 150 hours Contact hours: 60 hours (34 hours Lecture, 26 hours Practical lessons) Self-Assignments: 90 hours
Frequency/Duration	VI semester
Module objective	The “Computational Thinking and Introduction to Programming” course introduces students to the fundamental concepts of computational thinking and programming. This course is designed to develop students' skills in analyzing problems, designing solutions, and implementing them using programming languages. The course covers fundamental programming concepts, problem-solving methods, and practical applications.
Module Learning Outcomes	Knowledge: LO1. Applying the principles of computational thinking to analyze and solve problems; LO 2. Developing algorithms and expressing them using pseudocode or diagrams;

	LO3. Designing and implementing object-oriented programs; LO4. Have practical skills in software development and testing. Skills: LO5. Will have the skill to analyze practical problems using computational thinking methods and to develop their algorithmic solutions; LO 6. Become proficient in working with data structures such as stacks, queues, and graphs; LO 7. Becomes proficient in applying recursive, dynamic, and parallel algorithms to practical problems; LO 8. Learns to use modern tools for testing programs, detecting faults, and debugging. Competenceis: LO 9. Will have the competence to independently develop efficient algorithms and software solutions for problems of varying complexity; LO 10. Develops the ability to design and implement software systems based on object-oriented programming; LO 11. Will be able to create and develop practical projects using the Python and JavaScript programming languages; LO12. Applies computational thinking approaches in the fields of modern information technologies, artificial intelligence, and software development.
Content	Lectures Introduction. Subject matter, objectives, and key concepts of the course. Introduces the scope and objectives of studying computational thinking as a discipline. Discusses the key concepts that form the foundation of the course. Strategies and algorithms for problem analysis and solution. Reviews systematic strategies for analyzing complex problems. Discusses how algorithms provide structured approaches to problem solving. Computational Thinking Concepts: Abstraction and Decomposition. Introduces abstraction as a technique for managing complexity. Explains decomposition as a method for breaking problems into smaller parts. Computational Thinking Concept: Algorithms and Patterns. Reviews the role of algorithms as core computational thinking tools. Discusses pattern recognition as a technique for identifying similarities across problems. Analysis of algorithms. Introduces criteria for analyzing algorithm performance and correctness. Discusses common approaches to evaluating time and space efficiency.

Functions and Procedures. Reviews the role of functions and procedures in structuring code. Discusses parameter passing and scope considerations.

Data Structures: Stacks, Queues, and Graphs. Introduces the structure and operations of stacks and queues. Reviews the fundamentals of graph data structures. Discusses typical use cases for each structure.

Search and Sorting Algorithms. Reviews common search algorithms, including linear and binary search. Discusses standard sorting algorithms and their efficiency characteristics.

Parallel and Recursive Computation Algorithms. Introduces the principles of recursive algorithm design. Discusses parallel computation approaches and their benefits.

Dynamic Programming. Introduces the principles underlying dynamic programming techniques. Discusses how overlapping subproblems and optimal substructure enable this approach. Reviews classic examples illustrating dynamic programming solutions.

Programming Paradigms. Reviews major programming paradigms, including procedural, object-oriented, and functional approaches. Discusses the strengths and typical use cases of each paradigm. Explores how paradigm choice influences program design.

OOP: Classes, objects, inheritance, and polymorphism. Introduces the core building blocks of object-oriented programming. Explains inheritance as a mechanism for extending existing classes. Discusses polymorphism and its role in flexible program design.

OOP: Encapsulation and abstraction. Reviews encapsulation as a means of protecting internal object state. Discusses abstraction as a tool for simplifying complex systems. Explores how these principles contribute to robust object-oriented design.

Code analysis. Introduces techniques for analyzing code quality and structure. Discusses common tools used for static and dynamic code analysis. Reviews how code analysis supports maintainable software development.

Debug and testing tools. Reviews common debugging techniques and tools used in software development. Discusses testing approaches that complement debugging efforts.

Fundamentals of Data Analysis. Introduces the basic principles and goals of data analysis. Discusses common techniques for exploring and interpreting datasets.

Artificial Intelligence and Computational Thinking. Explains the relationship between computational thinking and artificial intelligence. Discusses how AI systems apply computational problem-solving principles.

Practical lessons

Designing algorithms using flowcharts and pseudocode.

Practices representing algorithmic logic using flowcharts. Writes pseudocode for sample algorithmic problems.

Creating functions and procedures. Working with parameters and arguments. Defines functions and procedures for sample computational tasks. Practices passing parameters and arguments correctly.

Working with stacks, queues, and graphs. Implements stack and queue data structures and their core operations. Constructs simple graph structures and practices basic traversal.

Implement search and sorting algorithms and compare their efficiency. Implements common search algorithms such as linear and binary search. Implements standard sorting algorithms and measures their execution time.

Creating parallel and recursive algorithms. Implements recursive algorithms for sample computational problems. Practices designing basic parallel computation approaches.

Solving dynamic programming problems. Identifies problems suitable for a dynamic programming approach. Implements solutions using memoization or tabulation techniques.

Assessing the time and memory complexity of algorithms. Analyzes sample algorithms to determine their time complexity. Assesses memory usage patterns for the same algorithms. Compares complexity results across different algorithmic approaches.

Applying programming paradigms in practical examples. Implements a sample problem using different programming paradigms. Compares code structure and readability across paradigms. Discusses the practical trade-offs of each paradigm choice.

Object-oriented programming: creating classes and objects. Defines classes representing sample real-world entities. Creates and manipulates objects instantiated from these classes. Tests class behavior through sample method calls.

Using inheritance, polymorphism, encapsulation, and abstraction. Implements inheritance hierarchies for related classes. Applies polymorphism to enable flexible method behavior.

Applying code analysis and refactoring techniques. Applies static code analysis tools to a sample codebase. Identifies areas for improvement based on analysis results.

Verifying programs using debugging and testing tools. Uses debugging tools to trace and resolve program errors. Writes test cases to verify program correctness.

Complete a small project using NumPy, Pandas, and elements of artificial intelligence. Designs a small project integrating NumPy for numerical computation. Uses Pandas to process and analyze relevant data. Incorporates basic artificial intelligence elements into the project's functionality.

Independent Study:

Computational Models: The Turing Machine and Its Importance. Studies the theoretical foundation of the Turing machine model. Researches its historical significance to the development of computer science.

Parallel Models in Programming Languages. Researches how different programming languages support parallel computation models. Studies practical examples of parallel programming constructs.

Functional Programming: Key Concepts and Applications. Studies the core principles underlying functional programming. Researches practical applications where functional programming is advantageous.

Creating programs for working with databases. Researches basic techniques for connecting programs to databases. Studies example code demonstrating database read and write operations.

Real-world applications of algorithms: navigation systems and robotics. Studies how algorithms are applied within navigation systems. Researches algorithmic principles underlying robotics applications. Prepares a summary connecting algorithmic concepts to these real-world domains.

Programming languages and their types: low-level and high-level languages. Reviews the distinction between low-level and high-level programming languages. Researches examples and typical use cases for each language type.

Integrated Development Environments and Debugging Tools. Studies features commonly found in modern integrated development environments. Researches debugging tools available within these environments.

Complexity and Efficiency of Algorithms. Researches additional complexity analysis techniques beyond those covered

	in class. Studies real-world examples where algorithm efficiency significantly impacts performance. Comparison of Imperative and Declarative Programming Paradigms. Studies the defining characteristics of imperative and declarative paradigms. Researches example languages and code representing each paradigm. Fundamentals of Python Programming Language. Studies basic Python syntax and programming constructs. Researches introductory examples demonstrating Python's core features. Recursive and Dynamic Programming Algorithms. Researches additional examples of recursive and dynamic programming problems. Studies how these techniques relate to broader algorithmic design. Artificial Intelligence and Computational Thinking: Connections and Differences. Studies the conceptual overlap between artificial intelligence and computational thinking. Researches key distinctions between the two fields. Data Analysis with Python: The NumPy and Pandas Libraries. Studies additional NumPy and Pandas functionality beyond those covered in class. Researches practical data analysis examples using these libraries. Prepares a summary demonstrating combined library usage for a sample dataset.
Teaching Methods	The module is delivered through interactive lectures, problem-based and inquiry-based learning, hands-on programming laboratories, algorithm design workshops, computer simulations, seminars, group discussions, and project-based activities. Students design algorithms, implement data structures, and apply object-oriented and computational thinking principles using specialized software tools, and develop analytical, programming, and problem-solving skills. Independent learning and digital educational resources further promote critical thinking, scientific communication, and the application of modern computational thinking and programming concepts.
Prerequisites	Programming Database Data Structures and Algorithms
Exams and Assessment	Student performance is assessed through continuous assessment (20%), midterm assessment (30%), and a final written examination (50%). The continuous assessment (20%) evaluates classroom participation, laboratory performance, algorithm implementation exercises, seminar presentations, and the successful completion of independent study assignments, including flowchart and

	pseudocode design reports, data structure implementation tasks, recursive and dynamic programming exercises, and code analysis and debugging reports. The midterm assessment (30%) consists of written assignments, short in-class quizzes, and the evaluation of independent study projects, such as a report comparing programming paradigms, an object-oriented design case study, a presentation on functional programming concepts, and an essay on artificial intelligence and computational thinking. The final assessment (50%) is a written examination consisting of theoretical questions and problem-solving tasks designed to assess students' understanding of computational thinking and programming, analytical skills, and their ability to apply algorithmic principles to practical problems. Successful completion of all independent study assignments is required as an essential component of the module assessment.
Recommended Literature	1. Thomas Mailund, "Introduction to Computational Thinking". Apress, 2021, 657 p. 2. Allen B. Downey, "Think Python: How to Think Like a Computer Scientist". 3rd Edition, O'Reilly Media, 2024, 325 p. 3. Martin R. C. Clean Architecture: A Craftsman's Guide to Software Structure and Design. – Boston: Pearson Education, 2018. – 364 pages. 4. David D. Riley, Kenny A. Hunt, "Computational Thinking for the Modern Problem Solver", CRC Press, 2014, 406 p. 5. Al Sweigart. "Automate the Boring Stuff with Python". 2nd Edition. No Starch Press, 2020. 568 p. 6. D. S. Malik, "C++ Programming: From Problem Analysis to Program Design". Pearson Education, 2020, 800 p.
Responsible Lecturer	Murodov Sardorbek Komilovich, Lecturer, Department of Mathematical Modeling and Computer Science, TerDU
Field	Description
Module Title	PYTHON PROGRAMMING LANGUAGE
Module Code	PDT305
Credit Points (ECTS)	5 Credits
Workload	Total workload: 150 hours Contact hours: 60 hours (34 hours Lecture, 26 hours Practical lessons) Self-Assignments: 90 hours
Frequency/Duration	VI semester
Module objective	The Python Programming Language course is designed to teach students the theoretical foundations and practical capabilities of

	the Python programming language. This course develops students' skills in algorithmic thinking, analyzing programming problems, designing efficient algorithms, and creating software solutions using the Python programming language. The course covers data types, control statements, functions, data structures, object-oriented programming, standard modules, and working with modern libraries such as NumPy, Pandas, Matplotlib, and PyTorch. The acquired knowledge and skills are applied to solving practical problems, analyzing data, and creating artificial intelligence systems.
Module Learning Outcomes	Knowledge: LO1. Knows the syntax, basic operators, and data types of the Python programming language; LO 2. Understands the principles of operation of functions, modules, and data structures in the Python programming language; LO3. Knows the basic principles of object-oriented programming and their application in the Python programming language; LO4. Understands the functions and application areas of the NumPy, Matplotlib, Pandas, and PyTorch libraries. Skills: LO5. Will have the skill to develop algorithms and create software solutions using the Python programming language; LO 6. Master the skills to work with the List, Tuple, Dictionary, and Set data structures; LO 7. Becomes proficient in solving practical problems using files, standard modules, and regular expressions; LO 8. Develops the ability to create parallel programs using multithreading and multiprocessing technologies; Competencias: LO 9. Becomes proficient in designing and developing software systems based on object-oriented programming; LO 10. Develops the ability to process and analyze data using the NumPy and Pandas libraries; LO 11. Becomes proficient in graphically representing data and visualizing results using the Matplotlib library; LO12. Gains proficiency in working with tensors and simple neural networks using the PyTorch library and solving artificial intelligence problems.
Content	Lectures Introduction. The subject, purpose, and objectives of the Python programming language course. Introduces the scope

and goals of studying the Python programming language. Discusses the history and design philosophy behind Python

Arithmetic and logical operations. Mathematical functions. Reviews basic arithmetic and logical operators in Python. Explains built-in mathematical functions and their usage.

Programming branching and iterative processes. Introduces conditional statements for controlling program flow. Explains loop constructs, including for and while loops.

Working with functions. Lambda functions. Reviews the syntax and purpose of user-defined functions in Python. Discusses parameter passing and return values.

Data structures in Python (List, Tuple, Dictionary, Set). Reviews the structure and characteristics of Python's core data types. Compares use cases for lists, tuples, dictionaries, and sets.

Strings and their usage. Reviews string creation, manipulation, and formatting in Python. Discusses common string methods and their applications. Explores practical examples of text processing using strings.

Regular Expression. Introduces the syntax and purpose of regular expressions in Python. Discusses pattern matching techniques using the re module. Reviews practical use cases such as text validation and extraction.

Standard modules in Python (datetime, random, json, os). Reviews the functionality provided by commonly used standard library modules. Discusses practical applications of datetime, random, json, and os modules. Explores examples demonstrating each module's core features.

Working with files in Python. Explains methods for reading from and writing to files in Python. Discusses file handling modes and best practices. Reviews error handling techniques relevant to file operations.

Introduction to Object-Oriented Programming. Class and Object. Introduces the fundamental principles of object-oriented programming. Explains the concepts of classes and objects in Python. Discusses how to define and instantiate simple classes.

Inheritance, Encapsulation, and Polymorphism. Explains the concept and implementation of inheritance in Python classes. Discusses encapsulation as a means of controlling access to data. Reviews polymorphism and its role in flexible object-oriented design.

Abstraction and Meta-class. Introduces the concept of abstraction in object-oriented design. Discusses abstract

classes and methods in Python. Explores the concept and use cases of metaclasses.

Parallel Programming in Python: Multithreading and Multiprocessing. Reviews the principles of multithreading and multiprocessing in Python. Discusses the differences between the two approaches and their appropriate use cases.

NumPy: Working with Numerical Computations and Arrays. Introduces the NumPy library for efficient numerical computation. Reviews array creation, indexing, and manipulation techniques.

Matplotlib: visualizing data. Introduces the Matplotlib library for data visualization. Reviews techniques for creating common plot types.

Pandas: data processing and analysis. Introduces the Pandas library for structured data processing. Reviews core data structures such as Series and DataFrame.

PyTorch Library: Working with Tensors and Neural Networks. Introduces the PyTorch library and its role in machine learning. Reviews tensor operations as the foundation of PyTorch computations.

Practical lessons

Installing the Python programming environment. Variables, data types. Installs Python and configures a working development environment. Practices declaring variables and working with basic data types.

Calculating mathematical functions, logical operations, and expressions. Writes Python code to evaluate arithmetic and logical expressions. Applies built-in mathematical functions to solve sample problems.

Programming branching and looping processes (if, for, while). Implements conditional statements to control program flow. Writes for and while loops to handle repetitive tasks. Combines branching and looping to solve sample programming problems.

Creating and using functions. Lambda functions. Defines and calls user-defined functions with various parameter configurations. Writes lambda functions for concise, single-use operations. Tests function behavior with different input scenarios.

Python data structures: Working with List, Tuple, Dictionary, and Set. Practices creating and manipulating lists, tuples, dictionaries, and sets. Applies common operations and methods specific to each data structure. Compares performance and use cases across the different structures.

Processing text using Strings and Regular Expressions. Applies string methods to manipulate and format text data. Writes regular expressions to match and extract specific patterns. Tests pattern matching against sample text inputs.

Working with Python standard modules (datetime, random, json, os). Uses the datetime module to handle date and time operations. Applies the random module to generate sample data. Practices reading and writing JSON data and performing basic os module operations.

File creation, reading, writing, and editing operations. Practices creating, reading, and writing text files in Python. Implements file editing operations such as appending and updating content. Handles common file-related errors using exception handling.

Object-oriented programming. Defines classes and creates objects representing sample entities. Implements methods and attributes within class definitions. Applies inheritance, encapsulation, and polymorphism to a sample program.

Using abstraction and metaclasses. Implements abstract classes and methods in a sample Python program. Explores the use of metaclasses to control class creation behavior. Tests the behavior of abstract and metaclass-based designs.

Parallel Programming in Python: Multithreading and Multiprocessing. Implements a multithreaded program to perform concurrent tasks. Implements a multiprocessing solution for CPU-bound operations. Compares performance outcomes between threading and multiprocessing approaches.

NumPy and Matplotlib libraries. Performs numerical computations and array operations using NumPy. Creates data visualizations using Matplotlib based on computed results. Tests different plot types to represent sample datasets.

Pandas and PyTorch libraries. Uses Pandas to load, clean, and analyze a sample dataset. Builds a simple tensor-based computation using PyTorch. Discusses the combined use of both libraries in a data-driven workflow.

Independent Study:

Installing the Python programming environment. Variables, data types. Researches alternative Python environments and package managers beyond those covered in class. Studies best practices for organizing a Python development workspace. Prepares a summary comparing environment setup options.

Calculating mathematical functions, logical operations, and expressions. Studies additional built-in mathematical functions

available in Python. Researches common pitfalls in evaluating complex logical expressions. Prepares a set of practice problems demonstrating correct usage.

Programming branching and looping processes (if, for, while). Researches advanced looping techniques such as list comprehensions and generator expressions. Studies best practices for writing efficient conditional logic. Prepares example programs demonstrating these techniques.

Creating and using functions. Lambda functions. Studies advanced function features such as default arguments, *args, and **kwargs. Researches functional programming patterns using lambda functions. Prepares a summary of practical function design techniques.

Python data structures: Working with List, Tuple, Dictionary, and Set. Studies performance characteristics of each data structure in greater depth. Researches advanced operations and comprehensions for each structure. Prepares a comparative summary of use cases and efficiency.

Processing text using Strings and Regular Expressions. Researches advanced regular expression patterns and techniques. Studies real-world text processing applications using Python. Prepares example scripts demonstrating complex pattern matching.

Working with Python standard modules (datetime, random, json, os). Studies additional standard library modules not covered in class. Researches practical applications combining multiple standard modules.

File creation, reading, writing, and editing operations. Researches advanced file handling techniques, including working with binary files. Studies best practices for robust error handling during file operations.

Object-oriented programming. Studies advanced object-oriented design principles beyond those covered in class. Researches real-world examples of well-structured Python class hierarchies.

Using abstraction and metaclasses. Researches advanced use cases for abstract base classes in Python. Studies practical applications of metaclasses in real-world frameworks.

Parallel Programming in Python: Multithreading and Multiprocessing. Studies additional concurrency approaches, such as asynchronous programming with asyncio. Researches performance benchmarking techniques for concurrent Python programs.

	NumPy and Matplotlib libraries. Studies advanced NumPy operations such as broadcasting and vectorization. Researches additional Matplotlib plot types and customization options. Prepares example visualizations demonstrating these techniques. Pandas and PyTorch libraries. Studies advanced Pandas operations for data cleaning and transformation. Researches introductory neural network concepts using PyTorch. Prepares a summary connecting data processing to basic model building.
Teaching Methods	The module is delivered through interactive lectures, problem-based and inquiry-based learning, hands-on coding laboratories, computer simulations, seminars, group discussions, and project-based activities. Students write, test, and debug Python programs covering data structures, object-oriented programming, and scientific libraries such as NumPy, Pandas, and PyTorch, and develop programming, analytical, and problem-solving skills. Independent learning and digital educational resources further promote critical thinking, scientific communication, and the application of modern Python programming concepts.
Prerequisites	Programming Database Data Structures and Algorithms
Exams and Assessment	Student performance is assessed through continuous assessment (20%), midterm assessment (30%), and a final written examination (50%). The continuous assessment (20%) evaluates classroom participation, laboratory performance, coding exercises, seminar presentations, and the successful completion of independent study assignments, including data structure implementation reports, regular expression exercises, object-oriented programming projects, and NumPy/Matplotlib visualization tasks. The midterm assessment (30%) consists of written assignments, short in-class quizzes, and the evaluation of independent study projects, such as a file handling application report, an abstraction and metaclass case study, a presentation on multithreading and multiprocessing, and an essay on Pandas-based data analysis. The final assessment (50%) is a written examination consisting of theoretical questions and problem-solving tasks designed to assess students' understanding of the Python programming language, analytical skills, and their ability to apply programming principles to practical problems. Successful completion of all independent study assignments is required as an essential component of the module assessment.

Recommended Literature	1. Eric Matthes, Python Crash Course. – 3rd Edition. – San Francisco: No Starch Press, 2023. -544 p. 2. Narzullayev Anvar, Fundamentals of Programming in Python. -Tashkent: Akademnashr, 2021. -336p. 3. Fundamentals of Programming in Python. A Textbook. – Samarkand: SamDU Publishing, 2020, – 180 p. 4. Allen B. Downey, "Think Python: How to Think Like a Computer Scientist". 3rd Edition, O'Reilly Media, 2024, 325 p. 5. David D. Riley, Kenny A. Hunt, "Computational Thinking for the Modern Problem Solver", CRC Press, 2014, 406 p. 6. Al Sweigart. "Automate the Boring Stuff with Python". 2nd Edition. No Starch Press, 2020. 568 p. 7. Thomas Mailund, "Introduction to Computational Thinking". Apress, 2021, 657 p.
Responsible Lecturer	Murodov Sardorbek Komilovich, Lecturer, Department of Mathematical Modeling and Computer Science, TerDU.
Field	Description
Module Title	R PROGRAMMING LANGUAGE
Module Code	RDT305
Credit Points (ECTS)	5 Credits
Workload	Total workload: 150 hours Contact hours: 60 hours (34 hours Lecture, 26 hours Practical lessons) Self-Assignments: 90 hours
Frequency/Duration	VI semester
Module objective	
Module Learning Outcomes	Knowledge LO1. Have in-depth knowledge of the basic concepts, syntax, data types, operators, functions, and packages of the R programming language; LO2. Understanding the methods for collecting, storing, processing, analyzing, and managing data in the R environment, as well as their practical applications; LO3. Knowledge of the fundamental data structures of the R programming language (vector, matrix, array, list, factor, data frame) and the principles for working with them; LO4. Knowledge of the theoretical foundations of statistical analysis, descriptive statistics, probability theory, and methods of correlation and regression analysis; LO5. Understanding modern data visualization methods and the capabilities of ggplot2, plotly, and other R packages;

	LO6. Knowledge of the fundamental concepts, algorithms, and applications in the R environment for Data Science, Business Analytics, and Machine Learning. Skills LO7. Have skills in importing, exporting, and managing data from various sources (CSV, Excel, SQL databases, APIs, and web resources); LO8. Develop skills in cleaning, transforming, and preparing data for analysis using the tidyverse, dplyr, tidyr, and data. table packages of the R programming language; LO9. Master the skills to analyze data, interpret results, and draw scientific conclusions using statistical analysis methods; LO10. Have skills in creating graphs, charts, and interactive visualizations, as well as preparing analytical reports in the R programming environment; LO11. Develop skills in applying machine learning algorithms, creating predictive models, and evaluating their effectiveness. Qualifications LO12. Be proficient in independently performing data processing, statistical analysis, and evidence-based decision-making using the R programming language; LO13. Develop the competence to solve practical problems in analyzing, modeling, and forecasting data related to education, economics, finance, healthcare, marketing, and other fields; LO14. Develop the competence to develop innovative solutions, prepare analytical reports, and support data-driven management decisions using modern R programming tools in data science and analytics projects.
Content	Lectures Introduction to the R Programming Language. Students study the fundamentals of the R programming language and its applications in data analysis and statistical computing. The topic provides a foundation for programming with R. Data Types and Variables. Students explore the basic data types, variables, and data structures in R. The topic develops skills in storing and manipulating data efficiently. Vectors and Their Operations. Students study vector creation, indexing, and vectorized operations in R. The topic introduces the primary data structure used for numerical and statistical computations. Matrices and Multi-Dimensional Arrays. Students examine matrices and arrays for storing and processing multidimensional

data. The topic develops skills in matrix operations and data manipulation.

Lists and Data Frames. Students study lists and data frames as flexible data structures for organizing complex datasets. The topic emphasizes efficient data storage and management in R.

Control Flow and Loops. Students explore conditional statements and looping structures used to control program execution. The topic develops logical programming and automation skills.

Functions and Functional Programming. Students learn to create reusable functions and apply functional programming concepts in R. The topic promotes modular, efficient, and maintainable code development.

Working with Strings and Regular Expressions. Students study string manipulation and regular expression techniques for processing textual data. The topic develops practical skills for text analysis and pattern matching.

File Operations and Input/Output. Students explore methods for reading, writing, importing, and exporting data files in R. The topic develops skills in handling data from various file formats.

Data Cleaning and dplyr. Students study data cleaning techniques using the *dplyr* package. The topic emphasizes filtering, transforming, summarizing, and organizing datasets for analysis.

Visualization: ggplot2 Basics. Students learn the fundamental principles of data visualization using the *ggplot2* package. The topic introduces the creation of clear and informative statistical graphics.

Visualization: Advanced Methods. Students explore advanced visualization techniques for presenting complex datasets. The topic develops skills in creating customized and interactive graphical representations.

Fundamentals of Statistical Analysis. Students study descriptive and inferential statistical methods using R. The topic provides a foundation for analyzing and interpreting quantitative data.

Regression Analysis. Students examine regression models for analyzing relationships between variables. The topic develops skills in model fitting, interpretation, and prediction using R.

Machine Learning: Classification and Clustering
Students explore machine learning techniques for classification and clustering tasks. The topic introduces algorithms used for pattern recognition and data-driven decision-making.

R Markdown and Quarto Reports. Students learn to create dynamic reports using R Markdown and Quarto. The topic develops skills in integrating code, analysis, and documentation into reproducible reports.

Shiny: Interactive Web Applications. Students study the development of interactive web applications using the Shiny framework. The topic provides practical experience in building data-driven web interfaces with R.

Practices

RStudio Environment and Basic Operations. Students become familiar with the RStudio integrated development environment and its basic features. The topic develops practical skills in creating, running, and managing R programs.

Practical Work with Vectors. Students perform practical exercises on creating, indexing, and manipulating vectors in R. The topic develops skills in using vector operations for data analysis.

Creating and Editing a Data Frame. Students create, modify, and manage data frames containing structured datasets. The topic emphasizes efficient data organization and manipulation techniques.

Creating Functions and Control Flow. Students develop user-defined functions and implement conditional statements and loops in R. The topic promotes structured programming and code reusability.

Text Processing and Regular Expressions. Students perform practical tasks involving string manipulation and regular expressions. The topic develops skills for processing and analyzing textual data.

Reading Data from CSV and Excel Files. Students import datasets from CSV and Excel files into R for analysis. The topic provides practical experience in handling external data sources.

Data Cleaning with dplyr. Students apply the **dplyr** package to clean, filter, transform, and summarize datasets. The topic develops practical data preprocessing skills for statistical analysis.

Graphical Visualization with ggplot2. Students create statistical graphs using the **ggplot2** package. The topic develops skills in presenting data through clear and informative visualizations.

Advanced Visualization Methods. Students apply advanced visualization techniques to represent complex datasets. The

topic emphasizes customization and effective communication of analytical results.

Statistical Tests and Hypothesis Testing. Students perform statistical tests and evaluate hypotheses using R. The topic develops practical skills in interpreting statistical results and making data-driven conclusions.

Linear and Logistic Regression

Students build and evaluate linear and logistic regression models using real datasets. The topic develops predictive modeling and data interpretation skills.

Machine Learning: Modeling with caret. Students implement machine learning models using the **caret** package. The topic introduces model training, validation, and performance evaluation techniques.

Shiny Application Development. Students develop interactive web applications using the Shiny framework. The topic provides practical experience in creating user-friendly, data-driven web applications with R.

Student Independent Work

Installation and Configuration of R and RStudio. Students install and configure the R programming language and the RStudio development environment. The topic provides the foundation for developing and executing R programs.

R Syntax and Basic Structures. Students practice the basic syntax, operators, and programming structures of R. The topic develops fundamental programming skills for data analysis.

Collection of Vector and Matrix Problems. Students solve practical problems involving vectors and matrices using R. The topic develops computational and analytical skills for numerical data processing.

Data Frame Exploration. Students explore, organize, and analyze datasets using data frames. The topic develops practical skills in managing structured data.

Creating a Function Library. Students design and implement reusable functions to build a personal function library. The topic promotes modular programming and code reusability.

Line Processing Project. Students develop a project involving text processing and regular expression techniques. The topic emphasizes practical methods for analyzing and manipulating textual data.

Open Data Analysis. Students import and analyze publicly available datasets using R. The topic develops skills in exploratory data analysis and interpretation.

	ggplot2 Visualization Album. Students create a collection of statistical charts using the ggplot2 package. The topic develops skills in designing clear and informative data visualizations. Statistical Analysis Report. Students perform statistical analyses and prepare professional analytical reports. The topic develops skills in interpreting and presenting statistical findings. Regression Model Project. Students develop and evaluate regression models using real-world datasets. The topic emphasizes predictive analysis and model interpretation. ML Model Comparison. Students compare the performance of different machine learning models using common evaluation metrics. The topic develops skills in model selection and performance assessment. R Markdown Computational Report. Students prepare reproducible computational reports using R Markdown. The topic integrates programming, analysis, and documentation into a single report. Final Shiny Project. Students design and develop an interactive web application using the Shiny framework. The topic demonstrates the practical application of R for interactive data analysis. Predictive Modeling Using Real Data. Students create predictive models using real-world datasets and visualize the analytical results. The topic develops practical skills in machine learning and data-driven decision-making. Complex Data Science Mini-Project in the R Ecosystem Students complete a comprehensive data science project involving data collection, cleaning, analysis, visualization, and the preparation of an interactive report. The topic integrates the knowledge and skills acquired throughout the course.
Teaching Methods	The module is delivered through interactive lectures, hands-on programming laboratories, inquiry-based and problem-based learning, computer-based data analysis, seminars, group discussions, and project-based activities. Students develop practical programming skills in the R language by working with real-world datasets, implementing statistical methods, creating data visualizations, and applying reproducible data analysis techniques. Independent learning and digital educational resources further promote computational thinking, problem-solving abilities, scientific communication, and the effective use of R for statistical computing, data science, and research applications.
Prerequisites	Programming (DAS1110)

	Linear Algebra (CHA1306) Algorithms and Data Structures (ABSB210) Algorithm Design (ALL1506) Database (BBZB209)
Exams and Assessment	Type of examination (written, oral, project) and its weight Student performance is assessed through continuous assessment (20%), midterm assessment (30%), and a final written examination (50%). The continuous assessment (20%) evaluates students' participation in lectures and practical classes, R programming activities, and completion of independent study assignments, including data processing tasks, statistical analysis, data visualization with ggplot2, data cleaning using tidyverse and dplyr, machine learning exercises, Shiny applications, presentations, analytical reports, and mini-projects related to data science and R programming. The midterm assessment (30%) consists of a written test and the evaluation of independent study projects, such as reports, presentations, analytical papers, and practical assignments covering R programming fundamentals, data structures, statistical analysis, regression, data visualization, machine learning, R Markdown, and Shiny application development. The final assessment (50%) is a written examination consisting of theoretical and practical questions designed to assess students' understanding of R programming, data analysis, statistical modeling, data visualization, machine learning, and their ability to develop data-driven solutions and analytical applications using the R programming environment. Successful completion of all independent study assignments is required as an essential component of the module assessment.
Recommended Literature	1. Wickham H. , Grolemund G. R for Data Science. O'Reilly Media, 2023. — 561 p. 2. Crawley M. J. The R Book. Wiley, 2023. — 950 p. 3. James G. et al. An Introduction to Statistical Learning with R. Springer, 2023. 4. Kabacoff R. R in Action. Manning Publications, 2022. — 590 p. 5. Chang W. R Graphics Cookbook. O'Reilly Media, 2021. — 455 p. 6. Gillespie C. , Lovelace R. Efficient R Programming. O'Reilly, 2021. 7. Wickham H. Advanced R. Chapman & Hall, 2019. (advr.hadley.nz)

	8. Healy K. Data Visualization: A Practical Introduction. Princeton UP, 2019.
Responsible Lecturer	Bakhtiyor Abdullayev Panjiyevich Senior lecturer
Field	Description
Module Title	BIG DATA FUNDAMENTALS AND DATA ANALYSIS
Module Code	BADK305
Credit Points (ECTS)	5 Credits
Workload	Total workload: 150 hours Contact hours: 60 hours (34 hours Lecture, 26 hours Practical lessons) Self-Assignments: 90 hours
Frequency/Duration	VI semester
Module objective	Designed for students of the Software Engineering program, the course is intended to prepare them to collect, store, manage, process, and analyze large volumes of data in their future professional activities, making data-driven decisions, and developing innovative solutions in various fields. Within the discipline, competencies are developed for the effective use of Big Data technologies, distributed computing systems, data warehouses, cloud platforms, data visualization, statistical analysis, and machine learning methods.
Module Learning Outcomes	Knowledge LO15. In-depth knowledge of the concept of Big Data, its key characteristics (5Vs: Volume, Velocity, Variety, Veracity, Value), modern approaches to data analysis, and their application areas; LO16. Understanding the technologies and architectures for collecting, storing, processing, and managing large volumes of data; LO17. Knowledge of the operating principles of Hadoop, HDFS, MapReduce, Apache Spark, and NoSQL databases; LO18. Knowledge of data warehouses, the ETL process, OLTP and OLAP systems, and data integration methods; LO19. Have an understanding of the basic methods of statistical analysis, data visualization, and machine learning; LO20. Knowledge of Big Data security, confidentiality assurance methods, data protection mechanisms, and ethical principles. Skills LO21. Have skills to collect, clean, transform, and prepare large volumes of data for analysis; LO22. Develop skills in data processing and management using Hadoop, Spark, and NoSQL technologies;

	LO23. Acquire skills in analyzing data and interpreting results using Python, SQL, and modern analytical tools; LO24. Develop skills in data visualization and preparing analytical reports using Tableau, Power BI, and Python libraries; LO25. Develop skills in applying statistical analysis, regression, clustering, and machine learning algorithms to practical problems. Qualifications LO26. Be proficient in implementing data management, analysis, and results-driven decision-making processes in Big Data projects; LO27. Develop the ability to solve practical problems and make forecasts based on large volumes of data from various fields (education, healthcare, finance, marketing, transportation, etc.); LO28. Develop the ability to create innovative solutions and support data-driven decision-making using Big Data technologies, statistical analysis, and machine learning methods.
Content	Lectures Introduction to Big Data Analysis Students study the fundamental concepts of big data and its role in modern information systems. The topic provides an overview of technologies and analytical approaches for processing large-scale datasets. Historical Development of Big Data Students examine the evolution of big data, including the growth of data volumes, technological advancements, and emerging challenges. The topic highlights the impact of big data across various industries. Big Data Analytics System (BDAS) Architecture and Components Students explore the architecture and core components of Big Data Analytics Systems. The topic introduces distributed storage, processing frameworks, and analytical workflows. Collection and Preprocessing of Big Data Students study methods for collecting, cleaning, transforming, and preparing large datasets for analysis. The topic emphasizes data quality and preprocessing techniques. Hadoop Technology.Students examine the Hadoop ecosystem, including HDFS and MapReduce. The topic introduces distributed storage and parallel data processing concepts. Apache Spark.Students study Apache Spark as a framework for fast, distributed data processing. The topic emphasizes in-memory computation and scalable analytics.

NoSQL Databases.Students explore NoSQL database models, including key-value, document, column-family, and graph databases. The topic develops an understanding of flexible data storage for big data applications.

Data Warehouse and Data Integration.Students study data warehouse architectures and techniques for integrating data from multiple sources. The topic supports efficient data management and business intelligence applications.

Cloud Technologies and Big Data.Students examine cloud computing platforms used for storing and processing big data. The topic introduces scalable cloud-based analytics solutions.

Data Visualization.Students study techniques for visualizing large datasets using modern analytical tools. The topic develops skills in presenting complex information through meaningful graphical representations.

Statistical Analysis and Modeling.Students explore statistical methods and analytical models for extracting knowledge from big data. The topic develops quantitative analysis and decision-making skills.

Fundamentals of Machine Learning.Students study the basic concepts and algorithms of machine learning applied to big data. The topic introduces predictive modeling and pattern recognition techniques.

Big Data in Real-World Domains.Students examine practical applications of big data in healthcare, finance, business, education, manufacturing, and smart cities. The topic demonstrates the value of data-driven decision-making.

Big Data Security and Privacy.Students study security challenges, privacy protection, and ethical issues in big data environments. The topic emphasizes secure data management and regulatory compliance.

Future Trends in Working with Big Data.Students explore emerging technologies and future developments in big data analytics, artificial intelligence, and cloud computing. The topic highlights current research directions and industry innovations.

Managing Big Data Projects and DataOps.Students examine project management principles and DataOps practices for developing and maintaining big data solutions. The topic emphasizes collaboration, automation, and continuous data delivery.

Practical Big Data Project and Case Study.Students apply big data technologies and analytical methods to solve real-world

problems through a practical project and case study. The topic integrates theoretical knowledge with hands-on experience.

Practics

Starting the Big Data Project in Python.Students initialize a big data project using Python and configure the required development environment. The topic provides practical experience in building data processing workflows.

Apache Spark and Databases (SQL).Students integrate Apache Spark with SQL databases to query and process structured data. The topic develops practical skills in distributed data analysis.

Machine Learning Model Creation and HDFS.Students build machine learning models using datasets stored in the Hadoop Distributed File System (HDFS). The topic introduces scalable model training in distributed environments.

MapReduce.Students implement MapReduce programs for distributed data processing. The topic develops practical understanding of parallel computation using the Hadoop framework.

Working with Apache Spark and RDD.Students create and manipulate Resilient Distributed Datasets (RDDs) using Apache Spark. The topic develops skills in distributed data transformation and analysis.

Data Analysis and Application of NoSQL Databases.Students analyze large datasets and apply NoSQL databases for efficient data storage and retrieval. The topic emphasizes flexible data management techniques.

Working with MongoDB and Cassandra.Students perform practical tasks using MongoDB and Cassandra for managing unstructured and distributed data. The topic develops hands-on experience with modern NoSQL technologies.

Data Engineering: ETL Processes and Transformation Practices.Students design and implement ETL (Extract, Transform, Load) workflows for preparing data for analysis. The topic develops practical skills in data integration and transformation.

Real-Time Data Processing.Students implement real-time data processing techniques using modern big data frameworks. The topic introduces methods for analyzing continuously generated data streams.

Streaming and Data Visualization.Students process streaming data and create visual representations of analytical results. The

topic develops skills in monitoring and presenting real-time information.

Dashboards and Machine Learning Practice. Students build interactive dashboards and apply machine learning techniques to large datasets. The topic integrates data visualization with predictive analytics.

Regression, Classification, or Clustering. Students develop regression, classification, and clustering models using real-world datasets. The topic provides practical experience in applying machine learning algorithms for data analysis.

Parallel Computing. Students implement parallel computing techniques to improve the performance of large-scale data processing tasks. The topic develops skills in efficient distributed computation.

Data Cleaning and Data Preparation Practice
Students perform practical exercises in cleaning, transforming, and preparing datasets for analysis. The topic emphasizes data quality and preprocessing techniques essential for big data analytics.

Student Independent Work

Introduction to the Fundamentals of Big Data (BD) and Comparison with Traditional Data. Students study the fundamental concepts of Big Data and compare its characteristics with traditional data management systems. The topic introduces the challenges and advantages of processing large-scale datasets.

Big Data Architecture and Data Storage and Management
Students examine the architecture of Big Data systems and methods for storing and managing distributed datasets. The topic provides a foundation for scalable data processing.

NoSQL Technologies. Students explore NoSQL database technologies and their applications in Big Data environments. The topic introduces flexible data models for handling structured and unstructured data.

Data Analysis and Processing. Students study techniques for processing and analyzing large datasets using modern analytical tools. The topic develops practical skills in extracting meaningful information from data.

Hadoop Ecosystem and MapReduce Technology
Students examine the Hadoop ecosystem and implement MapReduce concepts for distributed data processing. The topic introduces scalable computing for large datasets.

Working with Apache Spark and RDD. Students explore Apache Spark and its Resilient Distributed Dataset (RDD) model for parallel data processing. The topic develops practical skills in distributed analytics.

Stream Processing Technologies (Apache Kafka, Apache Flink, Apache Storm). Students study stream processing frameworks for handling real-time data. The topic introduces technologies for collecting, processing, and analyzing continuous data streams.

Data Storage and Management Strategies in Distributed Databases. Students examine strategies for storing, managing, and retrieving data in distributed database systems. The topic emphasizes reliability, scalability, and fault tolerance.

Data Analysis Using Data Science and Python. Students apply Python and data science techniques to analyze and interpret large datasets. The topic develops practical programming and analytical skills.

Machine Learning Algorithms and Machine Learning Models Students study fundamental machine learning algorithms and develop predictive models using real-world data. The topic introduces supervised and unsupervised learning techniques.

Data Cleaning and Data Preparation Practices. Students perform practical exercises in cleaning, transforming, and preparing datasets for analysis. The topic emphasizes the importance of data quality in Big Data projects.

Data Visualization, Reporting, and Dashboard Creation. Students create visualizations, analytical reports, and interactive dashboards to present data effectively. The topic develops skills in communicating analytical results.

Big Data Security and Privacy. Students examine security, privacy, and ethical issues related to Big Data systems. The topic introduces methods for protecting distributed data and ensuring regulatory compliance.

Practical Application of Big Data and Machine Learning Technologies (Based on Spark MLlib). Students apply Spark MLlib to develop and evaluate machine learning solutions for Big Data applications. The topic integrates distributed computing with predictive analytics.

The Future of Big Data and New Trends: AI, IoT, Quantum Computing, and Project Work. Students explore emerging technologies shaping the future of Big Data, including artificial intelligence, the Internet of Things, and quantum computing. The

	topic concludes with a practical project integrating modern Big Data technologies.
Teaching Methods	The module is delivered through interactive lectures, inquiry-based and problem-based learning, hands-on data analysis laboratories, computer-based practical sessions, seminars, group discussions, and project-based activities. Students collect, process, analyze, and visualize large-scale datasets using modern big data frameworks, statistical methods, and data analytics tools, while developing data management, computational, and decision-making skills. Independent learning and digital educational resources further promote critical thinking, data-driven problem solving, scientific communication, and the application of big data technologies and analytical techniques to real-world business, scientific, and engineering challenges.
Prerequisites	Programming (DAS1110) Linear Algebra (CHA1306) Algorithms and Data Structures (ABSB210) Algorithm Design (ALL1506) Database (BBZB209) Computer Networks (KOT1306) Operating Systems (OPT1506) Probability and Statistics (EVS1406)
Exams and Assessment	Type of examination (written, oral, project) and its weight Student performance is assessed through continuous assessment (20%), midterm assessment (30%), and a final written examination (50%). The continuous assessment (20%) evaluates students' participation in lectures and practical classes, Big Data laboratory activities, and completion of independent study assignments, including data collection and preprocessing, Hadoop and Apache Spark exercises, NoSQL database tasks, ETL processes, data visualization, machine learning applications, presentations, analytical reports, and mini-projects related to Big Data technologies and data analysis. The midterm assessment (30%) consists of a written test and the evaluation of independent study projects, such as reports, presentations, analytical papers, and practical assignments covering Big Data architecture, Hadoop, Apache Spark, NoSQL databases, data warehousing, data visualization, statistical analysis, machine learning, and Big Data security. The final assessment (50%) is a written examination consisting of theoretical and practical questions designed to assess students' understanding of Big Data technologies, distributed

	data processing, data analytics, machine learning, data visualization, and their ability to develop data-driven solutions using modern Big Data platforms and analytical tools. Successful completion of all independent study assignments is required as an essential component of the module assessment.
Recommended Literature	9. "Big Data: A Revolution That Will Transform How We Live, Work, and Think" – Viktor Mayer-Schönberger, Kenneth Cukier 10. Sirojiddin Komolov, Sherzod Raxmatov: Machine Learning. Tashkent – 2019. 11. Haykin S. Neural Networks: A Comprehensive Foundation. 22nd ed. Translated from English. Moscow: Williams Publishing House, 2006. 452 pp. 12. "Spark: The Definitive Guide" – Bill Chambers, Matei Zaharia // CRC 2018 -480 pp. ISBN 13:9781138502383 13. Laurence Moroney. AI and Machine Learning for Coders: Reilly Media 2020. -390 p. ISBN 13: 9781492078197. 14. F. X. Khazratov, J. J. Atamuradov, H. I. Eshonqulov. BIG DATA AND DATA ANALYSIS - Bukhara 2021. 160 pages. 15. Artificial Intelligence and Expert Systems: Itisha Gupta & Garima Nagpal. 2018. -425 p. ISBN: 978-1-68392-507-1 16. "Data Science for Business: What You Need to Know about Data Mining and Data-Analytic Thinking" – Foster Provost, Tom Fawcett 17. Stephen Marsland. Machine Learning and Algorithmic Perspective. 2015. ISBN-13: 978-1-4665-8333-7. 18. Kaggle: Data Science and Big Data Problems.
Responsible Lecturer	Bakhtiyor Abdullayev Panjiyevich Senior lecturer
Field	Description
Module Title	SOFTWARE PROJECT MANAGEMENT
Module Code	DTLB305
Credit Points (ECTS)	4 Credits
Workload	Total workload: 150 hours Contact hours: 60 hours (34 hours Lecture, 26 hours Practical lessons) Self-Assignments: 90 hours
Frequency/Duration	VI semester
Module objective	
Module Learning Outcomes	Knowledge LO1. The student will know the basic concepts, principles, and phases of software project management and be able to explain

	their content , and will also have a complete theoretical knowledge of the project life cycle. LO2. The student theoretically understands project planning processes, methods for effective allocation of time, cost, and resources, and can analyze and differentiate their impact on the project. LO3. The student knows the basic principles of risk management, quality control, and monitoring methods, and theoretically understands and can explain how to apply them in software projects. LO4. The student knows the essence, advantages, and differences of project management methodologies such as Agile, Scrum, and Waterfall, and can analyze their areas of application. Skills LO5. The student will have the skills to plan software projects, distribute tasks, and create project schedules, and will apply them in practical projects. LO6. The student can apply project resource management, team organization, and effective communication skills in practical activities. LO7. The student will be able to identify risks, assess them, and make practical decisions to mitigate them, as well as develop appropriate measures. LO8. The student will acquire skills in applying agile and waterfall methodologies in real projects, organizing sprints, and managing the project process. Qualifications LO9. The student can independently plan, manage, and complete software projects at a professional level, ensuring project efficiency. LO10. The student can make decisions regarding team management, optimal resource allocation, and risk minimization in complex software projects. LO11. The student will be qualified to apply project management methodologies, quality control, and analyze project outcomes in real business and IT environments.
Content	Lectures Project Concept. Students study the fundamental concepts, characteristics, and objectives of projects. The topic provides a foundation for understanding project management in software and digital technology environments.

Project Life Cycle. Students examine the phases of the project life cycle, from initiation to closure. The topic introduces the structured processes required for successful project management.

Role and Responsibilities of the Project Manager. Students explore the roles, responsibilities, and leadership skills of a project manager. The topic emphasizes effective planning, coordination, communication, and decision-making throughout the project.

Software Development Principles. Students study the fundamental principles of software development, including quality, maintainability, reliability, and scalability. The topic provides a foundation for developing efficient and sustainable software systems.

Software Project Management Methodologies. Students examine software project management methodologies such as Waterfall, Agile, Scrum, and Kanban. The topic introduces structured approaches for planning and delivering software projects.

Project Initiation Process. Students study the activities involved in initiating a project, including defining objectives, identifying stakeholders, and assessing feasibility. The topic establishes the foundation for effective project planning.

Project Planning Method. Students explore project planning methods used to define scope, schedules, resources, budgets, and deliverables. The topic develops systematic planning skills for successful project execution.

Project Evaluation Method. Students study methods for evaluating project performance based on objectives, quality, schedule, and cost. The topic emphasizes continuous improvement and project success assessment.

Project Costs. Students examine project cost estimation, budgeting, and financial control techniques. The topic develops skills in managing project resources efficiently.

Project Risk Management. Students study methods for identifying, assessing, and managing project risks. The topic emphasizes strategies for minimizing uncertainty and improving project outcomes.

Project Implementation. Students explore the processes involved in executing project plans and delivering project outcomes. The topic focuses on coordinating activities, resources, and team performance.

The Concept of Project Dimensions. Students study the key dimensions of projects, including scope, time, cost, quality, and resources. The topic highlights their relationships in achieving project success.

Project Scope. Students examine techniques for defining, documenting, and controlling project scope. The topic emphasizes the importance of managing project requirements and deliverables.

Project Schedule and Cost Estimate. Students study scheduling techniques and cost estimation methods used in project planning. The topic develops practical skills in allocating time and financial resources effectively.

Project Progress Checkpoints and Project Review. Students explore methods for monitoring project progress through checkpoints and performance reviews. The topic emphasizes timely evaluation and corrective actions during project execution.

Project Report Preparation. Students study the principles and structure of professional project reports. The topic develops skills in documenting project progress, outcomes, and recommendations.

Managing Communications in Software Projects. Students examine communication strategies and stakeholder management in software projects. The topic emphasizes effective information sharing, collaboration, and coordination throughout the project lifecycle.

Practices

Comparison of Project Management Methodologies. Students compare Agile, Scrum, Kanban, and Waterfall methodologies through practical examples and case studies. The topic develops the ability to select the most appropriate methodology for different software development projects.

Analysis of the Life Cycle Stages of DT Projects. Students analyze the major phases of digital technology (DT) project life cycles. The topic emphasizes the objectives, activities, and deliverables of each project stage.

Setting DT Project Objectives. Students learn to formulate clear, measurable, and achievable project objectives using the SMART principle. The topic develops effective project goal-setting skills.

Using Project Planning Tools. Students explore modern project planning tools for organizing project tasks, resources, and schedules. The topic develops practical skills in planning and monitoring project activities.

Creating a Project Plan Using the Gantt Chart and PERT Techniques. Students develop project plans using Gantt charts and PERT techniques. The topic provides practical experience in scheduling tasks, identifying dependencies, and managing project timelines.

Applying Team Management Strategies. Students study strategies for managing software development teams and improving collaboration. The topic emphasizes leadership, motivation, and effective teamwork.

Practical Exercises on Distributing Roles and Responsibilities to Build an Effective Team. Students participate in practical activities to assign project roles and responsibilities. The topic develops organizational, coordination, and team management skills.

Risk Identification and Assessment. Students identify and assess potential risks in software projects using practical scenarios. The topic develops analytical skills for evaluating project risks.

Identifying Project Risks and Developing Risk Management Strategies. Students develop appropriate strategies to prevent, mitigate, or respond to identified project risks. The topic emphasizes proactive risk management for successful project delivery.

DT Project Quality Control Methods. Students study quality control techniques and apply quality standards in practical digital technology projects. The topic develops skills in monitoring and improving project quality.

Project Communication Management. Students develop communication plans and evaluate the effectiveness of communication tools used in software projects. The topic emphasizes stakeholder communication and information management.

Preparing Project Reports. Students prepare professional project reports that summarize project progress, results, and recommendations. The topic develops technical writing and project documentation skills.

Managing Communications in Software Projects. Students examine communication practices that support collaboration among project stakeholders and development teams. The topic emphasizes effective communication throughout the software project lifecycle.

Student Independent Work

Project Concept. Students study the fundamental concepts, characteristics, and objectives of projects. The topic provides a foundation for understanding project management in software and digital technology environments.

The Role and Responsibilities of the Project Manager

Students examine the leadership roles and responsibilities of a project manager throughout the project lifecycle. The topic emphasizes planning, coordination, decision-making, and effective resource management.

Principles of Software Development. Students study the core principles of software development, including software quality, maintainability, reliability, and user-centered design. The topic establishes best practices for developing robust software systems.

Software Project Management Methodologies. Students explore software project management methodologies such as Waterfall, Agile, Scrum, and Kanban. The topic develops an understanding of selecting appropriate methodologies for different project requirements.

Project Initiation Process. Students study the activities involved in initiating a software project, including defining objectives, identifying stakeholders, and assessing project feasibility. The topic provides the basis for effective project planning.

Project Planning Method. Students examine methods for planning project activities, schedules, resources, budgets, and deliverables. The topic develops systematic planning skills required for successful project execution.

Project Evaluation Method. Students study techniques for evaluating project performance, quality, schedule, and overall outcomes. The topic emphasizes continuous improvement and project success measurement.

Project Costs. Students examine methods of project cost estimation, budgeting, and financial control. The topic develops skills in managing project expenses throughout the project lifecycle.

Project Execution. Students study the processes involved in implementing project plans and coordinating project activities. The topic focuses on resource utilization, teamwork, and achieving project objectives.

Concept of Project Measures. Students explore key project performance measures such as scope, time, cost, quality, and productivity. The topic emphasizes the use of performance indicators to monitor project success.

	Project Schedule and Project Cost Estimation. Students study scheduling techniques and cost estimation methods used in software project management. The topic develops practical skills for planning project timelines and budgets accurately. Project Performance Checkpoints and Project Review Students examine methods for monitoring project progress through checkpoints, reviews, and performance assessments. The topic emphasizes timely evaluation and corrective actions during project execution. Preparation of Project Reports. Students study the principles and structure of professional project reports. The topic develops skills in documenting project activities, results, and recommendations for stakeholders. Integration and Project Quality Management. Students examine project integration processes and quality management principles. The topic emphasizes coordinating project activities while ensuring compliance with quality standards and customer requirements. Managing Communications in Software Projects. Students study communication management strategies for software development projects. The topic develops skills in stakeholder communication, team collaboration, and effective information exchange throughout the project lifecycle.
Teaching Methods	The module is delivered through interactive lectures, case-based and project-based learning, practical workshops, software project simulations, seminars, group discussions, and collaborative team activities. Students apply project management methodologies, scheduling techniques, risk assessment, resource planning, and quality assurance practices using modern project management tools and software. They develop leadership, teamwork, communication, decision-making, and problem-solving skills while managing software projects in realistic development environments. Independent learning and digital educational resources further promote critical thinking, professional communication, and the application of contemporary software project management principles and industry best practices.
Prerequisites	Mathematical Logic (DMMB111) Basic knowledge of programming languages such as Python, Java, or C++ (DASB113) Data Structures (BBZB209) English Language (XJTB106)
Exams and Assessment	Type of examination (written, oral, project) and its weight

	Student performance is assessed through continuous assessment (20%), midterm assessment (30%), and a final written examination (50%). The continuous assessment (20%) evaluates students' participation in lectures and practical classes, project management activities, and completion of independent study assignments, including project planning, Gantt charts, PERT analysis, Agile and Scrum practices, risk assessment, communication plans, presentations, analytical reports, and mini-projects related to software project management. The midterm assessment (30%) consists of a written test and the evaluation of independent study projects, such as reports, presentations, analytical papers, and practical assignments covering project life cycles, project planning, Agile, Scrum, Kanban, Waterfall methodologies, project scheduling, cost estimation, quality management, risk management, and project communication. The final assessment (50%) is a written examination consisting of theoretical and practical questions designed to assess students' understanding of software project management principles, project planning, scheduling, risk and quality management, project monitoring, team coordination, and their ability to apply project management methodologies to real-world software development projects. Successful completion of all independent study assignments is required as an essential component of the module assessment.
Recommended Literature	1. Personal Project Handbook, The IB MYP. 2. Personal Project Guide, International School of Bydgoszcz, Middle Years Programme 2022-2023. 3. PERSONAL PROJECT GUIDE, Dhahran Ahliyya Schools 4. Khorikova L. V. , A Guide for Students on Writing an Individual Project, Borovichi, 2020. 5. Babajanov A. R. , Project Organization and Management, Textbook, Tashkent 2015.
Responsible Lecturer	Amirkulov Chori Jumayevich teacher
Field	Description
Module Title	PROGRAMMING STYLES AND PARADIGMS
Module Code	DUP305
Credit Points (ECTS)	5 Credits
Workload	Total workload: 150 hours

	Contact hours: 60 hours (34 hours Lecture, 26 hours Practical lessons) Self-Assignments: 90 hours
Frequency/Duration	VI semester
Module objective	The goal of the "Software Project Management" course is to develop students' theoretical knowledge and practical skills in effectively managing the processes of planning, organizing, controlling, and closing software projects. Through this course, students will learn methods for managing project schedules, resources, costs, and quality, and will master their application in managing real-world software projects.
Module Learning Outcomes	Knowledge: LO1. Understanding of programming paradigms and their main differences, and the nature of imperative and procedural programming and their areas of application. ; LO 2. Principles of Object-Oriented Programming (OOP): encapsulation, inheritance, polymorphism, and the fundamentals of functional programming, as well as functional languages (Haskell, Lisp); LO3. Advantages and limitations of parallel and distributed programming paradigms, popular programming languages for each paradigm, and their areas of application; LO4. Historical development and modern trends of programming styles, hybrid approaches that combine different paradigms; Skills: LO5. Expressing and programming simple problems based on various programming paradigms, creating efficient code in imperative and procedural programming approaches. ; LO 6. Applying functions, recursion, and lambda expressions in functional programming languages, modeling problems based on simple rules in declarative and logical programming environments; LO 7. Construct simple programs in parallel and distributed environments, comparing the advantages and disadvantages of different paradigms; LO 8. Integrating multiple paradigms in practical programming, applying logical reasoning and an analytical approach during the programming process; Competencies: LO 9. understanding the essence, advantages, and application areas of programming paradigms (imperative, object-oriented, functional, logical, etc.);

	LO 10. programming using various programming styles, writing code that conforms to the paradigms, and comparing and analyzing them LO 11. Analyze a problem, select an appropriate programming paradigm, develop efficient algorithms, and apply them in real-world projects.
Content	Lectures History of the development of programming languages and an introduction to paradigms. Traces the historical development of programming languages from early designs to modern languages. Introduces the concept of a programming paradigm. Formal Languages and Grammars. Introduces the theoretical foundations of formal languages. Reviews the structure and classification of formal grammars. Imperative programming paradigm. Reviews the core principles underlying the imperative programming paradigm. Discusses how imperative languages express computation through sequential statements. Explores examples illustrating imperative programming style. Procedural programming style. Introduces procedural programming as a specific style within the imperative paradigm. Discusses the use of procedures and functions to organize code. Reviews examples demonstrating procedural program structure. Object-oriented paradigm fundamentals. Introduces the core principles of object-oriented programming. Reviews foundational concepts such as classes, objects, and encapsulation. Discusses how these principles differ from procedural approaches. Application in object-oriented programming languages. Reviews practical applications of object-oriented principles across different languages. Discusses examples illustrating inheritance and polymorphism in practice. Explores how OOP concepts translate into real-world software design. Functional programming paradigm. Introduces the core principles underlying functional programming. Discusses concepts such as pure functions and immutability. Reviews how functional programming differs from imperative approaches. Programming languages in the functional paradigm. Reviews specific programming languages designed around the functional paradigm. Discusses their distinguishing features and typical use cases.

Logical programming paradigm. Introduces the principles underlying logical (logic-based) programming. Discusses how logic programming expresses computation through formal logic rules. Reviews examples illustrating logic programming concepts.

Declarative programming paradigm. Introduces the declarative paradigm and its focus on describing outcomes rather than steps. Discusses how declarative programming contrasts with imperative approaches. Reviews examples of declarative programming languages and constructs.

Parallel programming paradigms. Reviews principles underlying parallel programming paradigms. Discusses common models for expressing parallel computation.

Distributed programming paradigms. Introduces principles underlying distributed programming paradigms. Discusses challenges specific to programming across distributed systems. Reviews examples of distributed programming models and approaches.

Programming in hybrid paradigms. Explains how hybrid paradigms combine features from multiple programming approaches. Discusses examples of languages supporting multiple paradigms. Reviews the benefits and challenges of hybrid paradigm usage.

Programming in DSL paradigms. Introduces the concept of domain-specific languages (DSLs). Discusses how DSLs are designed to address specific problem domains. Reviews examples illustrating DSL design and application.

Multi-threaded Programming Paradigms. Reviews principles underlying multi-threaded programming approaches. Discusses common challenges such as synchronization and race conditions. Explores examples of multi-threaded programming constructs.

Trends in modern programming paradigms. Discusses emerging trends shaping the evolution of programming paradigms. Reviews how new paradigms address contemporary software development challenges. Explores examples of recently developed paradigm innovations.

Programming Paradigms: Analysis and Comparison. Reviews and compares the paradigms covered throughout the course. Discusses criteria for selecting an appropriate paradigm for a given problem. Reflects on the broader significance of paradigm diversity in programming.

Practical lessons

Formal languages and grammars. Constructs simple formal grammars to represent sample languages. Practices deriving strings from a defined grammar.

Imperative programming paradigm. Implements sample programs using imperative programming constructs. Practices writing sequential statements to solve computational problems. Analyzes program state changes throughout execution.

Procedural programming style. Writes procedural programs organized around functions and procedures. Practices decomposing a problem into reusable procedures. Tests procedure behavior across different input scenarios.

Object-oriented paradigm fundamentals. Implements simple classes and objects representing sample entities. Practices applying core OOP principles such as encapsulation. Tests object behavior through instantiation and method calls.

Application in object-oriented programming languages. Implements inheritance and polymorphism in a sample object-oriented program. Applies OOP concepts to solve a practical programming problem. Evaluates the design for adherence to OOP principles.

Functional programming paradigm. Writes functional programs using pure functions and immutable data. Practices applying functional constructs such as map and filter.

Programming languages in the functional paradigm. Explores a functional programming language through simple coding exercises. Compares its syntax and behavior to previously used languages.

Logical and declarative programming paradigm. Writes simple logic-based programs using declarative constructs. Practices expressing problem solutions through logical rules. Tests the program's ability to derive correct conclusions from given facts.

Parallel programming paradigms. Implements a simple parallel programming example using available language constructs. Practices managing concurrent execution of tasks. Analyzes performance improvements achieved through parallelization.

Distributed programming paradigms. Explores a basic distributed programming scenario across multiple processes or nodes. Practices implementing simple communication between distributed components. Discusses challenges encountered in coordinating distributed execution.

Programming in hybrid paradigms. Implements a sample program combining features from multiple paradigms. Practices selecting appropriate paradigm elements for different parts of the problem. Evaluates the resulting design for clarity and effectiveness.

Multi-threaded programming paradigms. Implements a multi-threaded program to perform concurrent tasks. Practices managing thread synchronization to avoid race conditions. Tests the program for correct and reliable concurrent behavior.

Trends in modern programming paradigms. Explores a recently developed or emerging programming paradigm through a small coding exercise. Compares its features to more established paradigms. Discusses its potential relevance to future software development.

Independent Study Topics

Declarative approach in SQL. Studies how SQL exemplifies the declarative programming paradigm. Researches example queries illustrating declarative problem specification. Prepares a summary connecting SQL's design to declarative principles.

Fundamentals of functional programming in Haskell. Studies the core features of Haskell as a functional programming language. Researches example Haskell code demonstrating functional concepts.

Drawbacks of object-oriented programming paradigms. Researches common criticisms and limitations of object-oriented programming. Studies example scenarios where OOP introduces unnecessary complexity. Prepares a written summary of OOP's practical drawbacks.

Java OOP Practices. Studies best practices for object-oriented programming within Java. Researches example Java code demonstrating effective OOP design.

How MapReduce Technology Works. Researches the underlying principles of the MapReduce programming model. Studies example applications of MapReduce in large-scale data processing. Prepares a summary explaining MapReduce's operational workflow.

Practical Parallel Computing in Python. Studies Python libraries and constructs supporting parallel computing. Researches example implementations of parallel tasks in Python. Prepares a summary of practical parallel computing techniques in Python.

Visualizing Objects Using UML Diagrams. Studies UML notation for representing object-oriented designs. Researches

examples of UML diagrams applied to sample systems. Prepares original UML diagrams for a self-chosen object-oriented design.

Chomsky Grammars and Their Application. Studies the Chomsky hierarchy of formal grammars. Researches practical applications of Chomsky grammars in language processing. Prepares a summary connecting grammar classification to programming language design.

Examples of the imperative paradigm in C. Studies example C programs demonstrating imperative programming style. Researches how C's design reflects imperative principles. Prepares a summary of imperative programming characteristics in C.

The Role of Formal Grammars in Programming. Researches the broader role formal grammars play in programming language design. Studies how grammars support parsing and compiler construction. Prepares a summary connecting formal grammar theory to practical programming tools.

Metaprogramming and its possibilities. Studies the concept and techniques of metaprogramming. Researches example applications of metaprogramming across different languages. Prepares a summary of metaprogramming's practical possibilities.

Predicate Logic and its Application in Programming. Studies the fundamentals of predicate logic. Researches how predicate logic underlies logic programming languages. Prepares a summary connecting predicate logic to programming applications.

Algorithmic approaches and programming paradigms. Studies the relationship between algorithmic design and paradigm choice. Researches examples where paradigm selection influences algorithm implementation. Prepares a summary connecting algorithmic thinking to paradigm application.

Context-free grammars. Studies the structure and properties of context-free grammars. Researches their application in defining programming language syntax. Prepares a summary of context-free grammar concepts and examples.

Application of mixed paradigms in C#. Studies how C# supports multiple programming paradigms within a single language. Researches example C# code demonstrating mixed paradigm usage. Prepares a summary of C#'s multi-paradigm capabilities.

Teaching Methods	The module is delivered through interactive lectures, problem-based and inquiry-based learning, hands-on coding laboratories, comparative programming exercises, computer simulations, seminars, group discussions, and project-based activities. Students implement and compare programs across imperative, object-oriented, functional, logical, and parallel programming paradigms using multiple languages and tools, and develop analytical, programming, and problem-solving skills. Independent learning and digital educational resources further promote critical thinking, scientific communication, and the application of modern programming paradigm concepts.
Prerequisites	Mathematical Logic Fundamental Knowledge in Programming Languages such as Python, Java, or C++ English Language
Exams and Assessment	Student performance is assessed through continuous assessment (20%), midterm assessment (30%), and a final written examination (50%). The continuous assessment (20%) evaluates classroom participation, laboratory performance, paradigm implementation exercises, seminar presentations, and the successful completion of independent study assignments, including formal grammar analysis reports, functional programming exercises in Haskell, parallel computing tasks in Python, and UML diagram design projects. The midterm assessment (30%) consists of written assignments, short in-class quizzes, and the evaluation of independent study projects, such as a report comparing object-oriented and functional paradigms, a MapReduce case study, a presentation on metaprogramming techniques, and an essay on mixed-paradigm programming in C#. The final assessment (50%) is a written examination consisting of theoretical questions and problem-solving tasks designed to assess students' understanding of programming styles and paradigms, analytical skills, and their ability to apply paradigm-specific principles to practical problems. Successful completion of all independent study assignments is required as an essential component of the module assessment.
Recommended Literature	1. Robert W. Sebesta, Concepts of Programming Languages, Boston, Pearson Education -2019 (12th Edition) 2. Michael L. Scott, Programming Language Pragmatics. Burlington, Morgan Kaufmann -2021 (5th Edition)

	3. Harold Abelson, Gerald Jay Sussman, Julie Sussman, Structure and Interpretation of Computer Programs. Cambridge, MIT Press -2022 (2nd Edition, reprint) 4. Bjarne Stroustrup, The C++ Programming Language. Boston, Addison-Wesley -2013 (4th Edition) 5. John C. Mitchell. Concepts in Programming Languages. Cambridge: Cambridge University Press, 2003. 6. Terrence W. Pratt, Marvin V. Zelkowitz, T. V. Gopal. Programming Languages: Design and Implementation. New Delhi: Pearson Education, 2006. 7. John E. Hopcroft, Rajeev Motwani, Jeffrey D. Ullman. Introduction to Automata Theory, Languages, and Computation. Boston: Addison-Wesley, 2006. 8. Allen B. Tucker, Robert E. Noonan. Programming Languages: Principles and Paradigms. Boston: McGraw-Hill, 2006.
Responsible Lecturer	Amirkulov Chori jumayevich teacher
Field	Description
Module Title	PRACTICAL APPLICATIONS SUITE
Module Code/	ADP305
Credit Points (ECTS)	4 Credits
Workload	Total workload: 120 hours Contact hours: 60 hours (34 hours Lecture, 26 hours Practical lessons) Self-Assignments: 60 hours
Frequency/Duration	VI semester
Module objective	The main objective of the "Package of Applied Programs" course is to develop in students theoretical knowledge and practical skills in downloading from modern software suites, namely the MS Excel, Maple, and Matlab environments. The course covers solving problems from linear algebra, mathematical analysis, differential equations, and numerical methods using practical software, building mathematical models, automating problem-analysis processes, creating programs, and methods for graphical representation. acquires the necessary foundational knowledge to solve practical software problems efficiently using modern information technologies, and aims to develop competencies in implementing software and applications.
Module Learning Outcomes	Knowledge: LO1. Have knowledge of the content, classification, development stages, and application areas of practical application packages.

	LO 2. Knowledge of the operating principles and functional capabilities of text editors, spreadsheets, presentation programs, and databases. LO3. Understanding the theoretical foundations of tools for document creation, editing, formatting, and automation. LO4. Knowledge of methods for processing, analyzing, sorting, filtering, and visualizing data. LO5. Knowledge of modern office technologies, cloud services, information exchange tools, and information security principles. Skills: LO 6. Have skills in creating, editing, and formatting text documents and working with tables and graphic objects. LO 7. Ability to use formulas, functions, charts, and data processing tools in spreadsheets. LO 8. Be able to create slides, design presentations, and display presentations using presentation software. LO 9. Create databases, enter data, search, sort, and prepare reports. LO 10. Collaborate and exchange information using cloud technologies and modern application suites. Competencies: LO 11. Be competent in independently creating documents, reports, and information resources using office software packages in professional activities. LO 12. Be proficient in performing calculations, analyses, and forecasting using spreadsheets to solve practical problems. LO 13. Be proficient in preparing presentation materials at a professional level and delivering them effectively to an audience. LO 14. Acquire the skills to work with databases, manage data, and use them effectively. LO 15. Be proficient in automating and digitizing professional activities using modern information and communication technologies.
Content	Lectures Introduction. The suite of practical applications, its main characteristics, and areas of application. Introduces the range of practical mathematical software applications covered in the course. Discusses the main characteristics distinguishing these applications. Reviews typical areas where such software is applied. Solving equations and systems of equations in MS Excel using numerical and graphical methods. Reviews techniques for solving equations using Excel's numerical tools. Discusses

graphical methods for visualizing equation solutions. Explores examples solving systems of equations within Excel.

Numerical solution of algebraic problems using MS Excel. Introduces numerical approaches for solving algebraic problems in Excel. Discusses relevant built-in functions and tools. Reviews practical examples of algebraic problem solving.

Using Visual Basic programming elements in the MS Excel environment. Introduces Visual Basic for Applications (VBA) within the Excel environment. Discusses how VBA extends Excel's built-in capabilities. Reviews simple examples of VBA scripting for automation.

Modern practical computer applications. Reviews contemporary software applications used for practical mathematical computation. Discusses their comparative strengths and typical use cases. Explores trends shaping the development of such applications.

Evaluating expressions, functions, and their values in the Maple environment. Introduces the Maple environment and its core computational capabilities. Discusses methods for evaluating mathematical expressions and functions. Reviews practical examples of expression evaluation in Maple.

Solving equations, inequalities, and their systems in the Maple environment. Reviews Maple's tools for solving equations and inequalities. Discusses methods for solving systems of equations. Explores practical examples demonstrating Maple's solving capabilities.

Numerical solution of algebraic problems in the Maple environment. Introduces numerical solution techniques available within Maple. Discusses scenarios where numerical methods are preferred over symbolic ones. Reviews practical examples of numerical algebraic problem solving.

Calculation of definite, improper, and multiple integrals. Reviews Maple's tools for calculating definite integrals. Discusses methods for handling improper and multiple integrals. Explores practical examples demonstrating integral calculation.

Expanding a function into series and power series. Introduces techniques for expanding functions into Taylor and power series. Discusses Maple's built-in series expansion capabilities. Reviews practical examples of series expansion applications.

Basic tools for working with graphics in Maple. Reviews Maple's core graphing and visualization tools. Discusses

methods for plotting functions and datasets. Explores practical examples of graphical output customization.

Numerical solution of ordinary differential equations and its practical applications. Introduces numerical methods for solving ordinary differential equations in Maple. Discusses practical applications where such solutions are needed. Reviews examples demonstrating differential equation solving.

Calculating simple arithmetic expressions in MathCAD. Introduces the MathCAD environment and its computational interface. Discusses methods for entering and evaluating arithmetic expressions. Reviews practical examples of basic MathCAD usage.

Solving linear algebra problems in MathCAD. Reviews MathCAD's tools for solving linear algebra problems. Discusses matrix operations and systems of linear equations. Explores practical examples of linear algebra problem solving.

Solving mathematical analysis problems in MathCAD. Introduces MathCAD's capabilities for mathematical analysis tasks. Discusses methods for solving calculus-related problems. Reviews practical examples of mathematical analysis in MathCAD.

MathCAD's graphical capabilities. Reviews MathCAD's tools for creating graphs and visualizations. Discusses customization options for presenting computational results. Explores practical examples of graphical output in MathCAD.

Solving Differential Equations in MathCAD. Introduces techniques for solving differential equations within MathCAD. Discusses both analytical and numerical solution approaches. Reviews practical examples demonstrating differential equation solving.

Practical lessons:

Calculating the values of expressions in MS Excel. Solving mathematical problems in MS Excel. Enters and evaluates mathematical expressions using Excel formulas. Applies built-in Excel functions to solve sample mathematical problems. Verifies calculated results against expected values.

Solving equations and systems of equations in MS Excel. Solving geometric problems in Excel. Solves equations using Excel's numerical and graphical tools. Applies Excel functions to solve sample geometric problems. Verifies solution accuracy through graphical representation.

Numerical solution of algebraic problems using MS Excel. Applies numerical methods to solve algebraic problems within

Excel. Uses relevant built-in functions to automate calculations. Analyzes results for accuracy and consistency.

Programming linear, branching, and iterative processes in the MS Excel environment using Visual Basic programming elements. Writes VBA code to implement linear, branching, and iterative logic. Tests the code within the Excel environment for correct execution.

Working with the Maple environment menu and in interactive mode. Explores Maple's menu structure and interactive computation mode. Practices entering and executing basic commands. Familiarizes with the overall Maple workspace and tools.

Functions and computing their values, substituting expressions, simplifying, and factoring. Defines functions and computes their values in Maple. Practices substituting values into expressions.

Solving equations and systems of equations, and inequalities and systems of inequalities in the Maple environment. Solves individual equations and systems of equations using Maple. Applies Maple's tools to solve inequalities and systems of inequalities. Verifies solutions against expected mathematical results.

Numerical solution of algebraic problems in the Maple environment. Applies numerical solving techniques to sample algebraic problems in Maple. Compares numerical results with symbolic solutions where applicable.

Operations on vectors and matrices. Performs vector and matrix operations using Maple's linear algebra tools. Practices operations such as addition, multiplication, and inversion.

Numerical solution of linear algebra problems. Applies numerical methods to solve linear algebra problems in Maple. Solves sample systems of linear equations using matrix techniques. Analyzes solution accuracy and computational approach.

Calculating simple arithmetic expressions in MathCAD. Enters and evaluates arithmetic expressions within the MathCAD interface. Practices formatting and organizing calculations clearly. Verifies results against expected values.

Expanding a function into a series. Testing functions. Applies MathCAD's series expansion tools to sample functions. Tests functions for specific properties or behaviors. Analyzes the accuracy of the resulting series expansions.

Solving mathematical analysis problems in MathCAD. Differentiation and integration. Applies MathCAD's differentiation tools to sample functions. Uses MathCAD to compute definite and indefinite integrals. Verifies results against known analytical solutions.

Independent Study:

Calculating the values of expressions in MS Excel. Solving mathematical problems in MS Excel. Researches additional Excel functions not covered in class for expression evaluation. Studies alternative approaches for solving mathematical problems in spreadsheets.

Solving equations and systems of equations in MS Excel. Solving geometric problems in Excel. Studies advanced graphical methods for equation solving in Excel. Researches examples of geometric problem-solving using spreadsheet tools.

Numerical solution of algebraic problems using MS Excel. Researches additional numerical methods applicable within Excel. Studies accuracy considerations when using spreadsheet-based numerical solutions. Prepares a written summary comparing numerical approaches.

Programming linear, branching, and iterative processes in the MS Excel environment using Visual Basic programming elements. Studies advanced VBA programming techniques beyond those covered in class. Researches practical VBA automation examples for spreadsheet tasks. Prepares a written summary of effective VBA programming practices.

Working with the Maple environment menu and in interactive mode. Researches additional Maple menu features and interactive commands. Studies best practices for efficient interactive computation in Maple. Prepares a summary of useful Maple interface techniques.

Functions and computing their values, substituting expressions, simplifying, and factoring. Studies advanced function manipulation techniques in Maple. Researches examples of complex expression simplification and factoring. Prepares additional practice examples demonstrating these operations.

Solving equations and systems of equations, and inequalities and systems of inequalities in the Maple environment. Researches advanced equation and inequality solving techniques in Maple. Studies examples involving more complex systems. Prepares a summary of solving strategies for challenging problems.

	Numerical solution of algebraic problems in the Maple environment. Studies numerical solving methods available in Maple in greater depth. Researches scenarios where numerical methods outperform symbolic ones. Prepares a written comparison of solving approaches. Operations on vectors and matrices. Researches advanced vector and matrix operations available in Maple. Studies applications of linear algebra operations in practical problems. Prepares additional worked examples demonstrating these operations. Numerical solution of linear algebra problems. Studies numerical techniques for solving linear algebra problems in greater depth. Researches accuracy and efficiency considerations for large systems. Prepares a summary comparing numerical linear algebra approaches. Calculating simple arithmetic expressions in MathCAD. Researches additional MathCAD features for arithmetic calculation. Studies best practices for organizing calculations within MathCAD worksheets. Prepares a summary of effective MathCAD usage techniques. Expanding a function into a series. Testing functions. Studies advanced series expansion techniques available in MathCAD. Researches methods for testing function properties computationally. Prepares additional practice examples demonstrating these techniques. Solving mathematical analysis problems in MathCAD. Differentiation and integration. Researches advanced differentiation and integration techniques in MathCAD. Studies practical applications of these tools to complex problems. Prepares a summary of effective mathematical analysis techniques in MathCAD.
Teaching Methods	The module is delivered through interactive lectures, problem-based and inquiry-based learning, computer-based laboratory sessions, hands-on software application exercises, computer simulations, seminars, group discussions, and project-based activities. Students solve mathematical, algebraic, and analytical problems using practical software applications such as MS Excel, Maple, and MathCAD, and develop computational, analytical, and problem-solving skills. Independent learning and digital educational resources further promote critical thinking, scientific communication, and the application of modern mathematical software tools.
Prerequisites	Numerical methods;

	Fundamentals of Algorithms and Programming; Mathematical modeling;
Exams and Assessment	Student performance is assessed through continuous assessment (20%), midterm assessment (30%), and a final written examination (50%). The continuous assessment (20%) evaluates classroom participation, laboratory performance, software application exercises, seminar presentations, and the successful completion of independent study assignments, including Excel-based problem-solving reports, Maple function manipulation exercises, matrix and vector operation tasks, and MathCAD calculation assignments. The midterm assessment (30%) consists of written assignments, short in-class quizzes, and the evaluation of independent study projects, such as a VBA programming report, a Maple equation-solving case study, a presentation on numerical linear algebra methods, and an essay on MathCAD's analytical capabilities. The final assessment (50%) is a written examination consisting of theoretical questions and problem-solving tasks designed to assess students' understanding of practical application suites, analytical skills, and their ability to apply mathematical software tools to practical problems. Successful completion of all independent study assignments is required as an essential component of the module assessment.
Recommended Literature	1. Karimov Q. M. , Razzoqov I. D. Working in the MathCAD and MatLab Environments. For students of physics, mathematics, and professional education faculties at higher education institutions. A methodological guide. Qarshi, "Nasaf" Publishing House, 2014. 80 pages. 2. Dadajanov T. , Muhitdinov M. Fundamentals of Matlab. Textbook. FAN Publishing House of the Academy of Sciences of the Republic of Uzbekistan. Tashkent. 2008. 628 pages. 3. J. Walkenbach, "Excel VBA Programming," Hoboken, New Jersey, 2013, 411 p. 4. Maplesoft, Maple User Manual. Waterloo, 2014. 354 p. 5. Eshtemirov S. , Aminov I. B. , Nomozov F. Fundamentals of Working in the Maple Environment. Methodological Guide. SamDU, Samarkand, 2010. 6. Aminov I. B. , Eshtemirov S. , Suyarov A. M. Solving Mathematical Problems in the Maple Environment. Methodological Guidebook. SamDU, Samarkand, 2010.

	7. K. Getz, M. Gilbert. Programming in Microsoft Office. A Complete Guide to VBA: Translated from the English. – Kyiv: BHV Publishing Group, 2000. – 768 pp. 8. Makárov, E. G. Engineering Calculations in Mathcad. Coursebook. St. Petersburg: Piter, 2003. 9. Programming in MS Office Suites: A Textbook / S. V. Nazaryov, P. P. Melnikov, L. P. Smolnikov, et al. ; ed. by S. V. Nazaryov. – Moscow: Finansy i statistika, 2007. – 656 pp. 10. V. P. Dyakonov, Mathcad 2000: A Textbook. Piter, 2002.
Responsible Lecturer	Yuldashev Shamsiddin Mamarajabovich, Head of the Department of Mathematical Modeling and Computer Sciences at TerDU
Field	Description
Module Title	ARDUINO PROGRAMMING LANGUAGE
Module Code	ADT405
Credit Points (ECTS)	5 Credits
Workload	Total workload: 150 hours Contact hours: 60 hours (34 hours Lecture, 26 hours Practical lessons) Self-Assignments: 90 hours
Frequency/Duration	VI semester
Module objective	To teach students the fundamental knowledge of the Arduino programming language and to assist them in applying it to modern technologies in the field of robotics.
Module Learning Outcomes	Knowledge: LO1. Will have knowledge of the architecture of the Arduino platform, the operating principles of microcontrollers, and the Arduino IDE software environment. LO 2. Understands the syntax, data types, operators, functions, and program structure of the Arduino programming language. LO3. Knows the theoretical foundations of working with sensors, actuators, and various electronic devices, as well as their fields of application. LO4. Can explain the basic principles and possibilities of using Arduino technologies in robotics, automation, and IoT (Internet of Things) systems. Skills: LO 5. Will have the skills to create, edit, compile, and upload program codes to the microcontroller memory using the Arduino IDE environment.

	LO 6. Acquires skills in connecting, configuring, and programmatically controlling various sensors, actuators, LED indicators, displays, and motors on Arduino boards. LO 7. They will acquire the skills to develop programs designed to solve practical problems using the operators, functions, libraries, and control structures of the Arduino programming language. LO 8. Develops skills in creating Arduino-based automation, robotics, and IoT projects, organizing data exchange between devices, and testing systems. Competencies: LO 9. Becomes proficient in independently designing, programming, and implementing automation, monitoring, and control systems based on the Arduino platform to solve real-world problems. LO 10. Becomes qualified to assemble, configure, and diagnose electronic devices using various sensors and actuators, and to identify and resolve any hardware and software malfunctions that arise. LO 11. Becomes qualified to develop Arduino-based robotics and IoT projects, collect and process data, analyze results, and prepare and present project documentation.
Content	Lectures Introduction to Arduino Programming Language. Introduces the scope and purpose of the Arduino platform and its programming language. Discusses the history and development of Arduino within embedded systems. Reviews common applications of Arduino in education and prototyping. Classification of Microcontrollers and Arduino Architecture. Reviews the classification of microcontrollers used in embedded systems. Explains the internal architecture of Arduino boards. Discusses how architectural components support programmable control tasks. Information about the basics of electronics and Arduino. Introduces fundamental electronics concepts relevant to Arduino programming. Discusses circuit basics such as voltage, current, and resistance. Reviews how these principles apply to Arduino-based projects. Installing the Arduino IDE programming environment. Explains the process of installing and configuring the Arduino IDE. Discusses key features of the development environment. Reviews steps for connecting and testing an Arduino board.

Syntax and operators of the Arduino programming language.

Reviews the basic syntax rules of the Arduino programming language. Discusses common operators used in Arduino sketches. Explores simple code examples demonstrating correct syntax usage.

Program Control Structures and Features. Reviews conditional and looping control structures within Arduino programs. Discusses program flow features relevant to embedded applications. Explores examples demonstrating structured program design.

Arduino Programming Language Functions and Libraries.

Introduces the use of functions for organizing Arduino code. Discusses commonly used Arduino libraries and their purposes. Reviews how libraries extend the platform's built-in capabilities.

Working with Digital Input and Output Ports on Arduino.

Explains the configuration and use of digital input and output pins. Discusses reading sensor states and controlling output devices. Reviews practical examples of digital I/O operations.

Working with analog signals and PWM technology on Arduino.

Introduces analog signal reading using Arduino's analog input pins. Explains pulse-width modulation (PWM) and its role in analog-like output. Discusses practical applications of PWM in controlling devices.

Working with Sensor Technologies. Reviews common sensor types used in Arduino projects. Discusses how sensors interface with Arduino for data collection. Explores practical examples of sensor integration.

Working with drive motors and actuators. Introduces common actuator types, including motors and servos. Discusses methods for controlling actuators through Arduino.

Data Exchange Interfaces. Reviews common data exchange interfaces used with Arduino, such as I2C, SPI, and UART. Discusses how these interfaces enable communication between devices.

Understanding Display and Visualization Devices. Introduces common display devices compatible with Arduino. Discusses methods for visualizing sensor data and program output. Reviews practical examples of integrating displays into projects.

Working with Wireless Communication Technologies via Arduino in Robotics.

Reviews wireless communication technologies compatible with Arduino, such as Bluetooth and Wi-Fi. Discusses their application within robotics projects. Explores practical considerations for wireless data transmission.

Application of Internet of Things (IoT) technologies on Arduino. Introduces the concept of IoT and its relevance to Arduino-based projects. Discusses common IoT protocols and platforms compatible with Arduino. Reviews practical examples of Arduino-based IoT applications.

Application of Arduino in robotics and automation systems. Reviews how Arduino is applied within robotics and automation contexts. Discusses integration of sensors, actuators, and control logic in such systems. Explores practical examples of Arduino-based automation projects.

About modern Arduino-based smart systems. Introduces contemporary examples of smart systems built on Arduino. Discusses trends shaping the development of such systems. Reviews the broader significance of Arduino within modern embedded and IoT ecosystems

Practical lessons

Installing the Arduino IDE and familiarization with its interface. Installs the Arduino IDE and configures it for use with an Arduino board. Explores the interface's key features, including the code editor and serial monitor.

Developing a traffic light model using LED indicators. Designs a simple circuit using LEDs to represent a traffic light system. Writes Arduino code to control the sequence and timing of LED states.

Receiving input signals with buttons and switches on Arduino. Connects buttons and switches to Arduino digital input pins. Writes code to read and respond to button and switch states.

Controlling LED brightness using PWM. Configures an Arduino output pin for pulse-width modulation control. Writes code to vary LED brightness using PWM signals.

Analog data acquisition and processing using a potentiometer. Connects a potentiometer to an Arduino analog input pin. Writes code to read and process the analog voltage values. Displays or uses the processed data within a sample application.

Working with DHT11/DHT22 sensor data. Connects a DHT11 or DHT22 sensor to the Arduino board. Writes code to read temperature and humidity data from the sensor. Displays the collected data through the serial monitor or another output method.

Measuring distance using sensors. Connects a distance-measuring sensor, such as an ultrasonic sensor, to Arduino.

Writes code to calculate and display measured distances. Tests the sensor's accuracy across different distance ranges.

Servo Motor Control and Robot Manipulator Model.

Connects a servo motor to the Arduino board. Writes code to control servo motor positioning. Builds a simple robotic manipulator model demonstrating servo-based movement.

Working with the process of transmitting data to displays.

Connects a display device to the Arduino board. Writes code to transmit and present sensor or program data on the display. Tests the display output for accuracy and readability.

Controlling devices using mobile devices. Establishes a wireless connection between Arduino and a mobile device. Writes code enabling mobile-based control of connected devices.

Developing an IoT monitoring system using sensors.

Integrates multiple sensors into an Arduino-based IoT monitoring system. Writes code to collect and transmit sensor data over a network. Tests the system's ability to monitor and report data remotely.

Creating a mobile robot that detects obstacles and navigates its way.

Assembles a mobile robot platform equipped with obstacle-detection sensors. Writes code enabling autonomous navigation and obstacle avoidance.

Development and protection of a smart home, smart greenhouse, security system, or robotic device.

Designs and builds a chosen Arduino-based smart system project. Integrates sensors, actuators, and control logic to achieve the project's functionality. Prepares and presents the completed project for evaluation.

Independent Study:

The development history of the Arduino platform and working with modern microcontrollers.

Researches the historical development of the Arduino platform since its inception. Studies how modern microcontrollers have evolved from early designs. Prepares a summary tracing key milestones in Arduino's development.

A comparative analysis of the features of the Arduino IDE programming environment and its modern versions.

Studies the evolution of features across different Arduino IDE versions. Researches improvements introduced in more recent releases.

Comparative analysis of Arduino and other microcontroller platforms (ESP32, Raspberry Pi Pico, etc.).

Compares the features, capabilities, and use cases of Arduino against ESP32

and Raspberry Pi Pico. Studies scenarios where each platform offers distinct advantages.

Effective use of control structures and libraries in Arduino programming. Researches best practices for structuring control logic in Arduino sketches. Studies commonly used libraries beyond those covered in class.

Use of digital and analog input-output ports in Arduino projects. Studies advanced techniques for managing digital and analog I/O in complex projects. Researches examples where careful port management is essential.

Development of an Arduino-based environmental monitoring system. Researches design approaches for environmental monitoring systems using Arduino. Studies relevant sensor combinations and data collection strategies.

Application of Arduino technologies in smart agriculture systems. Studies examples of Arduino applications within smart agriculture. Researches sensor and automation technologies relevant to this domain. Prepares a summary of practical smart agriculture use cases.

Exploring the capabilities of Arduino and ESP32 platforms in IoT technologies. Researches the IoT-specific capabilities of both Arduino and ESP32 platforms. Studies example IoT projects built on each platform. Prepares a comparative summary focused on IoT application suitability.

Creating a remote monitoring system by integrating Arduino and cloud technologies. Studies methods for connecting Arduino projects to cloud-based platforms. Researches examples of cloud-integrated remote monitoring systems. Prepares a proposed system design incorporating cloud connectivity.

Creating intelligent control systems using sensors and actuators. Researches design principles for intelligent control systems combining sensors and actuators. Studies examples of feedback-based control implementations. Prepares a summary of intelligent control system design approaches.

The Role and Importance of Arduino Microcontrollers in Creating Robotic Systems. Studies the specific contributions Arduino makes to robotic system development. Researches example robotic projects built using Arduino microcontrollers. Prepares a summary highlighting Arduino's role in robotics.

Designing an Arduino-based smart home system and developing its software. Researches design considerations for Arduino-based smart home systems. Studies software

	architecture approaches suitable for such systems. Prepares a proposed smart home system design with accompanying software outline. Application of TinyML and artificial intelligence algorithms on Arduino devices. Studies the concept and constraints of TinyML on resource-limited Arduino devices. Researches example applications combining Arduino with lightweight AI algorithms. Prepares a summary of TinyML application possibilities on Arduino.
Teaching Methods	The module is delivered through interactive lectures, problem-based and inquiry-based learning, hands-on laboratory experiments, circuit-building and hardware prototyping exercises, computer simulations, seminars, group discussions, and project-based activities. Students design, program, and test Arduino-based circuits and systems using sensors, actuators, and wireless communication modules, and develop technical, analytical, and problem-solving skills. Independent learning and digital educational resources further promote critical thinking, scientific communication, and the application of modern embedded systems and IoT concepts.
Prerequisites	Computer Networks C or C++ programming language Algorithms and Programming English Language
Exams and Assessment	Student performance is assessed through continuous assessment (20%), midterm assessment (30%), and a final written examination (50%). The continuous assessment (20%) evaluates classroom participation, laboratory performance, circuit-building exercises, seminar presentations, and the successful completion of independent study assignments, including sensor data collection reports, PWM and analog signal exercises, wireless communication demonstrations, and Arduino platform comparison analyses. The midterm assessment (30%) consists of written assignments, short in-class quizzes, and the evaluation of independent study projects, such as an environmental monitoring system design report, a smart agriculture application case study, a presentation on IoT-integrated Arduino systems, and an essay on TinyML applications on Arduino devices. The final assessment (50%) is a written examination consisting of theoretical questions and problem-solving tasks designed to assess students' understanding of Arduino programming,

	analytical skills, and their ability to apply embedded systems principles to practical problems. Successful completion of all independent study assignments is required as an essential component of the module assessment.
Recommended Literature	1. Banzi M. , Shiloh M. Getting Started with Arduino. 4th Edition. – Sebastopol: O'Reilly Media, 2022. – 168 p. 2. Blum J. Exploring Arduino: Tools and Techniques for Engineering Wizardry. 2nd Edition. – Indianapolis: Wiley, 2019. – 384 p. 3. Margolis M. Arduino Cookbook. 3rd Edition. – Sebastopol: O'Reilly Media, 2020. – 800 p. 4. Monk S. Programming Arduino: Getting Started with Sketches. 3rd Edition. – New York: McGraw-Hill Education, 2022. – 192 p. 5. McRoberts M. Beginning Arduino. 2nd Edition. – New York: Apress, 2020. – 416 p. 6. Scherz P. , Monk S. Practical Electronics for Inventors. 5th Edition. – New York: McGraw-Hill Education, 2023. – 1056 p.
Responsible Lecturer	Yuldashev Shamsiddin Mamarajabovich, Head of the Department of Mathematical Modeling and Computer Sciences at TerDU.

4th ACADEMIC YEAR

Field
Module Title
Module Code
Credit Points (ECTS)
Workload
Frequency/Duration
Module objective
Module Learning Outcomes

Content

Teaching Methods

Prerequisites

Exams and Assessment

--

Responsible Lecturer

Field

Module Title

Module Code

Credit Points (ECTS)	
-----------------------------	--

Workload

Frequency/Duration

Module objective	
-------------------------	--

Module Learning Outcomes

Content

Teaching Methods

Prerequisites

Exams and Assessment

Recommended Literature

Responsible Lecturer

Field

Module Title

Module Code

Credit Points (ECTS)

Workload

Frequency/Duration
Module objective
Module Learning Outcomes
Content

Prerequisites

Exams and Assessment

Recommended Literature

Responsible Lecturer

Field

Module Title

Module Code

Credit Points (ECTS)

Workload

Frequency/Duration

Module objective

Module Learning Outcomes

Content

Teaching Methods
Prerequisites
Exams and Assessment
Recommended Literature

Responsible Lecturer

Field

Module Title

Module Code

Credit Points (ECTS)

Workload

Frequency/Duration

Module objective

Module Learning Outcomes

Content

Teaching Methods
Prerequisites
Exams and Assessment
Recommended Literature

Responsible Lecturer

Field

Module Title

Module Code

Credit Points (ECTS)

Workload

Frequency/Duration

Module objective

Module Learning Outcomes

Content

Teaching Methods

Prerequisites

Exams and Assessment

Recommended Literature

Responsible Lecturer

Field

Module Title

Module Code

Credit Points (ECTS)

Workload

Frequency/Duration

Module objective

Module Learning Outcomes

Content

Teaching Methods

Prerequisites

Exams and Assessment

--

Recommended Literature

--

Responsible Lecturer

Field

Module Title

Module Code

Credit Points (ECTS)

Workload

--

Frequency/Duration

Module objective

--

Module Learning Outcomes

--

Content

Teaching Methods

Prerequisites

Exams and Assessment

Recommended Literature

Responsible Lecturer

Field

Module Title

Module Code

Credit Points (ECTS)
Workload
Frequency/Duration
Module objective
Module Learning Outcomes
Content

Teaching Methods

Prerequisites

Exams and Assessment

Recommended Literature

Responsible Lecturer

Field

Module Title

Module Code

Credit Points (ECTS)

Workload

Frequency/Duration

Module objective

Module Learning Outcomes

Content

Teaching Methods
Prerequisites
Exams and Assessment
Recommended Literature

Responsible Lecturer

Field

Module Title

Module Code

Credit Points (ECTS)

Workload

Frequency/Duration

Module objective

Module Learning Outcomes

Content

Teaching Methods

Prerequisites

Exams and Assessment

Recommended Literature
Responsible Lecturer